

<https://heropy.blog>



박영웅

학습 개요

웹 프론트엔드 개발의 **핵심 줄기**를 학습!

중요도가 높은 내용을 위주로 학습.

난이도가 높고 중요도가 낮은 내용은 가볍게 학습하거나 생략.

넵.....

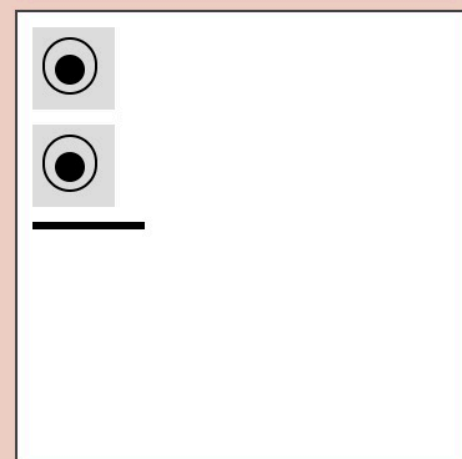
프론트엔드 개발

HTML, CSS, JS를 사용해 데이터를 그래픽 사용자 인터페이스(GUI)로 변환하고,
그것으로 사용자와 상호 작용할 수 있도록 하는 것.





HTML

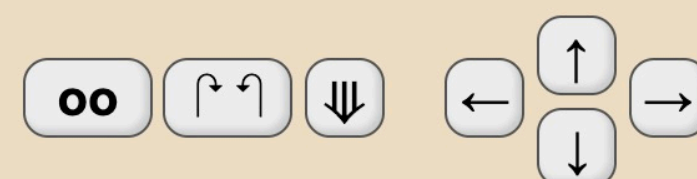


HTML + CSS



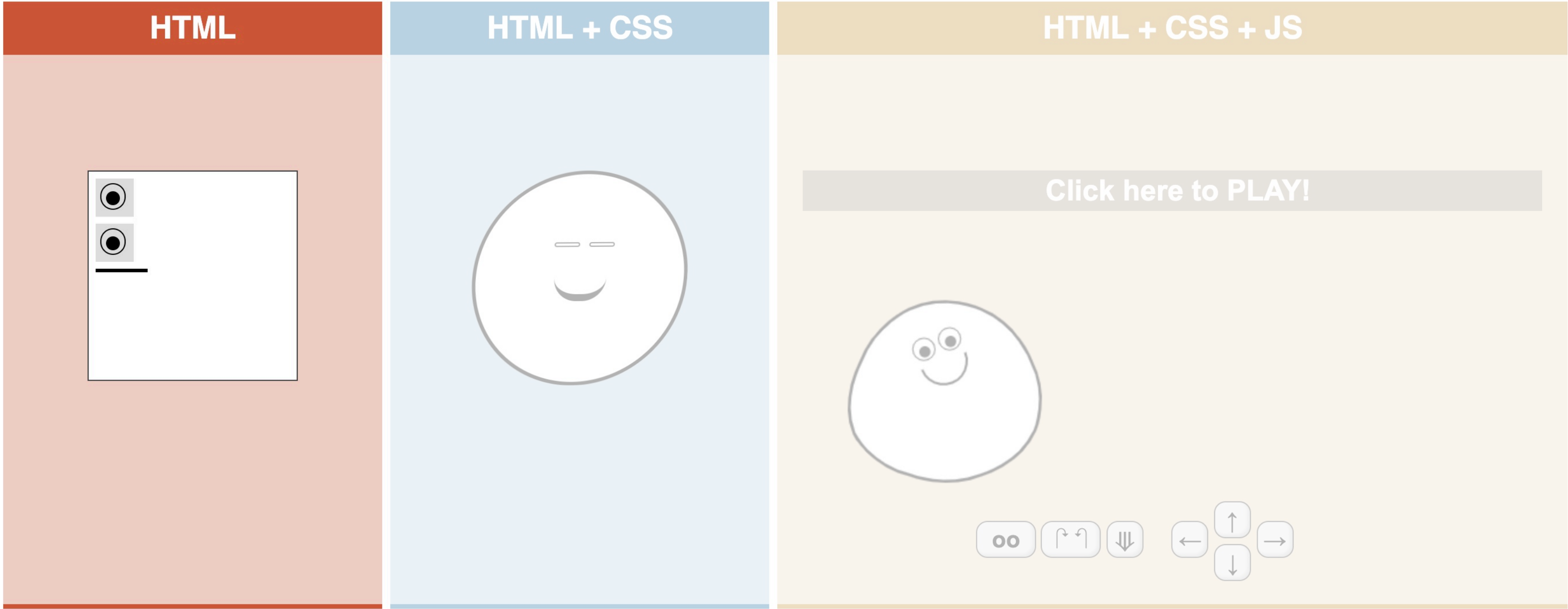
HTML + CSS + JS

Click here to PLAY!



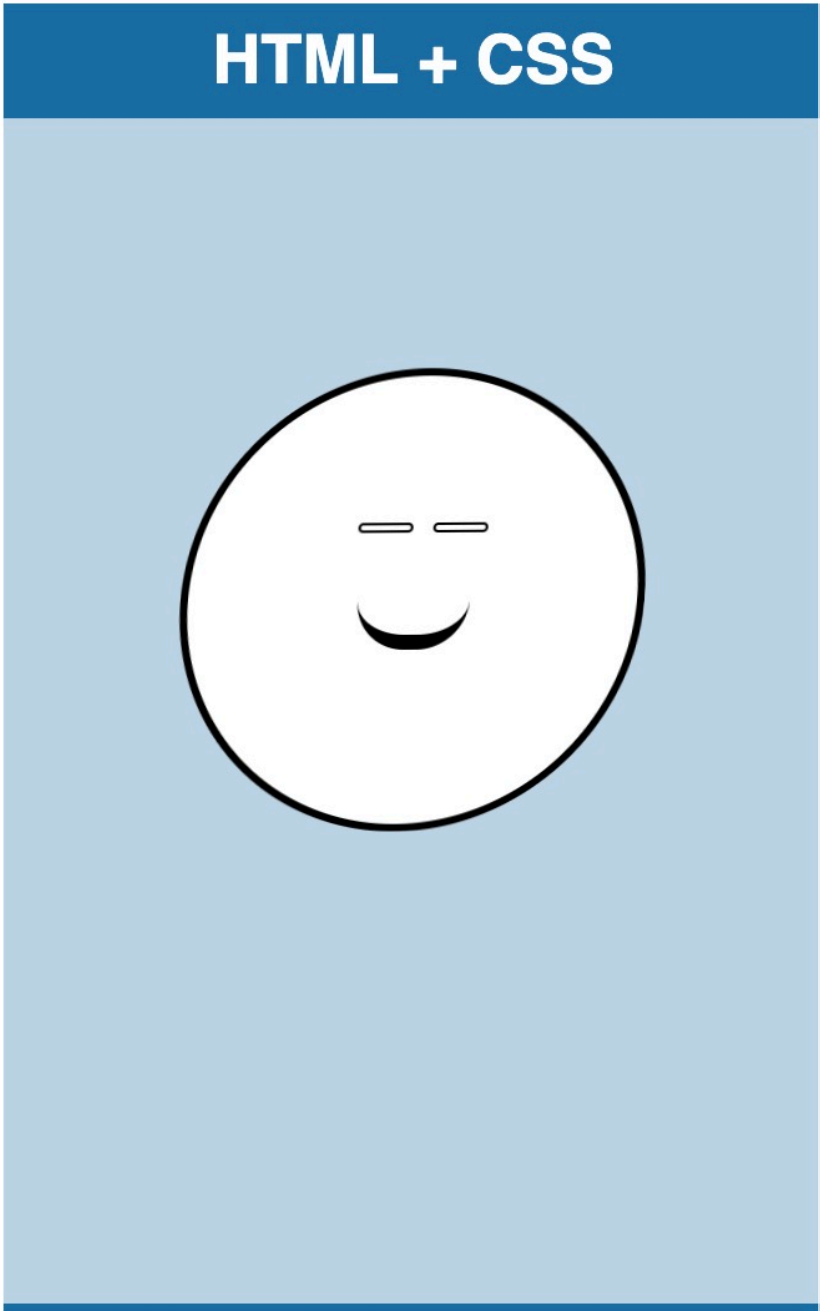
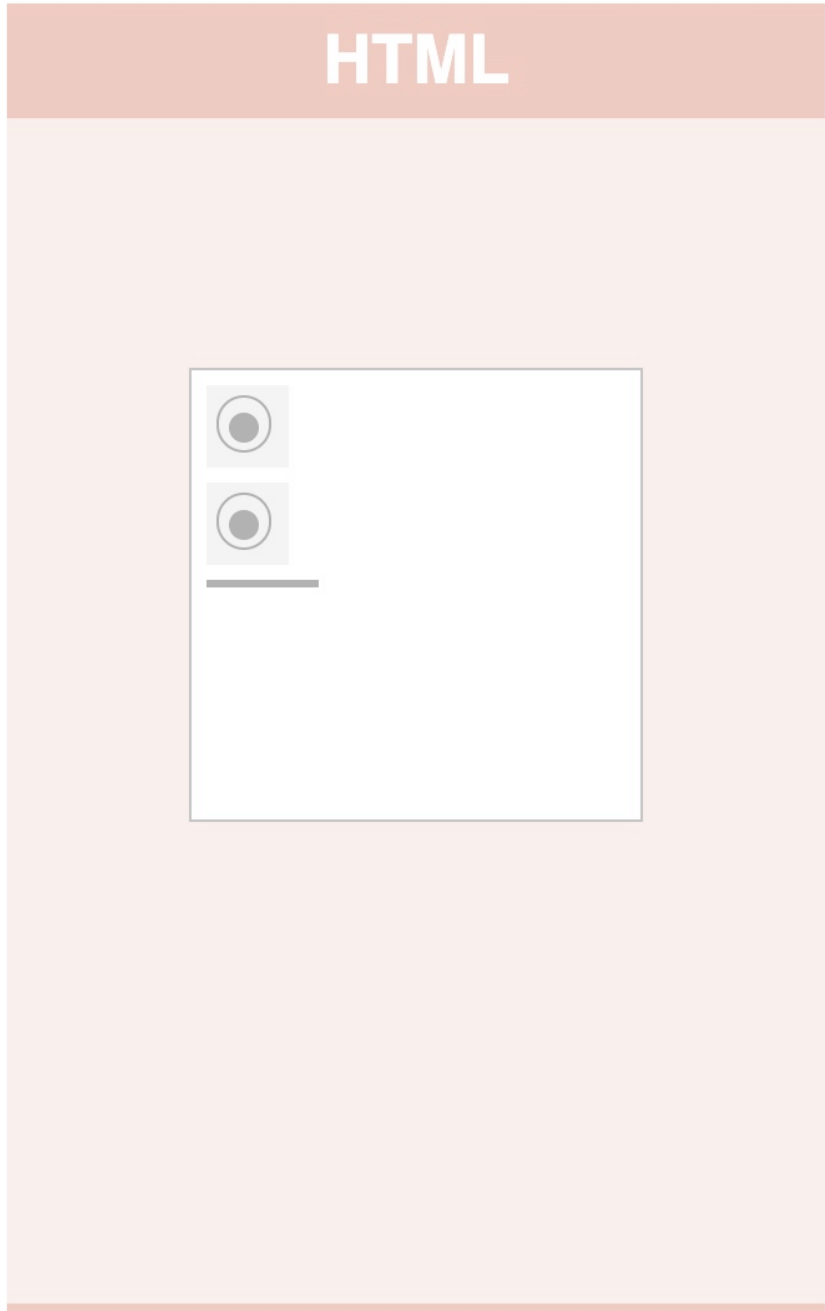
HTML (Hyper Text Markup Language)

페이지의 제목, 문단, 표, 이미지, 동영상 등 웹의 구조를 담당.



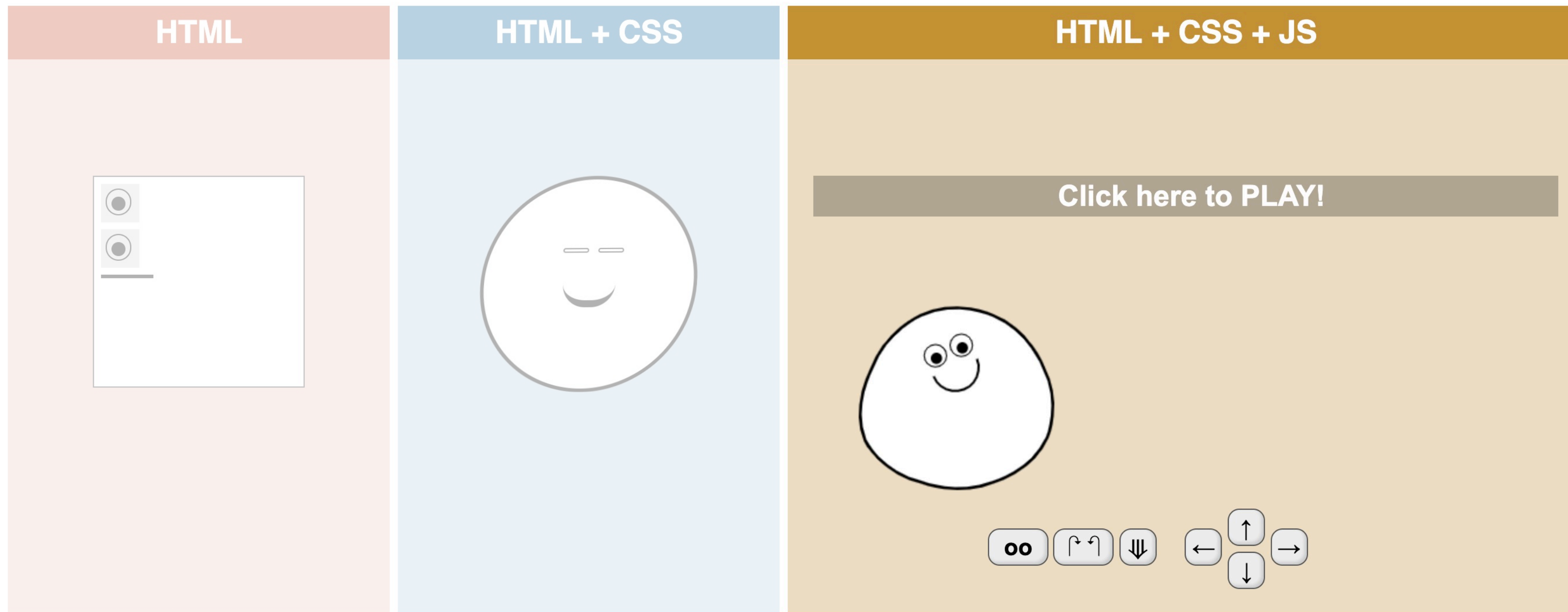
CSS (Cascading Style Sheets)

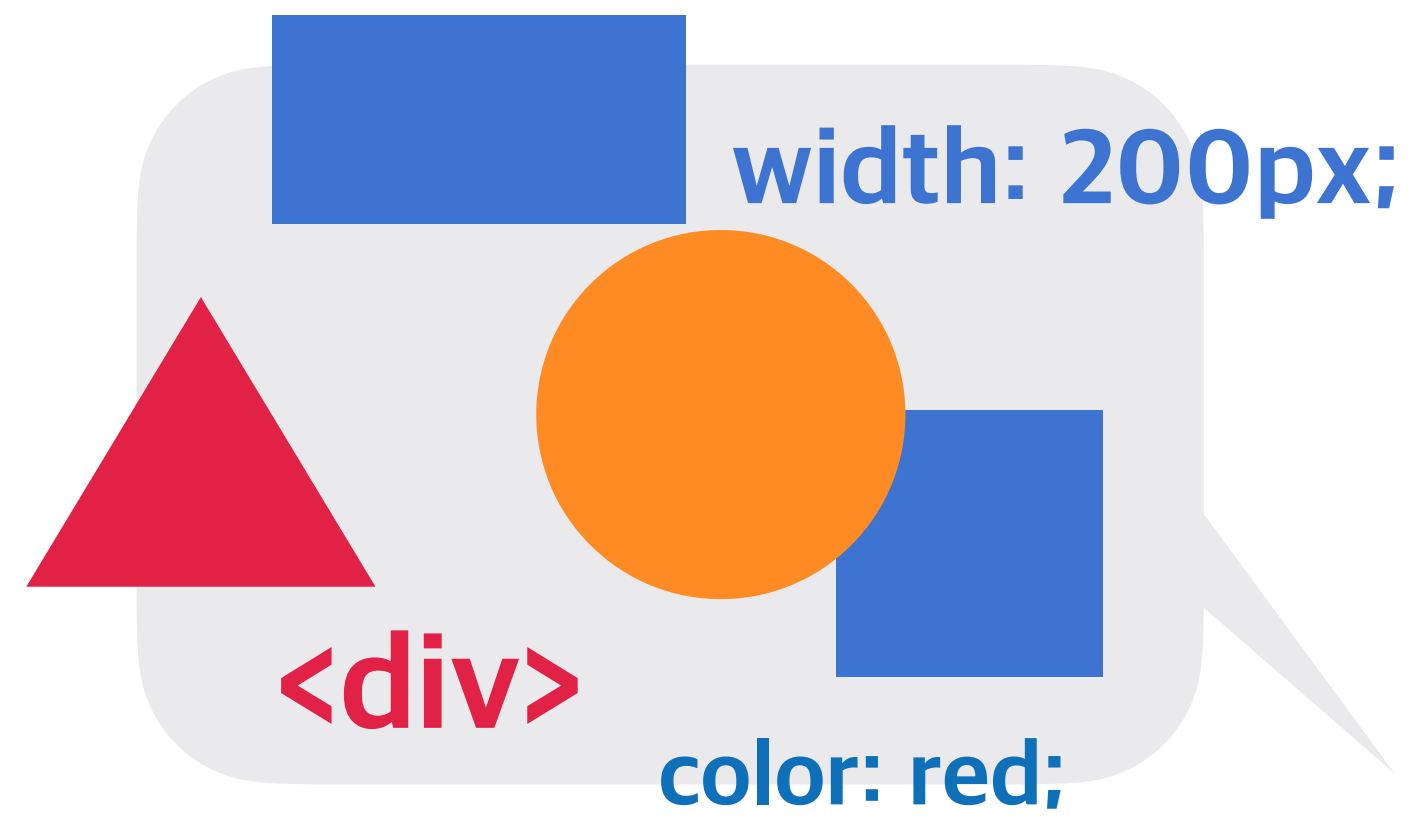
실제 화면에 표시되는 방법(색상, 크기, 폰트, 레이아웃 등)을 지정해 콘텐츠를 꾸며주는 시각적인 표현(정적)을 담당.

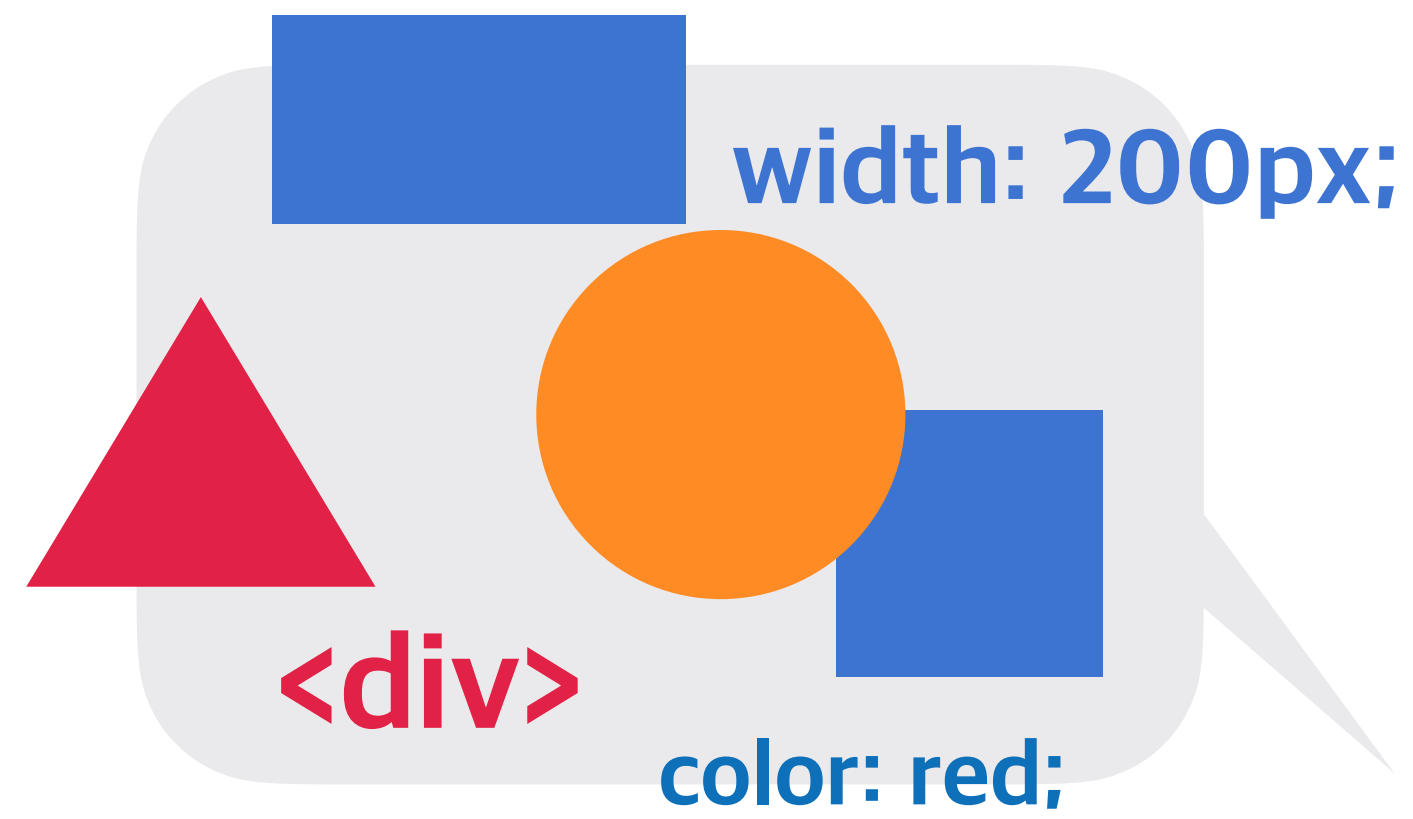


JS (JavaScript)

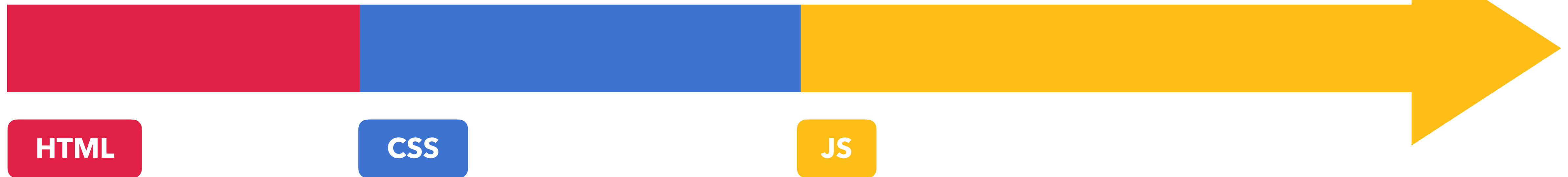
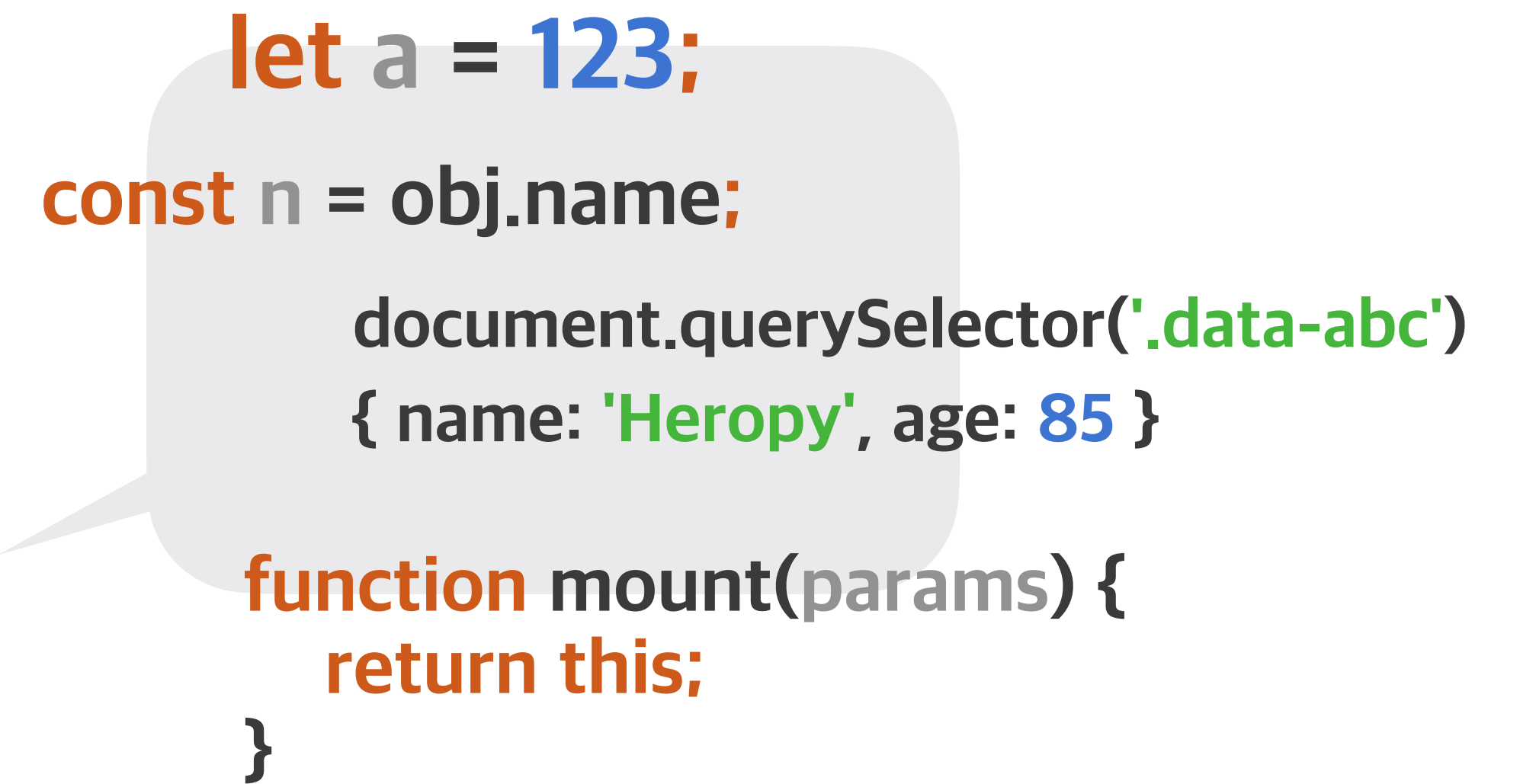
콘텐츠를 바꾸고 움직이는 등 페이지를 동작시키는 동적 처리를 담당.





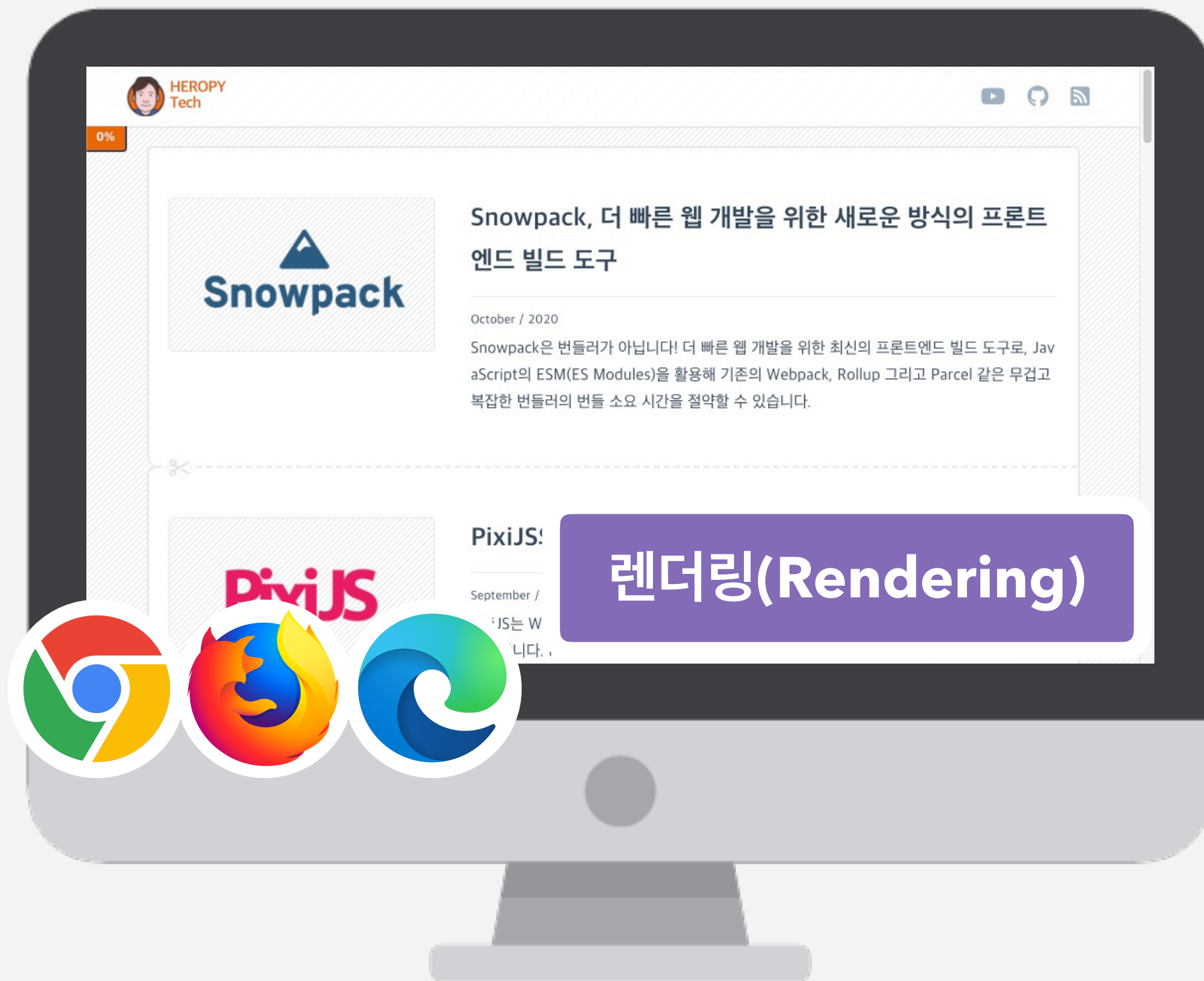


VS



https://heropy.blog

주소창에 페이지 주소 입력!



사용자 컴퓨터(브라우저)

1
최초 요청
(Request)

서버

2
최초 응답
(Response)

3
추가 요청

4
추가 응답

CSS JS JPG ...

웹 표준

웹 표준(Web Standard)이란 ‘웹에서 사용되는 표준 기술이나 규칙’을 의미,
W3C의 표준화 제정 단계의 ‘권고안(REC)’에 해당하는 기술.



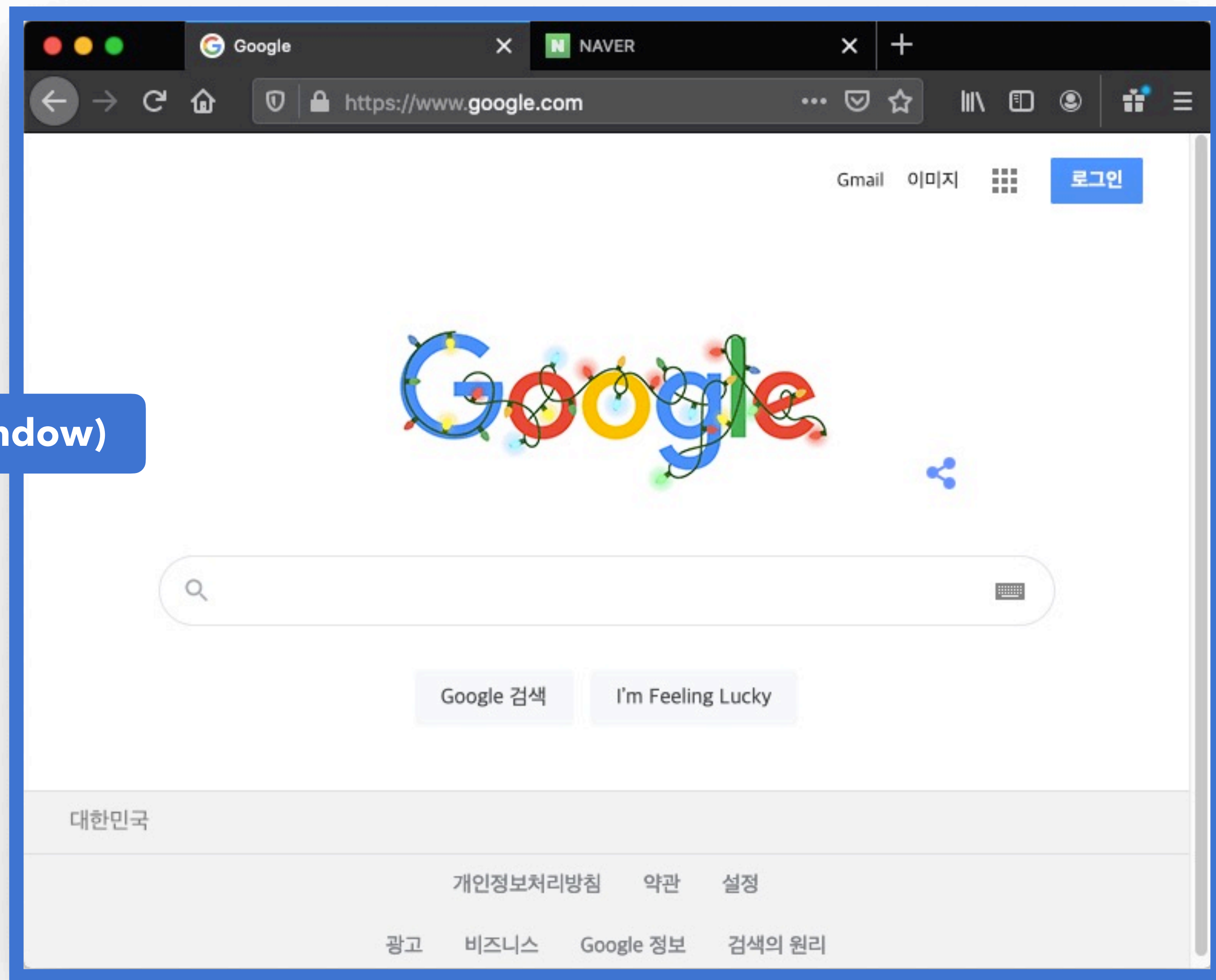
크로스 브라우징

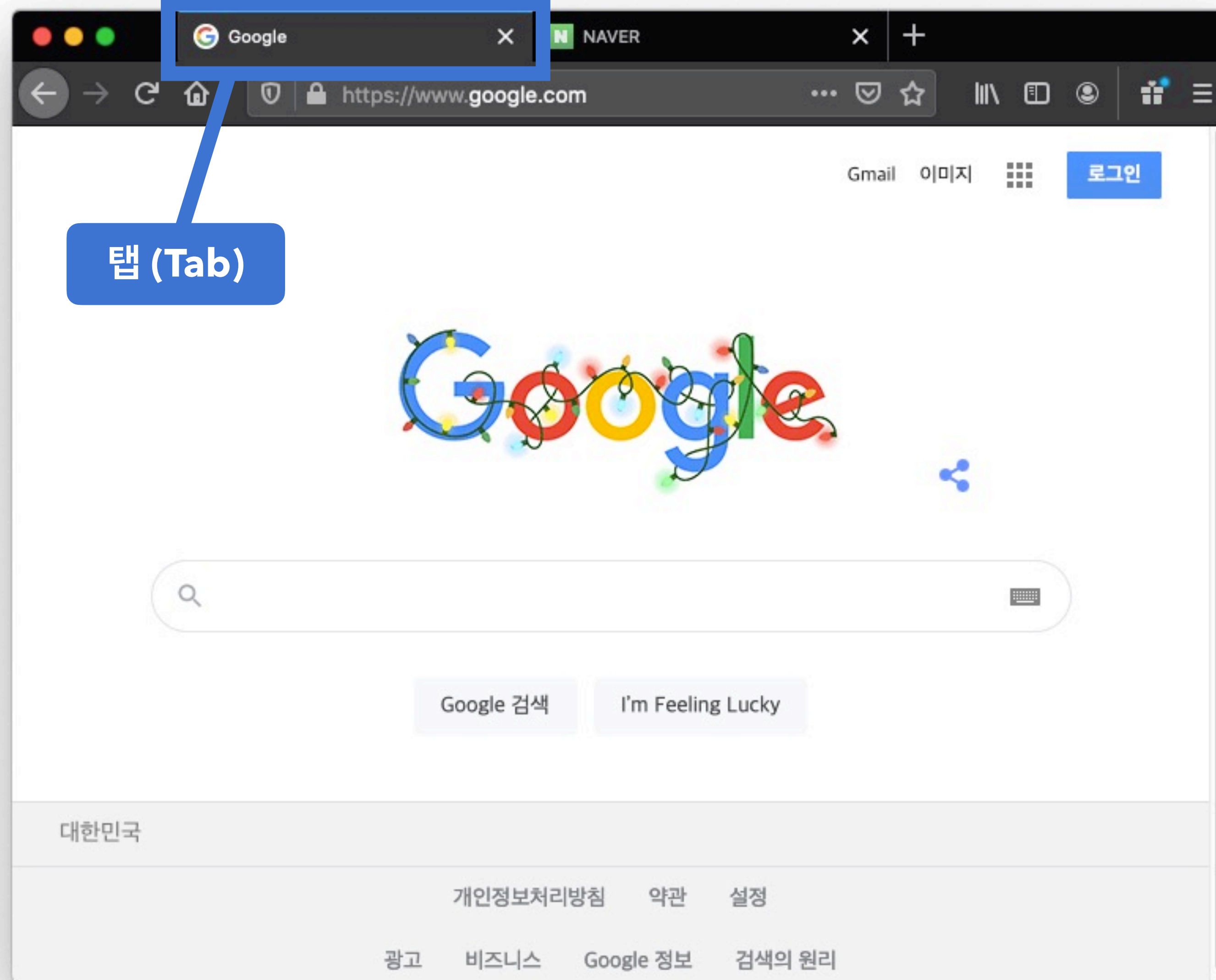
크로스 브라우징(Cross Browsing)이란 조금은 다르게 구동되는 여러 브라우저에서,
동일한 사용자 경험(같은 화면, 같은 동작 등)을 줄 수 있도록 제작하는 기술, 방법.



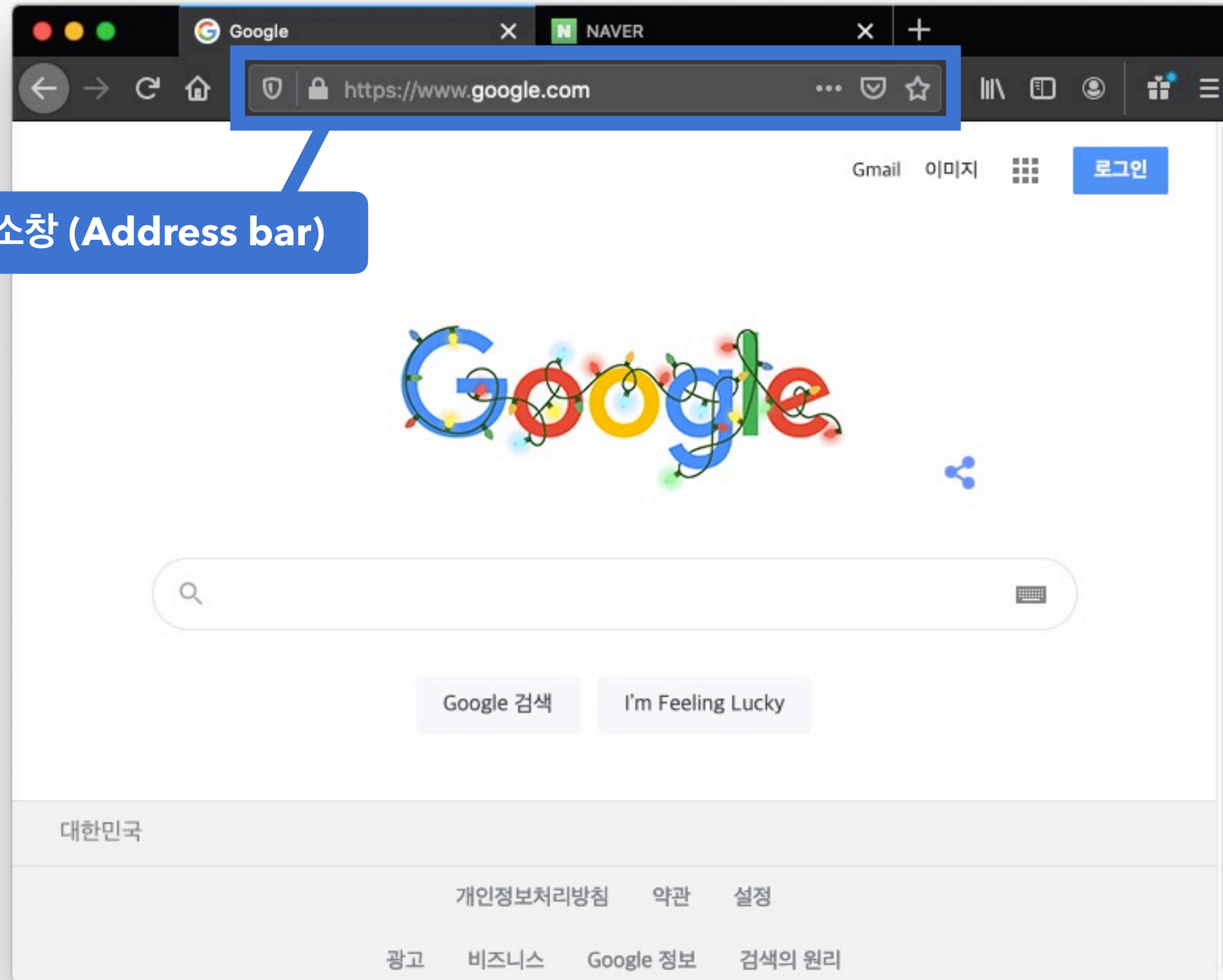
넵....

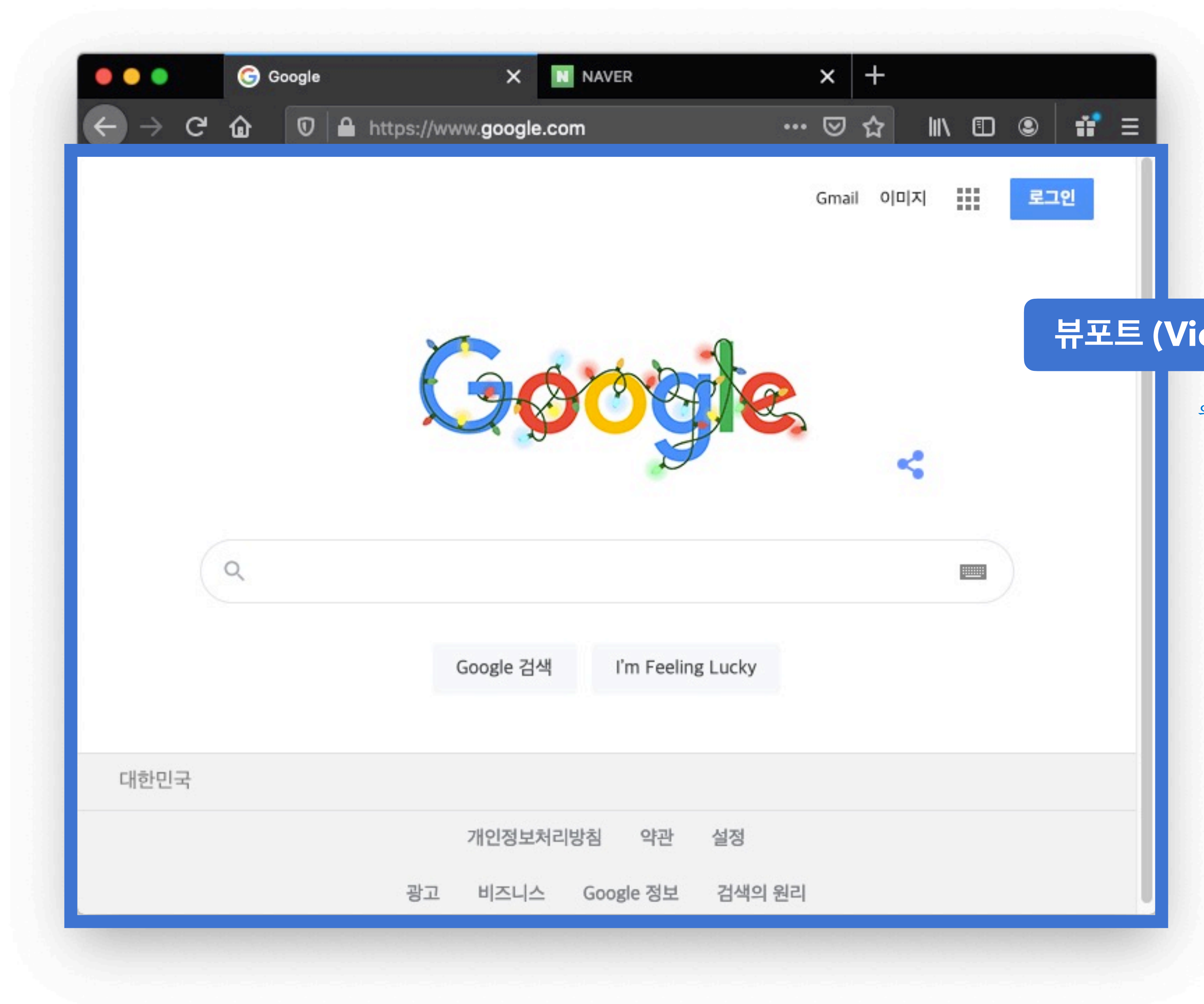
창 (Window)





주소창 (Address bar)





뷰포트 (Viewport)

웹페이지가 출력되는 영역(동적)

웹 접근성

웹 접근성이란 고령자, 장애인 같은 신체적, 환경적 조건에 제한이 있는 사용자를 포함한, 모든 사용자들이 동등하게 접근할 수 있는 웹 콘텐츠를 제작하는 방법.

ex. alt=" "



한손 사용자용 키보드



적외선 특수 마우스



큰 키 키보드

"웹의 힘은 그것의 보편성에 있다. 장애에 구애 없이,
모든 사람이 접근할 수 있는 것이 필수적인 요소이다."

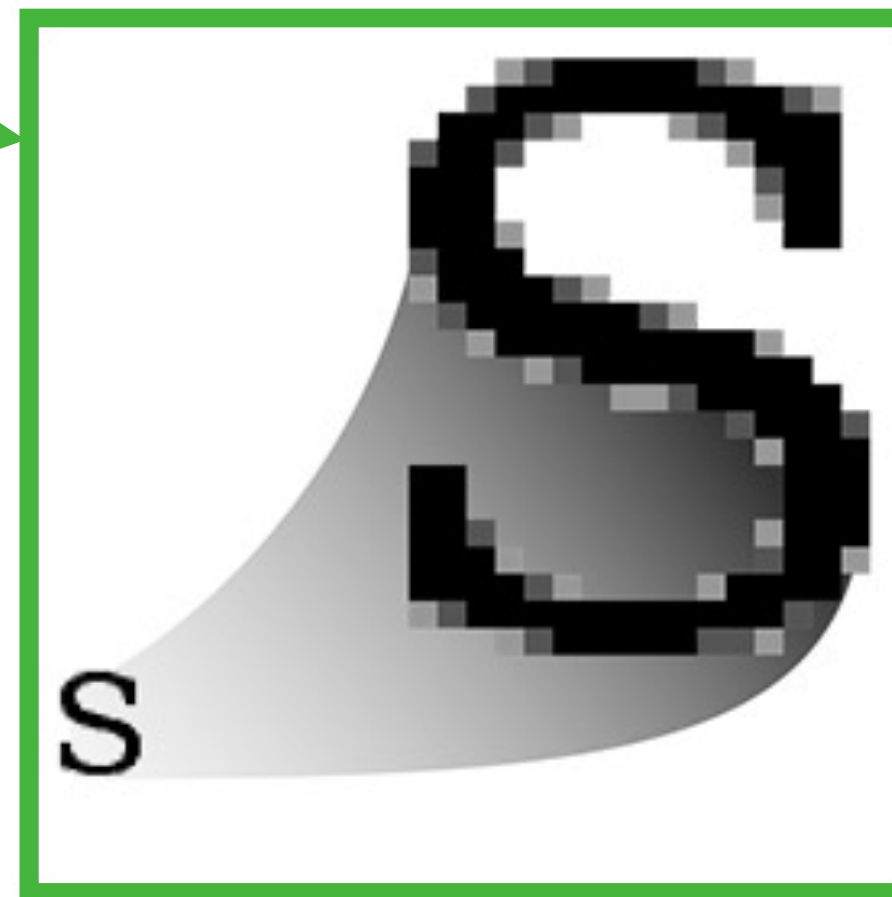


팀 버너스-리 경
- 웹의 창시자 -

웹 이미지

픽셀이 모여 만들어진 정보의 집합,
래스터(Raster) 이미지라고도 부름.

비트맵(Bitmap)과 벡터(Vector)



Raster
.jpeg .gif .png



Vector
.svg

점, 선, 면의 위치(좌표), 색상 등
수학적 정보의 형태(Shape)로
이루어진 이미지.

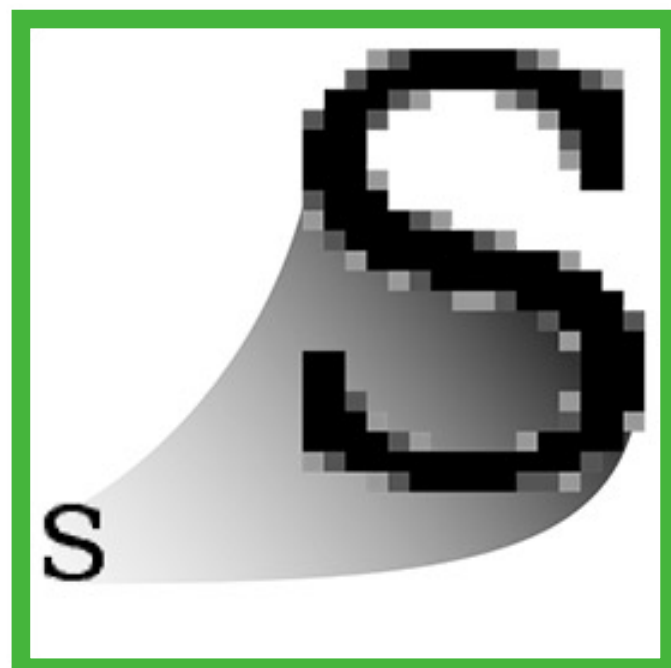


background.jpg

fox.png



비트맵



정교하고 다양한 색상을 자연스럽게 표현.
확대/축소 시 계단 현상, 품질 저하.

icon.svg



computer.svg



google-logo.svg

벡터



확대/축소에서 자유로움, 용량 변화가 없음.
정교한 이미지(인물, 풍경 사진 같은)를 표현하기 어려움.

JPG(Joint Photographic coding Experts Group)는 Full-color와 Gray-scale의 압축을 위해 만들어졌으며,
압축률이 훌륭해 사진이나 예술 분야에서 많이 사용.



JPG(JPEG)



손실 압축

표현 색상도(**24비트**, 약 1600만 색상)가 뛰어남

이미지의 품질과 **용량**을 쉽게 **조절** 가능

가장 널리 쓰이는 이미지 포맷

PNG(Portable Network Graphics)는 Gif의 대체 포맷으로 개발됨.



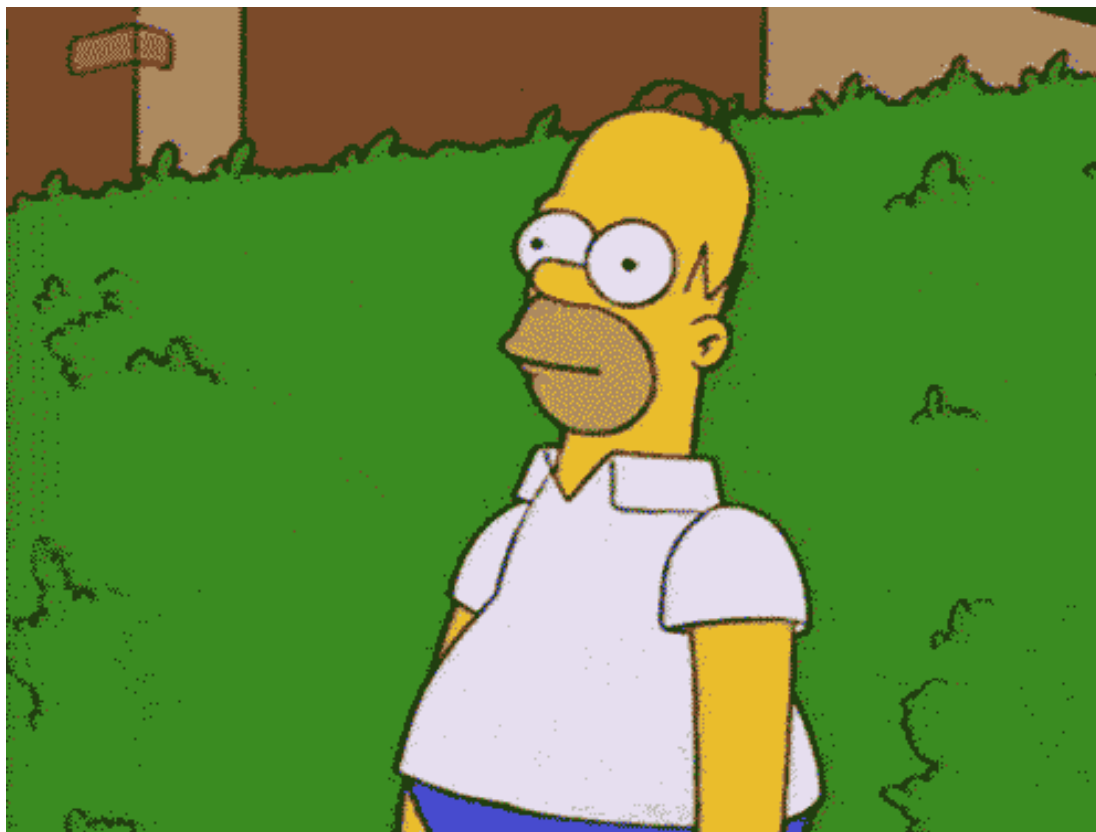
비손실 압축

8비트(256 색상) / **24비트**(약 1600만 색상) 컬러 이미지 처리

Alpha Channel 지원(**투명도**)

W3C 권장 포맷

GIF(Graphics Interchange Format)는 이미지 파일 내에 이미지 및 문자열 같은 정보들을 저장.



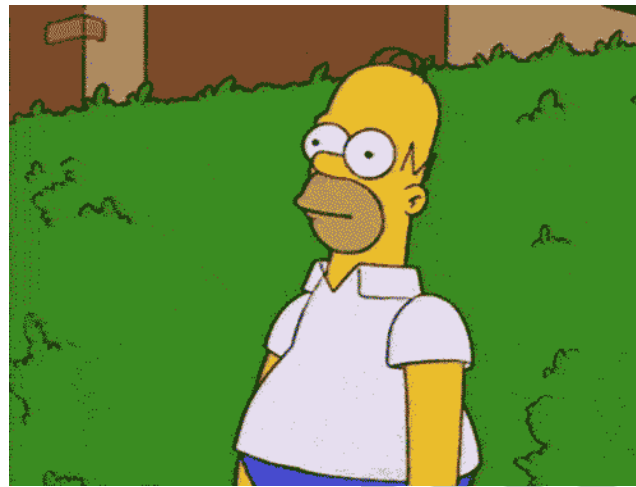
비손실 압축

여러 장의 이미지를 한 개의 파일에 담을 수 있음

(움짤, 애니메이션)

8비트 색상만 지원(다양한 색상 표현에는 적합하지 않음)

JPG, PNG, GIF를 모두 대체할 수 있는 구글이 개발한 이미지 포맷.



WEBP

완벽한 손실/비손실 압축 지원

GIF 같은 애니메이션 지원

Alpha Channel 지원(손실, 비손실 모두)



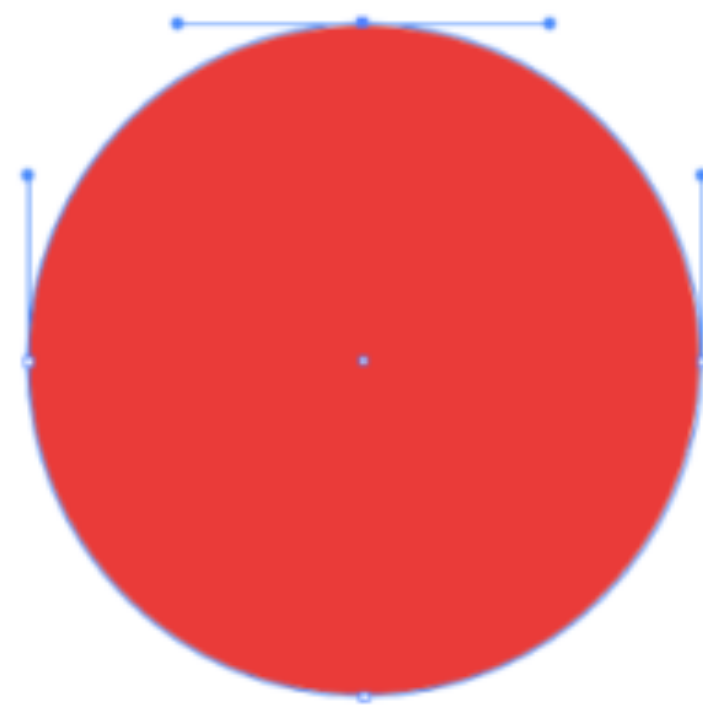
IE 지원 불가

[illegible]

SVG(Scalable Vector Graphics)는 마크업 언어(HTML/XML) 기반의 벡터 그래픽을 표현하는 포맷.



SVG



해상도의 영향에서 자유로움

CSS와 JS로 제어 가능

파일 및 코드 삽입 가능



코드로 되어 있어,
CSS, JS로 제어 가능!

google.svg ×

google.svg

```
1 <svg xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://www.w3.org/1999/xlink" viewBox="0 0 3995.04 512">
2   <g xmlns="http://www.w3.org/2000/svg" clip-path="url(#a)"><path d="M202.4,235.58V181.32H384a178.13,178.13,0
3   <path d="M663.64,269.32C663.64,343,606,397.2,535.32,397.2S407,343,407,269.32c0-74.11,57.63-127.88,128.34-12
4   <path d="M943.59,269.32C943.59,343,886,397.2,815.27,397.2S686.94,343,686.94,269.32c0-74.11,57.62-127.88,128
5   <path d="M1211.93,149.19v229.6c0,94.46-55.69,133.21-121.55,133.21-62,0-99.28-41.66-113.32-75.56L1026,416.09
6   <rect x="1250.9" y="14.82" width="56.17" height="374.63" fill="#34a853"/>
7   <path d="M1524,311.46l43.58,29.06c-14,20.83-47.94,56.68-106.54,56.68-72.64,0-126.88-56.19-126.88-127.88,0-7
8   </g>
9 </svg>
10
```

특수 문자 용어

특수 문자의 영어/한글 이름과 키보드 위치.





Backtick, Grave

백틱, 그레이브



~

Tilde

틸드, 물결 표시



!

Exclamation mark
엑스클러메이션, 느낌표



@

At sign
앳, 골뱅이



#

Sharp, Number sign

샵, 넘버, 우물 정



\$

Dollar sign

달러



%

Percent sign

퍼센트





Caret
캐럿



&

Ampersand

앰퍼센드



＊

Asterisk
에스터리스크, 별표



—

Hyphen, Dash 하이픈, 대시, 마이너스





Underscore, Low dash
언더스코어, 로대시, 밑줄



=

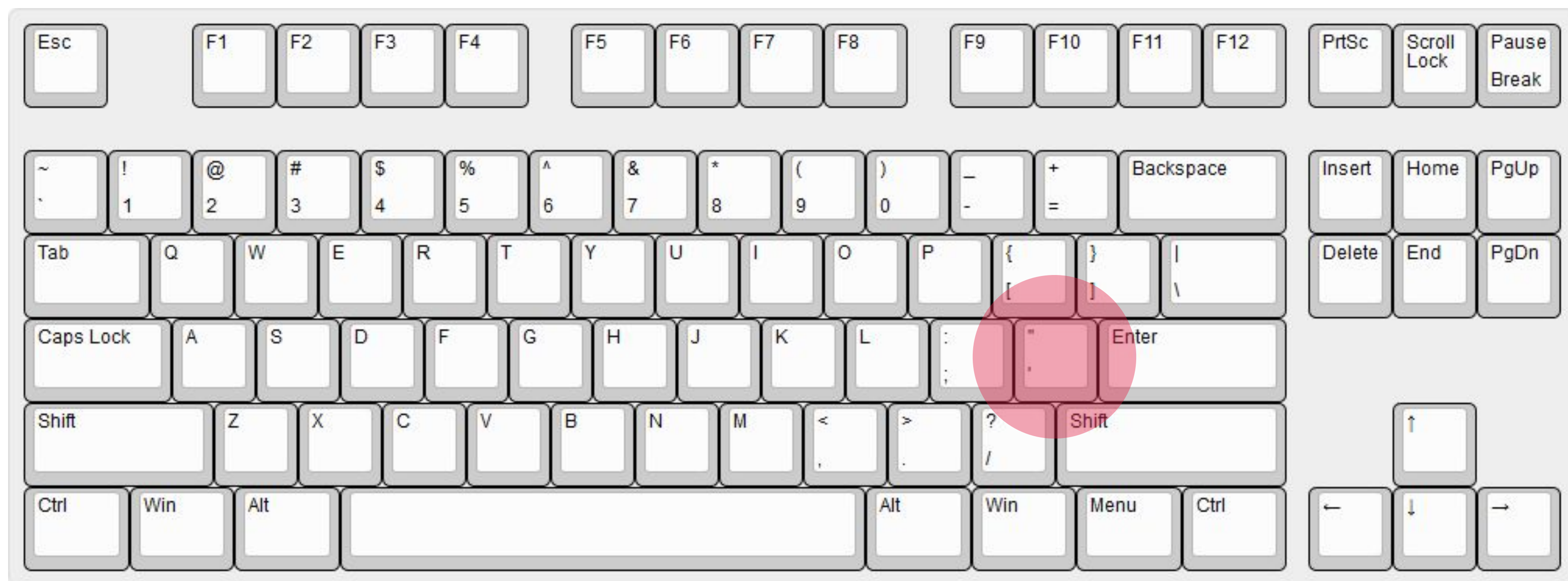
Equals sign

이퀄, 동등



“

Quotation mark
쿼테이션, 큰 따옴표



‘

Apostrophe

아포스트로피, 작은 따옴표



■
■

Colon
콜론



■
/

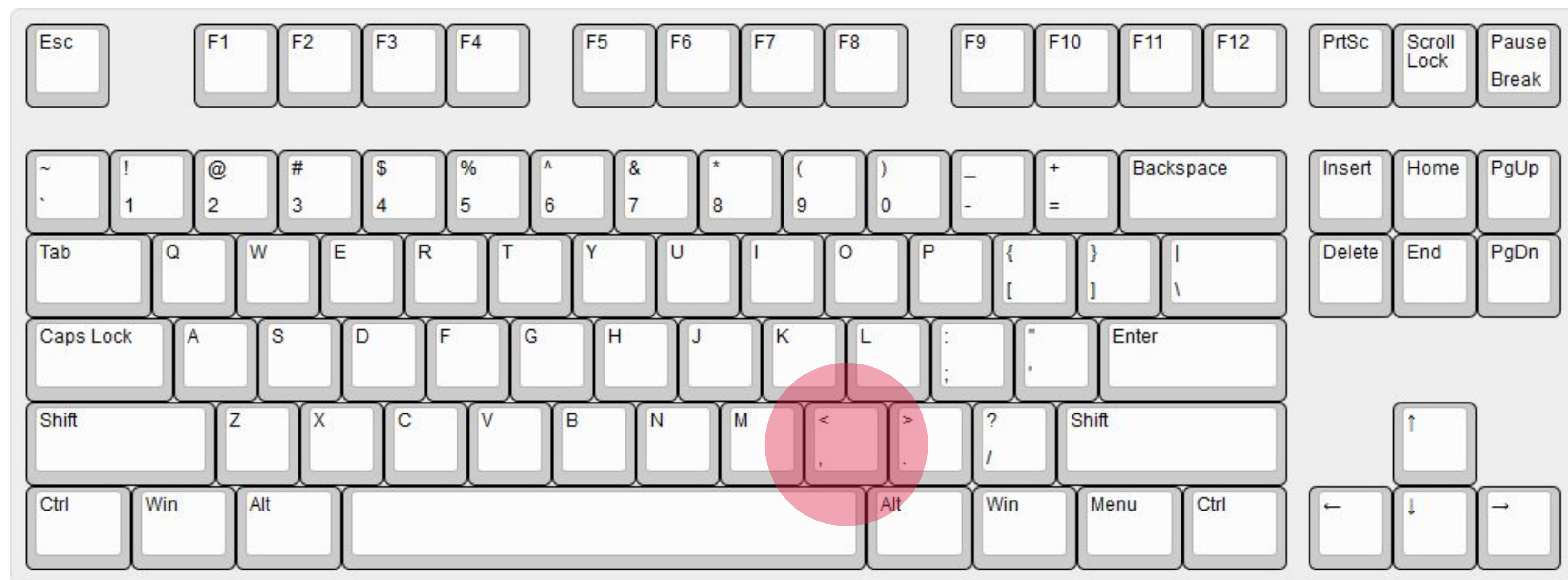
Semicolon

세미콜론



/

Comma
콤마, 쉼표





Period, Dot 피리어드, 닷, 점, 마침표



?

Question mark

퀘스천, 물음표



/

Slash
슬래시



|

Vertical bar
버티컬 바



\

Backslash

백슬래시, 역 슬래시



()

Parenthesis

퍼렌서시스, 소괄호, 괄호



{ }

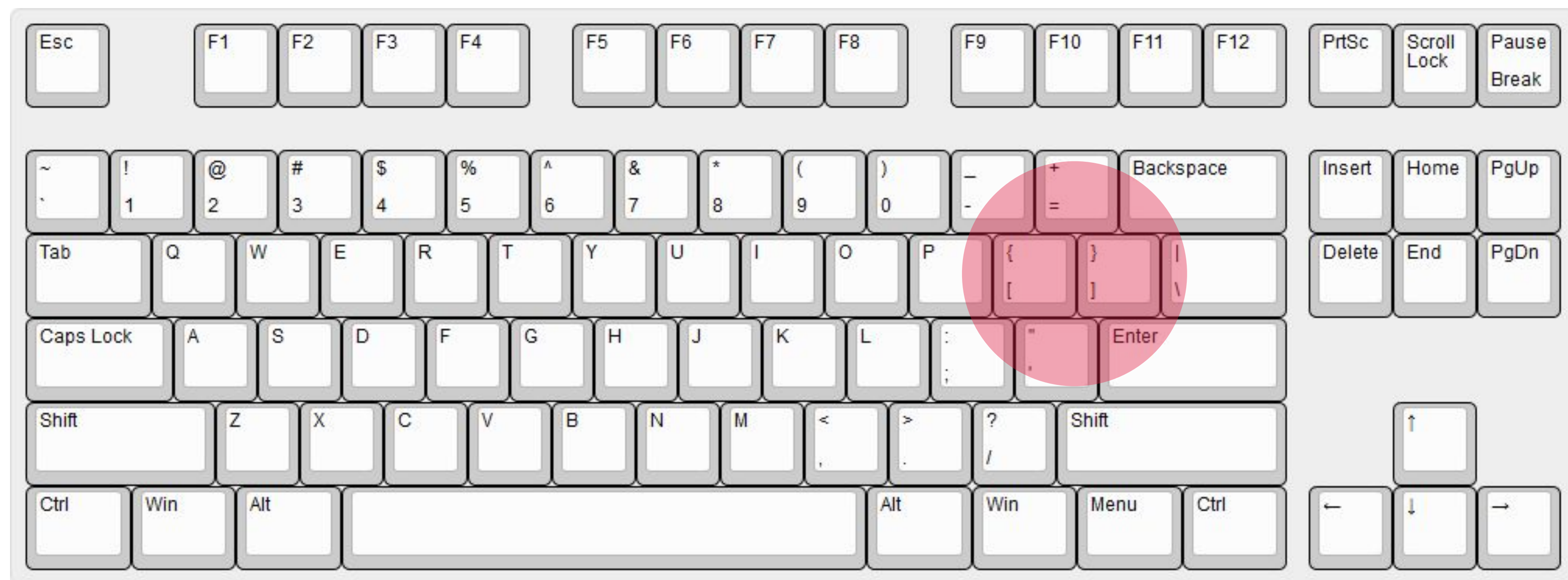
Brace

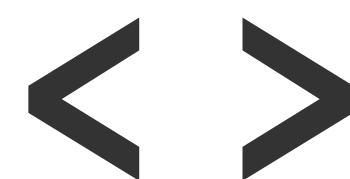
브레이스, 중괄호



[]

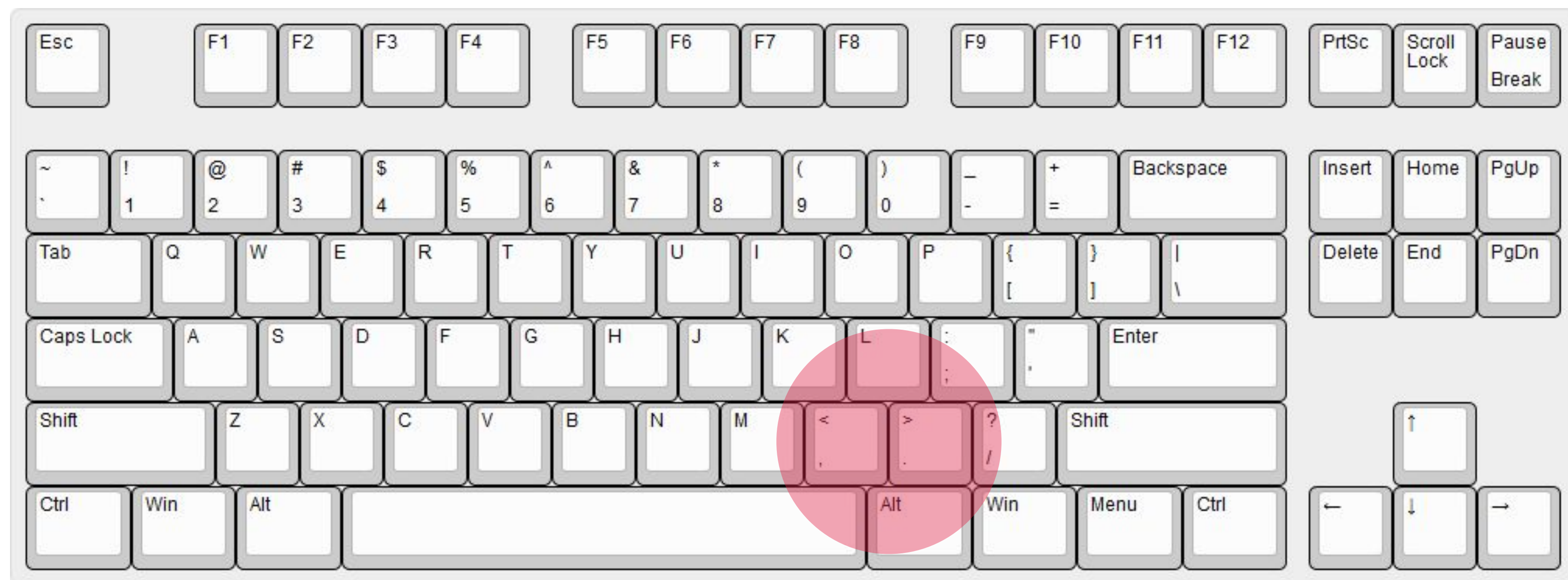
Bracket
브래킷, 대괄호





Angle Bracket

앵글 브래킷, 꺾쇠괄호



오픈 소스 라이선스

오픈소스란 어떤 제품을 개발하는 과정에 필요한 소스 코드나 설계도를,
누구나 접근해서 열람할 수 있도록 공개하는 것.

[OpenSource.org](https://opensource.org)

Apache License

아파치 소프트웨어 재단에서 자체 소프트웨어에 적용하기 위해 만든 라이선스,
개인적/상업적 이용, 배포, 수정, 특허 신청이 가능.



MIT License

매사추세츠공과대학(MIT)에서 소프트웨어 학생들을 위해 개발한 라이선스,
개인 소스에 이 라이선스를 사용하고 있다는 표시만 지켜주면 되며,
나머지 사용에 대한 제약은 없음.



BSD License

BSD(Berkeley Software Distribution)는 버클리 캘리포니아대학에서 개발한 라이선스,
MIT와 동일 조건.

BSD

Beerware

오픈소스 개발자에게 맥주를 사줘야 하는 라이선스.
물론 만날 수 있는 경우..



BEERWARE

<https://heropy.blog>



박영웅

상대 경로 vs 절대 경로

./ (생략 가능)

../

상위 경로로 이동

http (https)

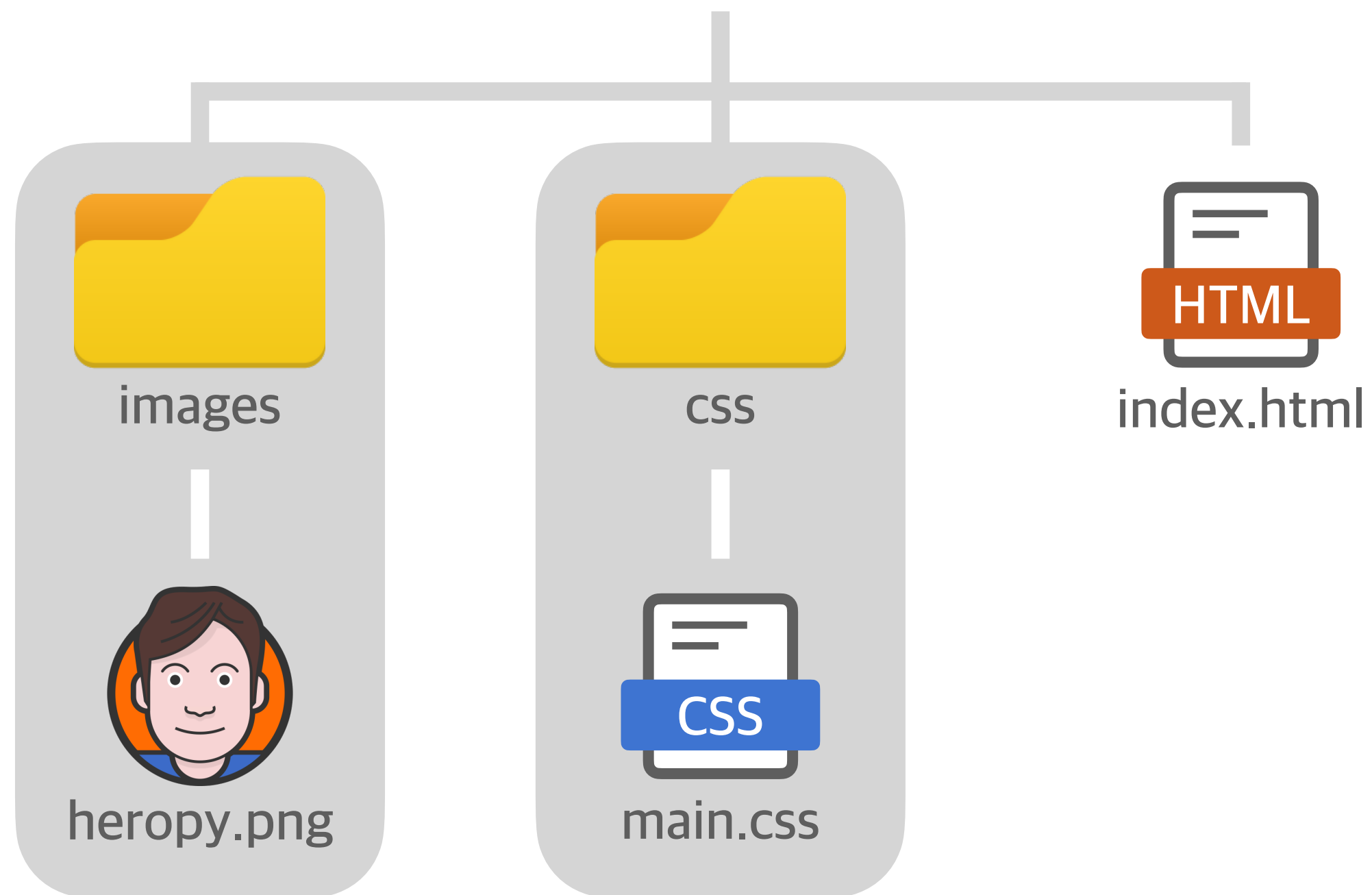
/ (//)

최상위 경로(root)

<http://localhost:5500>



my-project



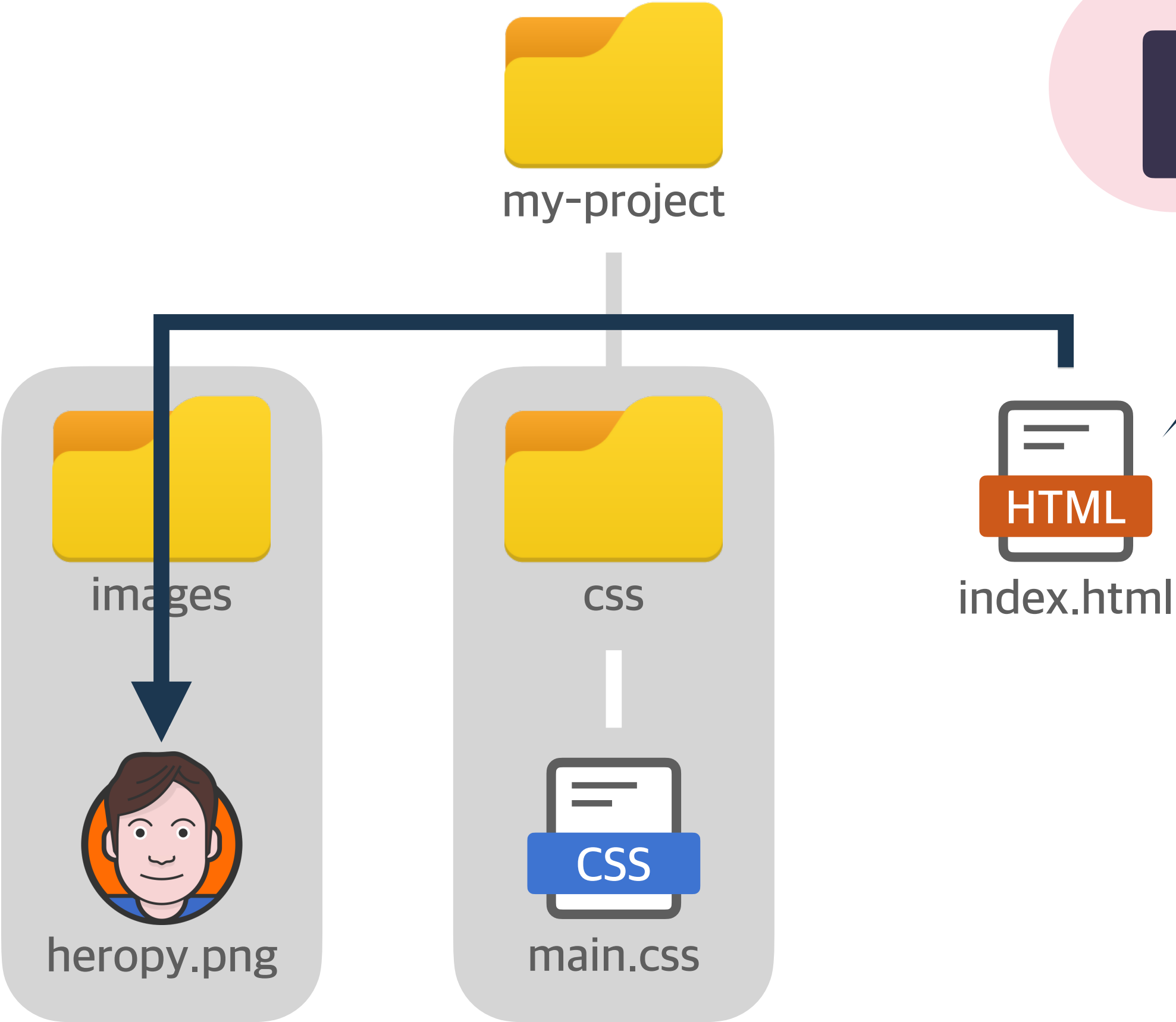
http://localhost:5500

images/heropy.png

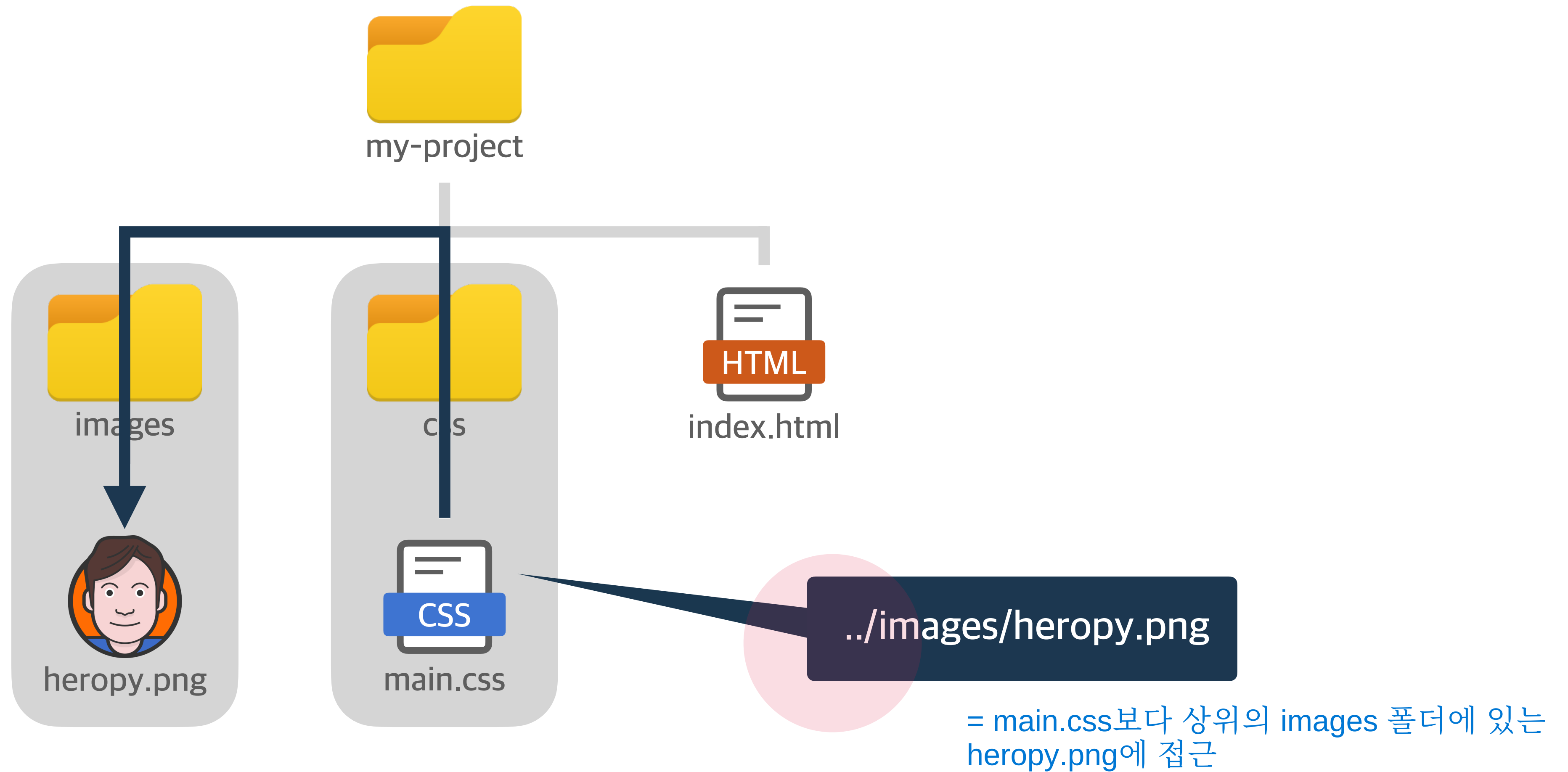
또는

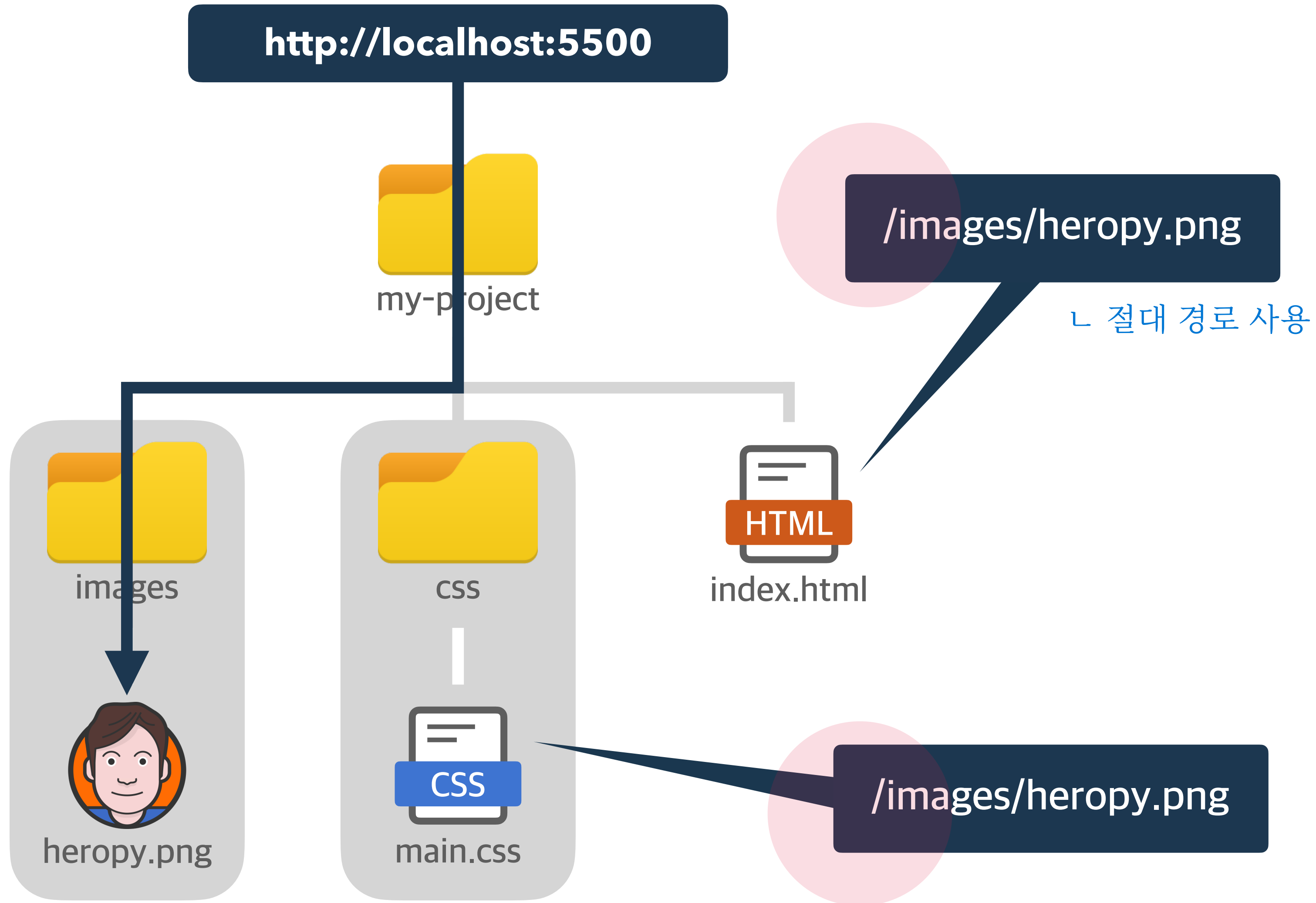
./images/heropy.png

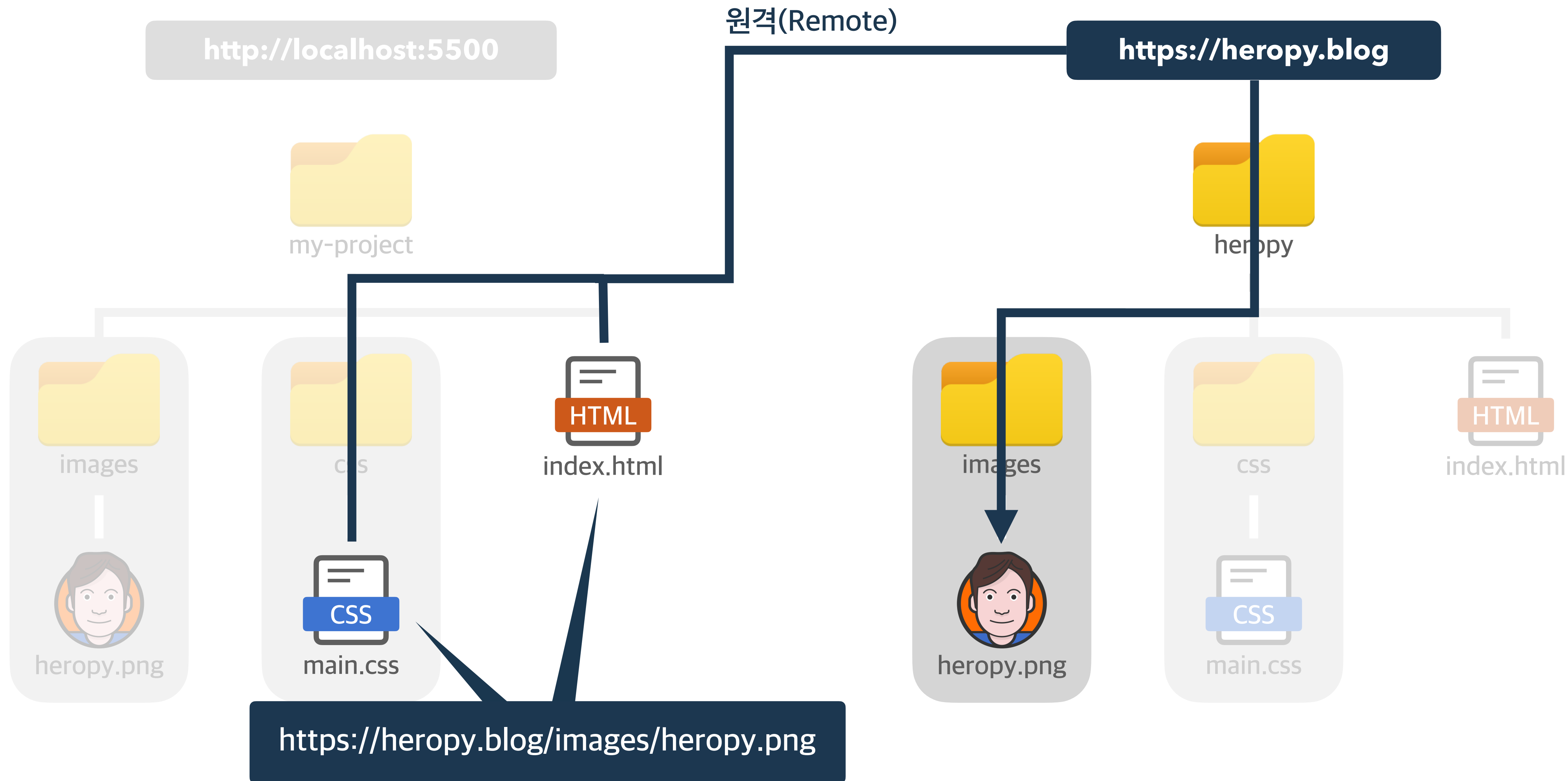
= 내 주변의 images라는 폴더의
heropy.png 파일에 접근



http://localhost:5500







<https://heropy.blog>



박영웅

HTML 기본 문법

<h1>Hello HTML~</h1>

<태그>내용</태그>

요소(Element)

<태그>내용</태그>

시작(열린) 태그(Tag)

<태그>내용</태그>

종료(닫힌) 태그

<태그>내용</태그>

요소의 내용(Contents)

부모 요소

자식 요소

<태그><태그>내용</태그></태그>

부모 요소

자식 요소

<태그>

<태그>내용</태그>

</태그>

Tab

들여쓰기
(Indent)

VS

내어쓰기
(Outdent)

Shift

Tab

<태그>

부모 요소

<태그>

<태그>내용</태그>

</태그>

</태그>

<태그>

상위(조상) 요소

<태그>

<태그>내용</태그>

</태그>

</태그>

<태그> 자식 요소

<태그>

<태그>내용</태그>

</태그>

</태그>

<태그>

하위(후손) 요소

<태그>

<태그>내용</태그>

</태그>

</태그>

빈(Empty) 태그!
: 시작 태그만 있는 애들;;

<태그></태그>

종료(닫힌) 태그???

두 방식 모두 html5 표준

편리함!

HTML 1/2/3/4/5

<태그>

VS

<태그 />

안전함!

XHTML / HTML5

Attribute

Value

<태그 속성="값">내용</태그>

기능의 확장

이미지를 삽입하는 요소(태그)!
어떤 이미지를 삽입해야 하지???
이미지 이름은 뭐지??

이미지의 경로

이미지의 이름
(대체 텍스트 / Alternate Text)

```

```

화면에 출력!



<input />

사용자가 데이터를 입력하는 요소(태그)!
어떤 데이터 타입을 입력 받을 거지?

hello world

화면에 출력!

데이터의 타입

text

사용자에게 일반 텍스트를 입력 받음!

```
<input type="text" />
```



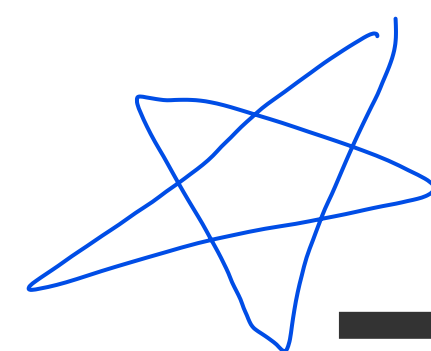
화면에 출력!

데이터의 타입

checkbox

사용자에게 체크 여부를 입력 받음!

```
<input type="checkbox" />
```



글자와 상자

요소가 화면에 출력되는 특성, 크게 2가지로 구분.

인라인(Inline) 요소 : 글자를 만들기 위한 요소들.

블록(Block) 요소 : 상자(레이아웃)를 만들기 위한 요소들.

`<span>Hello</span>`
`<span>World</span>`

`<span></span>`

대표적인 인라인 요소!

본질적으로 아무것도 나타내지 않는,
콘텐츠 영역을 설정하는 용도.

p, img 등과 다르게 태그 자체에
아무런 의미가 x

Hello World

요소가 수평으로 쌓임

Hello
World



요소의 가로 너비를 지정하는 CSS 속성

```
<span style="width: 100px;">Hello</span>
```

```
<span style="height: 100px;">World</span>
```

요소의 세로 너비를 지정하는 CSS 속성

글자처럼 취급하기 때문에, 요소의 크기를 지정해도 무시됨

반응 없음!

Hello

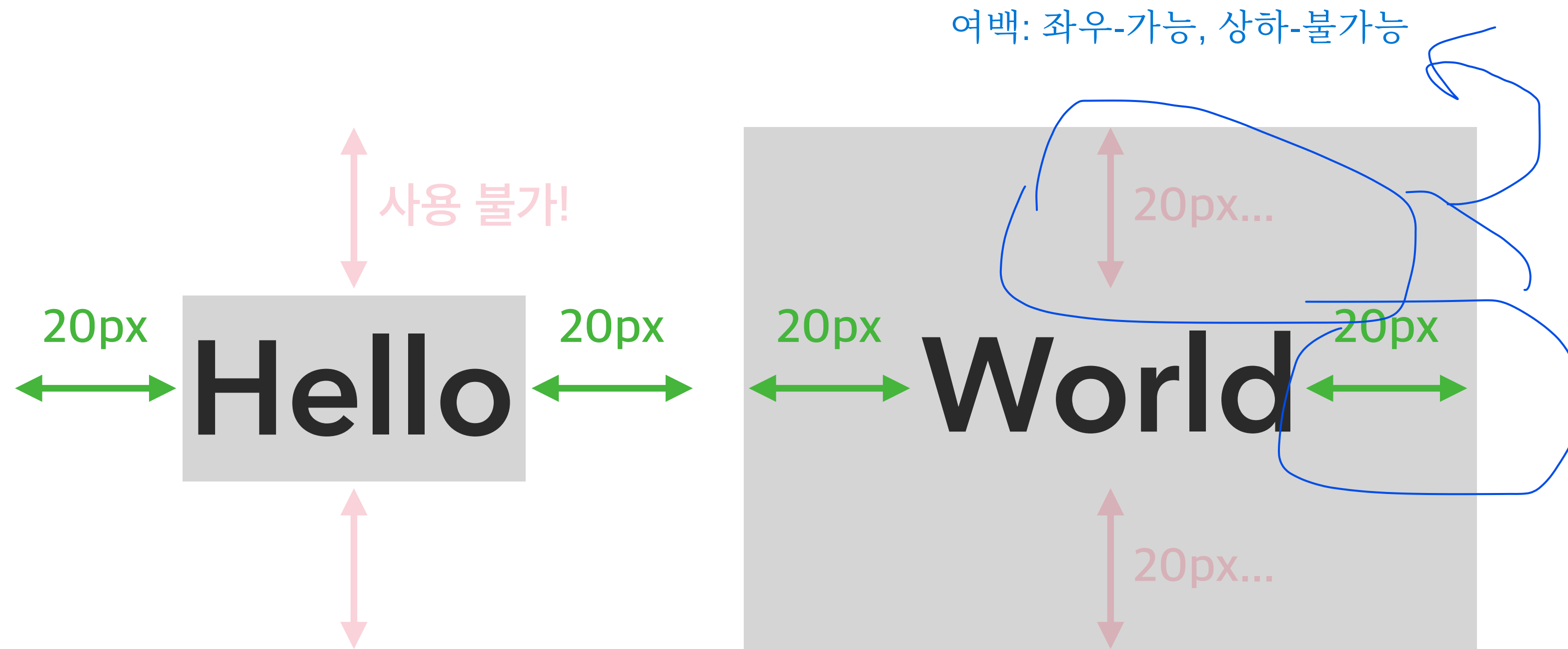
World

요소의 외부 여백을 지정하는 CSS 속성

```
<span style="margin: 20px 20px;">Hello</span>
```

```
<span style="padding: 20px 20px;">World</span>
```

요소의 내부 여백를 지정하는 CSS 속성





`<div>Hello</div>`
`<div>World</div>`

`<div></div>`

대표적인 블록 요소!

본질적으로 아무것도 나타내지 않는,
콘텐츠 영역을 설정하는 용도.



Hello

World

요소가
수직으로 쌓임



<div>Hello</div>
<div>World</div>

<div></div>

대표적인 블록 요소!
본질적으로 아무것도 나타내지 않는,
콘텐츠 영역을 설정하는 용도.

가로 길이: 부모 요소의 크기만큼 자동으로 늘어남!

Hello

World

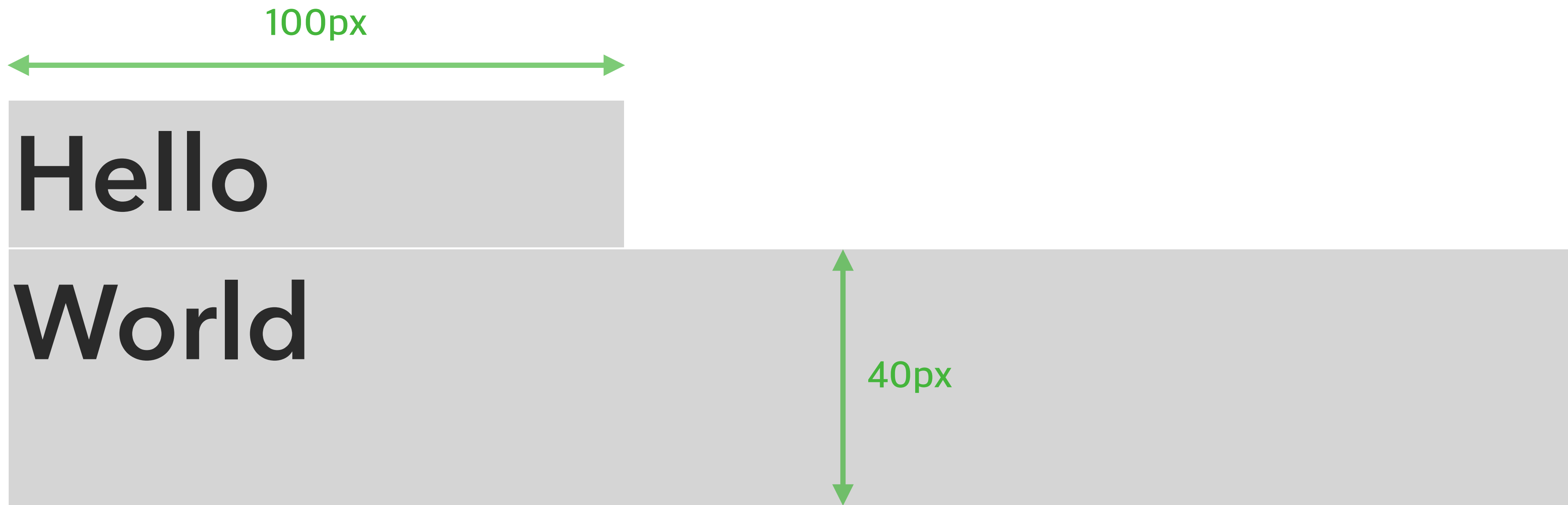
세로 길이: 포함된 콘텐츠 크기만큼
자동으로 줄어듦!

요소의 가로 너비를 지정하는 CSS 속성

```
<div style="width: 100px;">Hello</div>
```

```
<div style="height: 40px;">World</div>
```

요소의 세로 너비를 지정하는 CSS 속성

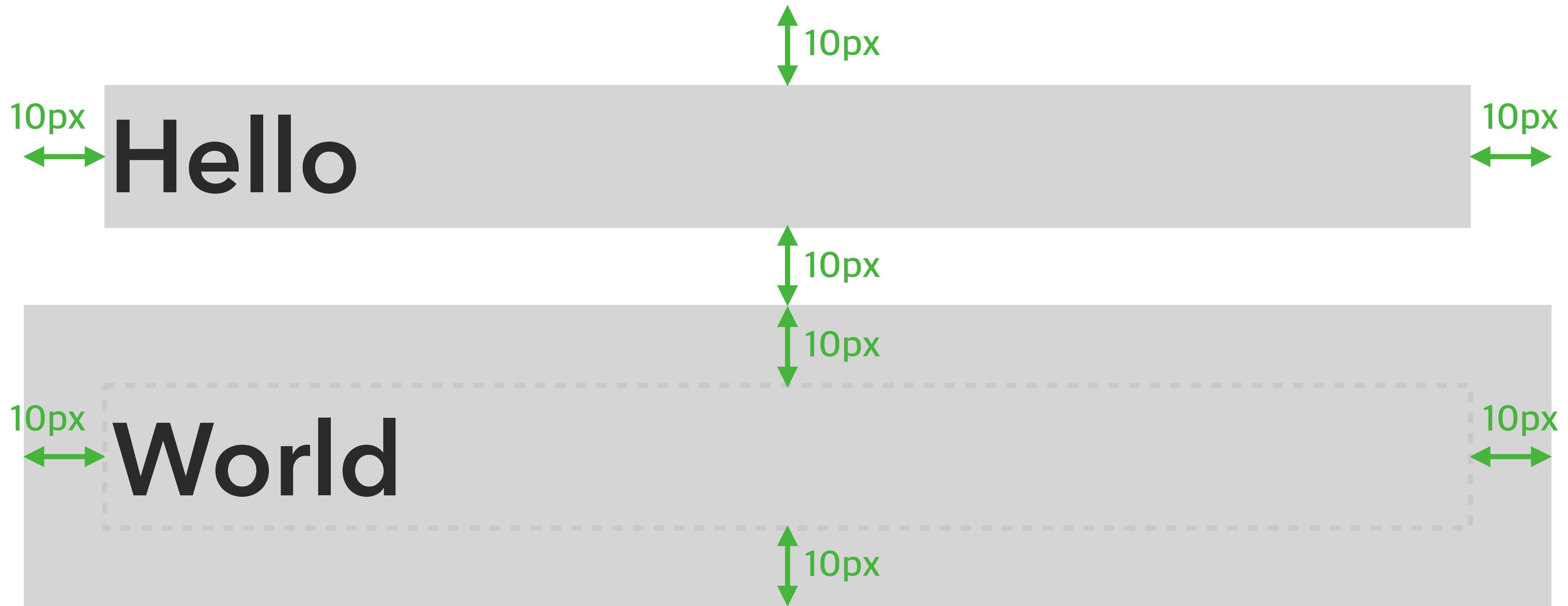


요소의 외부 여백을 지정하는 CSS 속성

```
<div style="margin: 10px;">Hello</div>
```

```
<div style="padding: 10px;">World</div>
```

요소의 내부 여백를 지정하는 CSS 속성



블록 요소

가능!

<div><div></div></div>

<div></div>

가능!

인라인 요소

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
  </head>
```

```
  <body>
```

```
  </body>
```

```
</html>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
  </head>
```

```
  <body>
```

```
  </body>
```

```
</html>
```

문서의 전체 범위

HTML 문서가 어디에서 시작하고, 어디에서 끝나는지 알려주는 역할.


```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
</head>
```

```
<body>
```

```
</body>
```

```
</html>
```

문서의 정보를 나타내는 범위

웹 브라우저가 해석해야 할
웹 페이지의 제목, 설명, 사용할 파일 위치, 스타일(CSS) 같은,
웹페이지의 보이지 않는 정보를 작성하는 범위.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
</head>
```

```
<body>
```

```
</body>
```

```
</html>
```

문서의 구조를 나타내는 범위

사용자 화면을 통해 보여지는
로고, 헤더, 푸터, 내비게이션, 메뉴, 버튼, 이미지 같은,
웹페이지의 보여지는 구조를 작성하는 범위.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
</head>
```

```
<body>
```

```
</body>
```

```
</html>
```

문서의 HTML 버전을 지정

HTML1

HTML2

HTML3

HTML4

XHTML

HTML5 (표준)

DOCTYPE(DTD, Document Type Definition)은 마크업 언어에서 문서 형식을 정의하며, 웹 브라우저가 어떤 HTML 버전의 해석 방식으로 페이지를 이해하면 되는지를 알려주는 용도.


```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Trans ...
```

```
<html>
```

```
<head>
```

```
</head>
```

```
<body>
```

```
</body>
```

```
</html>
```

문서의 HTML 버전을 지정

HTML1

HTML2

HTML3

HTML4

XHTML

HTML5 (표준)

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>      문자인코딩 방식
```

```
    <meta charset="UTF-8" />
```

```
    <meta name="author" content="HEROPY" />
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1" />
```

```
  </head>      정보의 종류
```

정보의 값

```
  <body></body>
```

```
</html>
```

<meta />는 HTML 문서(웹페이지)의 제작자, 내용, 키워드 같은, 여러 정보를 검색엔진이나 브라우저에게 제공.

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    <title>Naver</title>
```

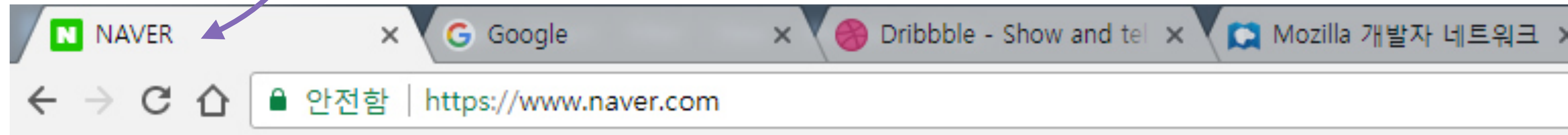
```
  </head>
```

```
  <body></body>
```

```
</html>
```

HTML 문서의 제목(title)을 정의.

웹 브라우저 탭에 표시됨!




```
<!DOCTYPE html>
```

```
<html>
```

필수 속성!

```
<head>
```

가져올 문서와 관계

가져올 문서의 경로

```
<link rel="stylesheet" href="./main.css" />
```

```
<link rel="icon" href="./favicon.png">
```

```
</head>
```

```
<body></body>
```

```
</html>
```

외부 문서를 가져와 연결할 때 사용.
(대부분 CSS 파일)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
  color: ■ red;
```

```
}
```

```
</style>
```

작성된 CSS

```
</head>
```

```
<body></body>
```

```
</html>
```

스타일(CSS)를 HTML 문서 안에서 작성하는 경우에 사용.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<script src="./main.js"></script>
```

```
<script>
```

```
console.log('Hello world!')
```

```
</script>
```

작성된 JS

```
</head>
```

```
<body></body>
```

```
</html>
```

1

자바스크립트(JS) 파일 가져오는 경우.

2

자바스크립트(JS)를 HTML 문서 안에서 작성하는 경우.


```
<!DOCTYPE html>
```

```
<html>
```

블록(상자) 요소

```
<head></head>
```

```
<body>
```

```
<div>
```

```
<h1>오늘의 날씨</h1>
```

```
<p>중부 집중호우, 남부는 열대야, 12호 태풍 북상 중..</p>
```

```

```

```
</div>
```

```
</body>
```

```
</html>
```

특별한 의미가 없는 구분을 위한 요소. (Division)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head></head>
```

```
<body>
```

```
<div>
```

```
<h1>오늘의 날씨</h1>
```

```
<p>중부 집중호우, 남부는 열대야, 12호 태풍 북상 중..</p>
```

```

```

```
</div>
```

```
</body>
```

```
</html>
```

블록(상자) 요소

제목을 의미하는 요소. (Heading)


```
<!DOCTYPE html>
```

```
<html>
```

```
  <head></head>
```

```
  <body>
```

```
    <div>
```

```
      <h1>오늘의 날씨</h1>
```

```
      <p>중부 집중호우, 남부는 열대야, 12호 태풍 북상 중..</p>
```

```
      
```

```
    </div>
```

```
  </body>
```

```
</html>
```

블록(상자) 요소

문장을 의미하는 요소. (Paragraph)

div를 써도 문제는 안되지만...의미가 있는 p 태그를 쓰는 것이 좋음


```
<!DOCTYPE html>
```

```
<html>
```

```
<head></head>
```

```
<body>
```

```
<div>
```

```
<h1>오늘의 날씨</h1>
```

```
<p>중부 집중호우, 남부는 열대야, 12호 태풍 북상 중..</p>
```

```

```

```
</div>
```

```
</body>
```

```
</html>
```

인라인(글자) 요소

이미지를 삽입하는 요소. (Image)

삽입할 이미지의 경로

필수 속성!

삽입할 이미지의 이름

필수 속성!

화면에 출력!



 12호 태풍

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head></head>
```

```
  <body>
```

```
    <h1>제목1</h1>
```

```
    <h2>제목2</h2>
```

```
    <h3>제목3</h3>
```

```
    <h4>제목4</h4>
```

```
    <h5>제목5</h5>
```

```
    <h6>제목6</h6>
```

```
  </body>
```

```
</html>
```

블록(상자) 요소

제목을 의미하는 요소. (Heading)
숫자가 작을수록 더 중요한 제목을 정의.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head></head>
```

```
<body>
```

```
<ul>
```

```
<li>사과</li>
```

```
<li>딸기</li>
```

```
<li>수박</li>
```

```
<li>오렌지</li>
```

```
</ul>
```

```
</body>
```

```
</html>
```

블록(상자) 요소

순서가 필요없는 목록의 집합을 의미. (Unordered List)

의 자식으로 다른 태그 들어갈 수 없음;;;

블록(상자) 요소

목록 내 각 항목 (List Item)


```
<!DOCTYPE html>
```

```
<html>
```

```
<head></head>
```

```
<body>
```

```
<a href="https://www.naver.com">NAVER</a>
```

```
<a href="https://www.google.com" target="_blank">Google</a>
```

```
</body>
```

```
</html>
```

인라인(글자) 요소

다른/같은 페이지로 이동하는 하이퍼링크를 지정하는 요소.
(Anchor)

새 탭에 띄우겠다는 뜻

링크 URL의 표시(브라우저 탭) 위치

화면에 출력!

Naver Google

밑줄?? 파란색 글자??


```
<!DOCTYPE html>
```

```
<html>
```

```
  <head></head>
```

```
  <body>
```

```
    <p>
```

```
      <span>동해물</span>과 백두산이 마르고 닳도록
```

```
    </p>
```

```
  </body>
```

```
</html>
```

인라인(글자) 요소

특별한 의미가 없는 구분을 위한 요소.

VS

<div></div>

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

스타일(CSS) 추가!

```
span { color: ■ red; }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p>
```

```
<span>동해물</span>과 백두산이 마르고 닳도록
```

```
</p>
```

```
</body>
```

```
</html>
```

스타일 적용

화면에 출력!

동해물과 백두산이 마르고 닳도록


```
<!DOCTYPE html>
```

```
<html>
```

```
<head></head>
```

```
<body>
```

```
<p>
```

동해물과 백두산이
마르고 닳도록

```
</p>
```

```
</body>
```

```
</html>
```

인라인(글자) 요소

줄바꿈 요소. (Break)

화면에 출력!

줄바꿈 됨!

동해물과 백두산이
마르고 닳도록

```
<!DOCTYPE html>
```

```
<html>
```

```
<head></head>
```

```
<body>
```

```
| <input type="text" />
```

```
</body>
```

```
</html>
```

인라인(글자) 요소

블록(상자) 요소

= Inline-block

사용자가 데이터를 입력하는 요소.

외부적인 규칙은 inline을,
내부적인 규칙은 box를 따름

입력받을 데이터의 타입

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head></head>
```

```
  <body>
```

```
    <input type="text" />
```

```
  </body>
```

```
</html>
```

입력받을 데이터의 타입

화면에 출력!




```
<!DOCTYPE html>
<html>
  <head></head>
  <body>
    <input type="text" value="HEROPY!" />
  </body>
</html>
```

미리 입력된 값(데이터)

화면에 출력!

HEROPY!

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head></head>
```

```
  <body>
```

```
    <input type="text" placeholder="이름을 입력하세요!" />
```

```
  </body>
```

```
</html>
```

사용자가 입력할 값(데이터)의 힌트

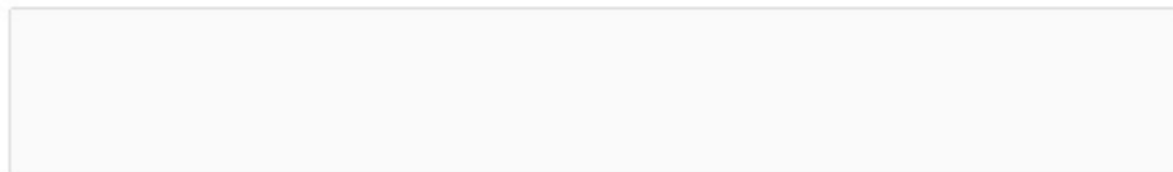
화면에 출력!

이름을 입력하세요!

```
<!DOCTYPE html>
<html>
  <head></head>
  <body>
    <input type="text" disabled />
  </body>
</html>
```

입력 요소 비활성화

화면에 출력!




```
<!DOCTYPE html>
```

```
<html>
```

```
<head></head>
```

```
<body>
```

```
<label>
```

```
<input type="checkbox" /> Apple
```

```
</label>
```

```
<label>
```

```
<input type="checkbox" /> Banana
```

```
</label>
```

```
</body>
```

```
</html>
```

인라인(글자) 요소

라벨 가능 요소(input)의 제목.

input 태그를 label 태그로 묶는 것이 웹 표준

사용자에게 체크 여부를 입력 받음!

☐ Apple ☐ Banana

화면에 출력!

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head></head>
```

```
  <body>
```

```
    <label>
```

```
      <input type="checkbox" /> Apple
```

```
    </label>
```

```
    <label>
```

```
      <input type="checkbox" checked="" /> Banana
```

```
    </label>
```

```
  </body>
```

```
</html>
```

☐ Apple ☒ Banana

화면에 출력!

체크박스 입력 요소 체크됨

```
<!DOCTYPE html>
```

```
<html>
```

```
<head></head>
```

```
<body>
```

```
<label>
```

```
<input type="radio" name="fruits" /> Apple
```

```
</label>
```

```
<label>
```

```
<input type="radio" name="fruits" /> Banana
```

```
</label>
```

```
</body>
```

```
</html>
```

사용자에게 체크 여부를
그룹에서 1개만 입력 받음!

'fruits'란 이름의 그룹

1개만 선택 가능 (택1)

☒ Apple ☐ Banana

화면에 출력!


```
<!DOCTYPE html>
```

```
<html>
```

테이블 요소 = table

```
<head></head>
```

표 요소, 행(Row)과 열(Column)의 집합.

```
<body>
```

```
<table>
```

화면에 출력!

```
<tr>
```

```
<td>A</td><td>B</td>
```

```
</tr>
```

```
<tr>
```

```
<td>C</td><td>D</td>
```

```
</tr>
```

```
</table>
```

```
</body>
```

```
</html>
```

A B
C D

```
<!DOCTYPE html>
```

```
<html>
```

```
<head></head>
```

```
<body>
```

```
<table>
```

```
<tr>
```

```
<td>A</td><td>B</td>
```

```
</tr>
```

```
<tr>
```

```
<td>C</td><td>D</td>
```

```
</tr>
```

```
</table>
```

```
</body>
```

```
</html>
```

테이블 요소 = table-row

행(Row)을 지정하는 요소. (Table Row)

화면에 출력!

A B

C D

```
<!DOCTYPE html>
```

```
<html>
```

```
<head></head>
```

```
<body>
```

```
<table>
```

```
<tr>
```

```
<td>A</td><td>B</td>
```

```
</tr>
```

```
<tr>
```

```
<td>C</td><td>D</td>
```

```
</tr>
```

```
</table>
```

```
</body>
```

```
</html>
```

테이블 요소 = table-cell

열(Column)을 지정하는 요소. (Table Data)

화면에 출력!

A	B
C	D


```
<!DOCTYPE html>
```

```
<html>
```

```
  <head></head>
```

```
  <body>
```

```
    <table>
```

```
      <tr>
```

```
        <td>A</td><td>B</td>
```

```
      </tr>
```

```
      <tr>
```

```
        <td>C</td><td>D</td>
```

```
      </tr>
```

```
    </table>
```

```
  </body>
```

```
</html>
```

화면에 출력!

A B

C D

```
<!DOCTYPE html>
```

```
<html>
```

```
<head></head>
```

```
<body>
```

```
<!--애국가-->
```

```
<div>
```

```

```

```
<h1>애국가</h1>
```

```
<p>동해물과 백두산이 마르고 닳도록</p>
```

```
</div>
```

```
</body>
```

```
</html>
```

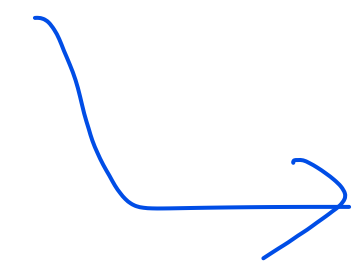


<!-- Comment -->

수정사항이나 설명 등을 작성(주석).

브라우저는 이 태그를 해석하지 않기 때문에 화면에 내용이 표시되지 않음.

HTML 전역 속성



모든 태그에 적용 가능

<태그 title="설명"></태그>

요소의 정보나 설명을 지정

요소에 마우스를 가져다 대면 설명이 띄워짐

<태그 style="스타일"></태그>

요소에 적용할 스타일(CSS)을 지정

<태그 class="이름"></태그>

요소를 지칭하는 중복 가능한 이름

<태그 id="이름"></태그>

요소를 지칭하는 고유한 이름

사용자 정의 속성
태그에 데이터 저장 가능

<태그 data-이름="데이터"></태그>

요소에 데이터를 지정

<https://heropy.blog>



박영웅

CSS 기본 문법

선택자 { 속성: 값; }

스타일(CSS)을 적용할 대상 (Selector)

선택자 {속성: 값; }

스타일(CSS)의 종류 (Property)

선택자 { 속성: 값; }

스타일(CSS)의 값 (Value)

선택자 { 속성: 값; }

속성은 값이다

선택자 {속성: 값; 속성: 값; }

스타일 범위의 시작

스타일 범위의 끝

글자색

빨강

```
div { color: red; }
```

The diagram illustrates the mapping of CSS properties to their Korean equivalents. It features the CSS rule `div { color: red; }` centered on the page. Two light gray rectangular boxes are positioned behind the text: one behind `color` and another behind `red`. An arrow points from the Korean text '글자색' (text color) to the `color` box. Another arrow points from a red box containing the Korean text '빨강' (red) to the `red` box.

요소 외부 여백

20픽셀

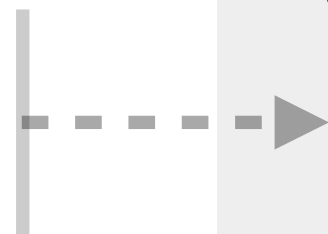
```
div { color: red; margin: 20px; }
```



div {



들여쓰기
(Indent)

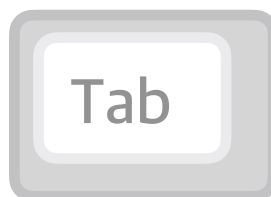


color: red;
margin: 20px;

}

VS

내어쓰기
(Outdent)




```
/* 설명 작성 */  
div {  
    color: red;  
    margin: 20px;  
}
```

주석 시작

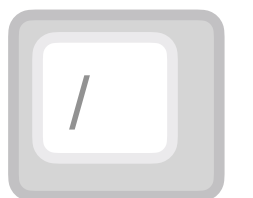
주석 끝

`/* 설명 작성 */`

브라우저는 이 범위를 해석하지 않음



혹은



`div {`

`color: red;`

`margin: 20px;`

`}`

CSS 선언 방식

내장 방식

링크 방식

인라인 방식

@import 방식

내부 스타일 시트

내장 방식

<style></style>의 내용(Contents)으로
스타일을 작성하는 방식

```
<style>
  div {
    color: ■ red;
    margin: 20px;
  }
</style>
```

인라인 스타일

인라인 방식

요소의 style 속성에 직접 스타일을 작성하는 방식
(선택자 없음)

```
<div style="color: ■ red; margin: 20px;"></div>
```

```
<link rel="stylesheet" href="./css/main.css">
```

링크 방식

외부 스타일 시트

<link /> 로 외부 CSS 문서를
가져와서 연결하는 방식

main.css

```
div {  
  color: ■ red;  
  margin: 20px;  
}
```

```
<link rel="stylesheet" href="./css/main.css">
```

<link>

main.css

box.css

```
@import url("./box.css");
```

```
div {  
  color: ■ red;  
  margin: 20px;  
}
```

```
.box {  
  background-color: ■ red;  
  padding: 20px;  
}
```

@import 방식

CSS의 @import 규칙으로 CSS 문서 안에서
또 다른 CSS 문서를 가져와 연결하는 방식

CSS 선택자

기본

복합

가상 클래스

가상 요소

속성

*

기본

전체 선택자 (Universal Selector)

모든 요소를 선택.

```
<div>
  <ul>
    <li>사과</li>
    <li>딸기</li>
    <li>오렌지</li>
  </ul>
  <div>당근</div>
  <p>토마토</p>
  <span>오렌지</span>
</div>
```

선택

*

```
{
  color: ■ red;
}
```

ABC

기본

태그 선택자 (Type Selector)

태그 이름이 ABC인 요소 선택.

```
<div>
  <ul>
    <li>사과</li>
    <li>딸기</li>
    <li>오렌지</li>
  </ul>
  <div>당근</div>
  <p>토마토</p>
  <span>오렌지</span>
</div>
```

선택

li {

color: ■ red;

}

.ABC

기본

클래스 선택자 (Class Selector)

HTML class 속성의 값이 ABC인 요소 선택.

```
<div>
  <ul>
    <li>사과</li>
    <li>딸기</li>
    <li class="orange">오렌지</li>
  </ul>
  <div>당근</div>
  <p>토마토</p>
  <span class="orange">오렌지</span>
</div>
```

선택

```
.orange {
  color: ■ red;
}
```


#ABC

기본

아이디 선택자 (ID Selector)

HTML id 속성의 값이 ABC인 요소 선택.

```
<div>
  <ul>
    <li>사과</li>
    <li>딸기</li>
    <li id="orange" class="orange">오렌지</li>
  </ul>
  <div>당근</div>
  <p>토마토</p>
  <span class="orange">오렌지</span>
</div>
```

선택

```
#orange {
  color: ■ red;
}
```



ABCXYZ

복합

일치 선택자 (Basic Combinator)

선택자 ABC와 XYZ를 동시에 만족하는 요소 선택.

```
<div>
  <ul>
    <li>사과</li>
    <li>딸기</li>
    <li class="orange">오렌지</li>
  </ul>
  <div>당근</div>
  <p>토마토</p>
  <span class="orange">오렌지</span>
</div>
```

선택

`span.orange` {
 color: ■ red;
}

붙여쓸 때, 태그 선택자는 항상 앞에..
(뒤에 쓸 경우, .orangespan 이 선택됨)

ABC > XYZ

복합

자식 선택자 (Child Combinator)

선택자 ABC의 자식 요소 XYZ 선택.

```
<div>
  <ul>
    <li>사과</li>
    <li>딸기</li>
    <li class="orange">오렌지</li>
  </ul>
  <div>당근</div>
  <p>토마토</p>
  <span class="orange">오렌지</span>
</div>
```

선택

```
ul > .orange {
  color: ■ red;
}
```


ABC XYZ

복합

하위(후손) 선택자 (Descendant Combinator)

선택자 ABC의 하위 요소 XYZ 선택.
'띄어쓰기'가 선택자의 기호!

```
<div>
  <ul>
    <li>사과</li>
    <li>딸기</li>
    <li class="orange">오렌지</li>
  </ul>
  <div>당근</div>
  <p>토마토</p>
  <span class="orange">오렌지</span>
</div>
<span class="orange">오렌지</span>
```

The diagram illustrates the descendant selector relationship. A box labeled '선택' (Selector) has two arrows pointing to the 'class="orange"' attributes in the HTML code. One arrow points to the 'class="orange"' attribute in the third list item, and the other points to the 'class="orange"' attribute in the span element. This indicates that the selector targets all elements with the class 'orange' that are descendants of the root div.

```
div .orange {
  color: ■ red;
}
```


ABC + XYZ

복합

인접 형제 선택자 (Adjacent Sibling Combinator)

선택자 ABC의 다음 형제 요소 XYZ 하나를 선택.

```
.orange + li {  
  color: ■ red;  
}
```

선택

```
<ul>  
  <li>딸기</li>  
  <li>수박</li>  
  <li class="orange">오렌지</li>  
  <li>망고</li>  
  <li>사과</li>  
</ul>
```

ABC ~ XYZ

복합

일반 형제 선택자 (General Sibling Combinator)

선택자 ABC의 다음 형제 요소 XYZ 모두를 선택.

```
.orange ~ li {  
  color: red;  
}
```

선택

```
<ul>  
  <li>딸기</li>  
  <li>수박</li>  
  <li class="orange">오렌지</li>  
  <li>망고</li>  
  <li>사과</li>  
</ul>
```

ABC: hover

가상 클래스 선택자 (Pseudo-Classes)

HOVER

선택자 ABC 요소에 마우스 커서가 올라가 있는 동안 선택.

화면에 출력!

NAVER

NAVER

선택

```
a:hover {  
  color: ■ red;  
}
```

```
<a href="https://www.naver.com">NAVER</a>
```


ABC:active

가상 클래스 선택자 (Pseudo-Classes)

ACTIVE

선택자 ABC 요소에 마우스를 클릭하고 있는 동안 선택.

화면에 출력!

NAVER

NAVER

선택

```
a:active {  
  color: ■ red;  
}
```

```
<a href="https://www.naver.com">NAVER</a>
```

ABC:focus

가상 클래스 선택자 (Pseudo-Classes)

FOCUS

선택자 ABC 요소가 포커스되면 선택.

= 선택이 되어 활성화되면

화면에 출력!

선택

```
input:focus {  
  background-color: orange;  
}
```

```
<input type="text" />
```


ABC:first-child

가상 클래스 선택자 (Pseudo-Classes)

FIRST CHILD

선택자 ABC가 형제 요소 중 첫째라면 선택.

선택

```
.fruits span:first-child {  
  color: red;  
}
```

배도 문제 없음,
하지만 구체적으로 선택자를 붙일수록
우선순위가 올라감

```
<div class="fruits">  
  <span>딸기</span>  
  <span>수박</span>  
  <div>오렌지</div>  
  <p>망고</p>  
  <h3>사과</h3>  
</div>
```


ABC:first-child

가상 클래스 선택자 (Pseudo-Classes)

FIRST CHILD

선택자 ABC가 형제 요소 중 첫째라면 선택.

?

```
.fruits div:first-child {  
  color: ■ red;  
}
```

```
<div class="fruits">  
  <span>딸기</span>  
  <span>수박</span>  
  <div>오렌지</div>  
  <p>망고</p>  
  <h3>사과</h3>  
</div>
```

ABC:last-child

가상 클래스 선택자 (Pseudo-Classes)

LAST CHILD

선택자 ABC가 형제 요소 중 막내라면 선택.

선택

```
.fruits h3:last-child {  
  color: ■ red;  
}
```

```
<div class="fruits">  
  <span>딸기</span>  
  <span>수박</span>  
  <div>오렌지</div>  
  <p>망고</p>  
  <h3>사과</h3>  
</div>
```

ABC:nth-child(n)

가상 클래스 선택자 (Pseudo-Classes)

NTH CHILD

선택자 ABC가 형제 요소 중 (n)째라면 선택.

```
.fruits *:nth-child(2) {  
  color: ■ red;  
}
```

선택

```
<div class="fruits">  
  <span>딸기</span>  
  <span>수박</span>  
  <div>오렌지</div>  
  <p>망고</p>  
  <h3>사과</h3>  
</div>
```


ABC:nth-child(n)

가상 클래스 선택자 (Pseudo-Classes)

NTH CHILD

선택자 ABC가 형제 요소 중 (n)째라면 선택.

```
<div class="fruits">
  <span>딸기</span>
  <span>수박</span>
  <div>오렌지</div>
  <p>망고</p>
  <h3>사과</h3>
</div>
```

선택

```
.fruits *:nth-child(2n) {
  color: red;
}
```

n은 0부터 시작!
(Zero-Based Numbering)

0
2
4
...

ABC:nth-child(n)

가상 클래스 선택자 (Pseudo-Classes)

NTH CHILD

선택자 ABC가 형제 요소 중 (n)째라면 선택.

```
<div class="fruits">
  <span>딸기</span>
  <span>수박</span>
  <div>오렌지</div>
  <p>망고</p>
  <h3>사과</h3>
</div>
```

선택

```
.fruits *:nth-child(2n+1) {
  color: red;
}
```

n은 0부터 시작!
(Zero-Based Numbering)

}
f
i

ABC:nth-child(n)

가상 클래스 선택자 (Pseudo-Classes)

NTH CHILD

선택자 ABC가 형제 요소 중 (n)째라면 선택.

```
<div class="fruits">
  <span>딸기</span>
  <span>수박</span>
  <div>오렌지</div>
  <p>망고</p>
  <h3>사과</h3>
</div>
```

선택

```
.fruits *:nth-child(n+2) {
  color: ■ red;
}
```

n은 0부터 시작!
(Zero-Based Numbering)

2
3
4
...

ABC:not(XYZ)

부정 선택자 (Negation)

NOT

선택자 XYZ가 아닌 ABC 요소 선택.

선택

```
.fruits *:not(span) {  
  color: ■ red;  
}
```

뒤에 , 로 여러가지 요소 추가 가능

```
<div class="fruits">  
  <span>딸기</span>  
  <span>수박</span>  
  <div>오렌지</div>  
  <p>망고</p>  
  <h3>사과</h3>  
</div>
```

: 만 입력해도 동작은 하지만, 웹 표준 에러

ABC**::**before

인라인(글자) 요소

가상 요소 선택자 (Pseudo-Elements)

BEFORE

선택자 ABC 요소의 내부 앞에 내용(Content)을 삽입.

화면에 출력!

앞! Content!

```
.box::before {  
  content: "앞!";  
}
```

필수
내용을 비우더라도 무조건 필요

::before

```
<div class="box">
```

Content!

```
</div>
```

ABC::after

인라인(글자) 요소

화면에 출력!

Content! 뒤!

```
<div class="box">
```

Content!

```
</div>
```

가상 요소 선택자 (Pseudo-Elements)

AFTER

선택자 ABC 요소의 내부 뒤에 내용(Content)을 삽입.

```
.box::after {  
  content: "뒤!";  
}
```

::after

[ABC]

속성 선택자 (Attribute)

ATTR

속성 ABC를 포함한 요소 선택

```
[disabled] {  
  color: red;  
}
```

선택

```
<input type="text" value="HEROPY">  
<input type="password" value="1234">  
<input type="text" value="ABCD" disabled>
```


[ABC]

속성 선택자 (Attribute)

ATTR

속성 ABC를 포함한 요소 선택

선택

```
[type] {  
  color: ■ red;  
}
```

```
<input type="text" value="HEROPY">  
<input type="password" value="1234">  
<input type="text" value="ABCD" disabled>
```

[ABC="XYZ"]

속성 선택자 (Attribute)

ATTR=VALUE

속성 ABC를 포함하고 값이 XYZ인 요소 선택.

[type="password"] {
color: ■ red;
}

선택

```
<input type="text" value="HEROPY">  
<input type="password" value="1234">  
<input type="text" value="ABCD" disabled>
```


스타일 상속

```
.animal {  
  color: ■ red;  
}
```

선택

```
<div class="ecosystem">생태계  
  <div class="animal">동물  
    <div class="tiger">호랑이</div>  
    <div class="lion">사자</div>  
    <div class="elephant">코끼리</div>  
  </div>  
  <div class="plant">식물</div>  
</div>
```

화면에 출력!

생태계
동물
호랑이
사자
코끼리
식물

모든 속성들이 상속되는게 아님

상속되는 CSS 속성들..

모두 글자/문자 관련 속성들!

(모든 글자/문자 속성은 아님 주의!)

font-style : 글자 기울기

font-weight : 글자 두께

font-size : 글자 크기

line-height : 줄 높이

font-family : 폰트(서체)

color : 글자 색상

text-align : 정렬

...

강제 상속

inherit 사용

선택자 우선순위

우선순위란, 같은 요소가 여러 선언의 대상이 된 경우,
어떤 선언의 CSS 속성을 우선 적용할지 결정하는 방법

1, 점수가 높은 선언이 우선함!

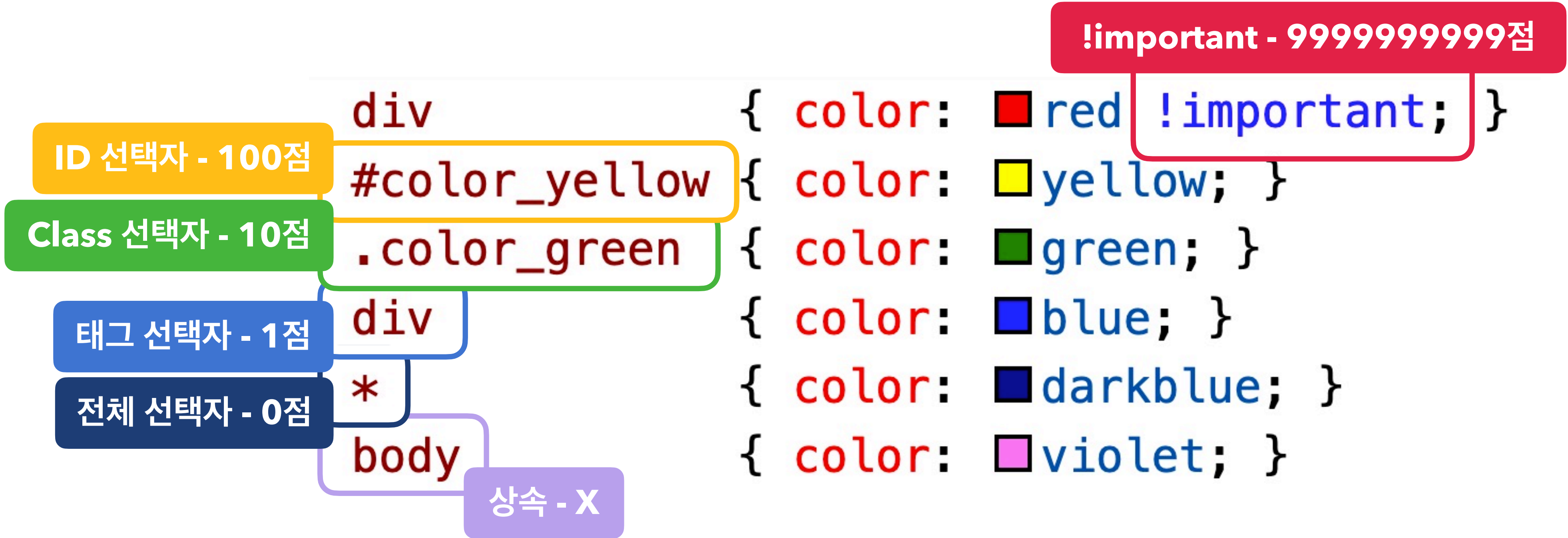
2, 점수가 같으면, 가장 마지막에 해석된 선언이 우선함!

div	{ color: ■ red !important; }
#color_yellow	{ color: ■ yellow; }
.color_green	{ color: ■ green; }
div	{ color: ■ blue; }
*	{ color: ■ darkblue; }
body 상속	{ color: ■ violet; }

선택

```
<div
  id="color_yellow"
  class="color_green"
  style="color: ■ orange;">
  Hello world!
</div>
```

과연 글자색은??



```
<div
  id="color_yellow"
  class="color_green"
  style="color: orange;">
  Hello world!
</div>
```

인라인 선언 - 1000점

21점 `.list li.item { color: red; }`

21점 `.list li:hover { color: red; }`

11점 `.box::before { content: "Good "; color: red; }`

101점 `#submit span { color: red; }`

22점 `header .menu li:nth-child(2) { color: red; }`

1점 `h1 { color: red; }`

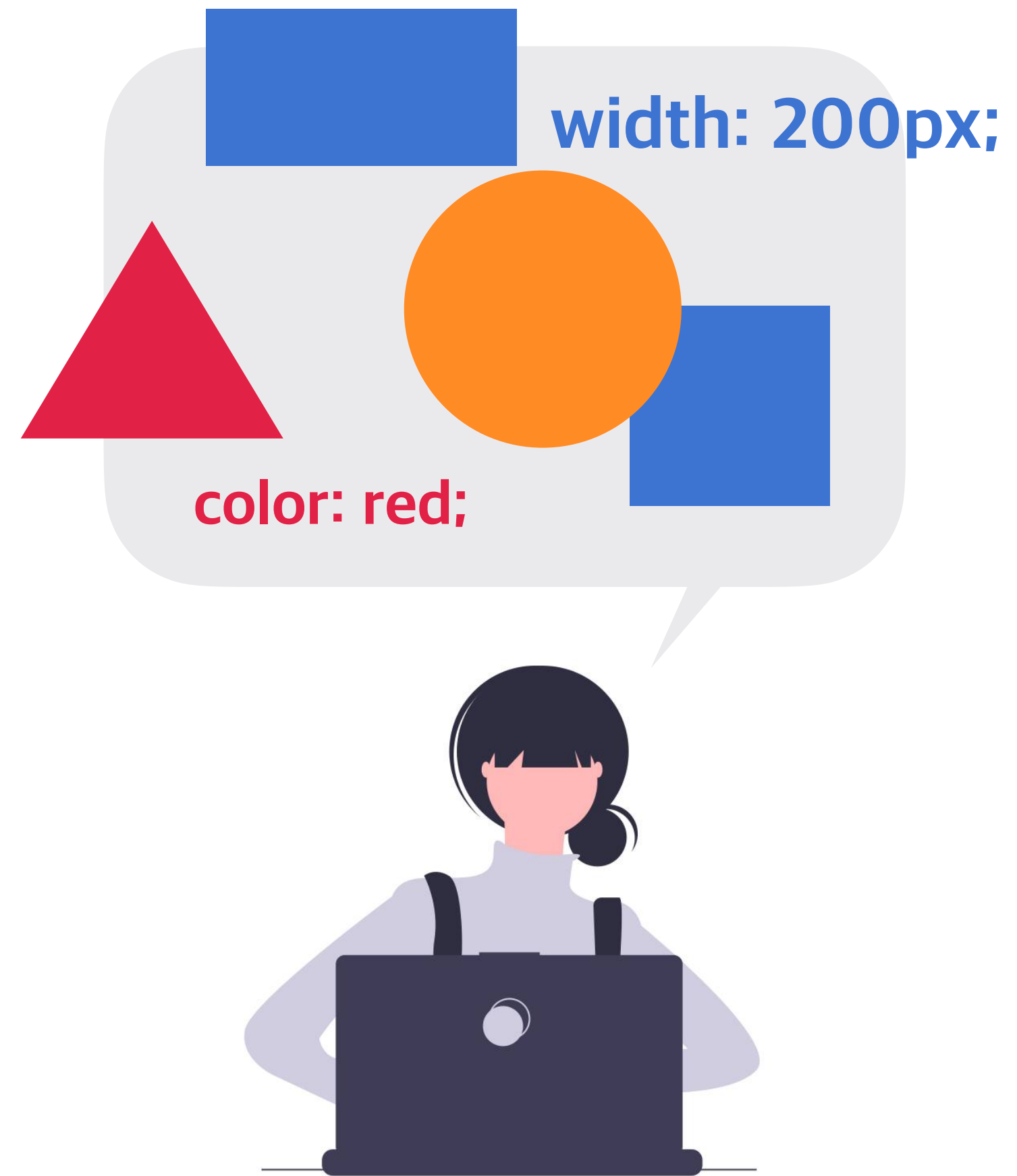
10점 `:not(.box) { color: red; }`

<https://heropy.blog>



박영웅

속성(Properties)



박스 모델
글꼴, 문자
배경
배치
플렉스(정렬)
전환
변환
띄움

자주 씬

애니메이션
그리드
다단
필터

박스 모델

글꼴, 문자

배경

배치

플렉스(정렬)

전환

변환

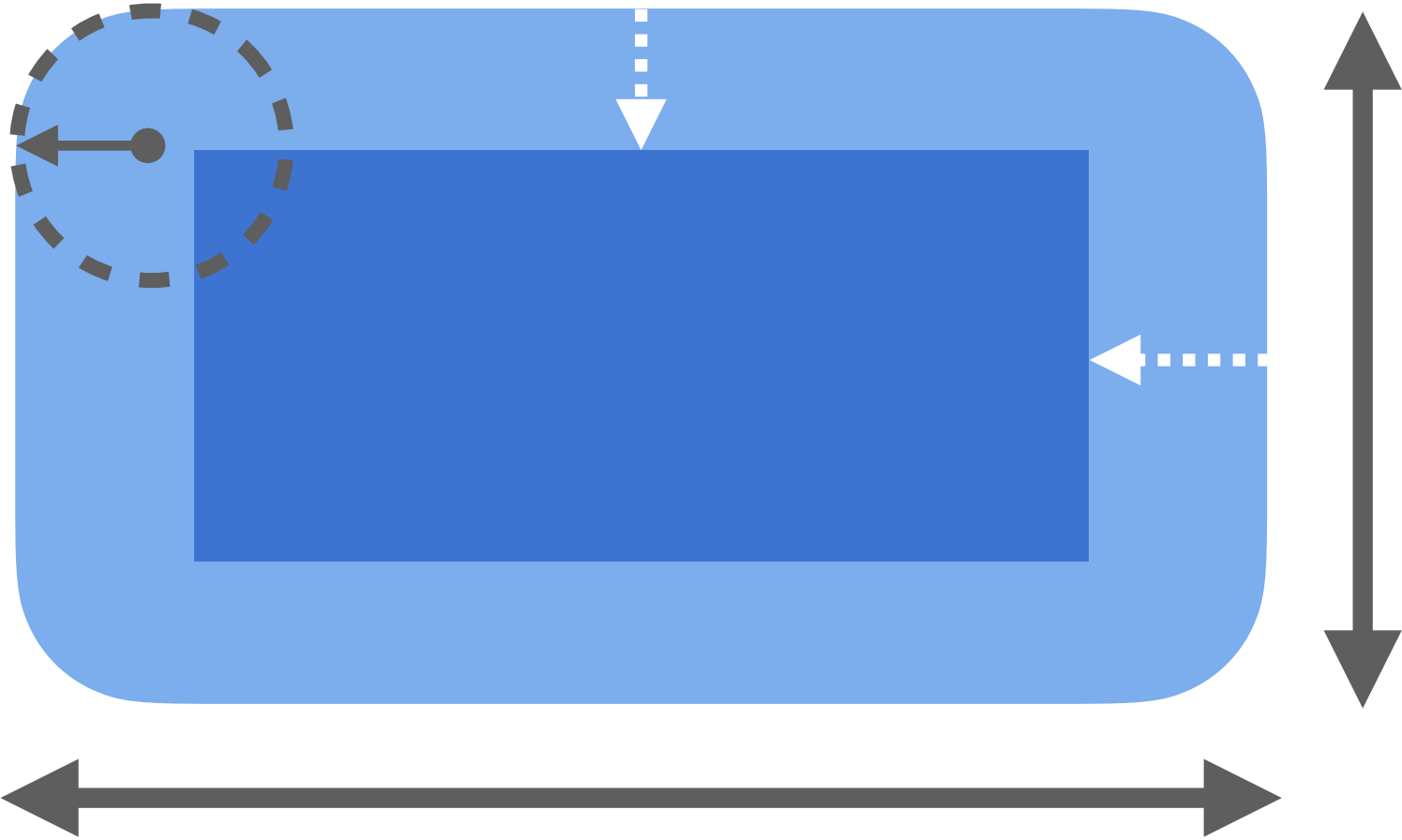
띄움

애니메이션

그리드

다단

필터



박스 모델

글꼴, 문자

배경

배치

플렉스(정렬)

전환

변환

띄움

애니메이션

그리드

다단

필터

font-
text-
로 시작

Hello world
Good morning~

박스 모델
글꼴, 문자

배경

배치

플렉스(정렬)

전환

변환

띄움

애니메이션

그리드

다단

필터



박스 모델
글꼴, 문자
배경

배치 position

플렉스(정렬)

전환

변환

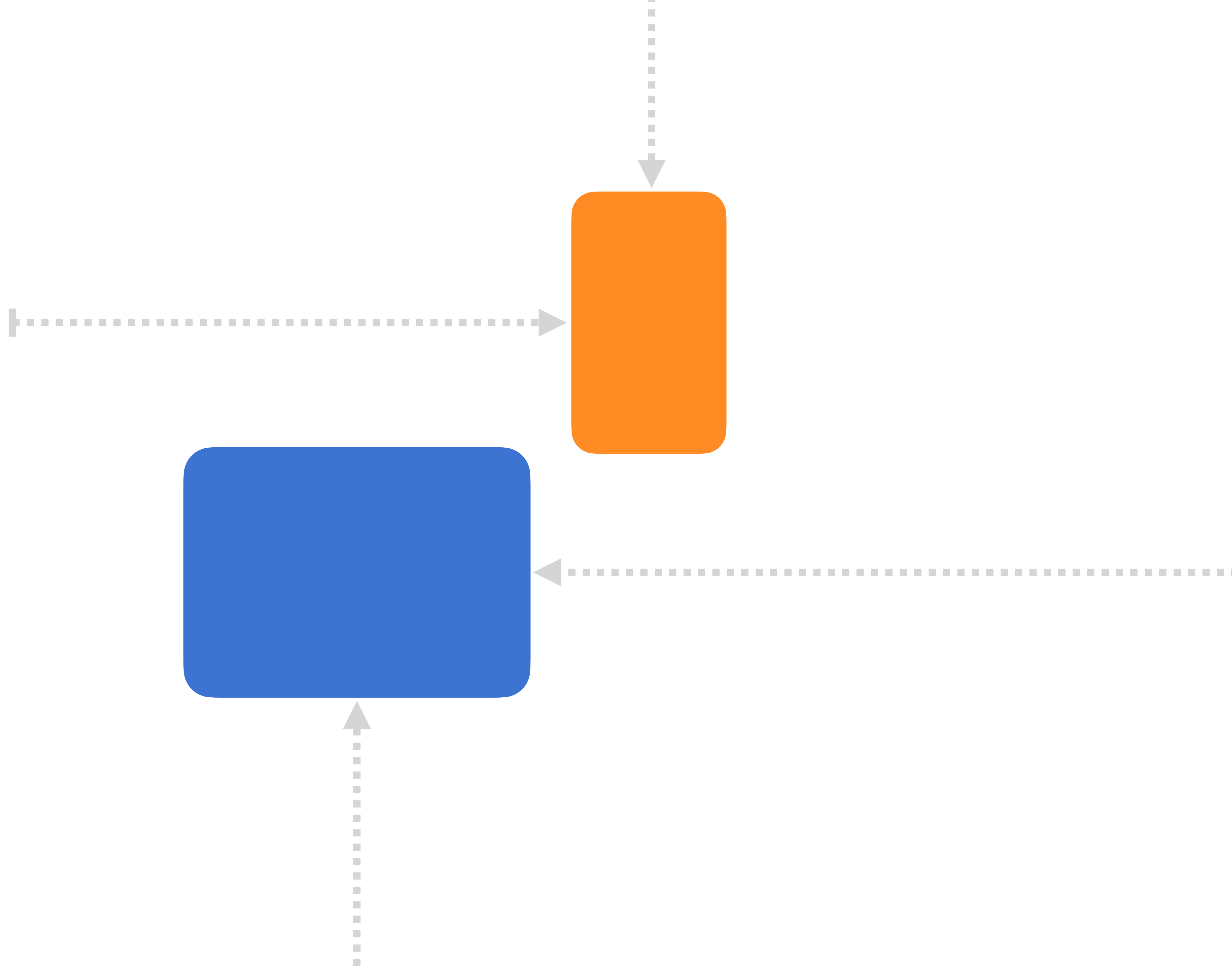
띄움

애니메이션

그리드

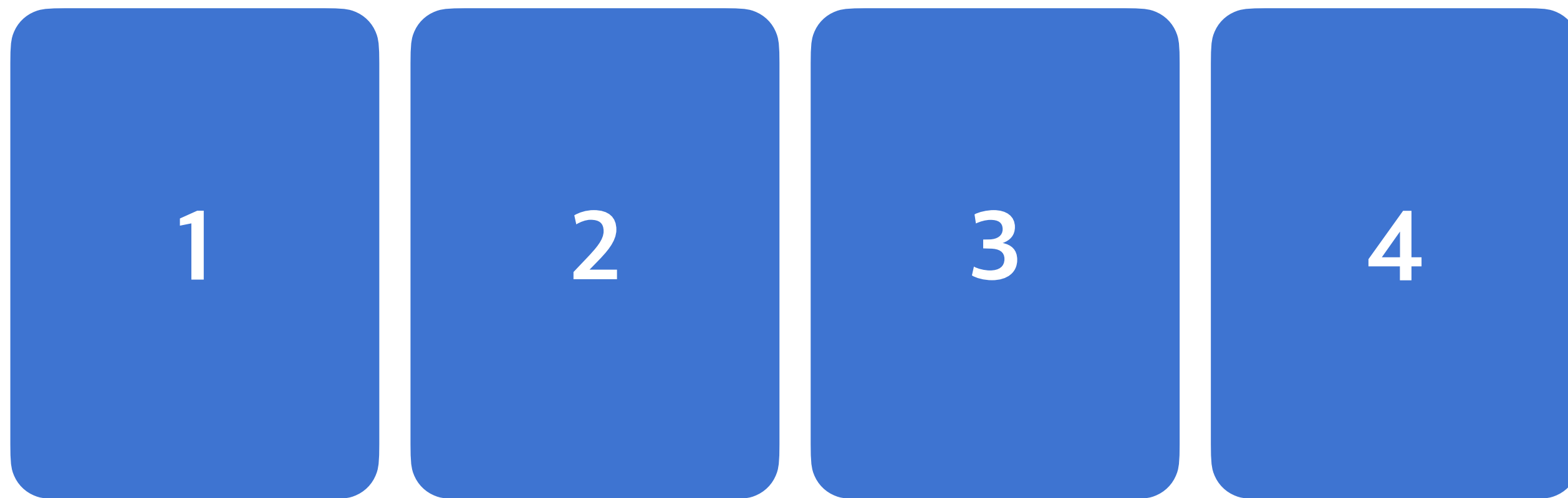
다단

필터



박스 모델
글꼴, 문자
배경
배치
플렉스(정렬)
전환
변환
띄움
애니메이션
그리드
다단
필터

1차원 레이아웃 만들 때 많이 씬



예전에는 레이아웃에 float를 사용했으나,
2014년 플렉스 등장 이후로는 레이아웃에 float를 사용하면 안 됨
(용도가 따로 있음)

박스 모델
글꼴, 문자
배경

배치

플렉스(정렬)

전환 transition

변환

띄움

애니메이션

그리드

다단

필터



박스 모델
글꼴, 문자

배경

배치

플렉스(정렬)

전환

변환 transform < 2d 변환
3d 변환

띄움

애니메이션

그리드

다단

필터



박스 모델
글꼴, 문자

배경

배치

플렉스(정렬)

전환

변환

띄움 float

애니메이션

그리드

다단

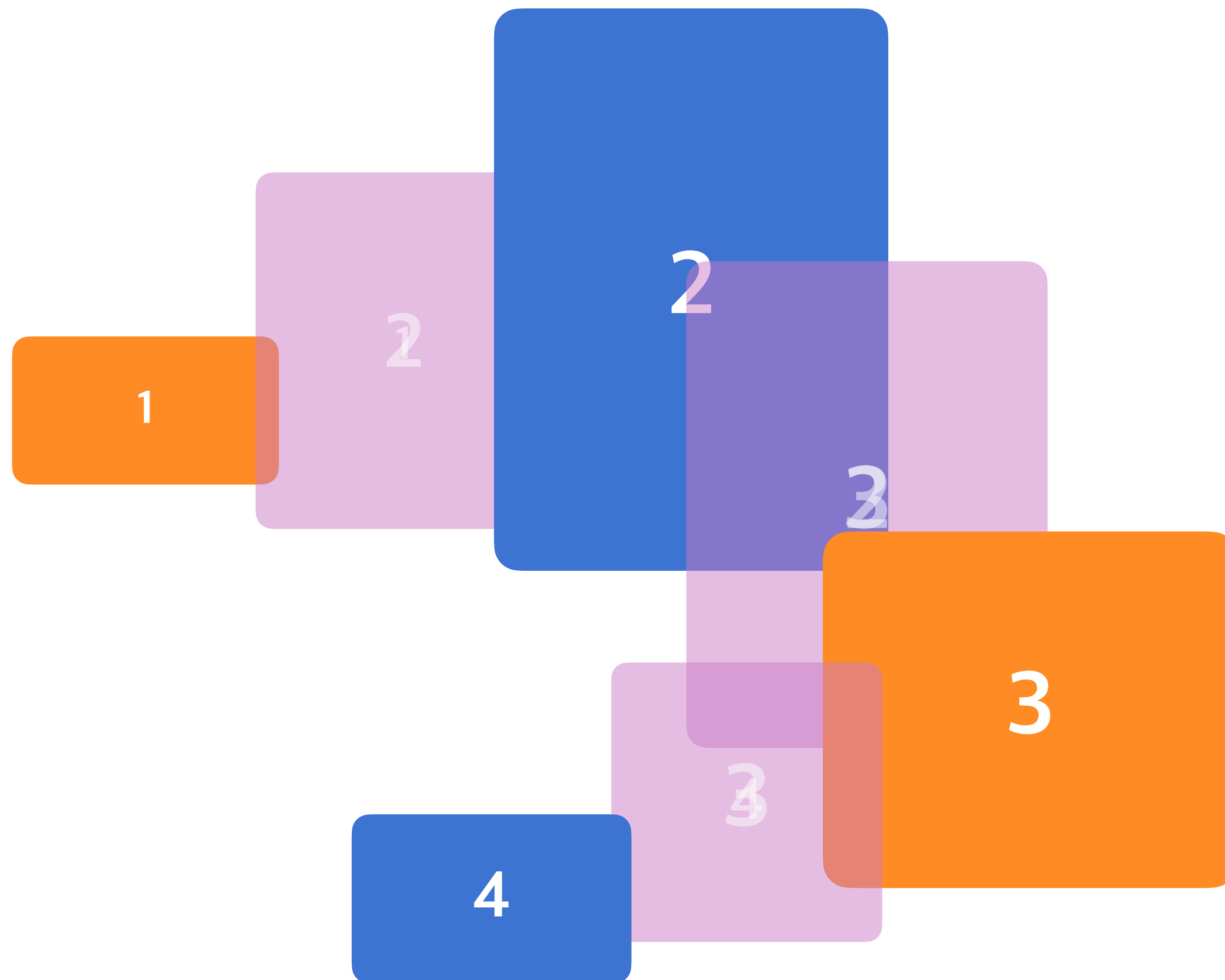
필터



Lorem
Ipsum is
simply dummy
text of the
printing and

Ipsum has been the industry's
standard dummy text ever
since the 1500s, when an

박스 모델
글꼴, 문자
배경
배치
플렉스(정렬)
전환
변환
띄움
애니메이션
그리드
다단
필터



박스 모델
글꼴, 문자

배경

배치

플렉스(정렬) 1차원

전환

변환

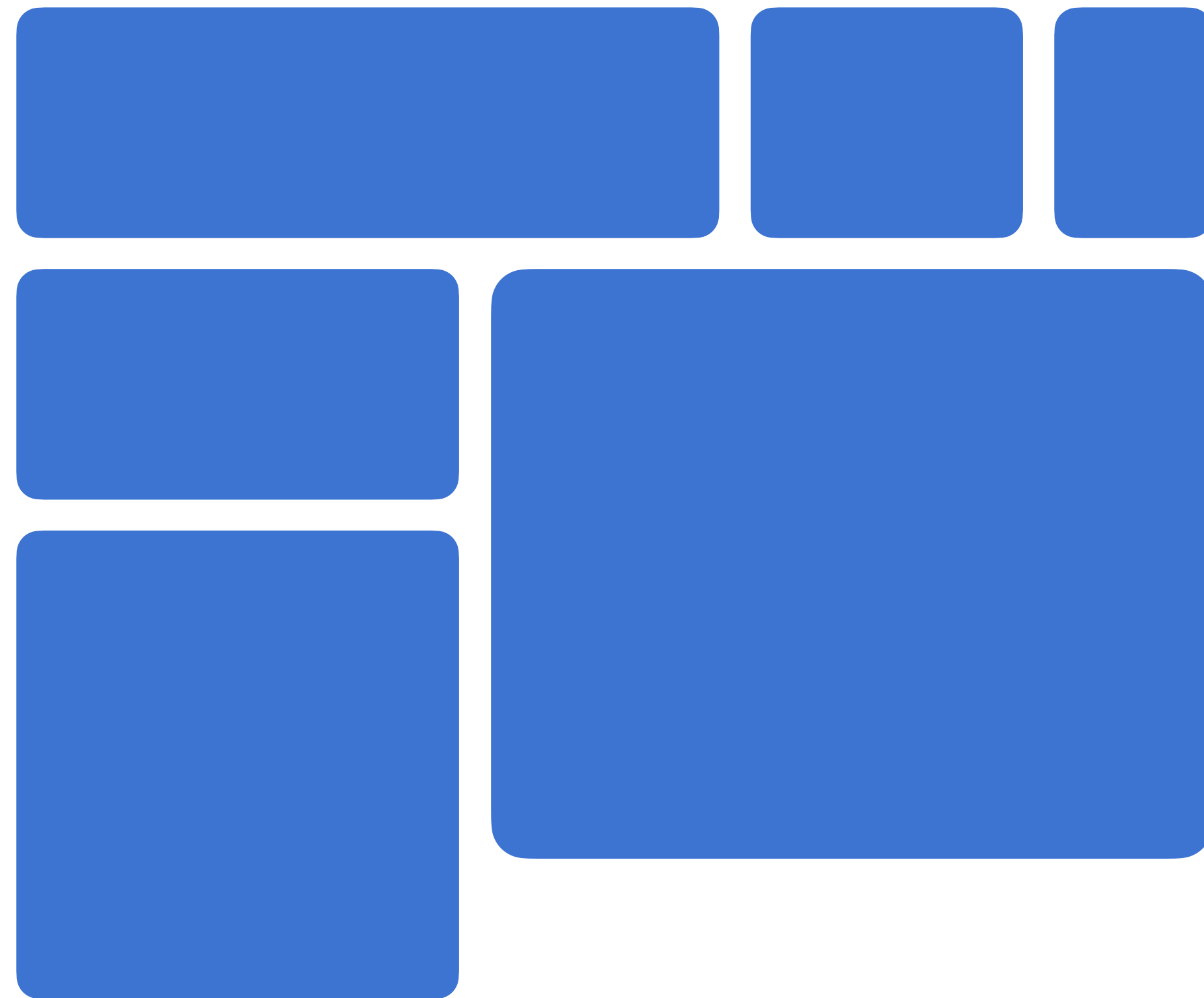
띄움

애니메이션

그리드 2차원

다단

필터



잘 안씀...사용법 복잡
2차원 레이아웃을 만드는 건 flex로도 해결 가능

박스 모델

글꼴, 문자

배경

배치

플렉스(정렬)

전환

변환

띄움

애니메이션

그리드

다단 columns

필터

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been

the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type

박스 모델
글꼴, 문자
배경

배치

플렉스(정렬)

전환

변환

띄움

애니메이션

그리드

다단

필터



박스 모델

요소의 가로/세로 너비

width, height

기본값
(요소에 이미 들어있는 속성의 값)



auto

브라우저가 너비를 계산 block, inline 특성에 따라

단위

px, em, vw 등 단위로 지정

`<span>Hello</span>`
`<span>World</span>`

`<span></span>`

대표적인 인라인 요소!
본질적으로 아무것도 나타내지 않는,
콘텐츠 영역을 설정하는 용도.



<div>Hello</div>
<div>World</div>

<div></div>

대표적인 블록 요소!
본질적으로 아무것도 나타내지 않는,
콘텐츠 영역을 설정하는 용도.

auto

부모 요소의 크기만큼 자동으로 늘어남!

Hello

포함한 콘텐츠 크기만큼
자동으로 줄어듬!

World

auto

요소가 커질 수 있는 최대 가로/세로 너비

max-width, max-height

기본값

none

최대 너비 제한 없음

auto

브라우저가 너비를 계산

단위

px, em, vw 등 단위로 지정

요소가 작아질 수 있는 최소 가로/세로 너비

min-width, min-height

기본값

0

최소 너비 제한 없음

auto

브라우저가 너비를 계산

단위

px, em, vw 등 단위로 지정

표현 단위

단위

px	픽셀
%	상대적 백분율
em	요소의 글꼴 크기
rem	루트 요소(html)의 글꼴 크기
vw	뷰포트 가로 너비의 백분율
vh	뷰포트 세로 너비의 백분율

요소의 외부 여백(공간)을 지정하는 단축 속성

가로(세로) 너비가 있는 요소의
가운데 정렬에 활용해요!

margin

음수를 사용할 수 있어요!

요소 배치를 위해 사용하기도 함

0

외부 여백 없음

auto

브라우저가 여백을 계산

단위

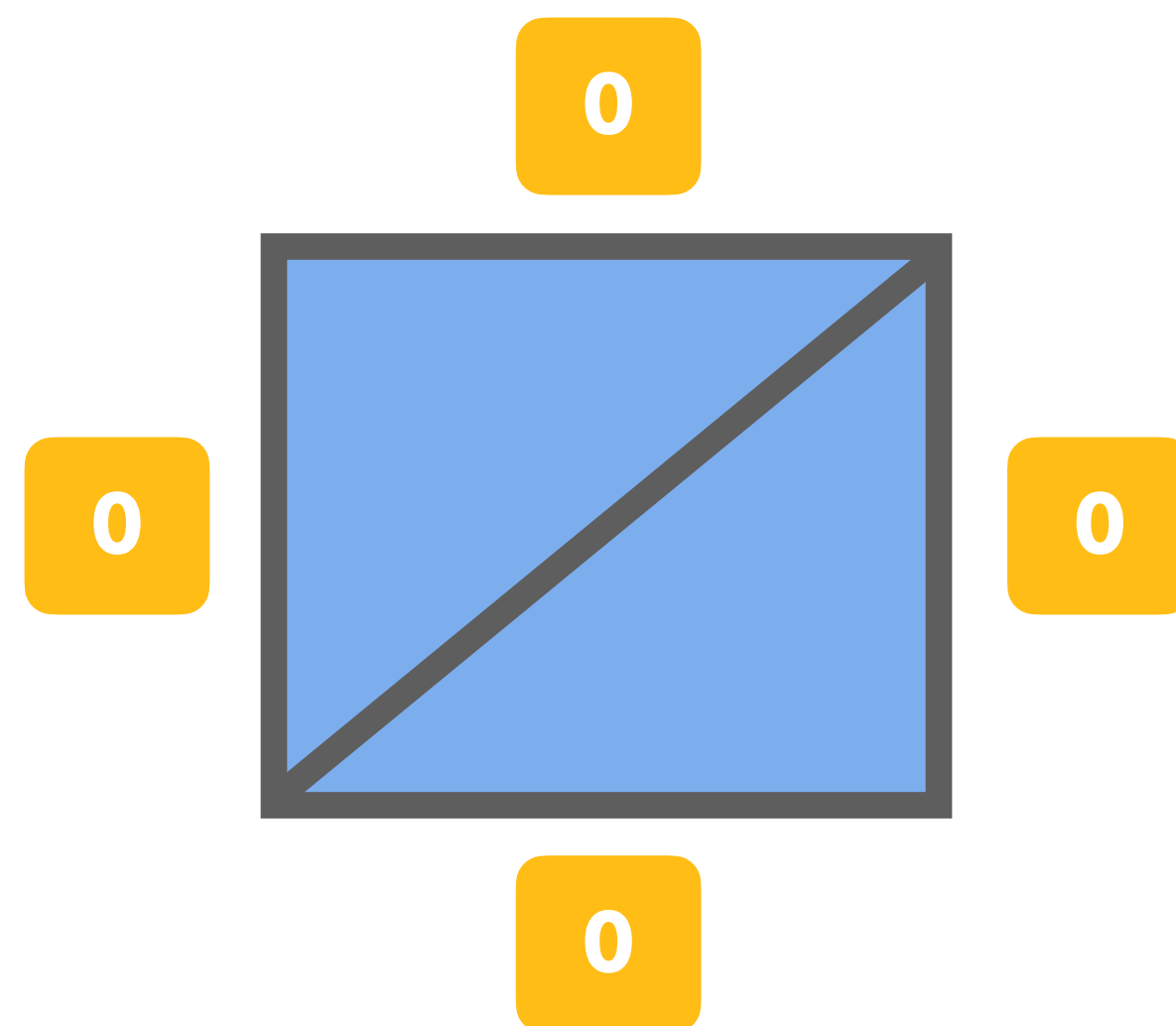
px, em, vw 등 단위로 지정

%

부모 요소의 가로 너비에 대한 비율로 지정

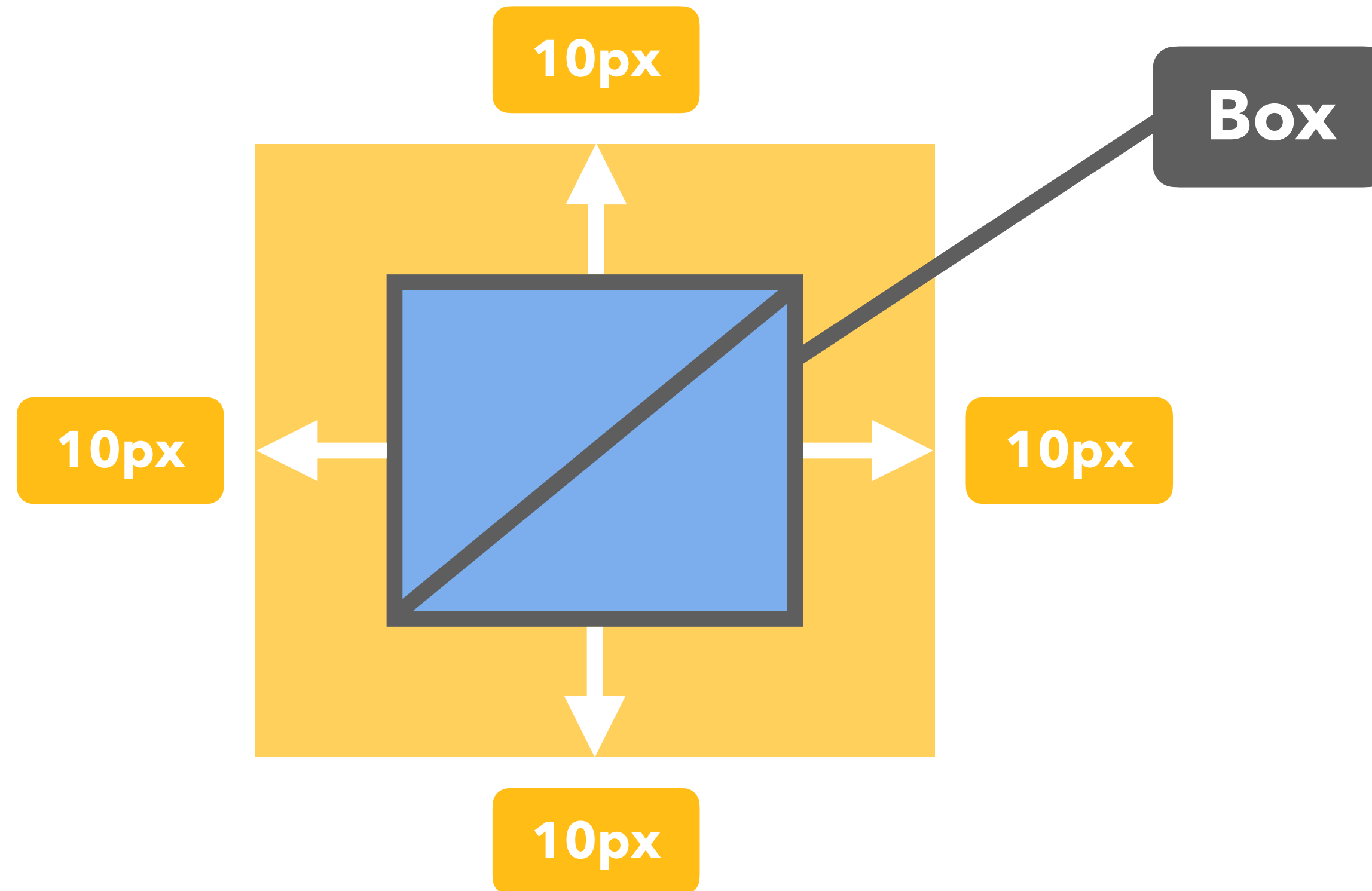
margin: 0;

top, right, bottom, left



margin: 10px;

top, right, bottom, left

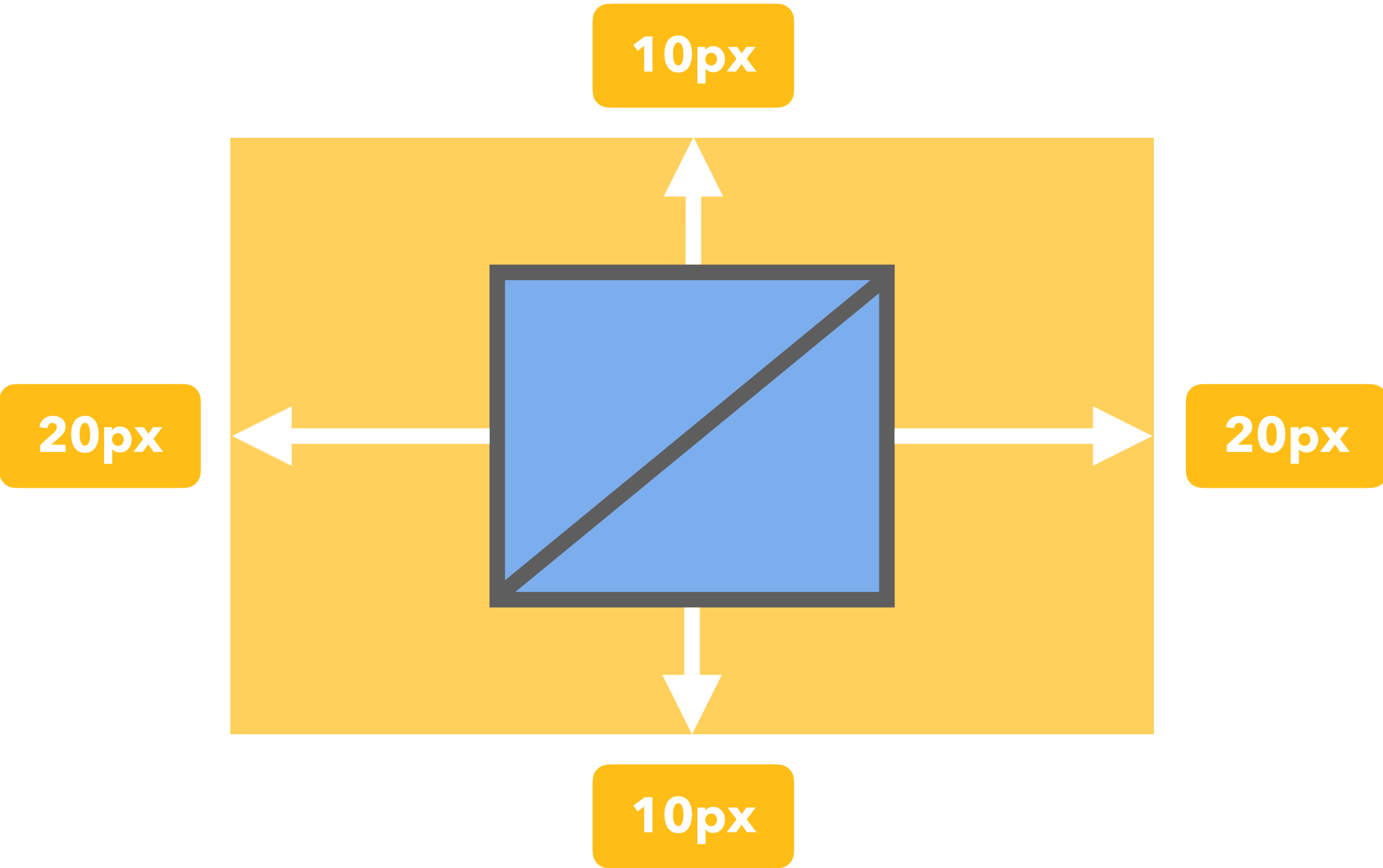


margin: 10px 20px;

top, bottom

left, right

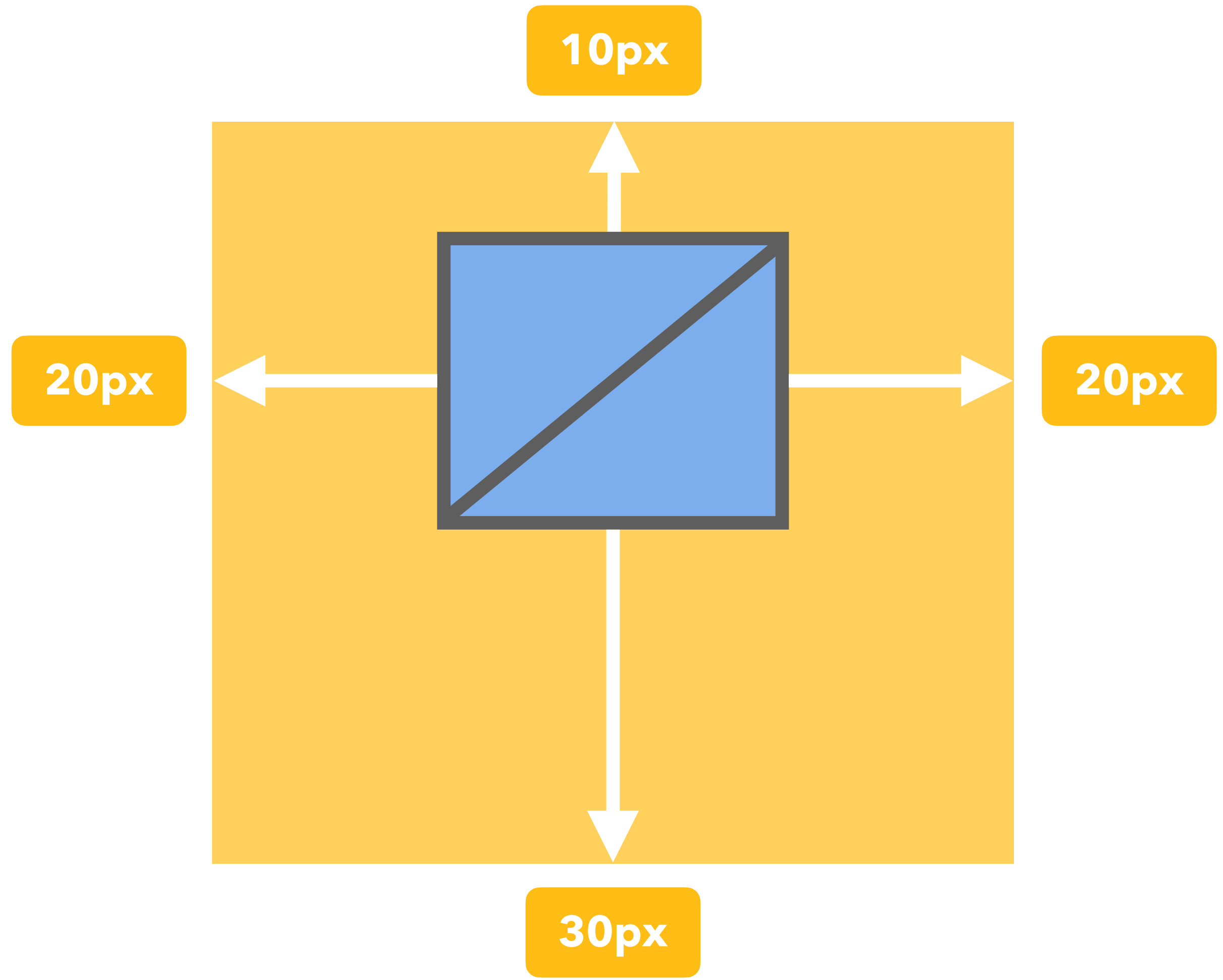
상하.좌우.



margin: 10px 20px 30px;

top left, right bottom

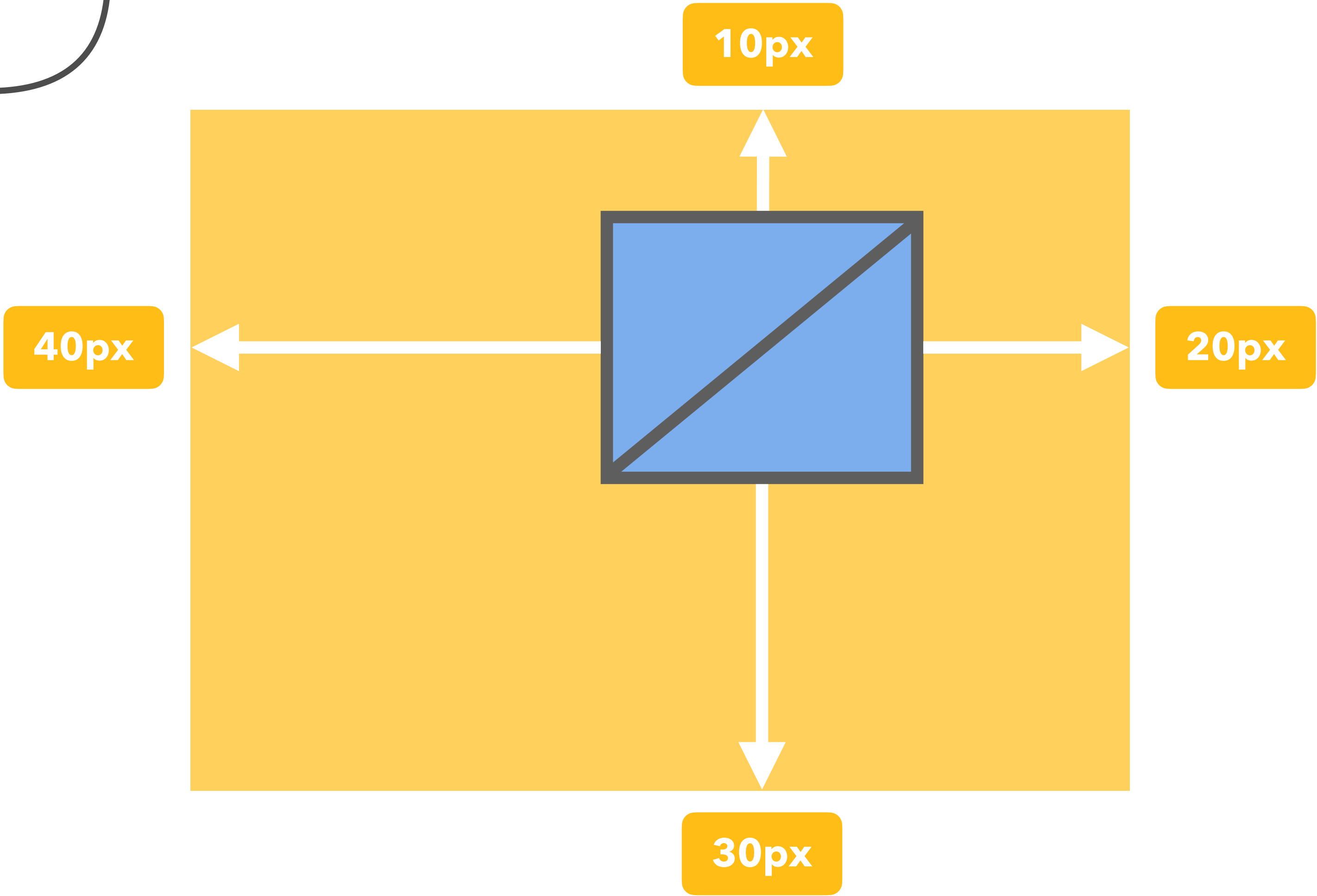
상.중.하



margin: 10px 20px 30px 40px;

top right bottom left

상.시계 방향



margin: top, right, bottom, left ;

margin: top, bottom left, right ;

margin: top left, right bottom ;

margin: top right bottom left ;

요소의 외부 여백(공간)을 지정하는 기타 개별 속성들

margin-방향

margin-top
margin-bottom
margin-left
margin-right

요소의 내부 여백(공간)을 지정하는 단축 속성

음수 사용 불가
box-size와 관련

padding

요소의 크기가 커져요!



0

내부 여백 없음

단위

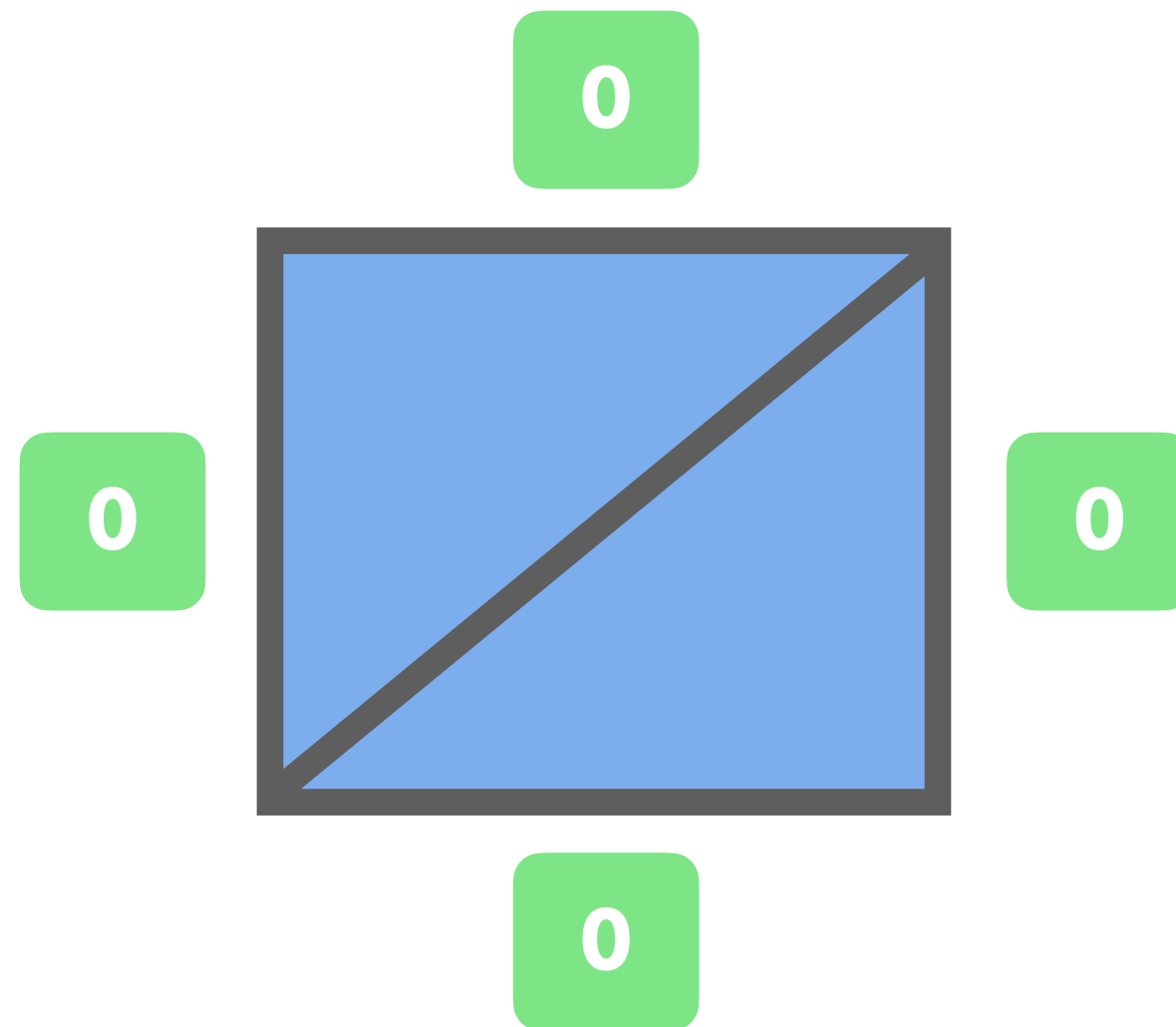
px, em, vw 등 단위로 지정

%

부모 요소의 가로 너비에 대한 비율로 지정

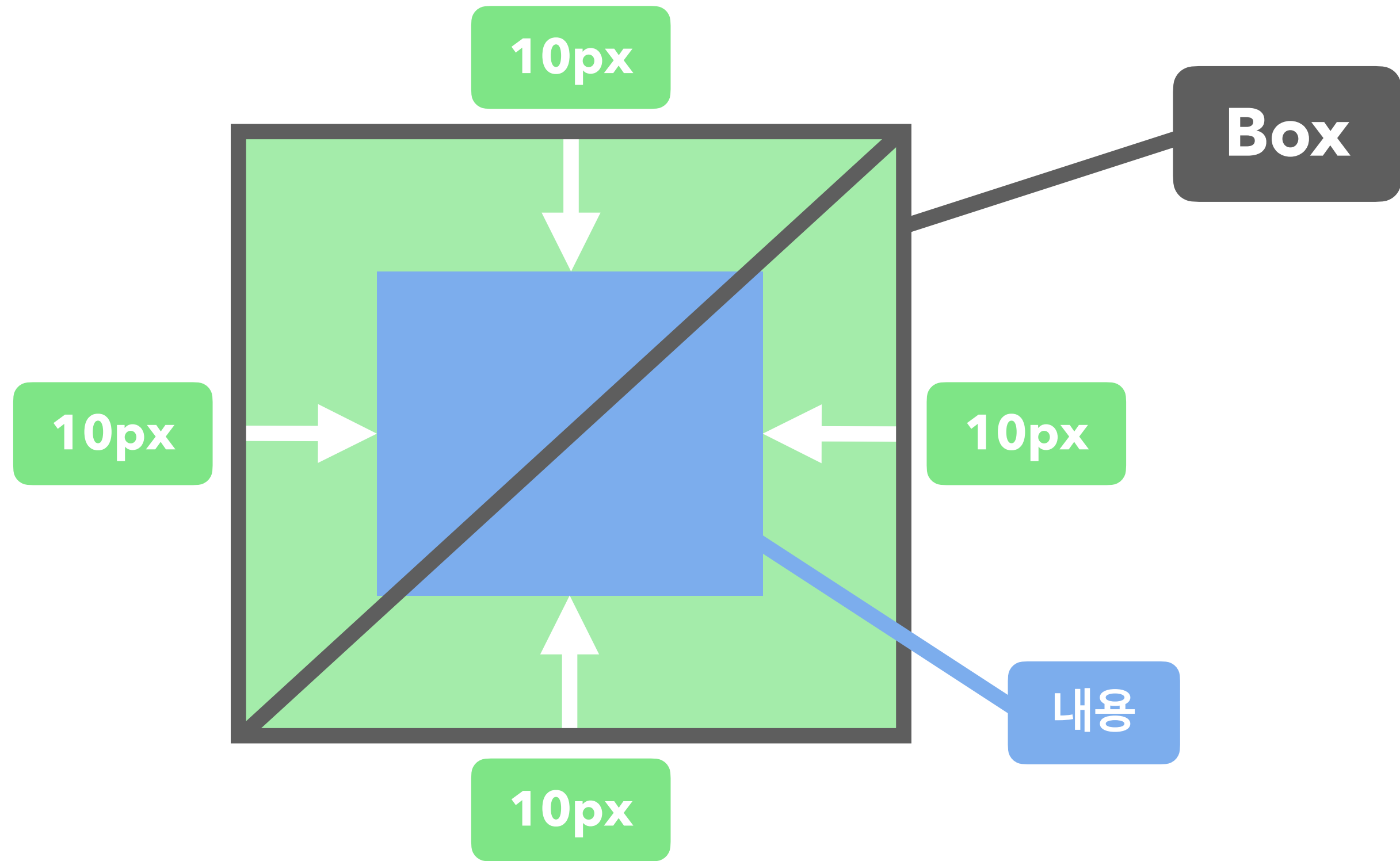
padding: 0;

top, right, bottom, left



padding: 10px;

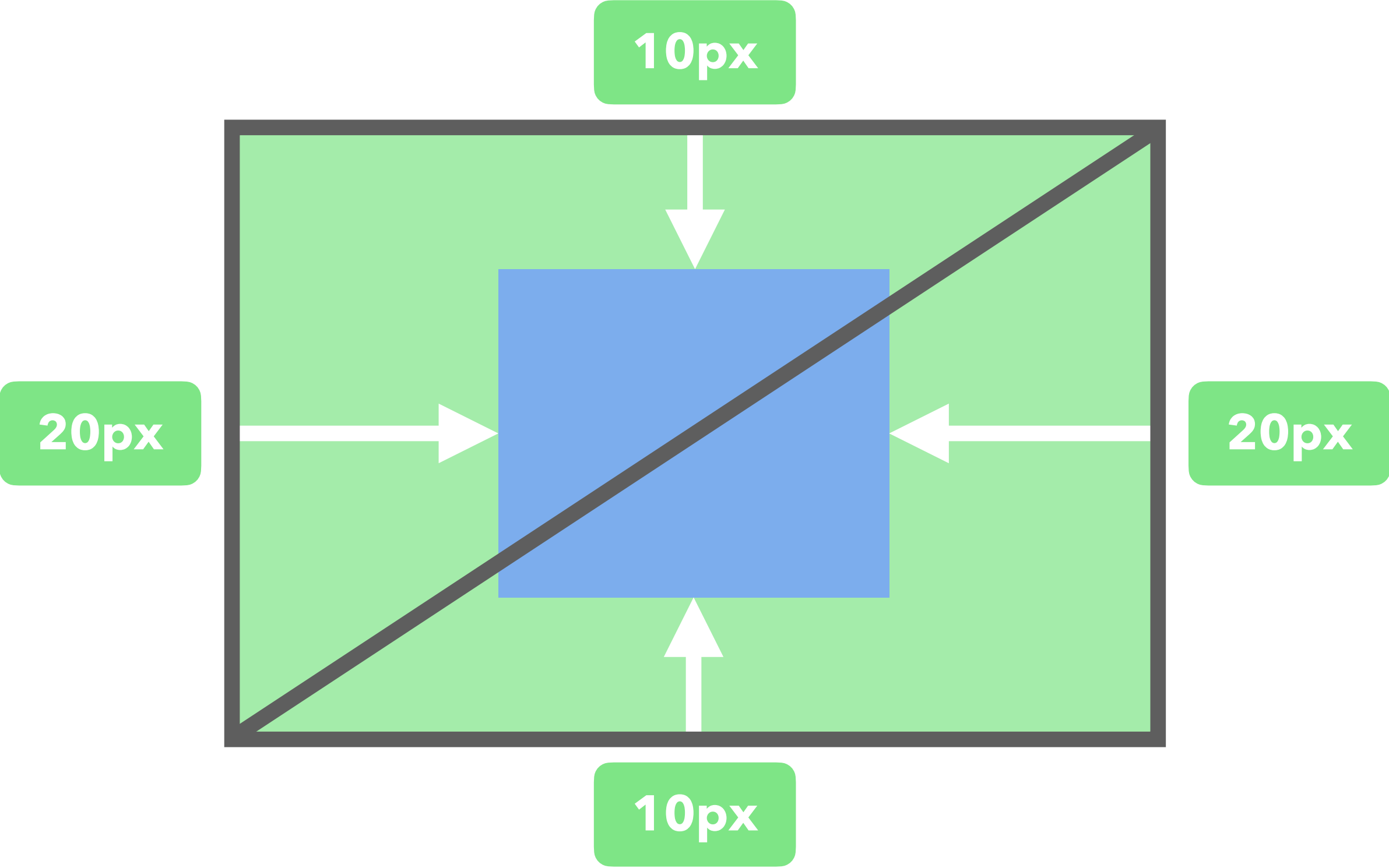
top, right, bottom, left



padding: 10px 20px;

top, bottom

left, right

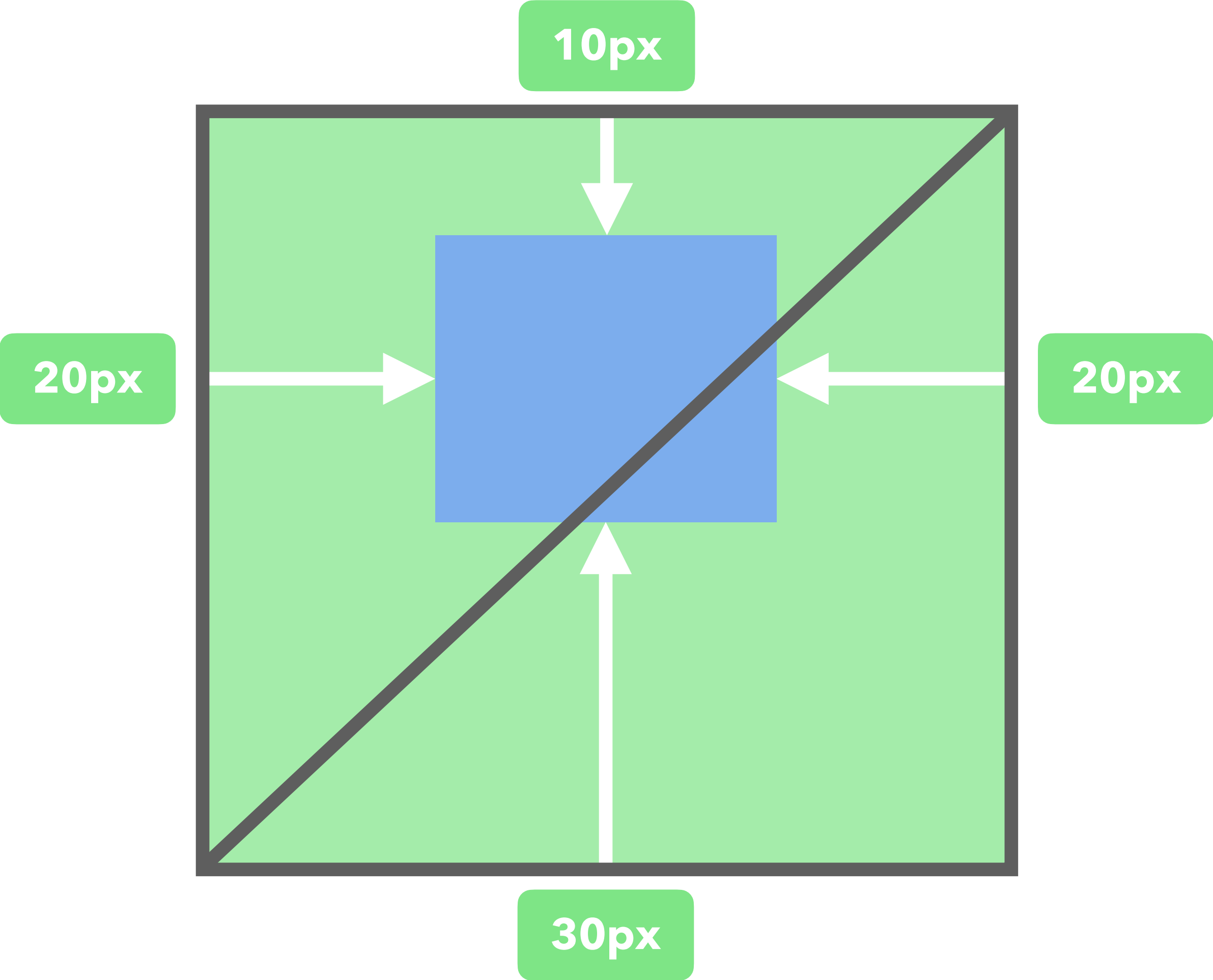


padding: 10px 20px 30px;

top

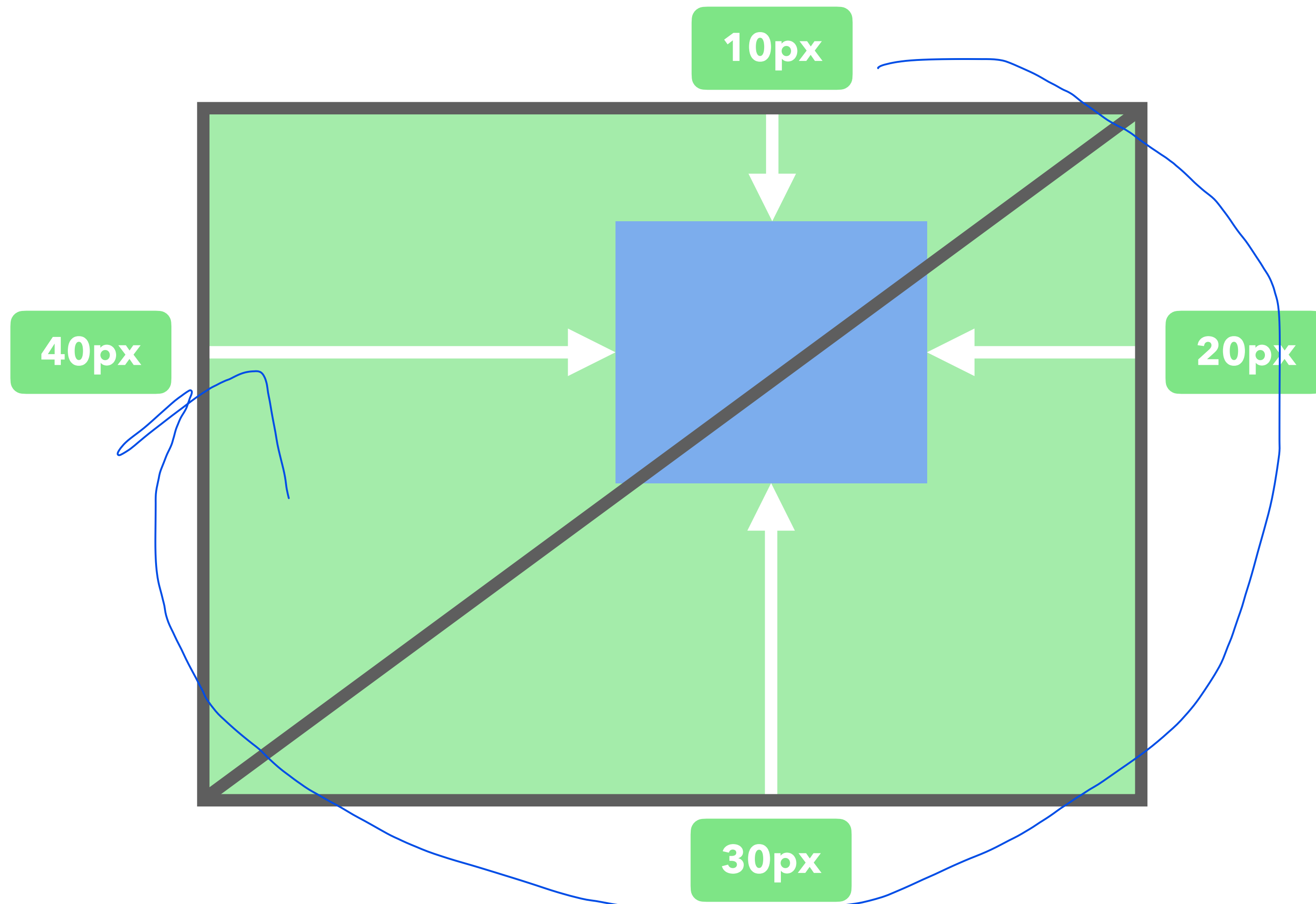
left, right

bottom



padding: 10px 20px 30px 40px;

top right bottom left



padding: top, right, bottom, left ;

padding: top, bottom left, right ;

padding: top left, right bottom ;

padding: top right bottom left ;

요소의 내부 여백(공간)을 지정하는 기타 개별 속성들

padding-방향

padding-top
padding-bottom
padding-left
padding-right

요소의 크기가 커져요!

요소의 테두리 선을 지정하는 단축 속성

border: 선-두께 선-종류 선-색상;

border-color

border-width

border-style

이 순서로 안 써도 동작은 하지만, 관례임!

border: medium none ■ black;

border-width

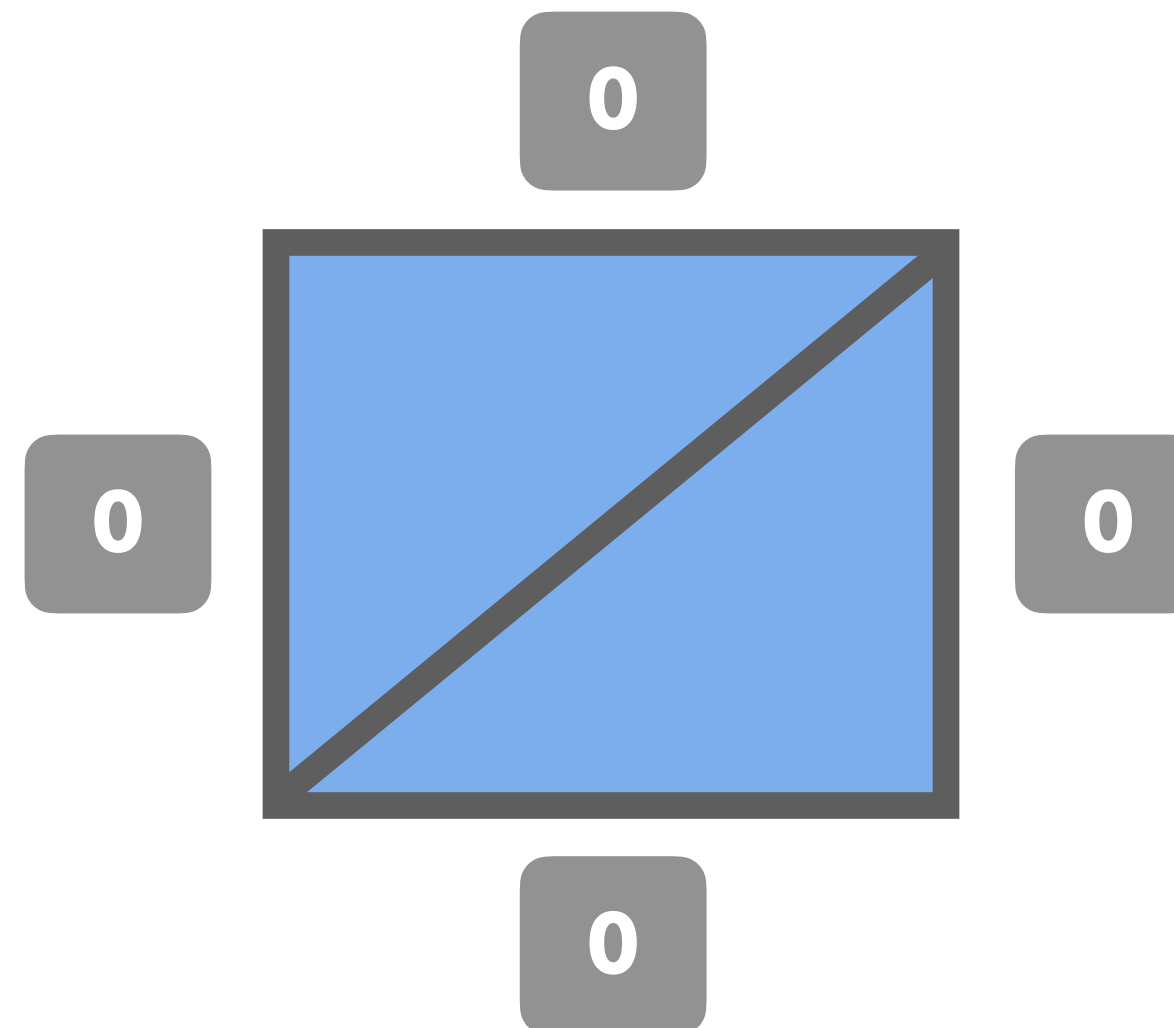
border-style

border-color

기본값

(요소에 이미 들어있는 속성의 값)

선의 종류가 없어서(none)
출력되지 않아요!

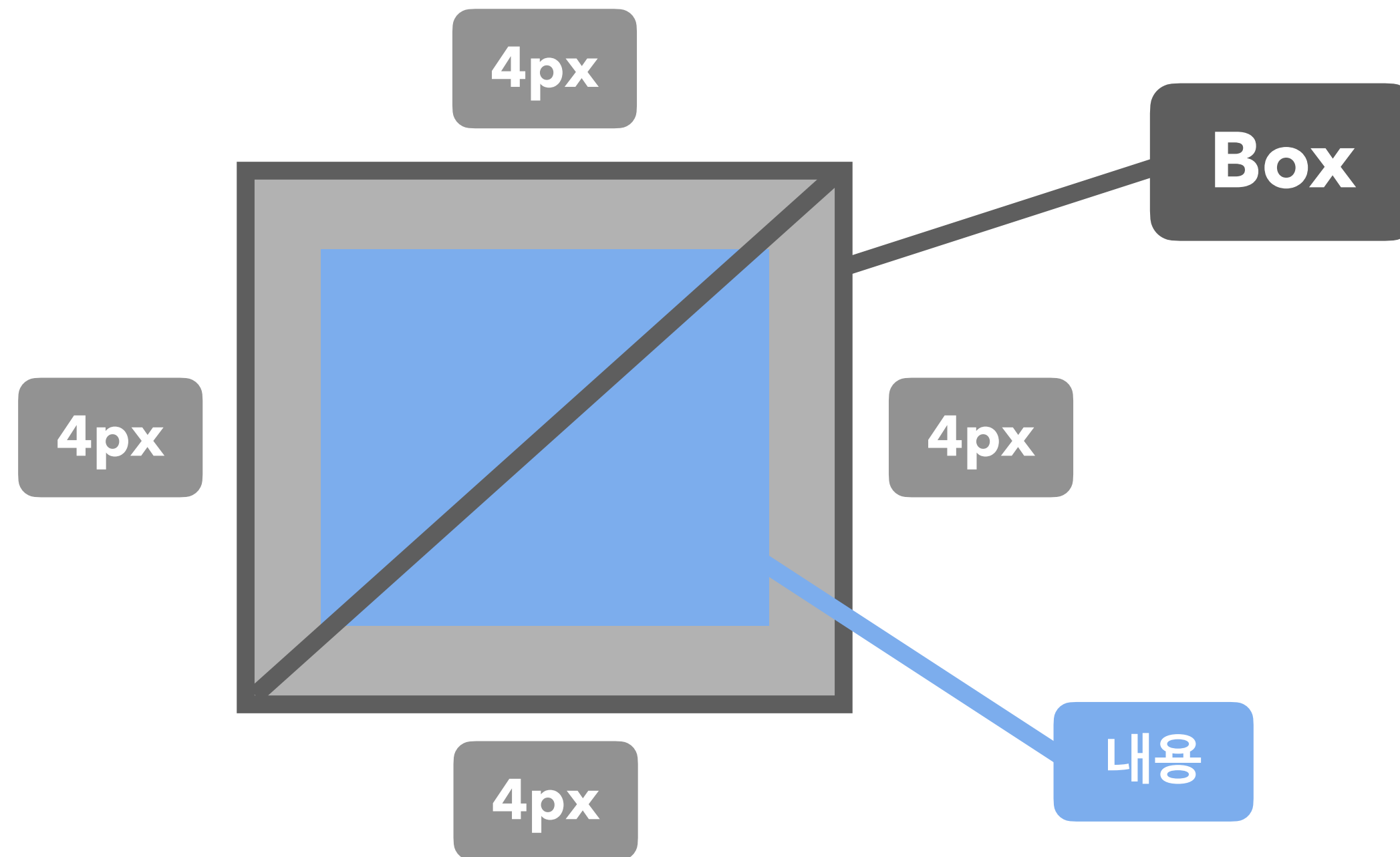


border: 4px solid ■ black;

border-width

border-style

border-color

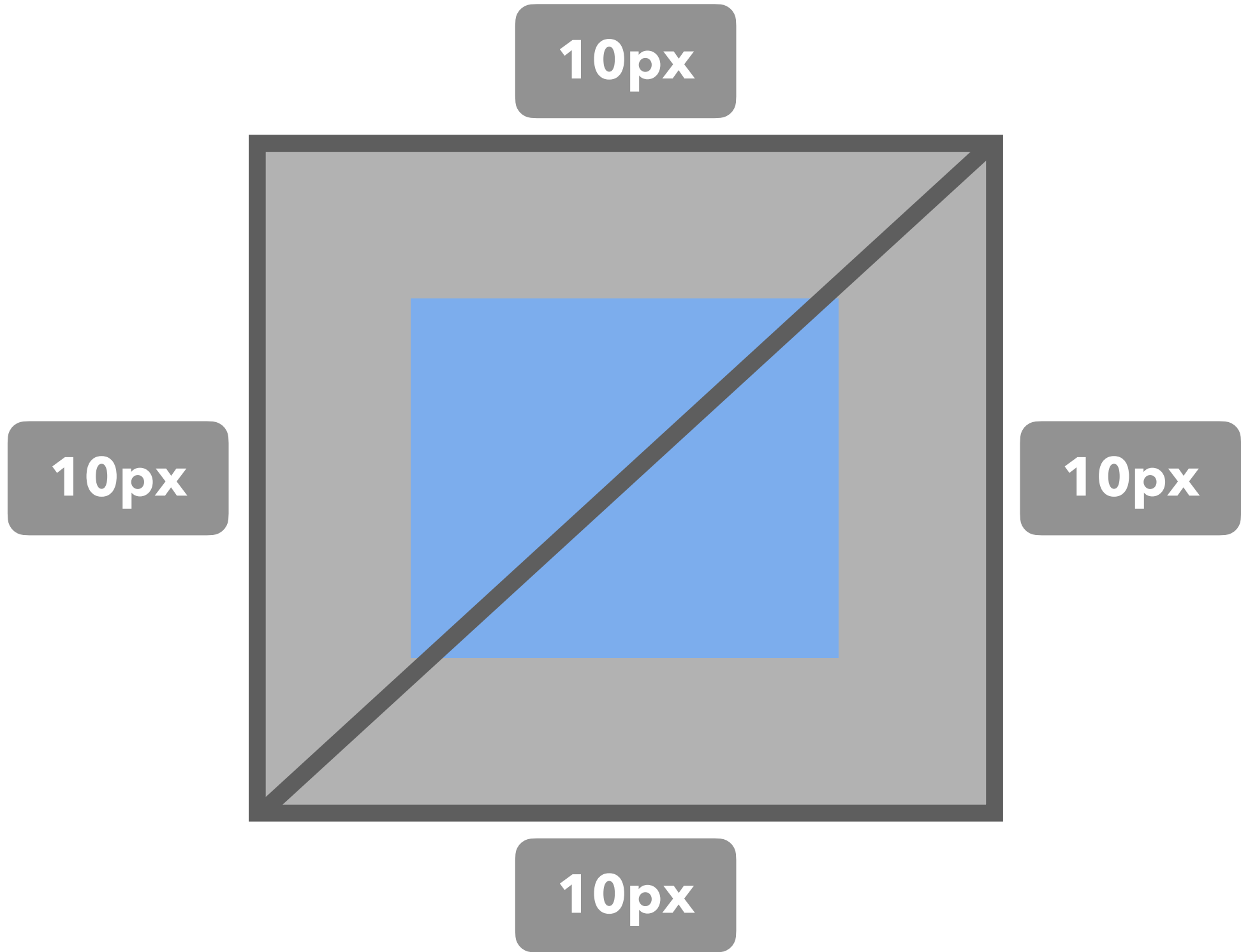


border: 10px solid ■ black;

border-width

border-style

border-color



요소 테두리 선의 두께

border-width



키워드는 잘 안 씀
(브라우저마다 이 키워드를 정의하는 단위가 다를 수 있기 때문)

border-width: top, right, bottom, left ;

border-width: top, bottom left, right ;

border-width: top left, right bottom ;

border-width: top right bottom left ;

요소 테두리 선의 종류

border-style

none

선 없음

solid

실선 (일반 선)

dotted

점선

dashed

파선

double

두 줄 선

groove

홈이 파여있는 모양

ridge

숫은 모양 (groove의 반대)

inset

요소 전체가 들어간 모양

outset

요소 전체가 나온 모양

border-style: **top, right, bottom, left** ;

border-style: **top, bottom** **left, right** ;

border-style: **top** **left, right** **bottom** ;

border-style: **top** **right** **bottom** **left** ;

요소 테두리 선의 색상을 지정하는 단축 속성

border-color

black 검정색

색상 선의 색상

transparent 투명

border-color: top, right, bottom, left ;

border-color: top, bottom left, right ;

border-color: top left, right bottom ;

border-color: top right bottom left ;

색을 사용하는 모든 속성에 적용 가능한 색상 표현

색상 표현

실무에선 사용 안 함

~~색상 이름~~

브라우저에서 제공하는 색상 이름

red, tomato, royalblue

Hex 색상코드

16진수 색상(Hexadecimal Colors)

#000, #FFFFFF

RGB

빛의 삼원색

rgb(255, 255, 255)

RGBA

빛의 삼원색 + 투명도

rgba(0, 0, 0, 0.5)

HSL

색상, 채도, 명도

hsl(120, 100%, 50%)

HSLA

색상, 채도, 명도 + 투명도

hsla(120, 100%, 50%, 0.3)

권장

요소의 테두리 선을 지정하는 기타 속성들

border-방향

border-방향-속성

border-top: 두께 종류 색상;

border-top-width: 두께;

border-top-style: 종류;

border-top-color: 색상;

border-bottom: 두께 종류 색상;

border-bottom-width: 두께;

border-bottom-style: 종류;

border-bottom-color: 색상;

border-left: 두께 종류 색상;

border-left-width: 두께;

border-left-style: 종류;

border-left-color: 색상;

border-right: 두께 종류 색상;

border-right-width: 두께;

border-right-style: 종류;

border-right-color: 색상;

요소의 모서리를 둥글게 깎음

border-radius

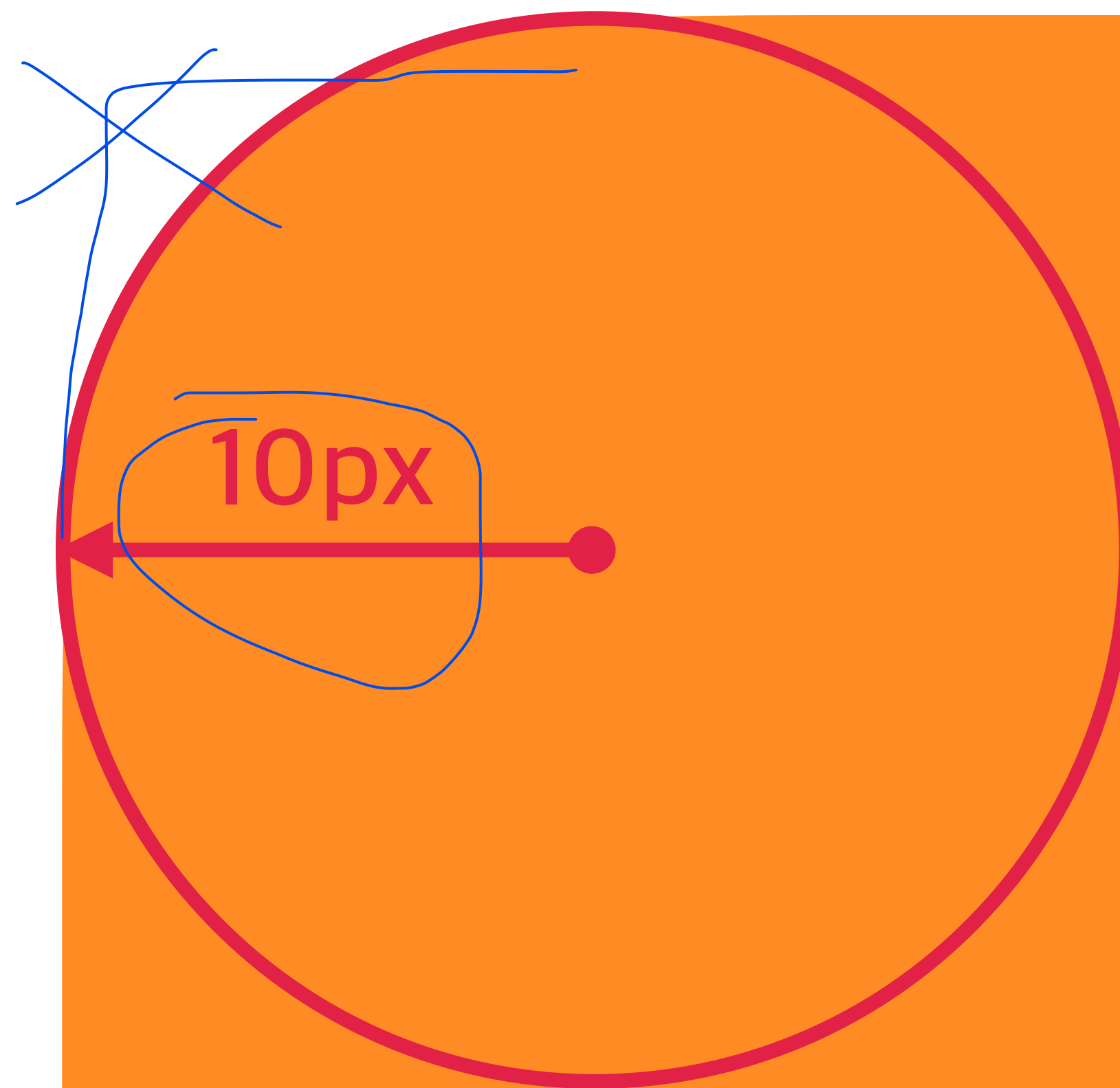
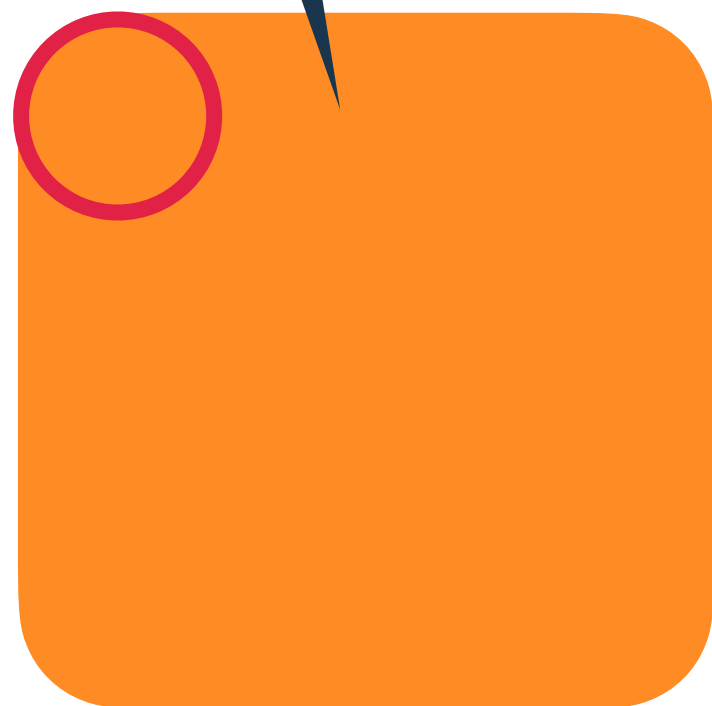
0

둥글게 없음

단위

px, em, vw 등 단위로 지정

`border-radius: 10px;`



`border-radius: 0;`

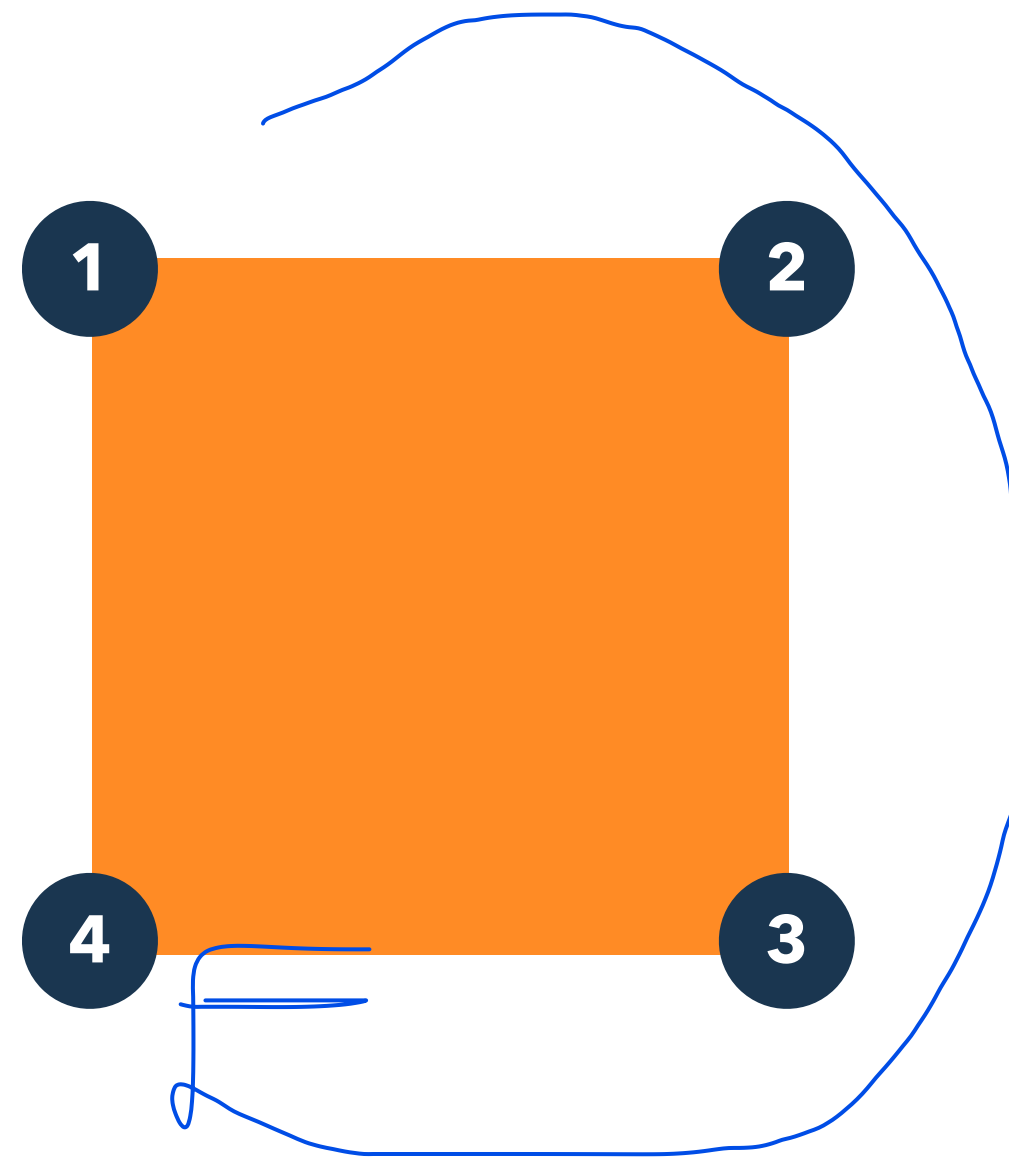


`border-radius: 10px;`



`border-radius: 0 10px 0 0;`





`border-radius: 0 10px 0 0;`

요소의 크기 계산 기준을 지정

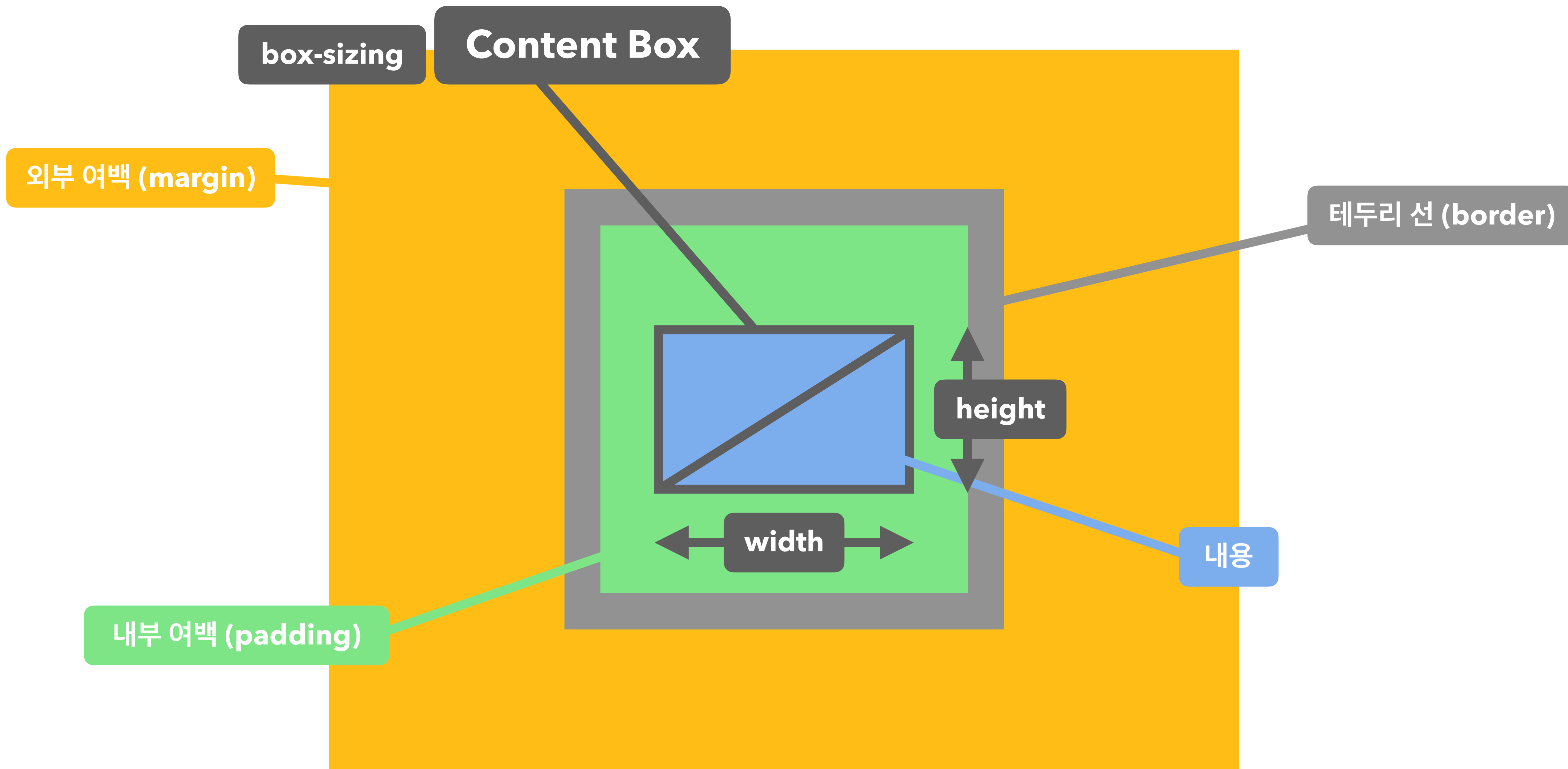
box-sizing

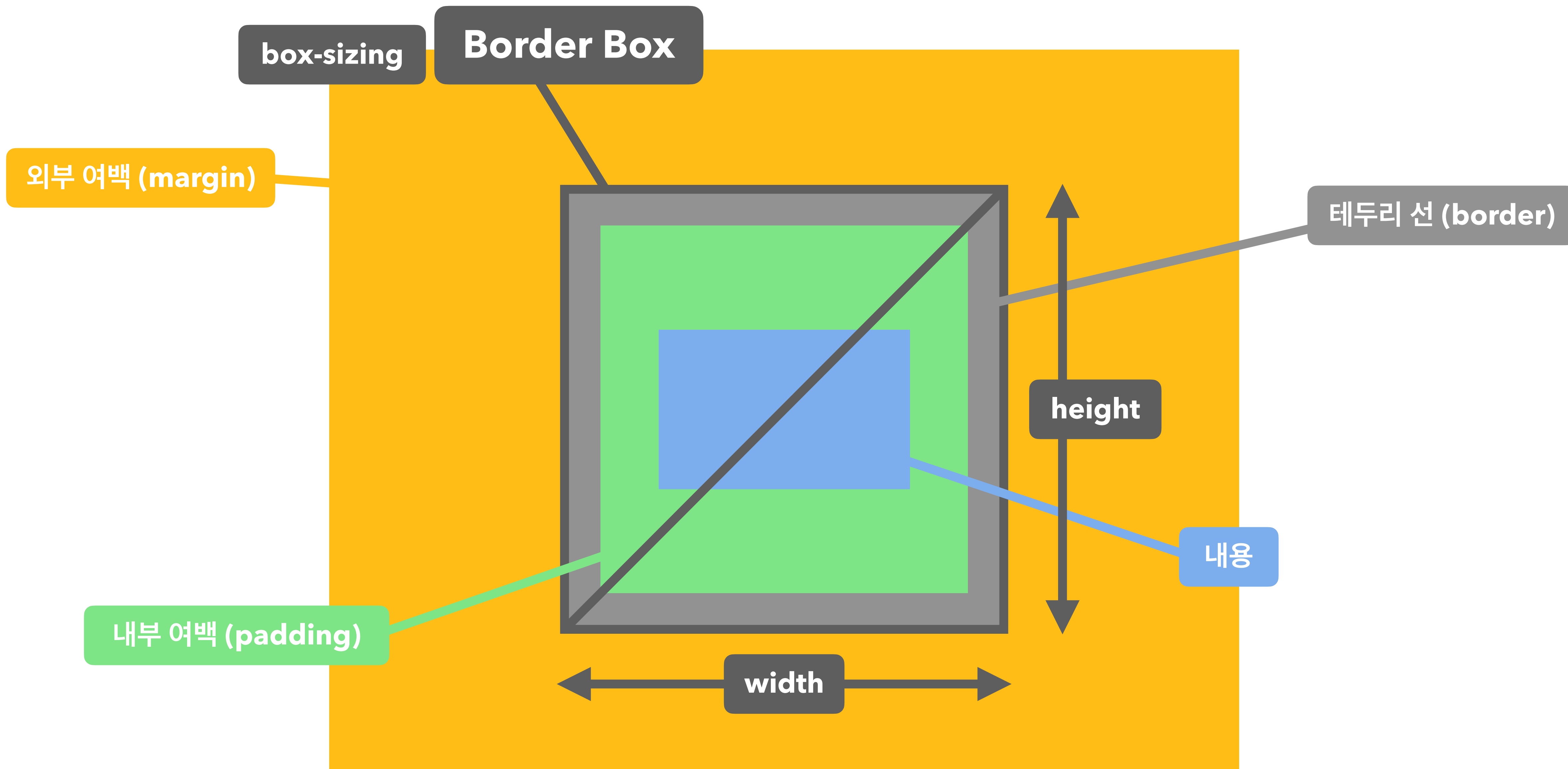
content-box

요소의 내용(content)으로 크기 계산

border-box

요소의 내용 + padding + border로 크기 계산





요소의 크기 이상으로 내용이 넘쳤을 때, 보여짐을 제어하는 단축 속성

overflow

visible

넘친 내용을 그대로 보여줌

hidden

넘친 내용을 잘라냄

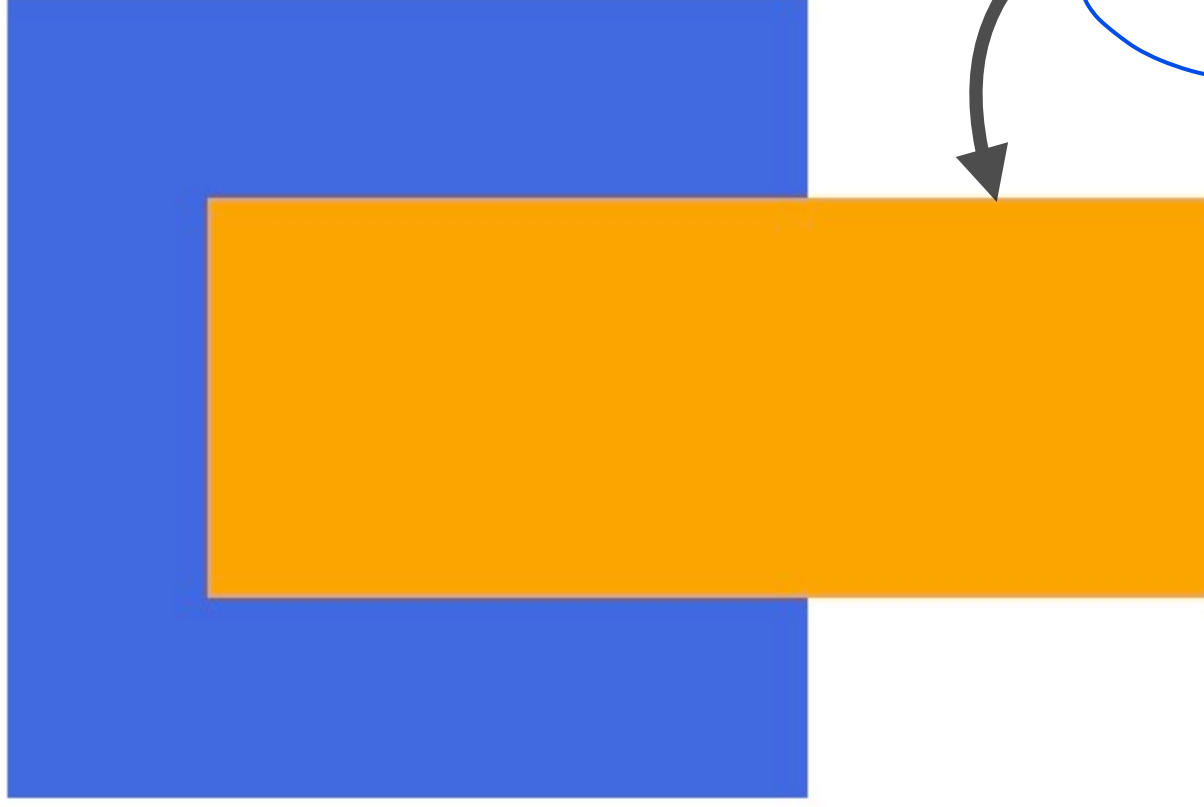
scroll

넘친 내용을 잘라냄, 스크롤바 생성

auto

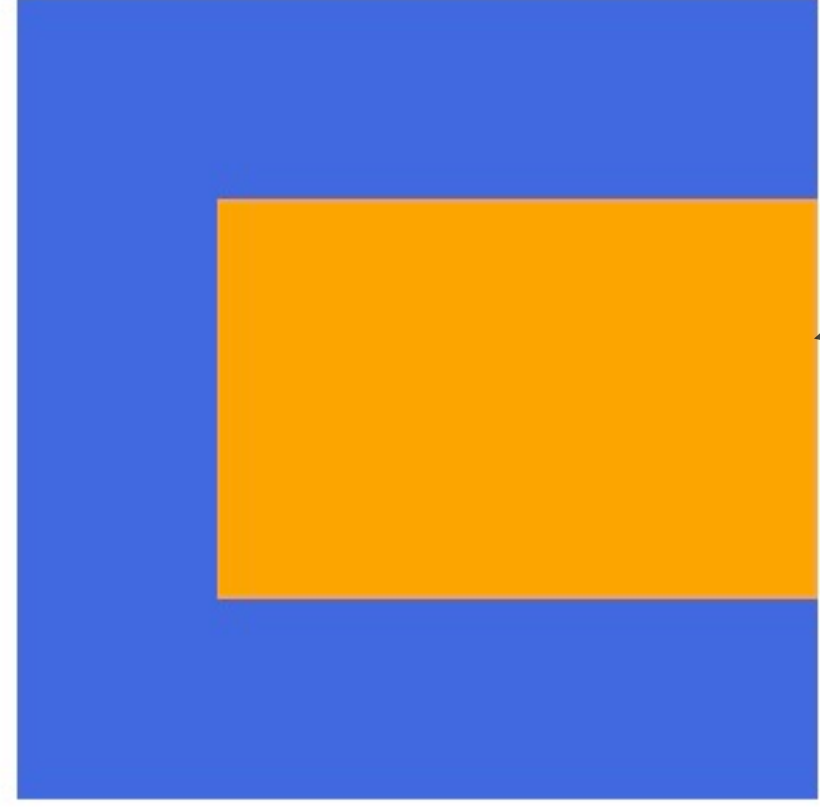
넘친 내용이 있는 경우에만 잘라내고 스크롤바 생성

visible



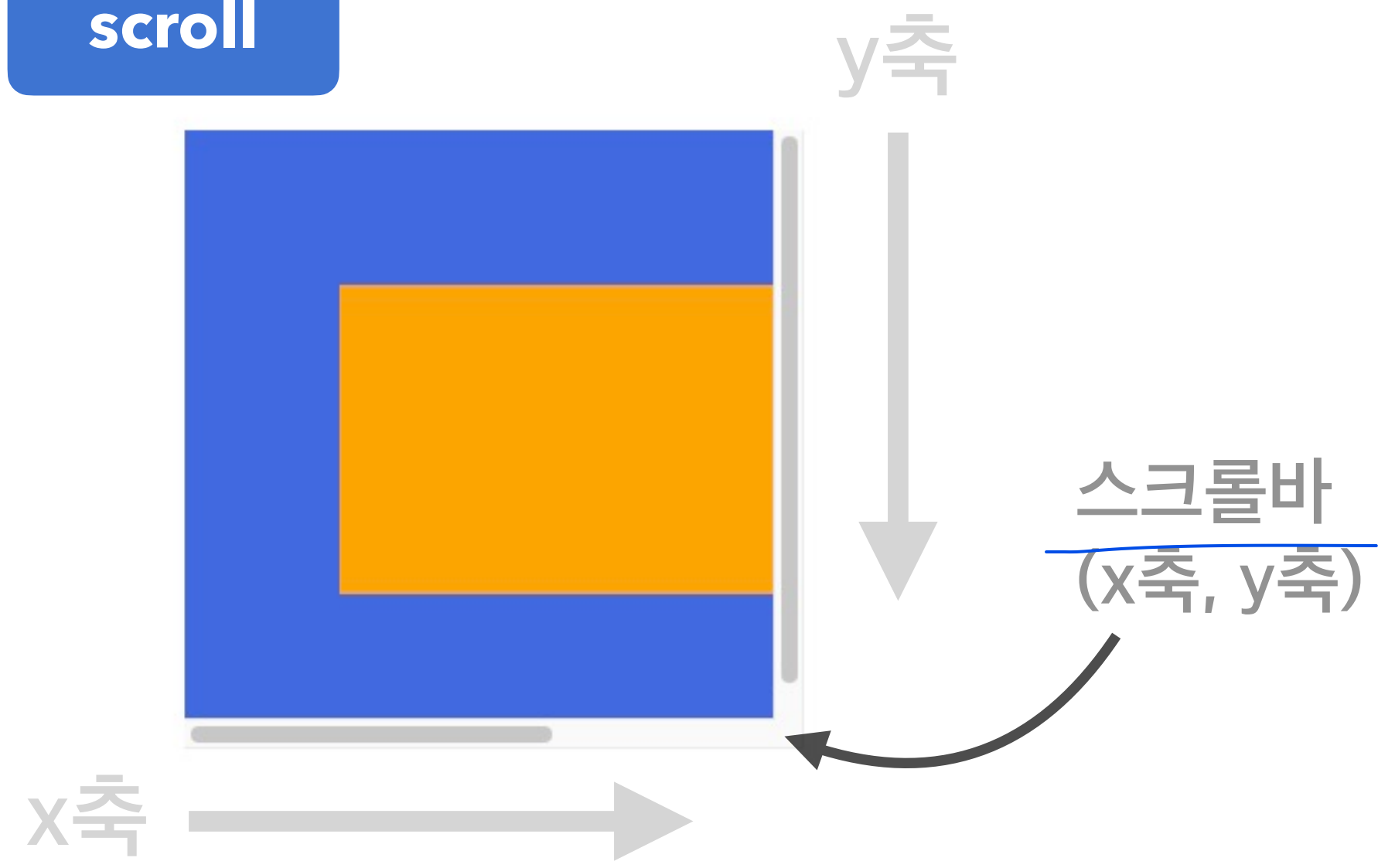
내용 넘침

hidden

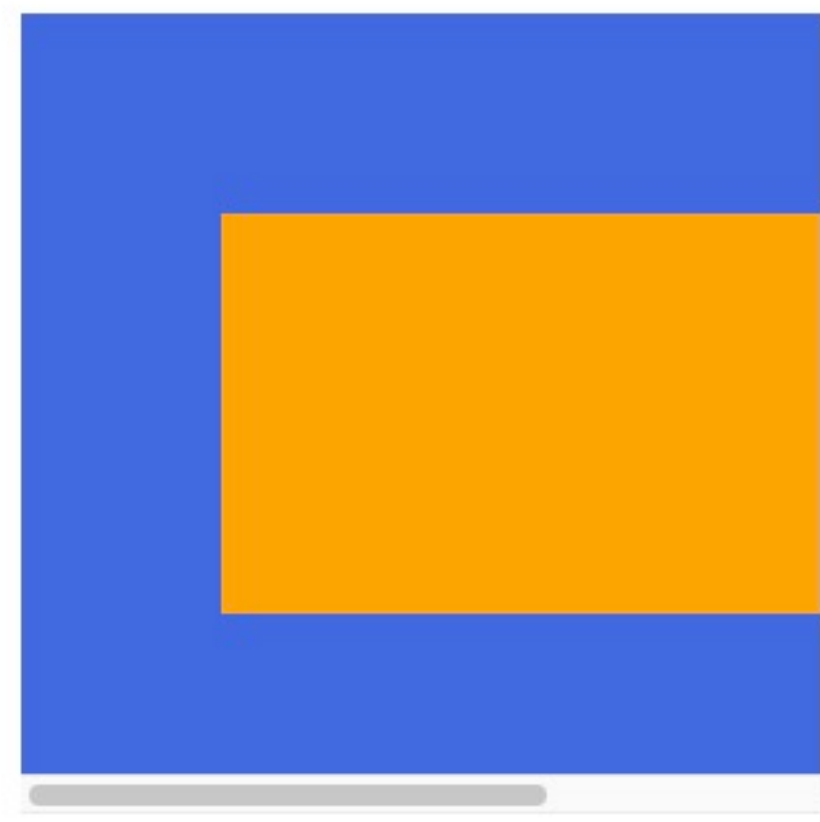


잘라냄

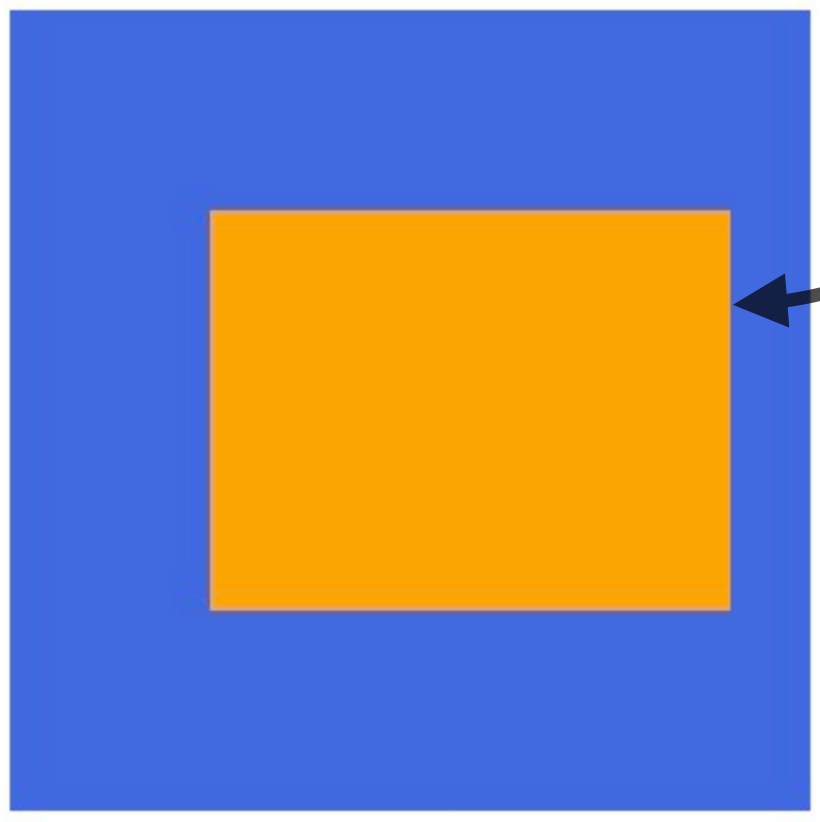
scroll



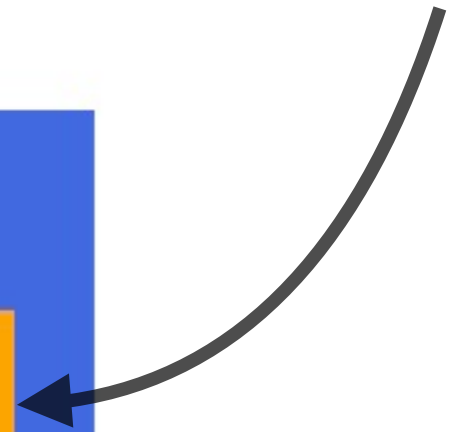
auto



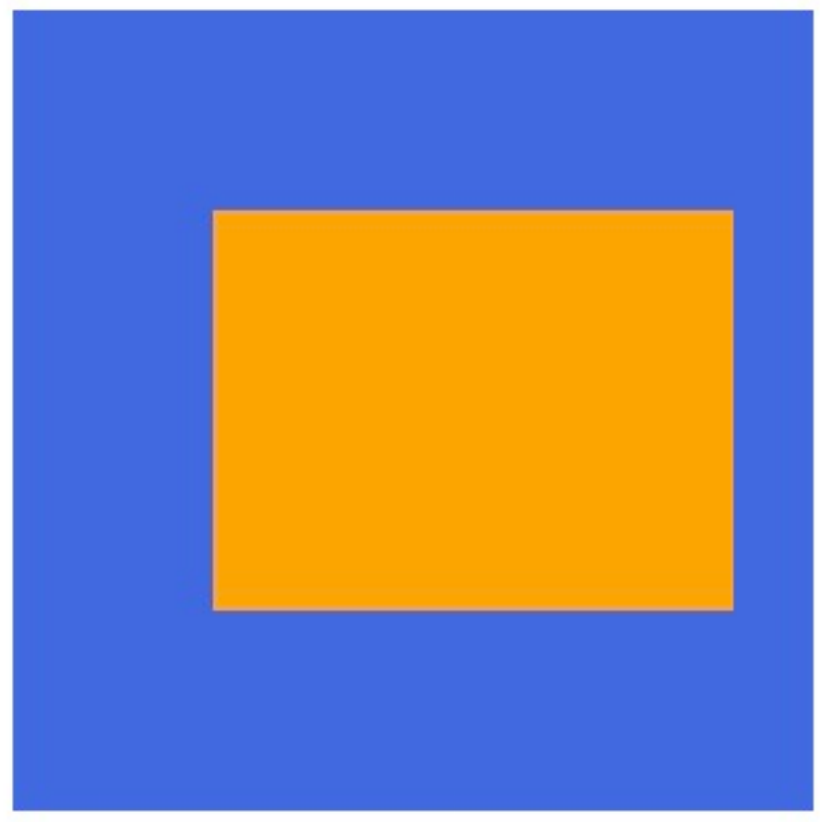
visible



내용 넘치지 않음!



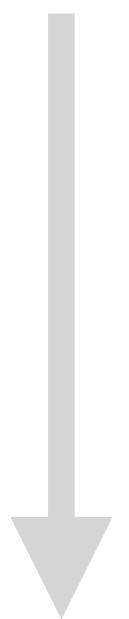
hidden



scroll



y축



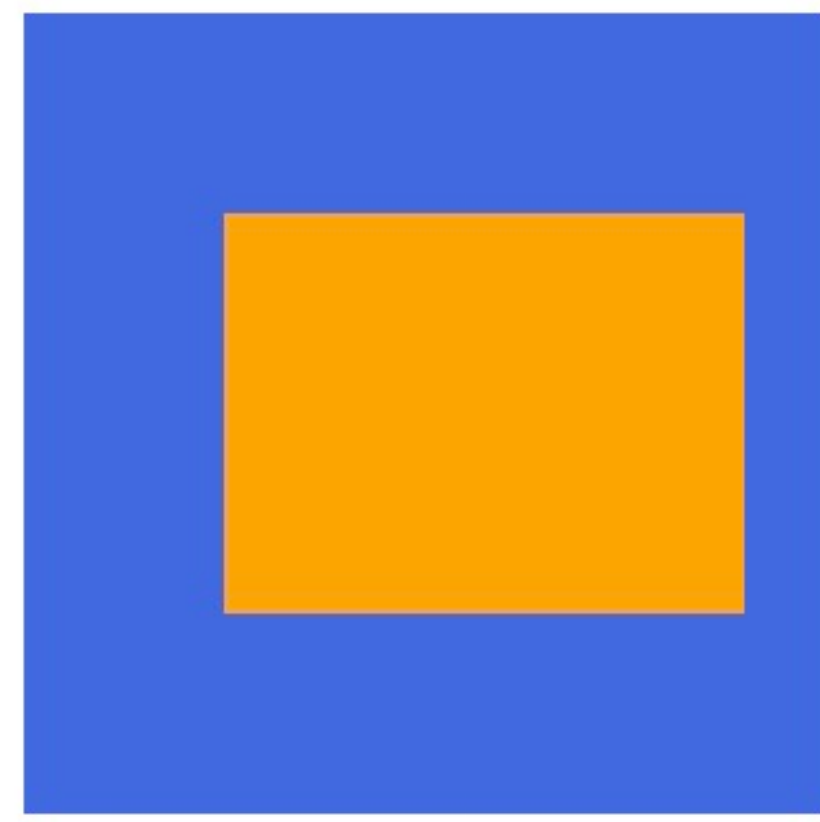
스크롤바
(x축, y축)



x축



auto



요소의 크기 이상으로 내용이 넘쳤을 때, 보여짐을 제어하는 개별 속성들

overflow-x
overflow-y

요소의 화면 출력(보여짐) 특성

display

각 요소에 이미 지정되어 있는 값

block

상자(레이아웃) 요소

inline

글자 요소

inline-block

글자 + 상자 요소

flex

플렉스 박스 (1차원 레이아웃)

grid

그리드 (2차원 레이아웃)

none

보여짐 특성 없음, 화면에서 사라짐

기타

table, table-row, table-cell 등..

따로 지정해서 사용하는 값

요소 투명도

opacity

1

불투명

0~1

0부터 1 사이의 소수점 숫자

opacity: 0.07;



opacity: 0.4;



opacity: 0.7;



opacity: 1;



글꼴

글자의 기울기

font-style

normal

기울기 없음

italic

이텔릭체

oblique

기울어진 글자

글자의 두께(가중치)

font-weight

normal, 400

기본 두께

bold, 700

두껍게

bolder

상위(부모) 요소보다 더 두껍게

lighter

상위(부모) 요소보다 더 얇기

100 ~ 900

100단위의 숫자 9개,
normal과 bold 이외 두께

폰트를 300인 것과 800인 것을 가져온 다음
400으로 두께를 지정하면? -> 300이 적용
(더 가까운 쪽의 것이 적용됨)

글자의 크기

font-size

16px

기본 크기

단위

px, em, rem 등 단위로 지정

%

부모 요소의 폰트 크기에 대한 비율

smaller

상위(부모) 요소보다 작은 크기

larger

상위(부모) 요소보다 큰 크기

xx-small ~ xx-large

가장 작은 크기 ~ 가장 큰 크기까지,
7단계의 크기를 지정

한 줄의 높이, 행간과 유사

line-height

normal

브라우저의 기본 정의를 사용

숫자

요소의 글꼴 크기의 배수로 지정

단위

px, em, rem 등의 단위로 지정

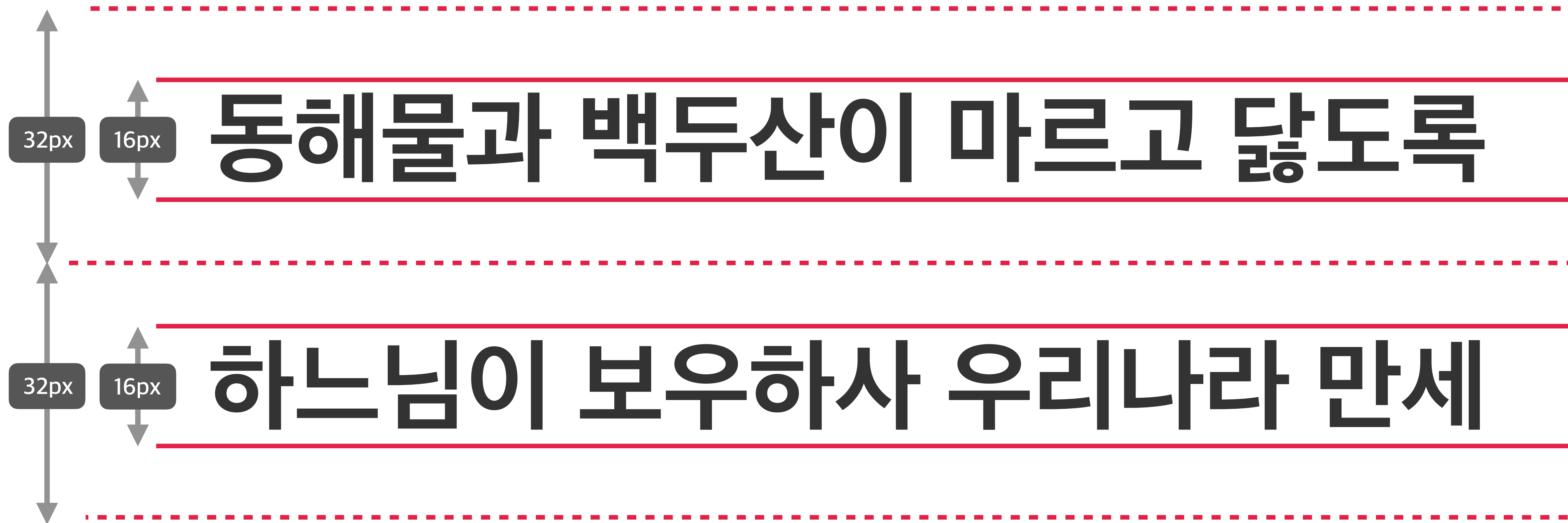
%

요소의 글꼴 크기의 비율로 지정

동해물과 백두산이 마르고 닳도록

하느님이 보우하사 우리나라 만세

```
font-size: 16px;  
line-height: 32px;  
/* line-height: 2; */  
/* line-height: 200%; */
```



```
font-size: 16px;  
line-height: 32px;  
/* line-height: 2; */  
/* line-height: 200%; */
```

2배 차이

후보들이 다 없다면,
마지막에는 글꼴 계열을 보고
로컬에 설치된 폰트를 불러 오

필수로 작성!

1순위 글꼴(서체) 지정 2순위

font-family: 글꼴1, "글꼴2", ... **글꼴계열;**

띄어쓰기 등 특수문자가 포함된
글꼴 이름은 큰 따옴표로 묶어야 합니다~

글꼴 후보들 작성 가능
(무제한으로 작성 가능)

네트워크나 서버 등의 문제로
글꼴을 불러오지 못할 경우에 대비

Hello World!

serif

바탕체 계열

Hello World!

sans-serif

고딕체 계열

He l l o W o r l d !

monospace

고정너비(가로폭이 동등) 글꼴 계열

Hello World!

cursive

필기체 계열

EMPIRE

fantasy

장식 글꼴 계열

문자

글자의 색상

color

rgb(0,0,0)

검정색

색상

기타 지정 가능한 색상

(수평)
문자의 정렬 방식

text-align

-  **left** 왼쪽 정렬
-  **right** 오른쪽 정렬
-  **center** 가운데 정렬
-  **justify** 양쪽 정렬

문자의 장식(선)

text-decoration

화면에 출력!

동해물과 백두산이 마르고 닳도록

none 장식 없음

동해물과 백두산이 마르고 닳도록

underline 밑줄

동해물과 백두산이 마르고 닳도록

overline 윗줄

~~동해물과 백두산이 마르고 닳도록~~

line-through 중앙 선

동해물과 백두산이 마르고 닳도록 하느
님이 보우하사 우리나라 만세 무궁화 삼천리 화
려 강산 대한 사람 대한으로 길이 보전하세

들여쓰기(50px)

문자 첫 줄의 들여쓰기

음수를 사용할 수 있어요!
반대는 내어쓰기(outdent)입니다.

text-indent

0

들여쓰기 없음

단위

px, em, rem 등 단위로 지정

%

요소의 가로 너비에 대한 비율

배경

요소의 배경 색상

background-color

transparent 투명함

색상 지정 가능한 색상

background-color: orange;



요소의 배경 이미지 삽입

background-image

none

이미지 없음

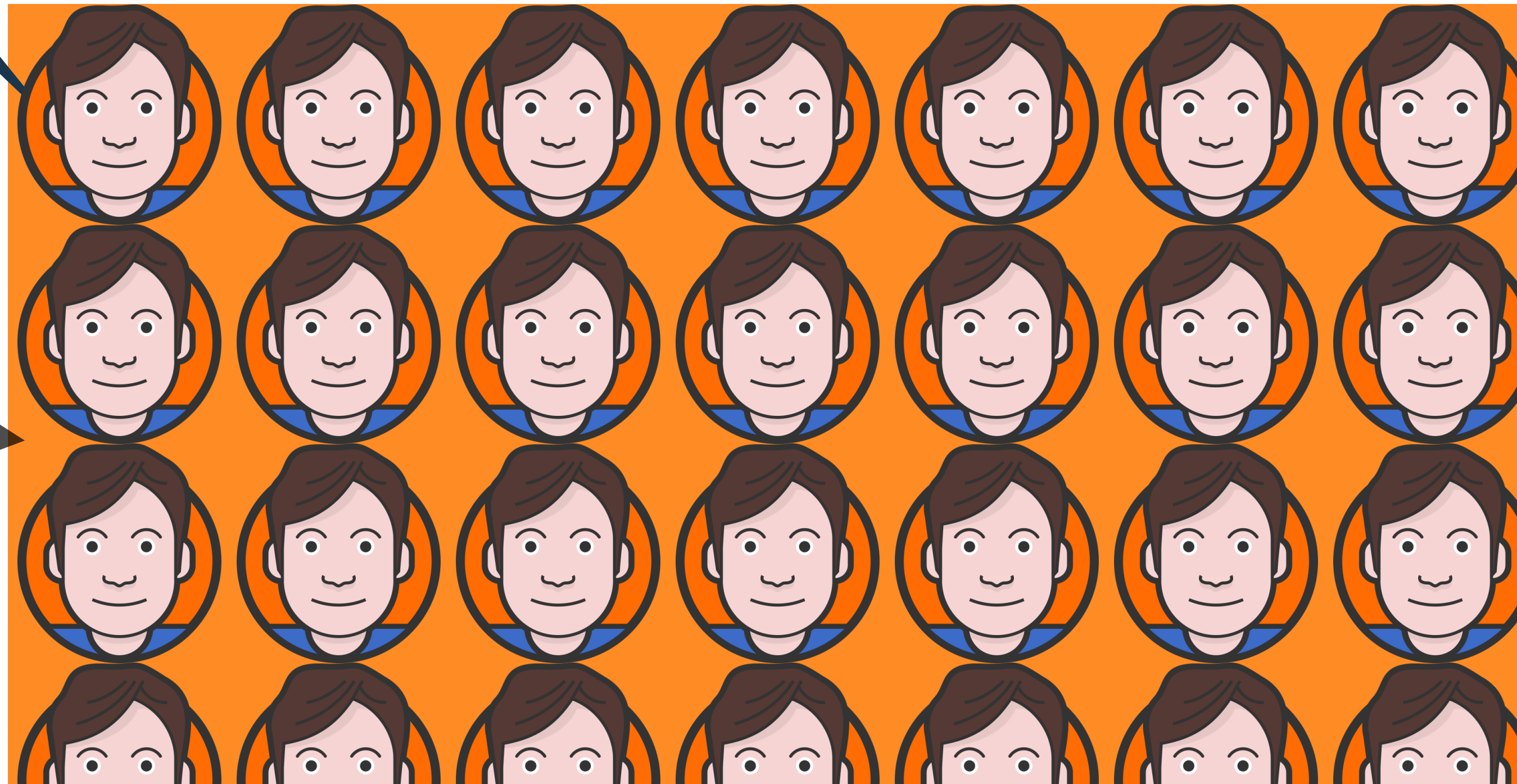
url("경로")

이미지 경로


```
background-color: orange;  
background-image: url("../images/heropy.png");
```



배경 색상은
이미지 뒤에 나와요!



요소의 배경 이미지 반복

background-repeat

repeat 이미지를 수직, 수평 반복

repeat-x 이미지를 수평 반복

repeat-y 이미지를 수직 반복

no-repeat 반복 없음

```
background-color: orange;  
background-image: url("../images/heropy.png");  
background-repeat: repeat-x;
```




```
background-color: orange;  
background-image: url("../images/heropy.png");  
background-repeat: repeat-y;
```

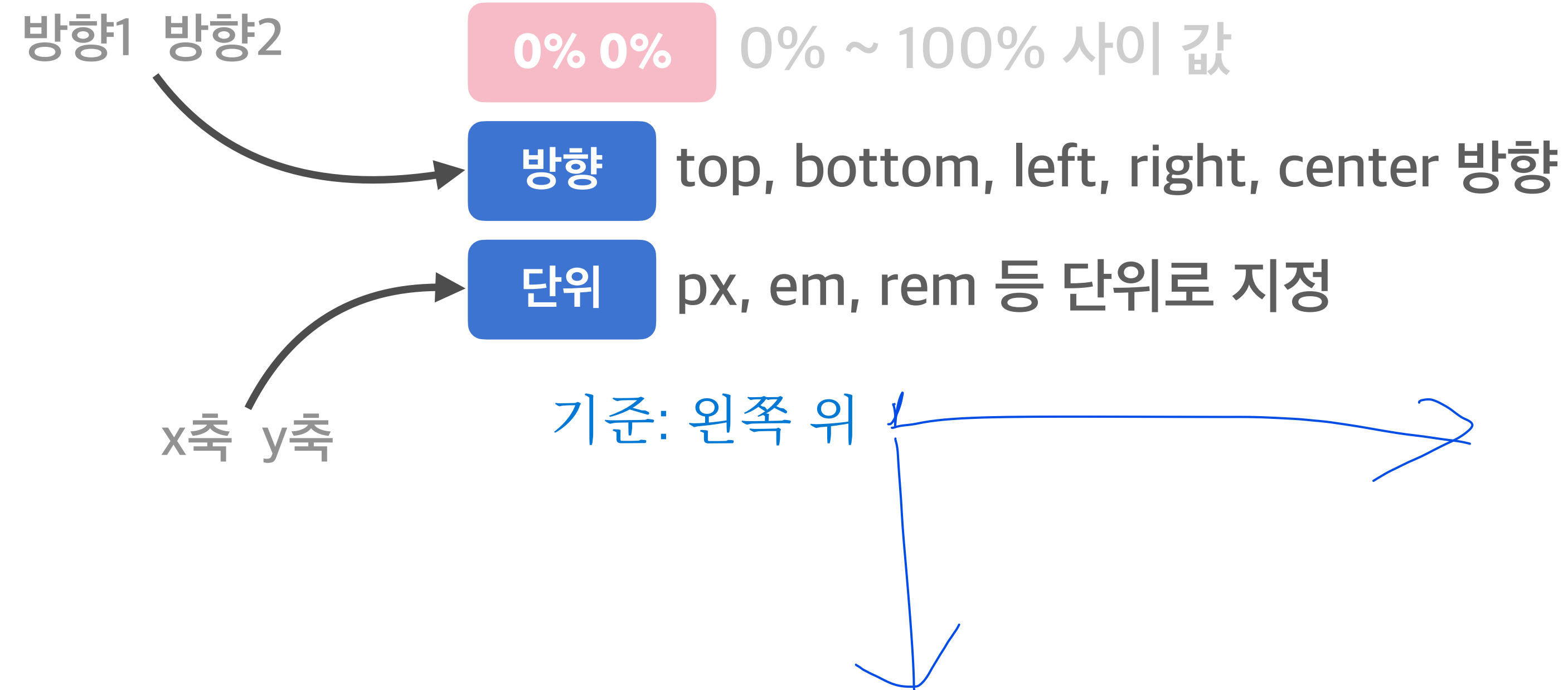


```
background-color: orange;  
background-image: url("../images/heropy.png");  
background-repeat: no-repeat;
```



요소의 배경 이미지 위치

background-position





```
background-color: orange;  
background-image: url("../images/heropy.png");  
background-repeat: no-repeat;  
background-position: top right;
```



```
background-color: orange;  
background-image: url("../images/heropy.png");  
background-repeat: no-repeat;  
background-position: center;
```




```
background-color: orange;  
background-image: url("../images/heropy.png");  
background-repeat: no-repeat;  
background-position: 100px 30px;
```

요소의 배경 이미지 크기

background-size

auto

이미지의 실제 크기

단위

px, em, rem 등 단위로 지정

cover

비율을 유지, 요소의 더 넓은 너비에 맞춤

contain

비율을 유지, 요소의 더 짧은 너비에 맞춤

```
background-color: orange;  
background-image: url("../images/heropy.png");  
background-repeat: no-repeat;  
background-size: cover;
```



```
background-color: orange;  
background-image: url("../images/heropy.png");  
background-repeat: no-repeat;  
background-size: contain;
```



요소의 배경 이미지 스크롤 특성

background-attachment

scroll

이미지가 요소를 따라서 같이 스크롤

fixed

이미지가 뷰포트에 고정, 스크롤 X

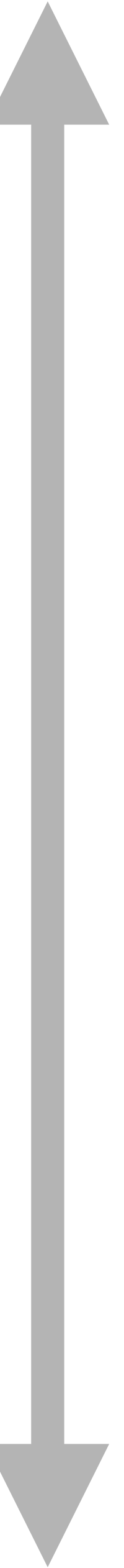
local

요소 내 스크롤 시 이미지가 같이 스크롤

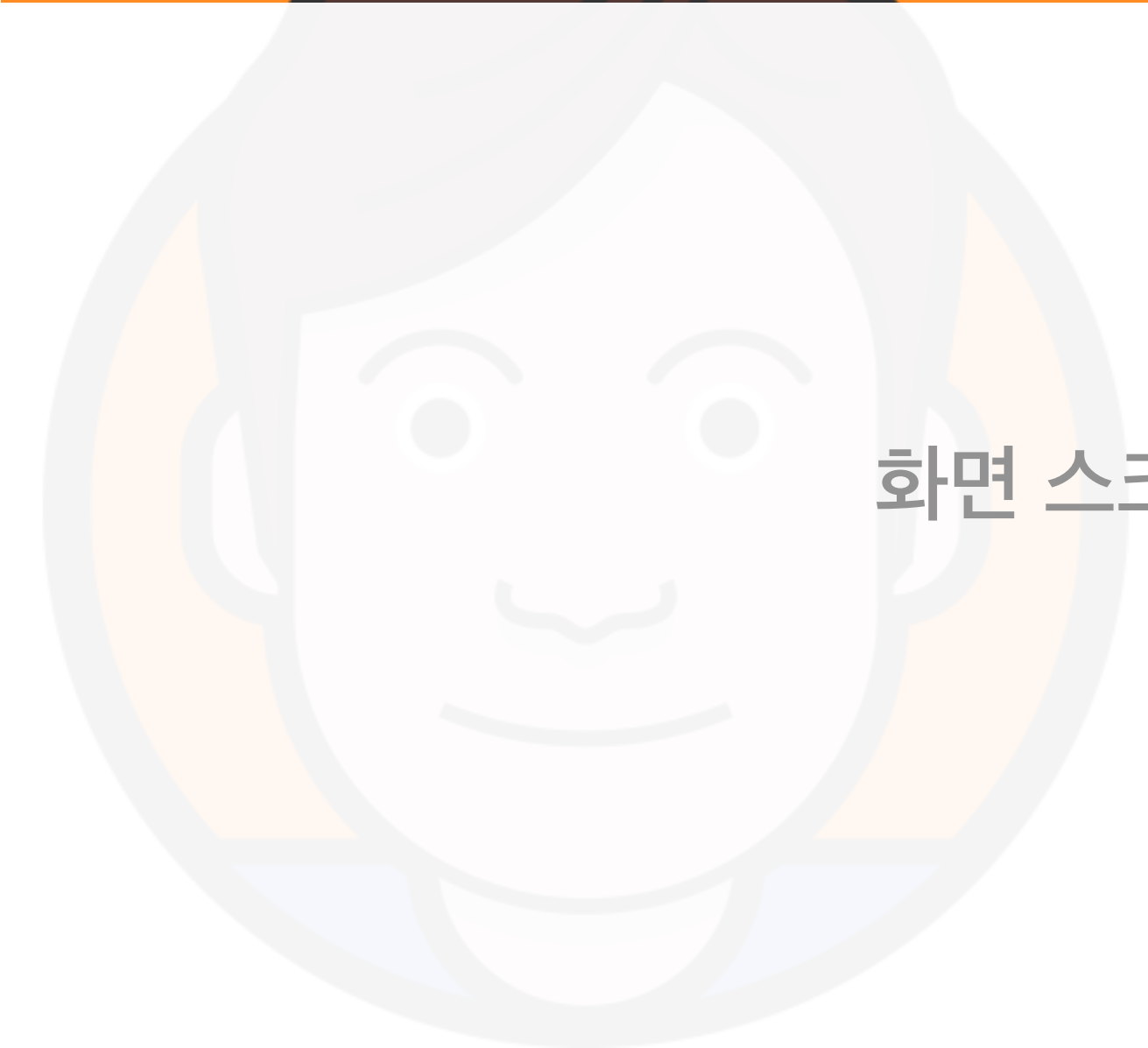
```
background-color: orange;  
background-image: url("../images/heropy.png");  
background-repeat: no-repeat;  
background-size: cover;  
background-attachment: scroll;
```



화면 스크롤



```
background-color: orange;  
background-image: url("../images/heropy.png");  
background-repeat: no-repeat;  
background-size: cover;  
background-attachment: fixed;
```



화면 스크롤



배치

position과 같이 사용하는 CSS 속성들!
모두 음수를 사용할 수 있어요!

top
bottom
left
right
z-index

요소의 위치 지정 기준

position

- static** 기준 없음
- relative** 요소 자신을 기준
- absolute** 위치 상 부모 요소를 기준
- fixed** 뷰포트(브라우저)를 기준
- sticky** 스크롤 영역 기준

위치 상 부모 요소를
꼭 확인해야 해요!

부모 중에 position을 갖고있는 요소를 찾아서,
그걸 기준으로 배치 됨
만약 없으면? 더 상위로 올라가서 position이 있
는 요소를 찾음 (position이 있는 요소를 만날 때
까지)

요소의 각 방향별 거리 지정

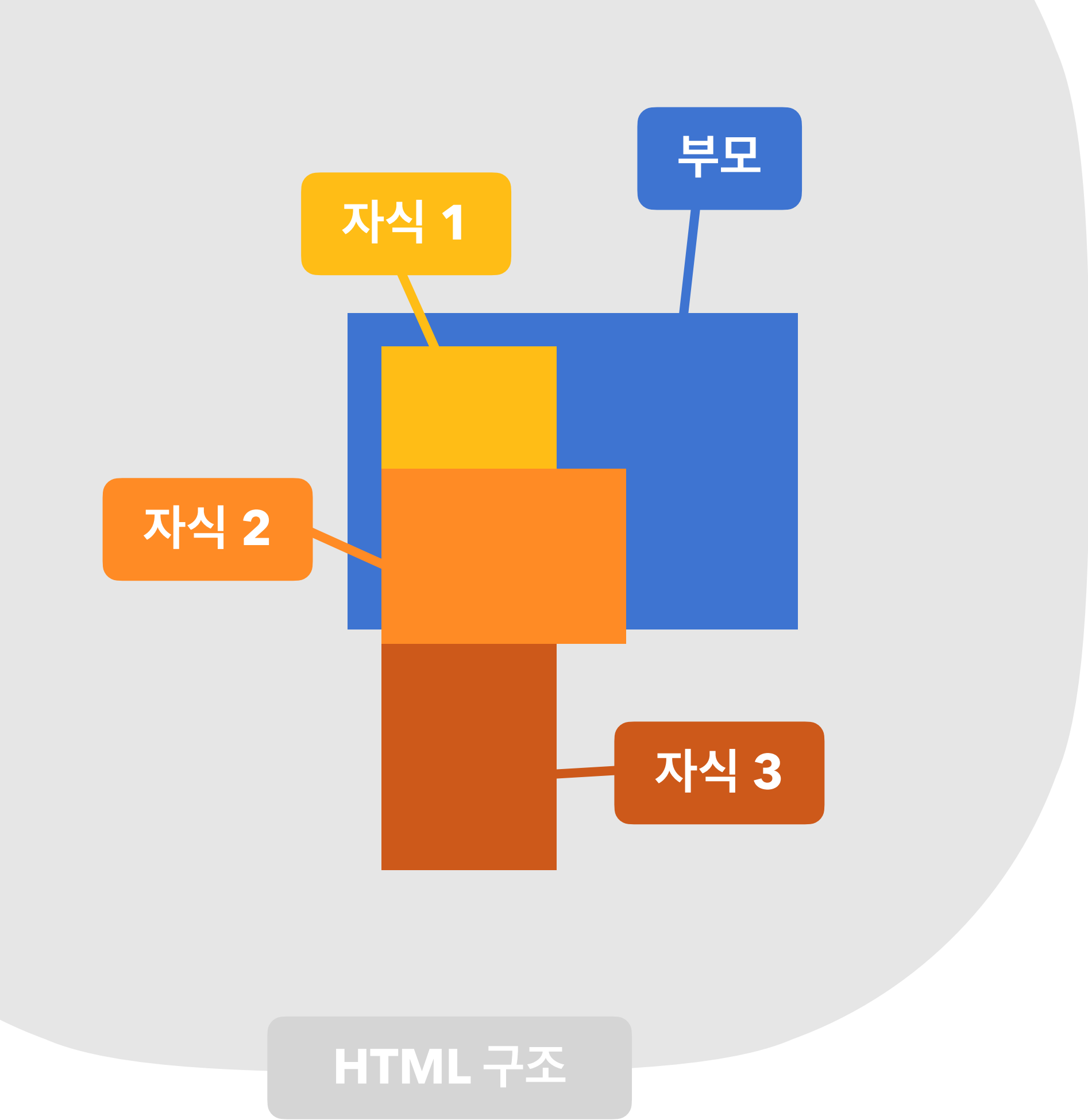
top, bottom, left, right

auto

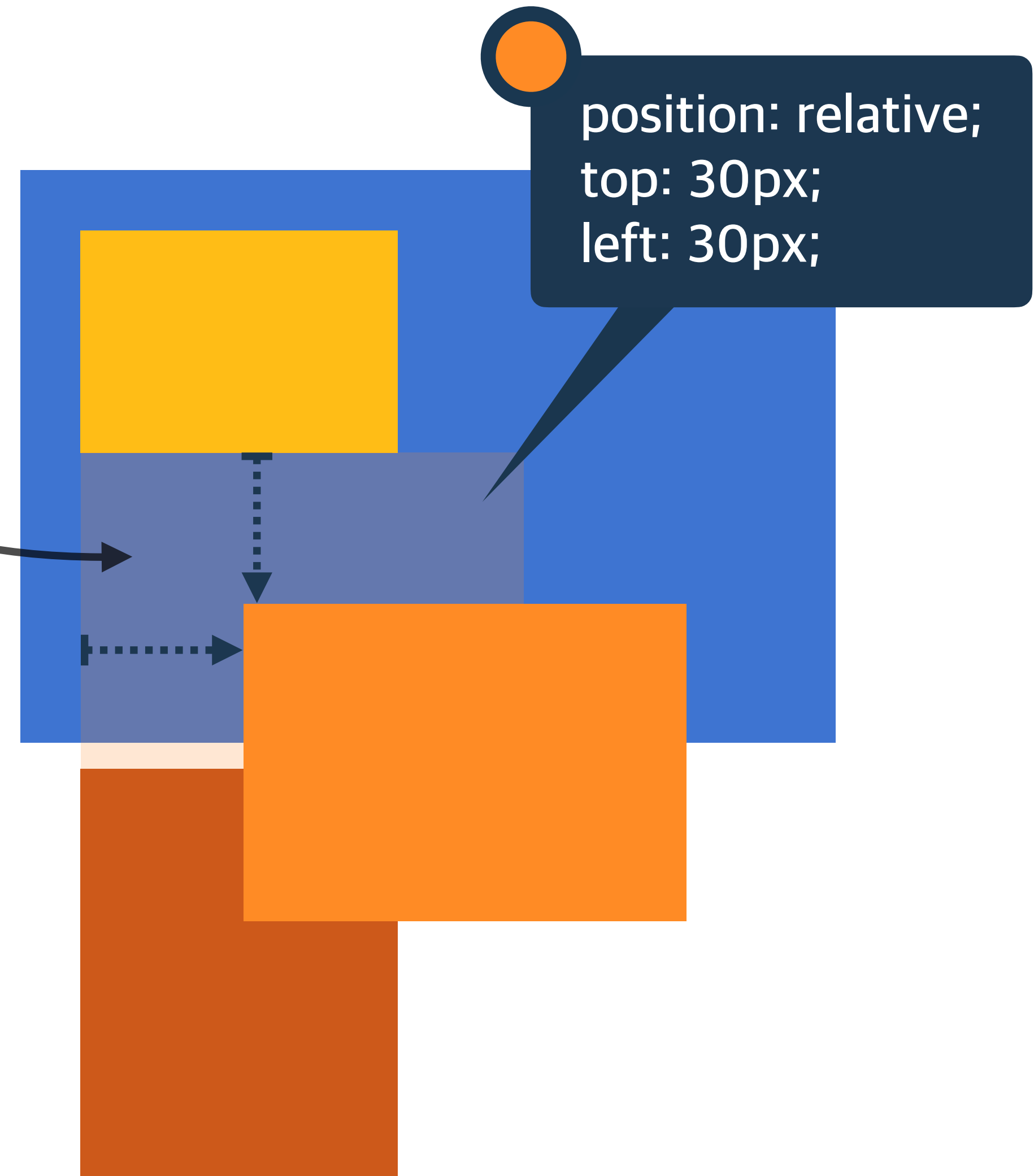
브라우저가 계산

단위

px, em, rem 등 단위로 지정



배치 전 자리는
비어 있어요!

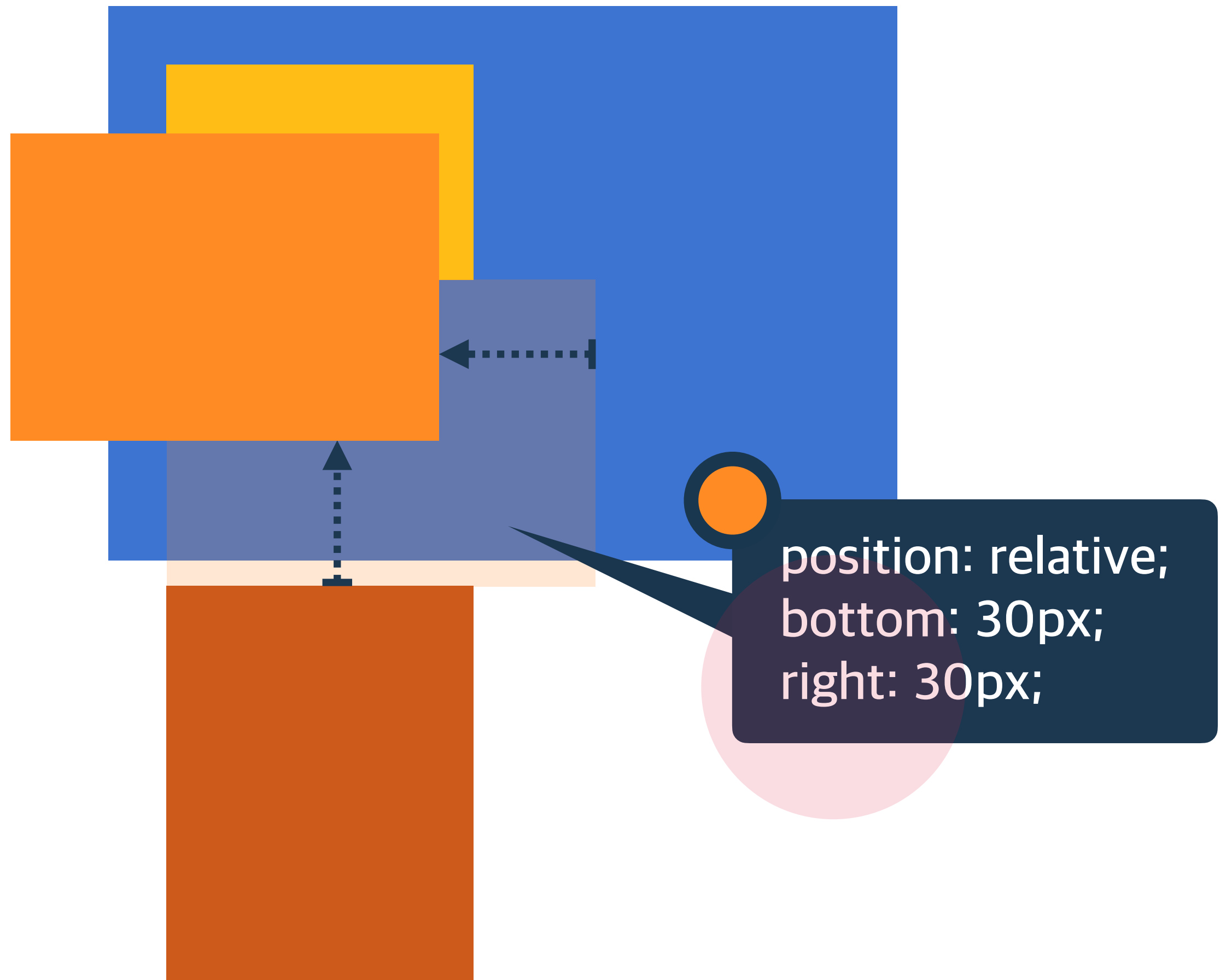
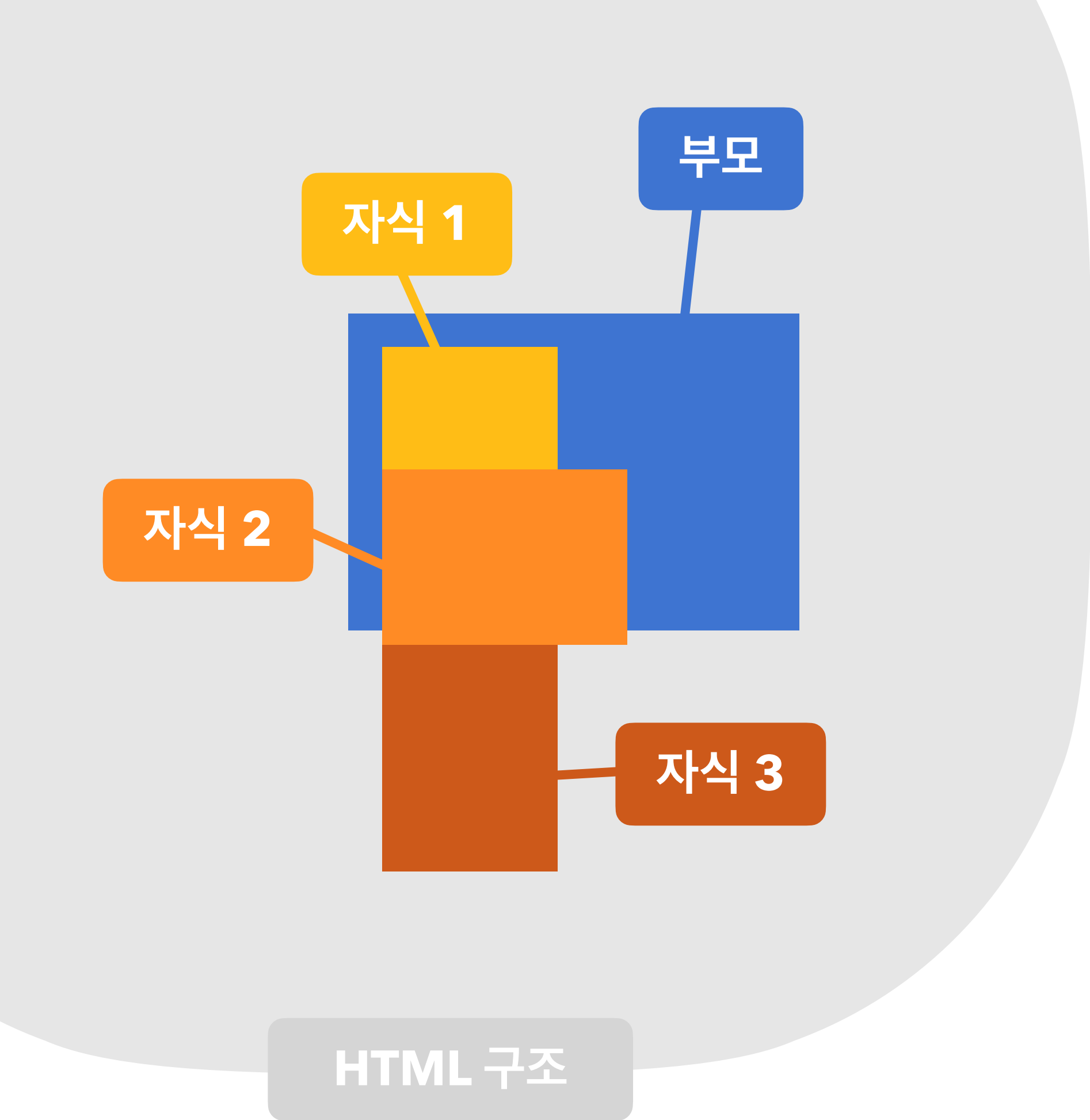


relative

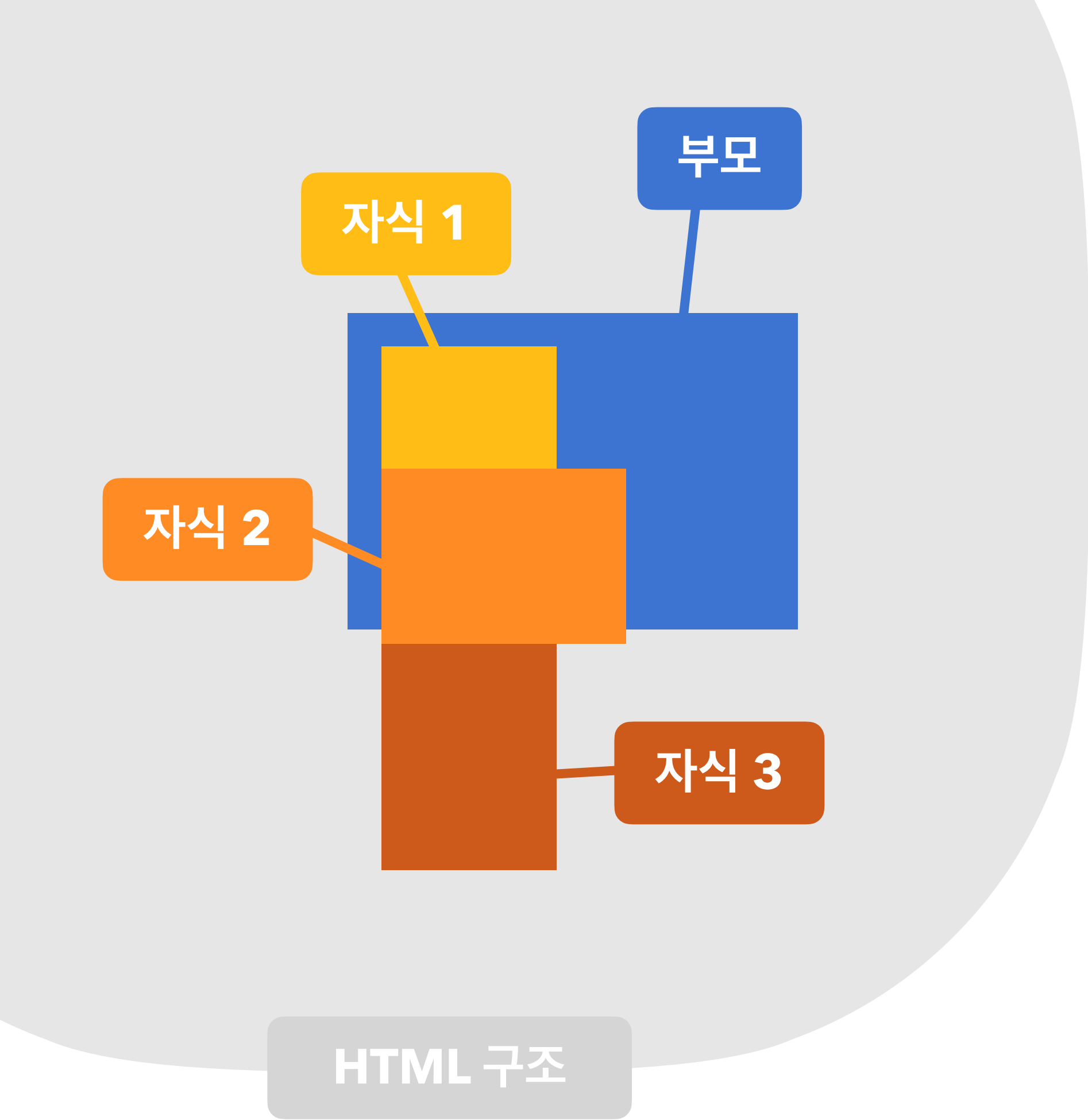
요소 자신을 기준으로 배치!

나의 원래 자리를 기준으로 함

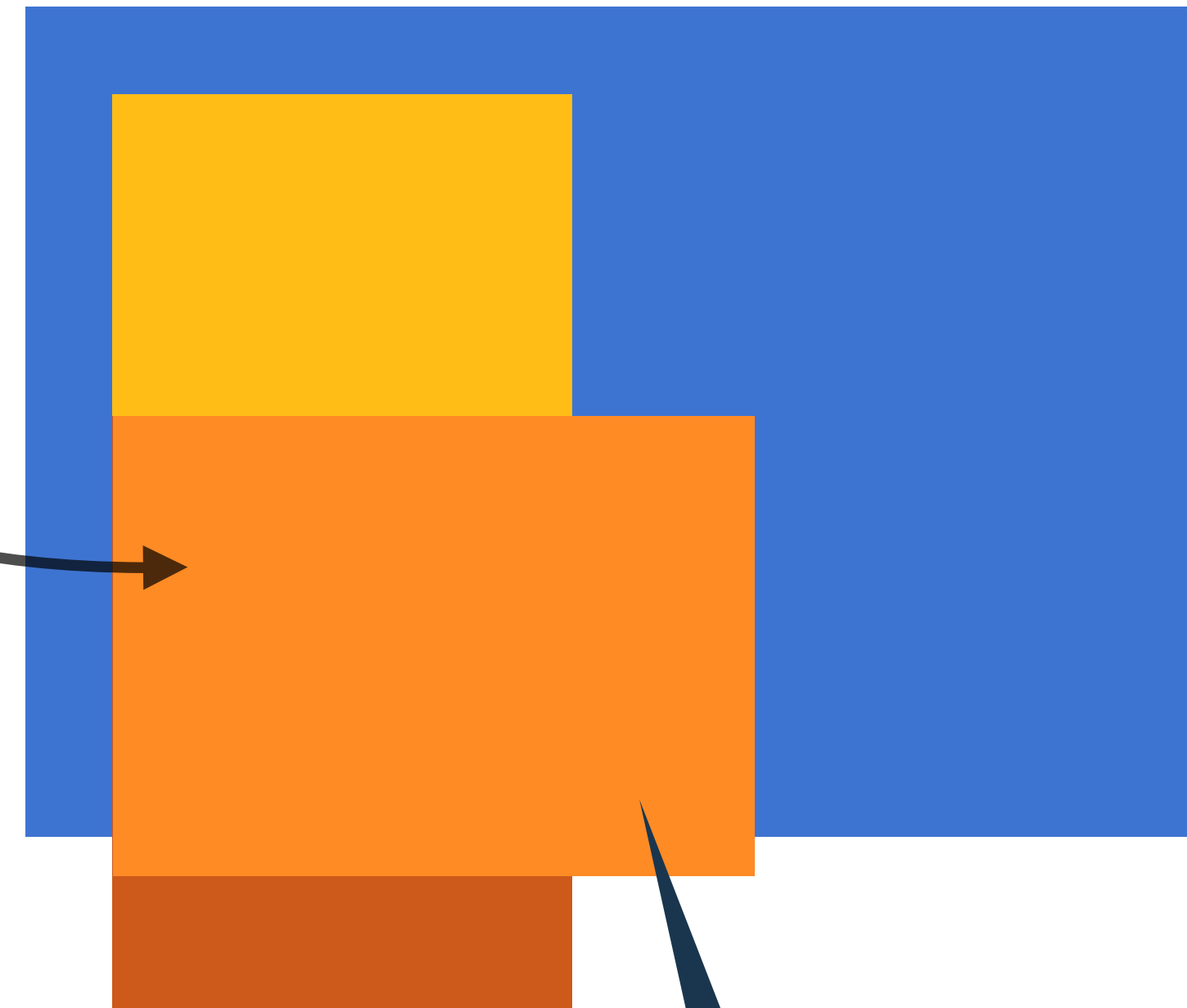
but 원래 위치의 영역은 계속 차지하고 있고, 눈에 보이는 위치만 바뀌는



relative 요소 자신을 기준으로 배치!

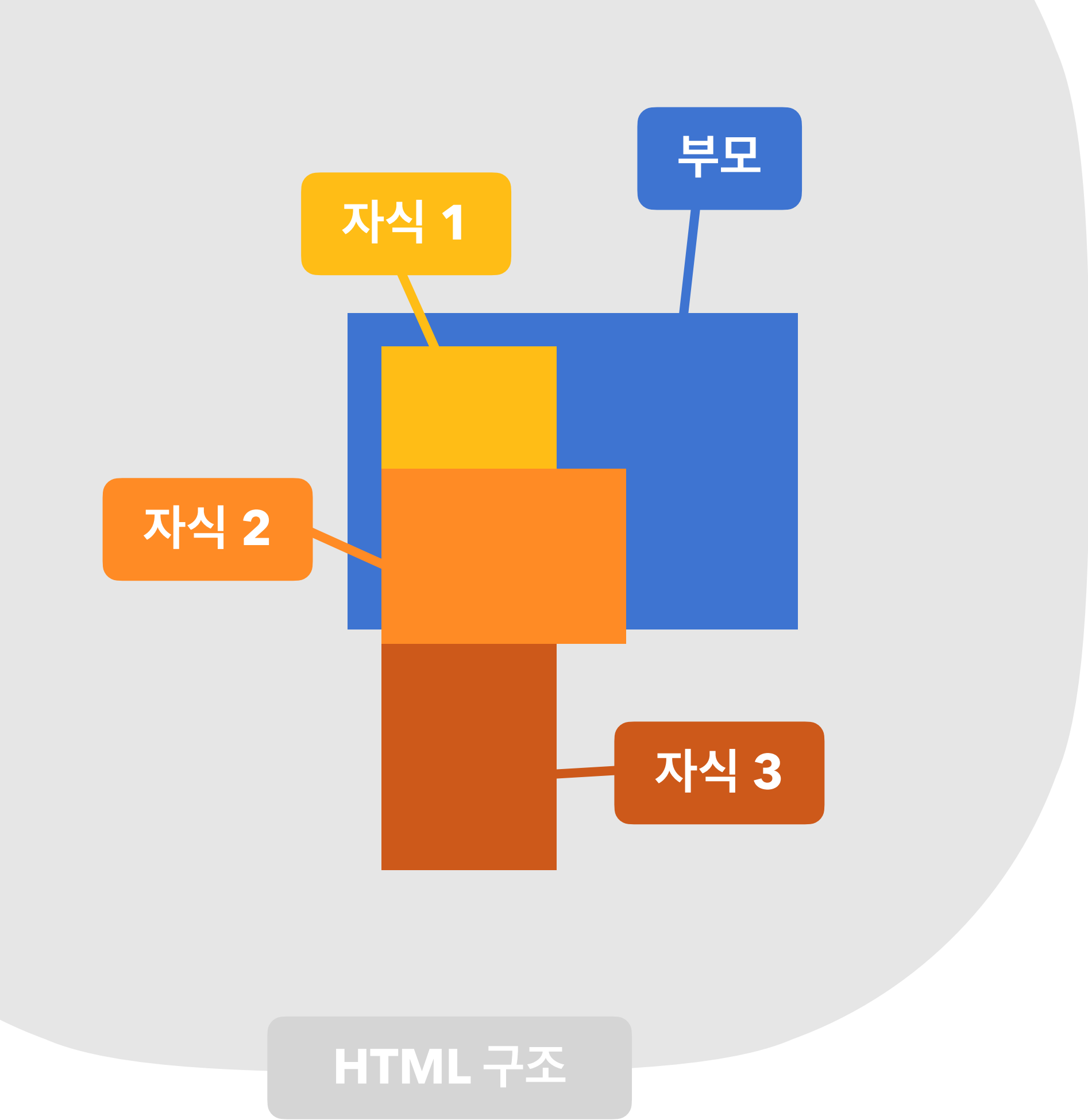


붕~ 뜨면서
요소가 겹쳐요

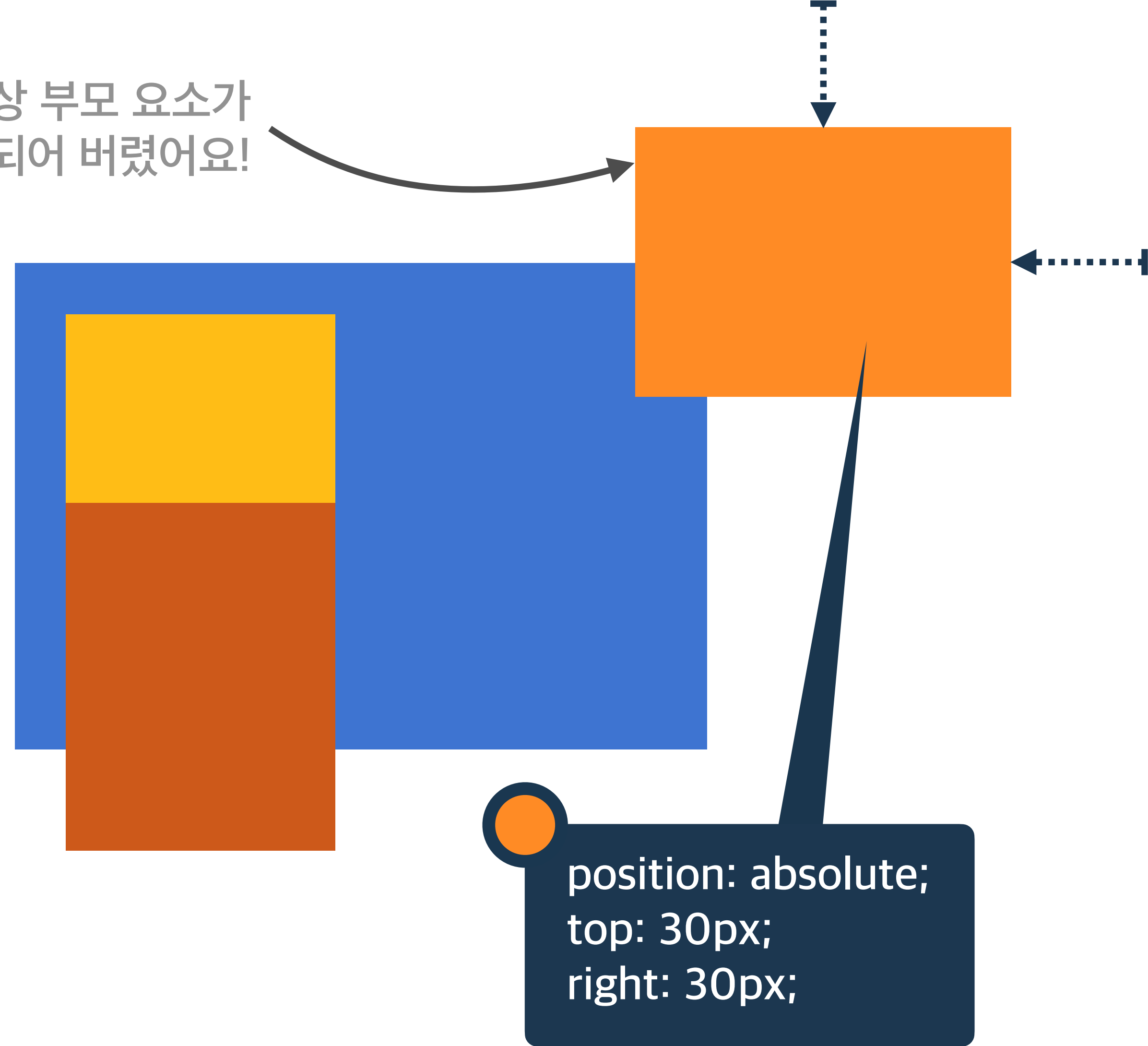


position: absolute;

absolute 위치 상 부모 요소를 기준으로 배치!

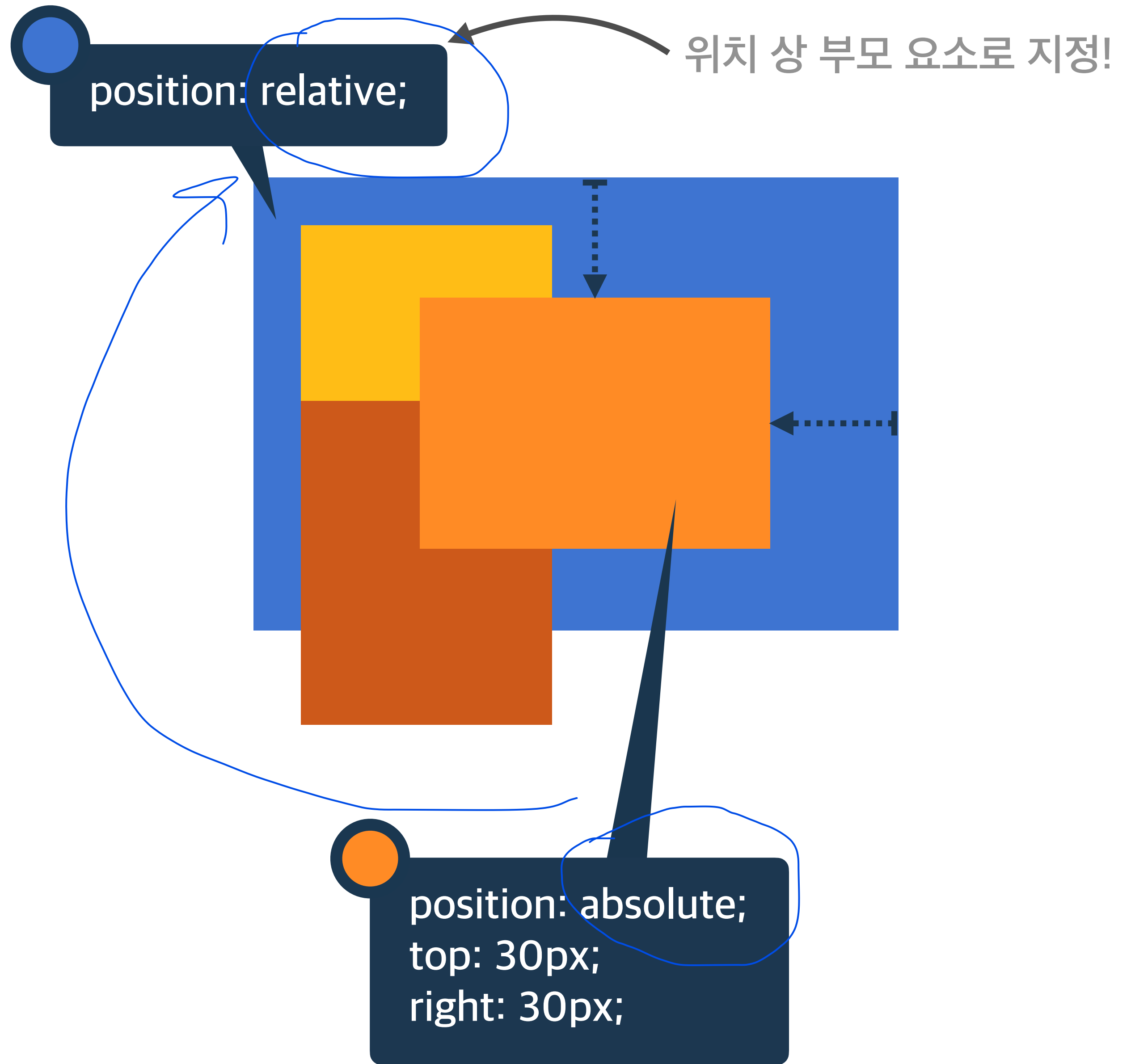
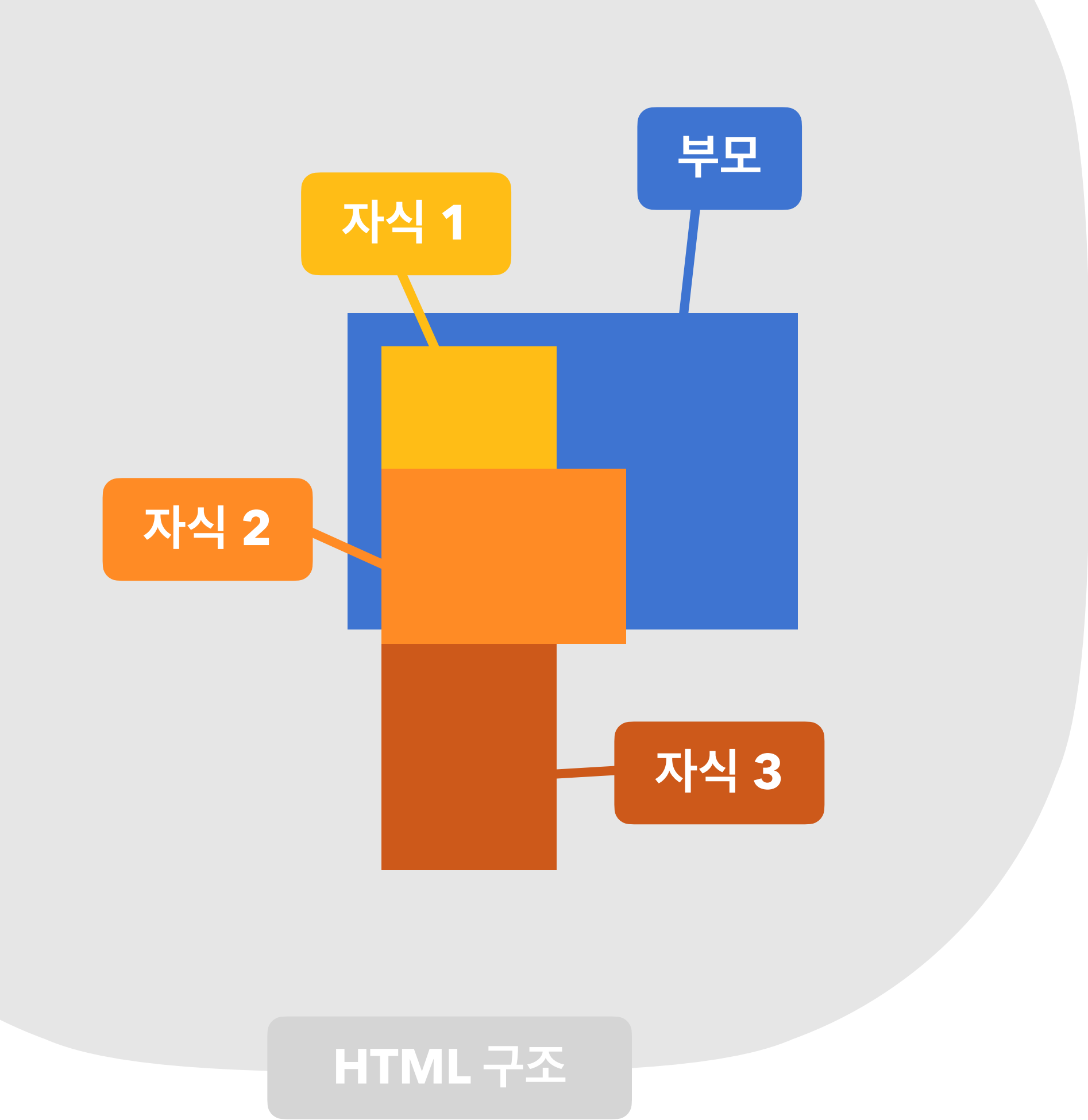


위치 상 부모 요소가
뷰포트가 되어 버렸어요!

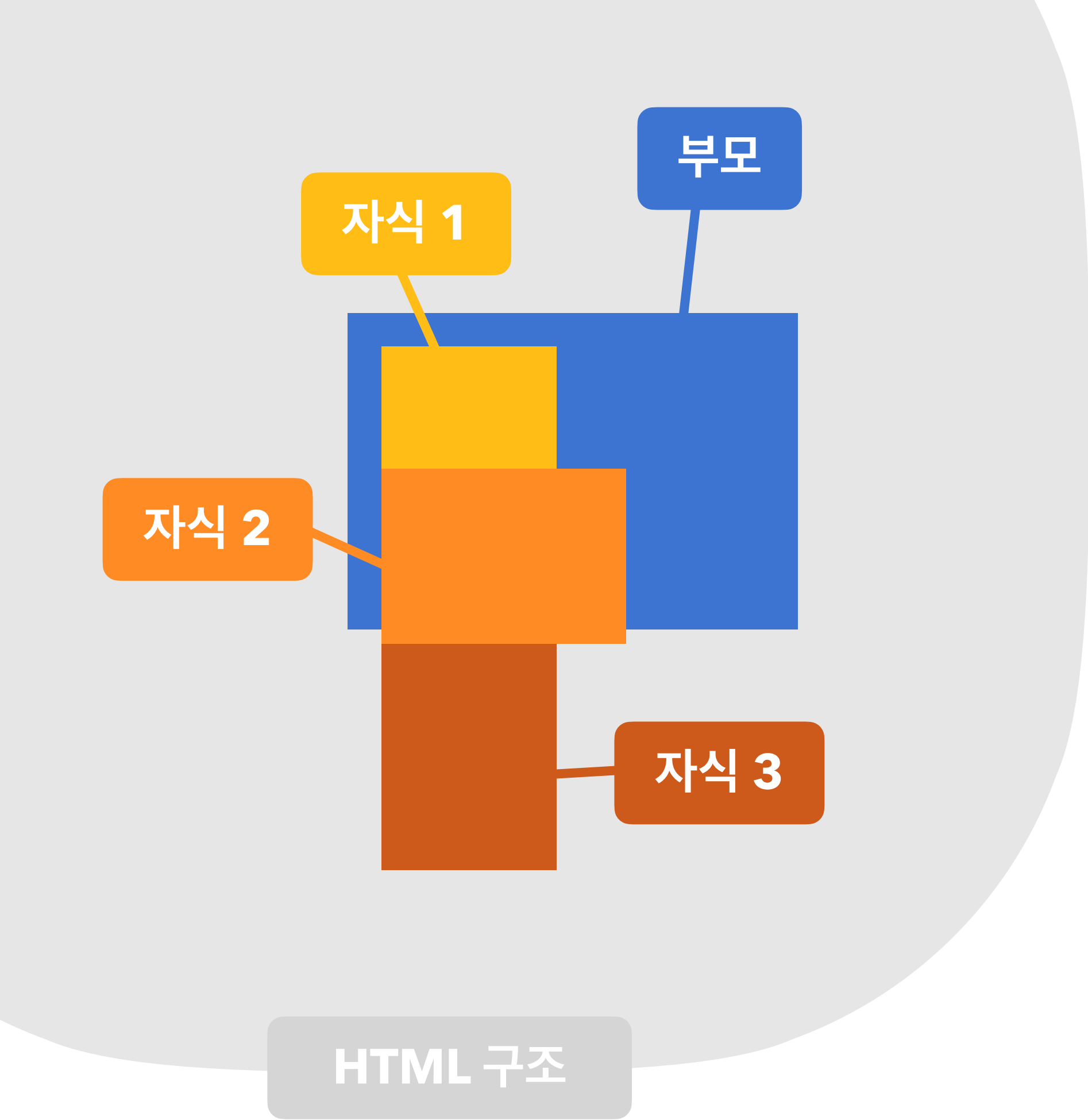


absolute

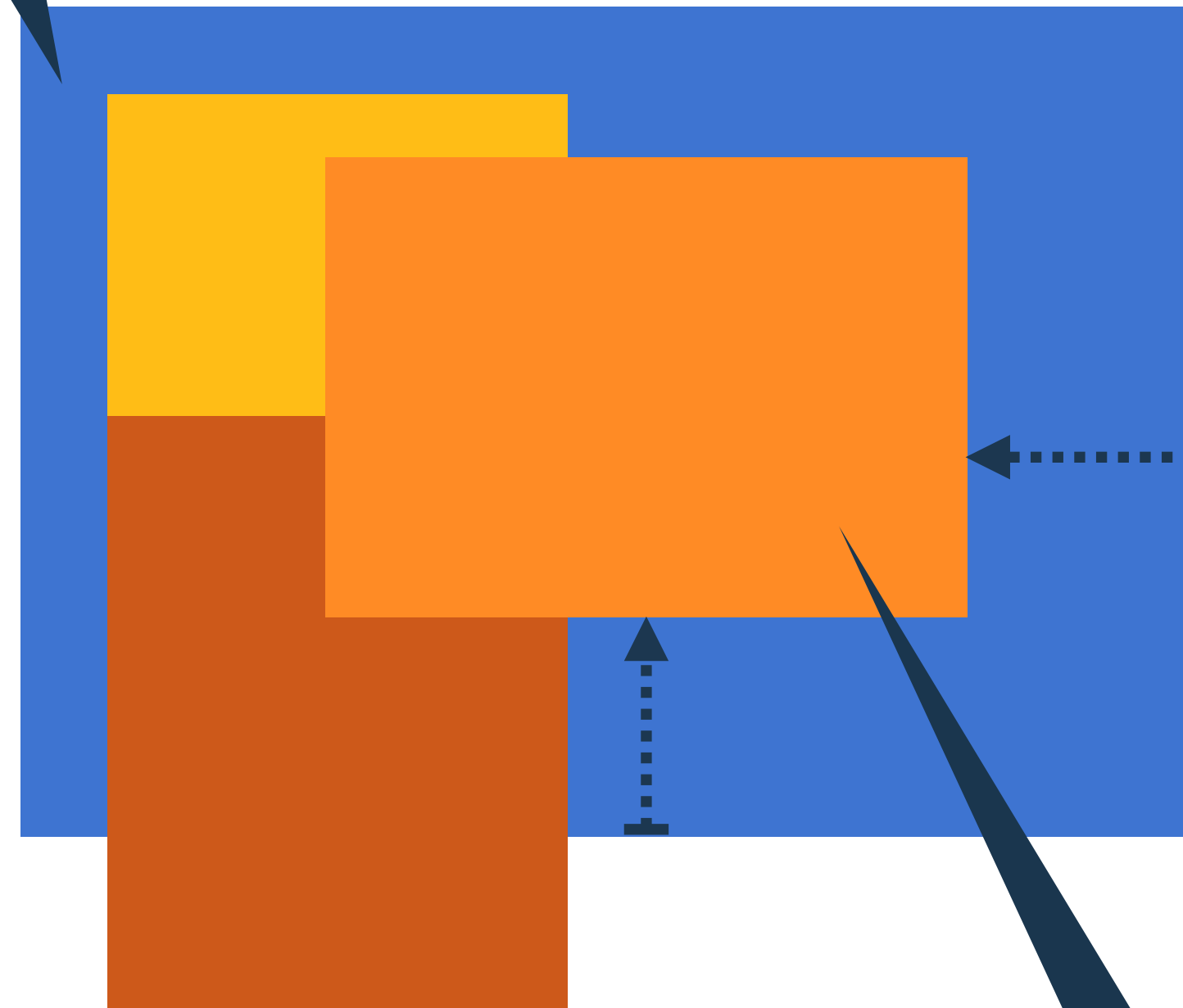
위치 상 부모 요소를 기준으로 배치!



absolute 위치 상 부모 요소를 기준으로 배치!

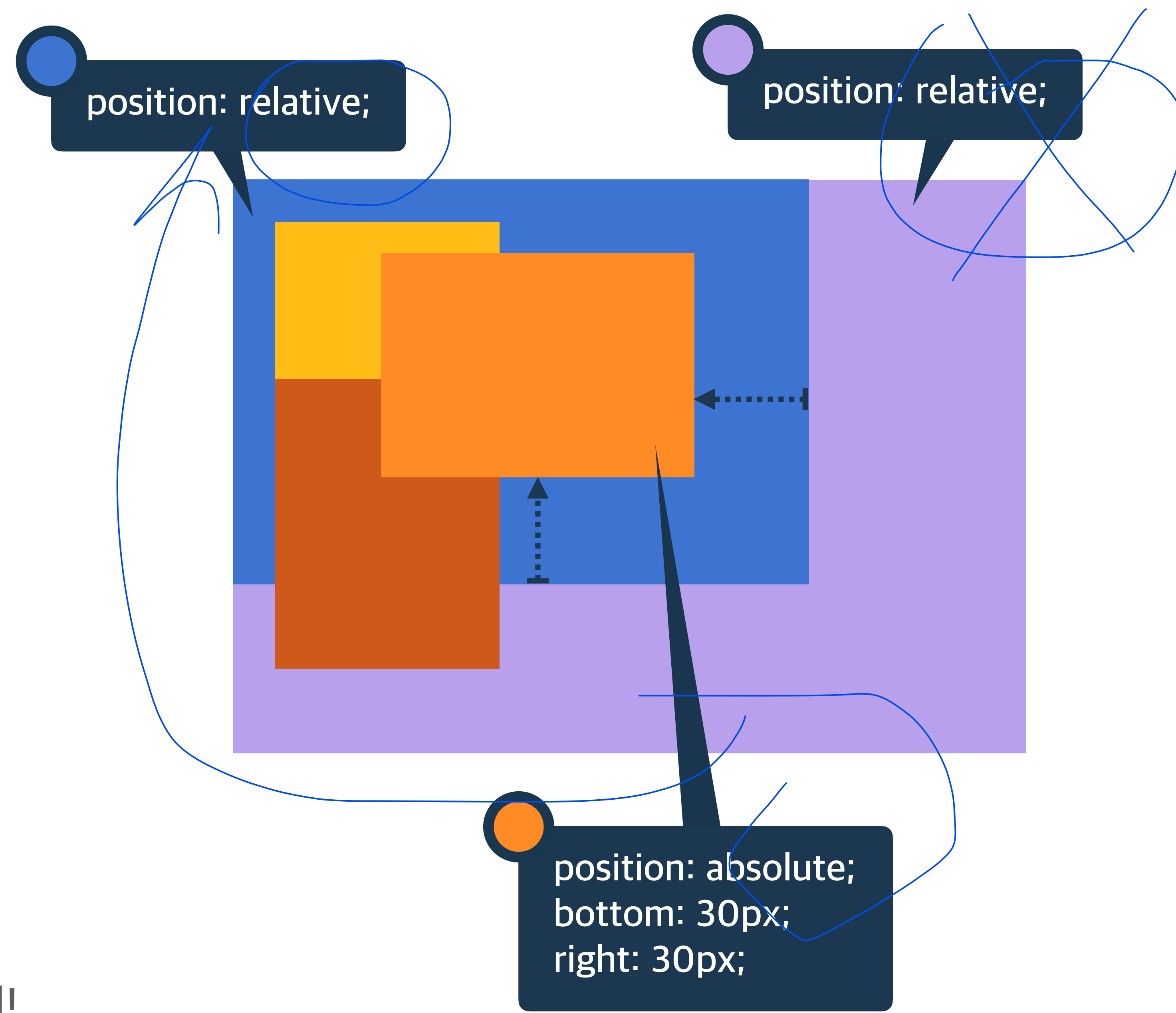
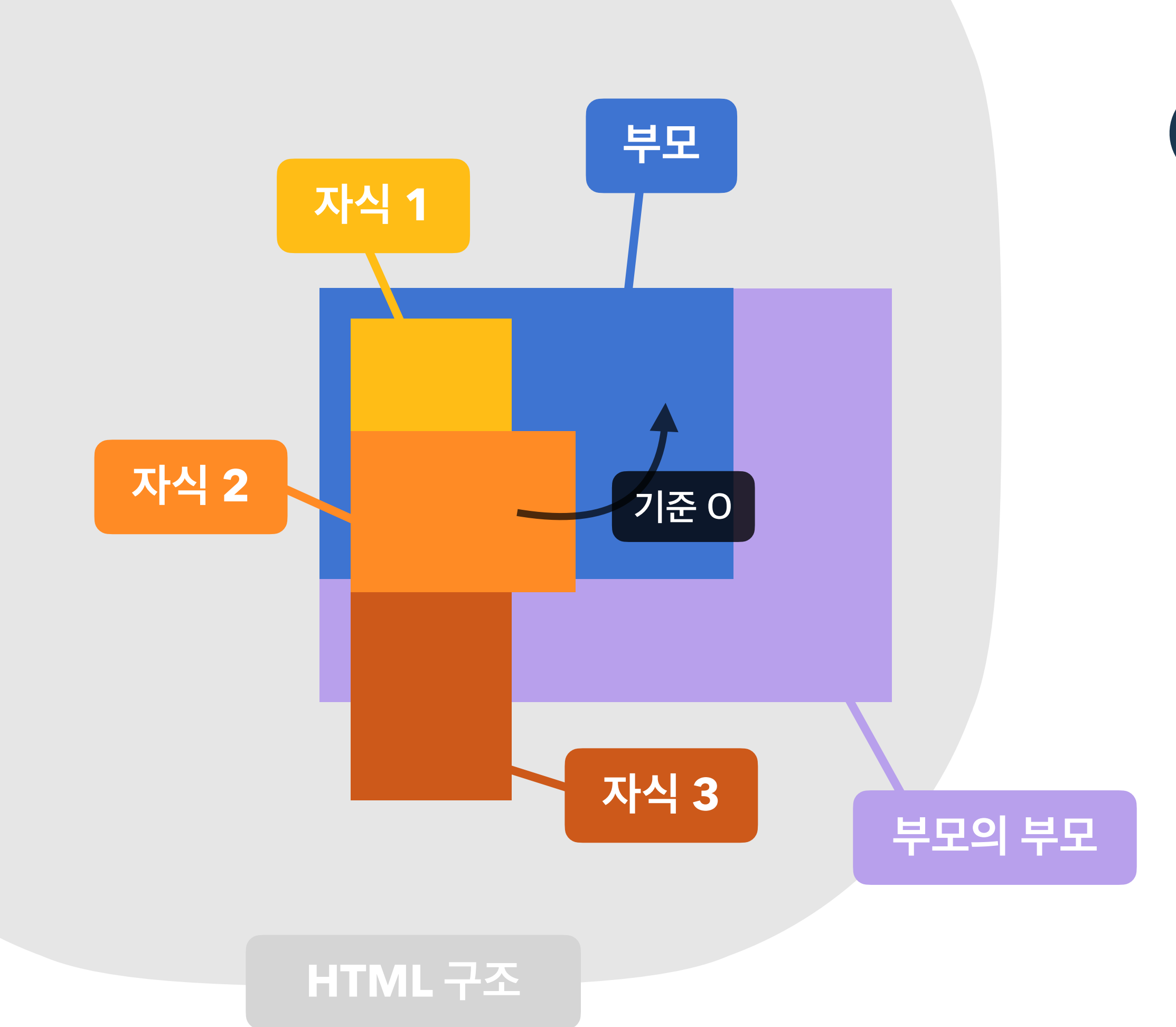


position: relative;

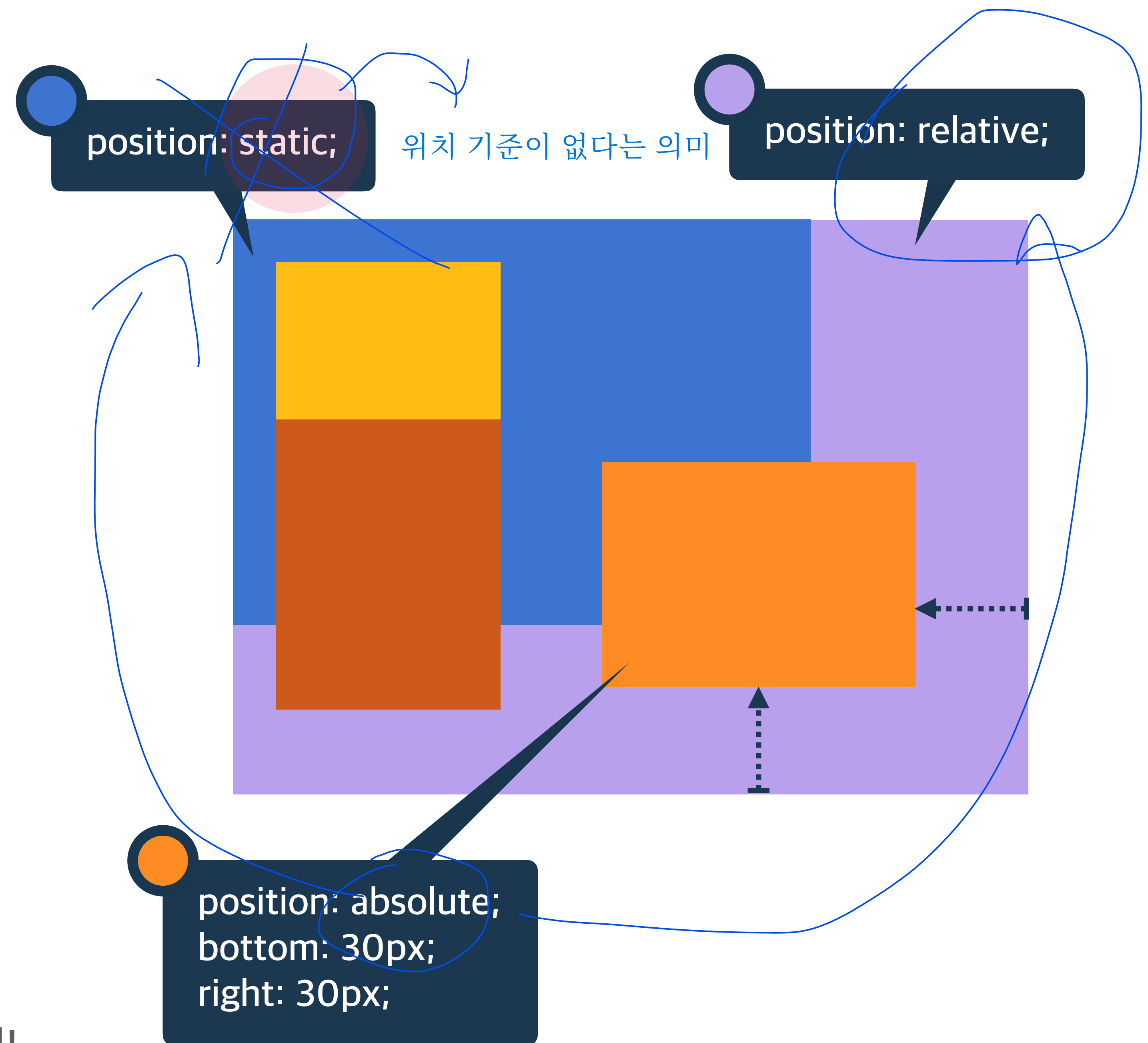
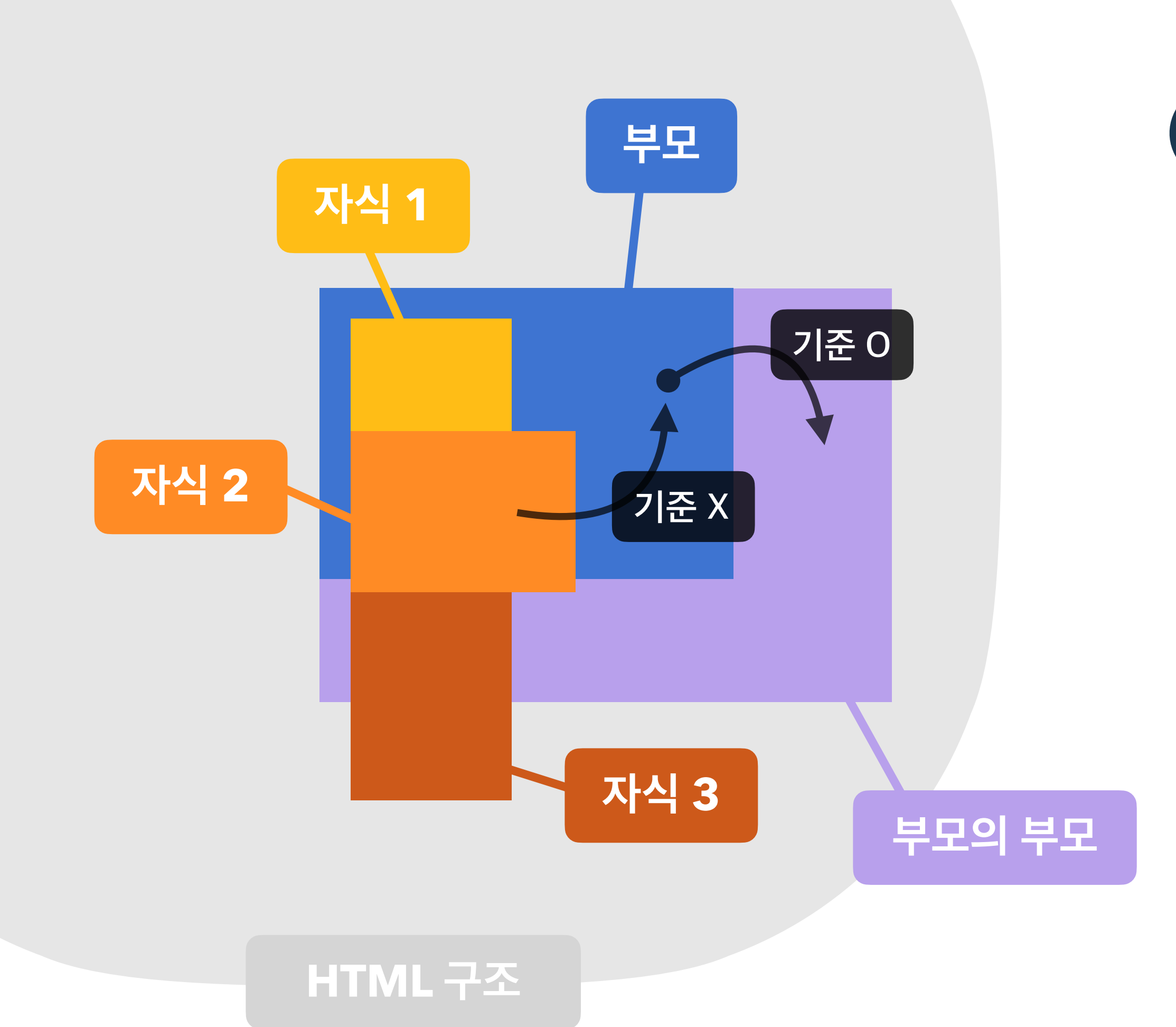


position: absolute;
bottom: 30px;
right: 30px;

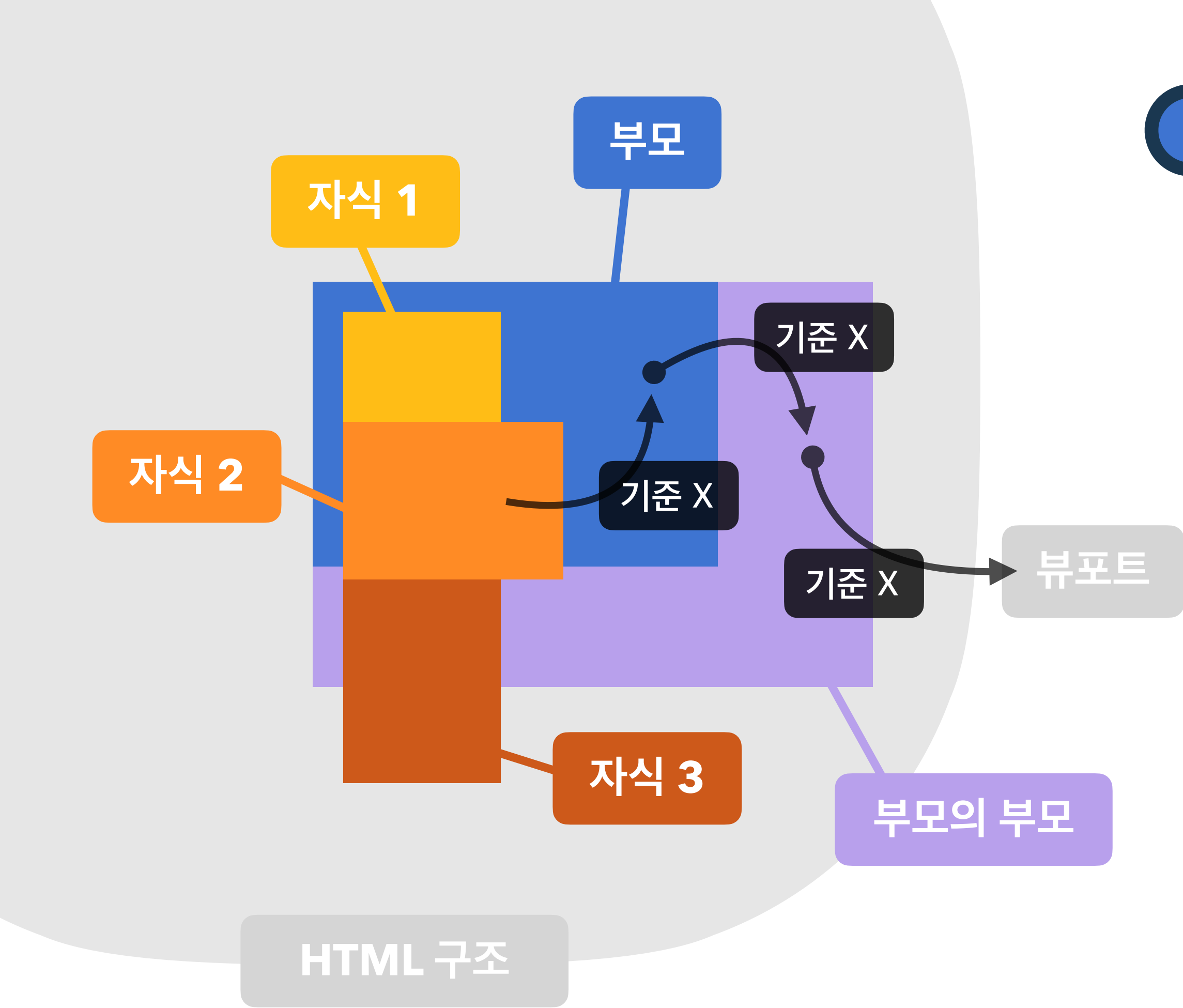
absolute 위치 상 부모 요소를 기준으로 배치!



absolute 위치 상 부모 요소를 기준으로 배치!

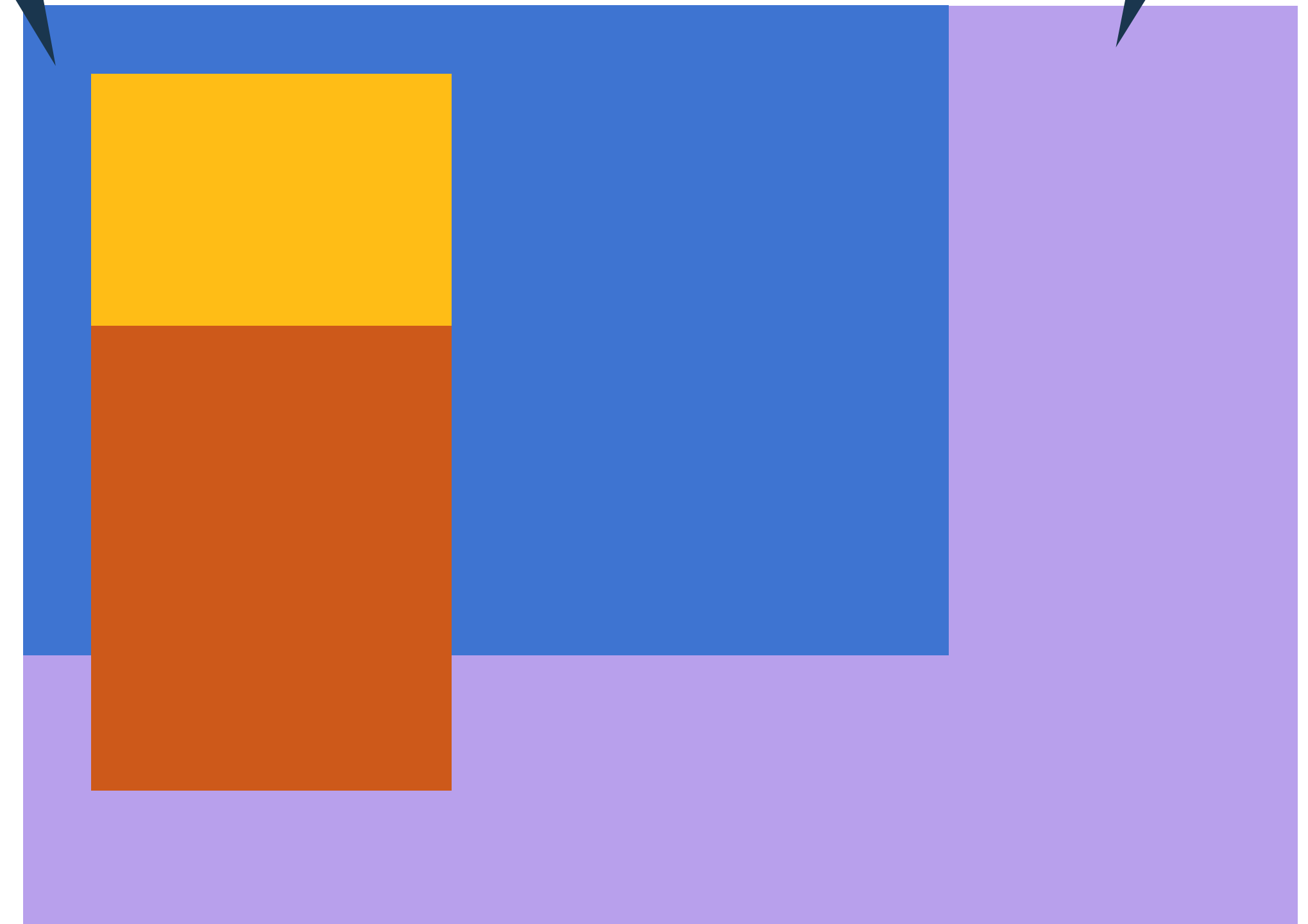


absolute 위치 상 부모 요소를 기준으로 배치!

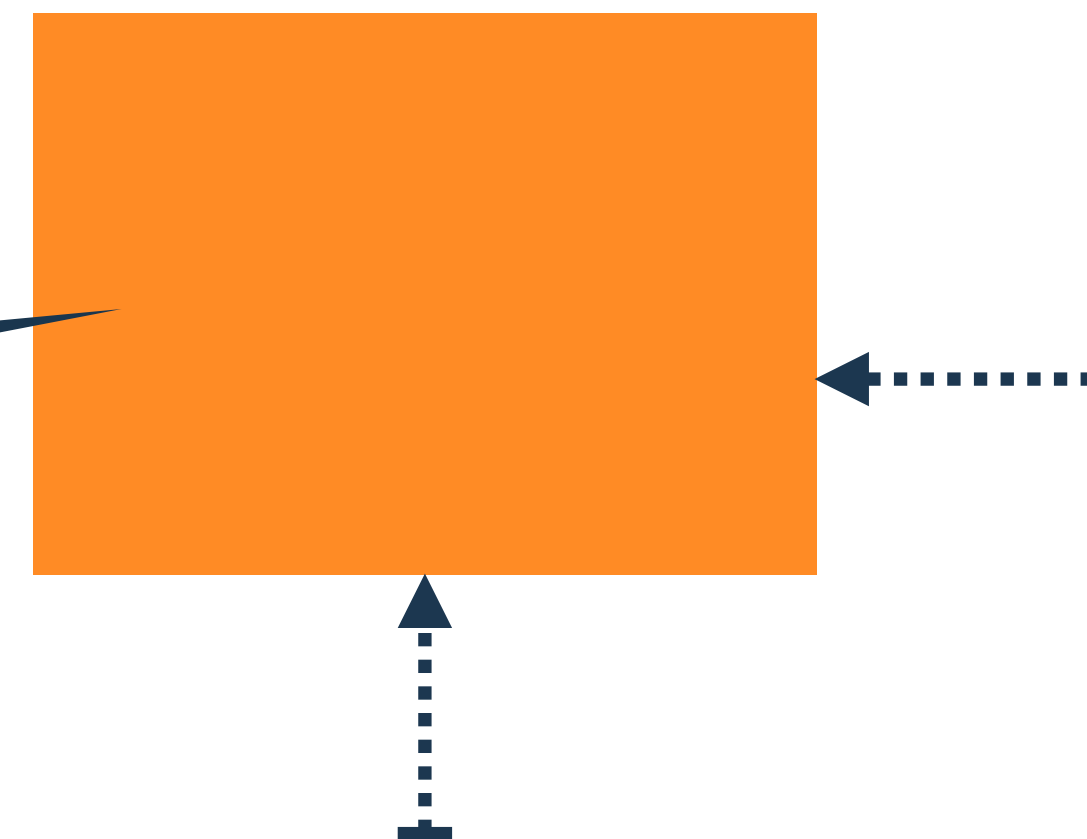


position: static;

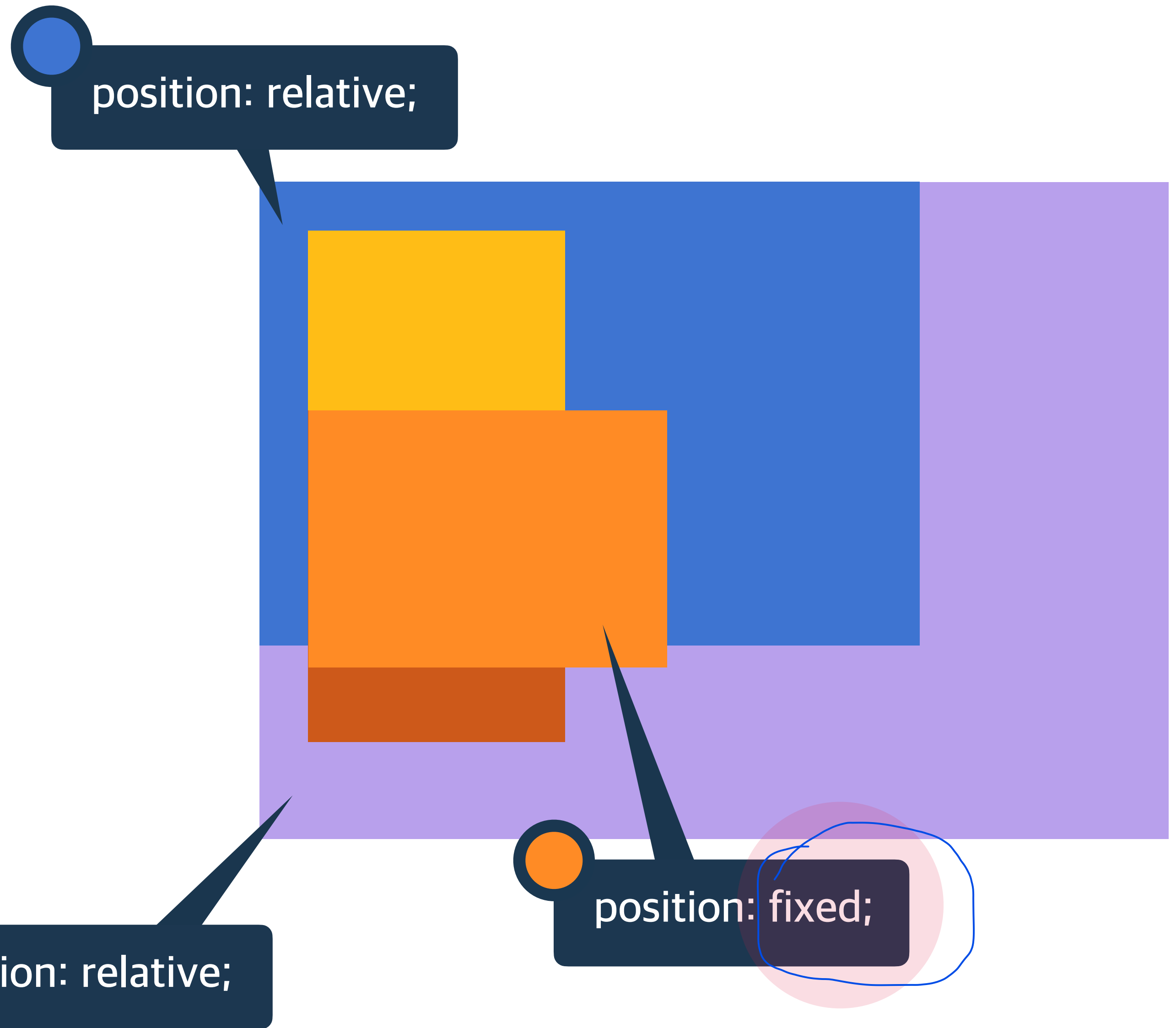
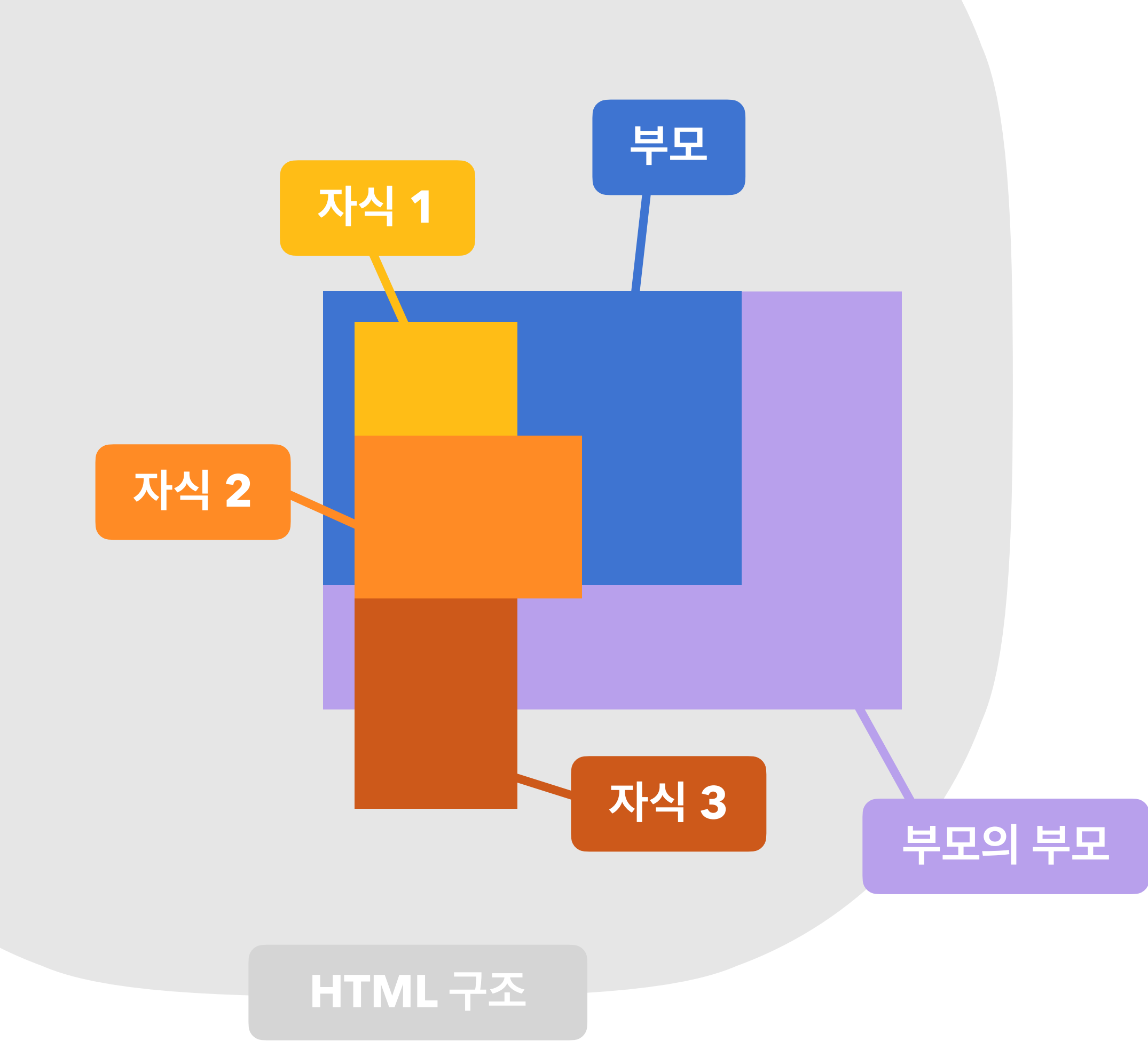
position: static;



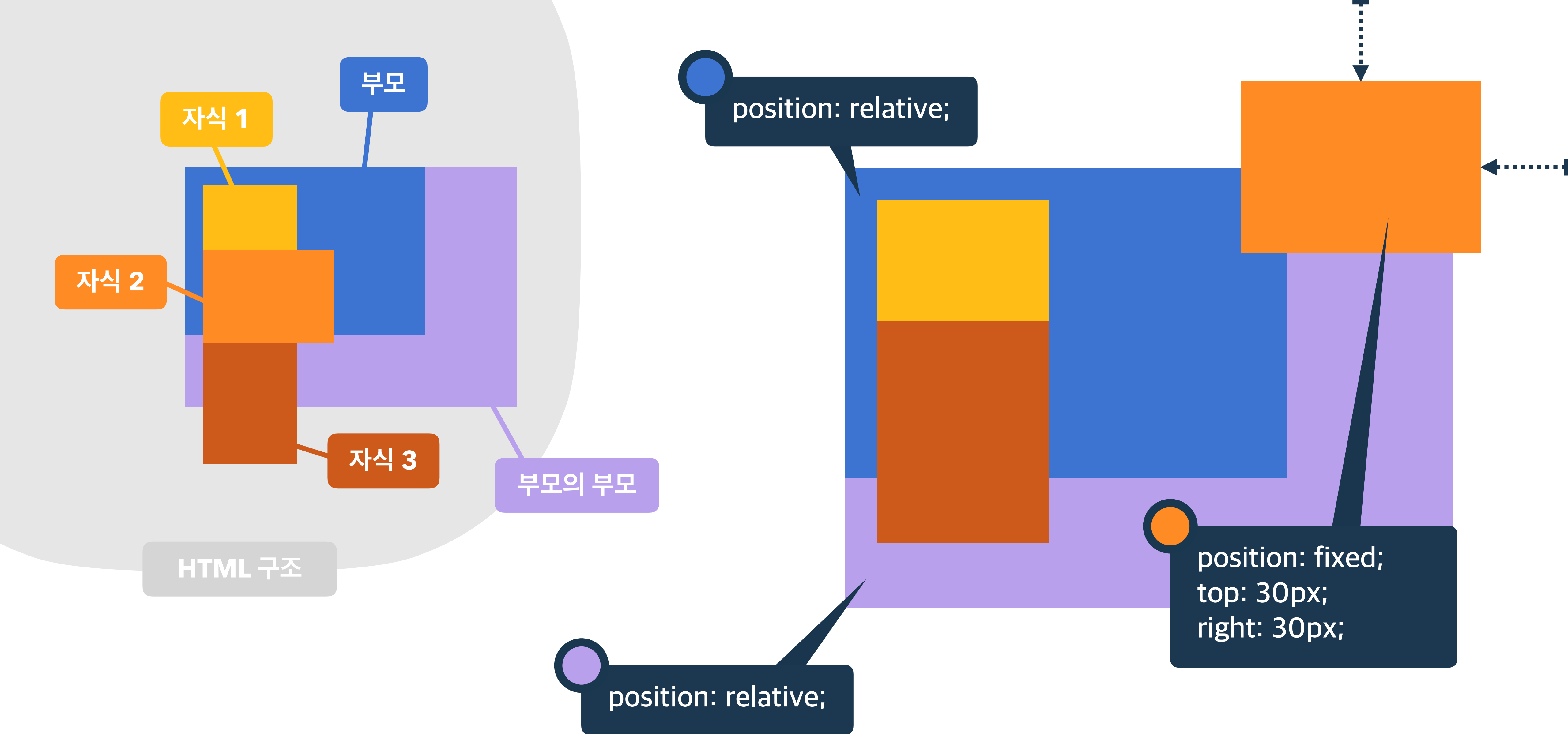
position: absolute;
bottom: 30px;
right: 30px;



absolute 위치 상 부모 요소를 기준으로 배치!



fixed 뷰포트(브라우저)를 기준으로 배치!



fixed

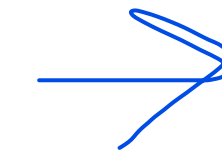
뷰포트(브라우저)를 기준으로 배치!

요소 쌓임 순서(Stack order)

어떤 요소가 사용자와 더 가깝게 있는지(위에 쌓이는지) 결정

1. 요소에 position 속성의 값이 있는 경우 위에 쌓임.(기본값 static 제외)
2. 1번 조건이 같은 경우, z-index 속성의 숫자 값이 높을 수록 위에 쌓임.
3. 1번과 2번 조건까지 같은 경우, HTML의 다음 구조일 수록 위에 쌓임.

요소의 쌓임 정도를 지정



position 값이 있어야(static 제외) 사용 가능

z-index

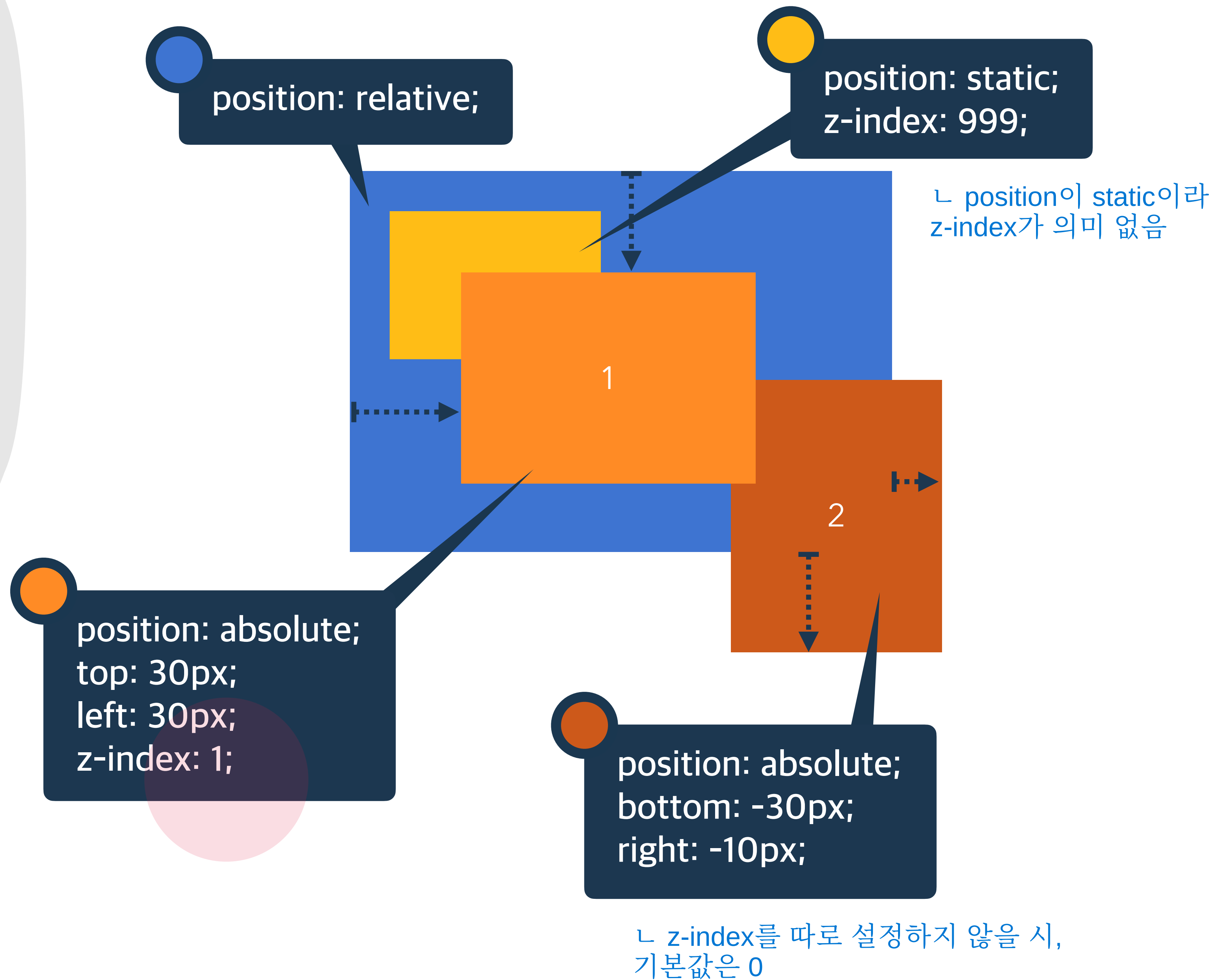
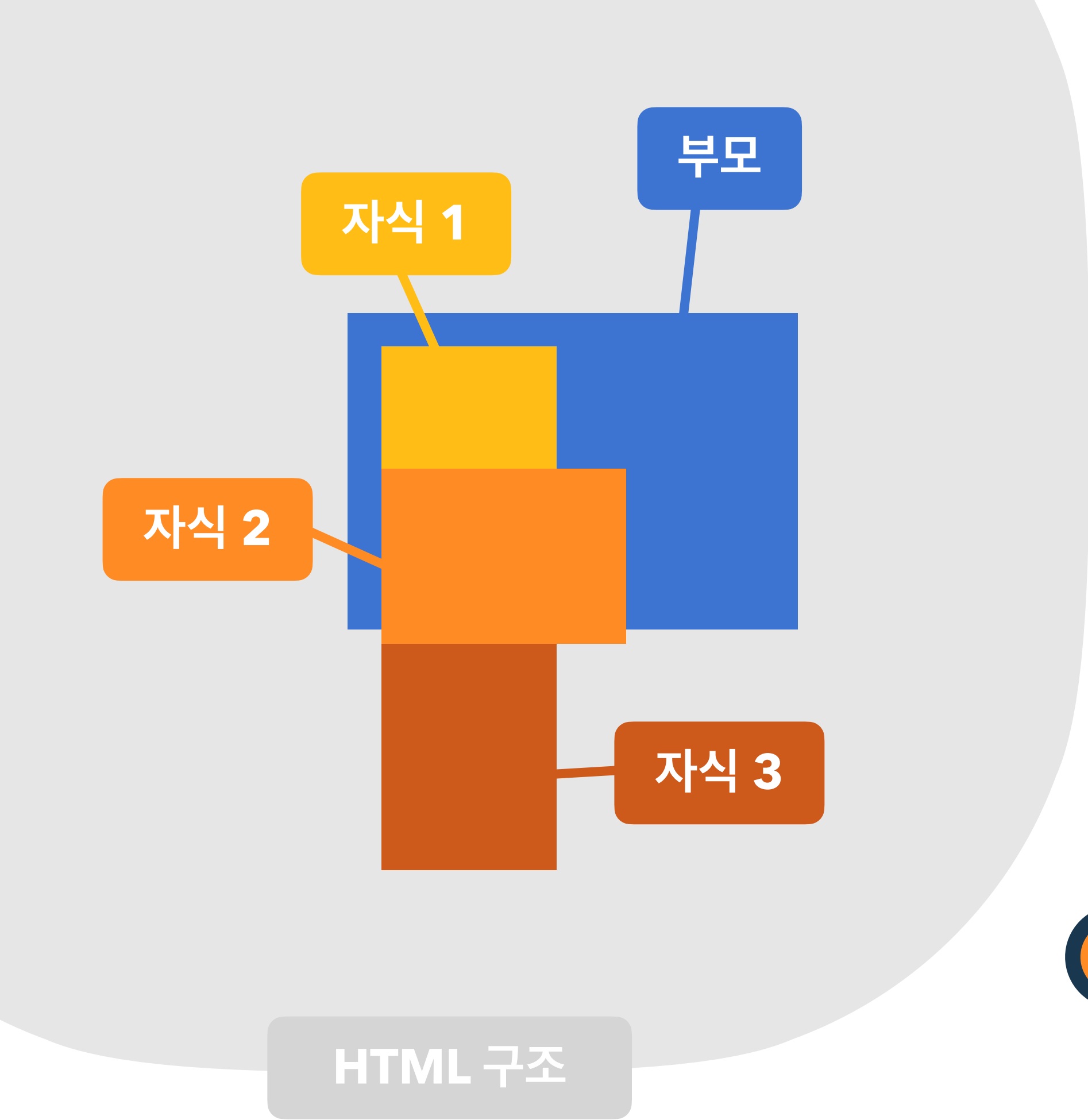
auto

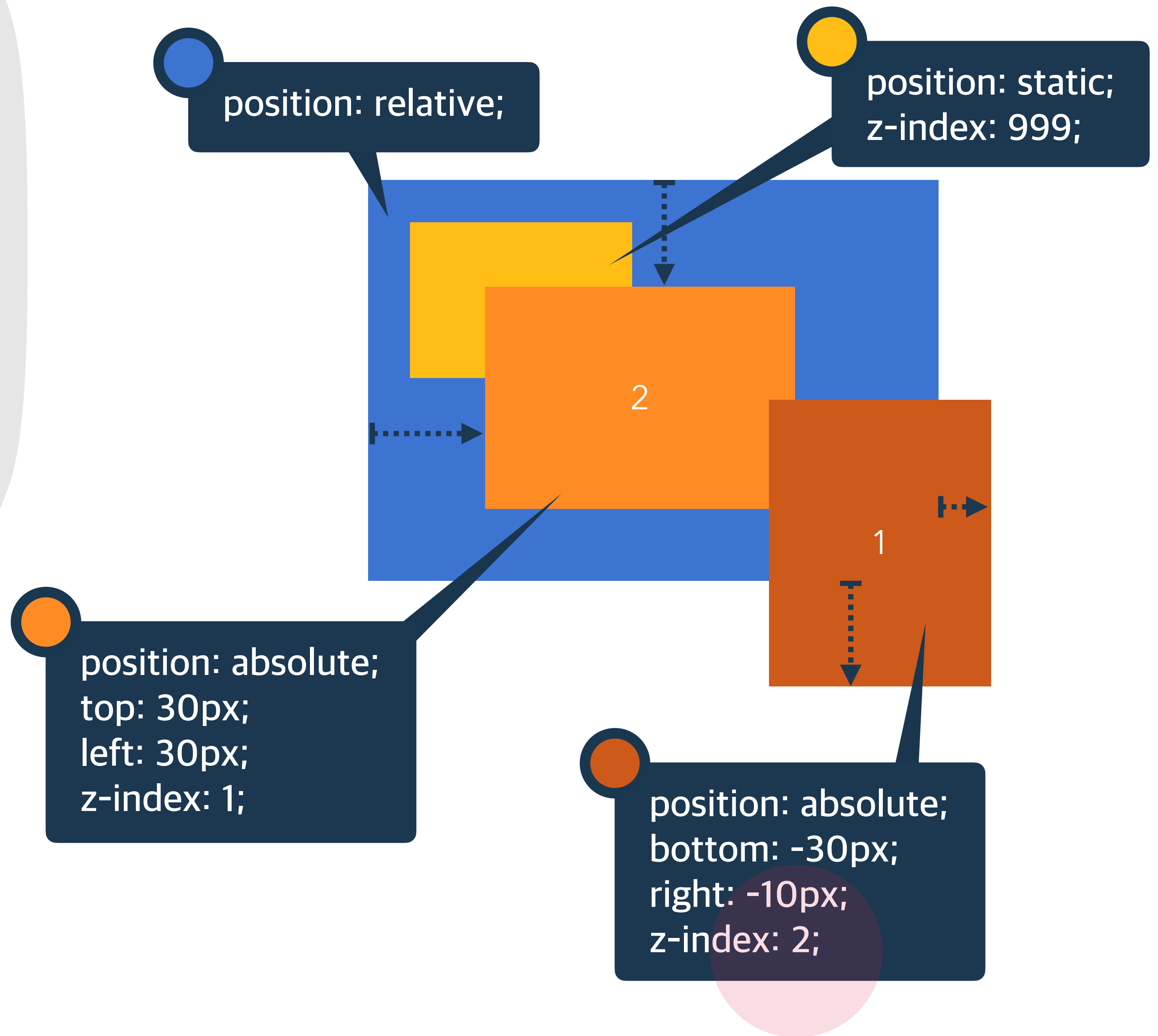
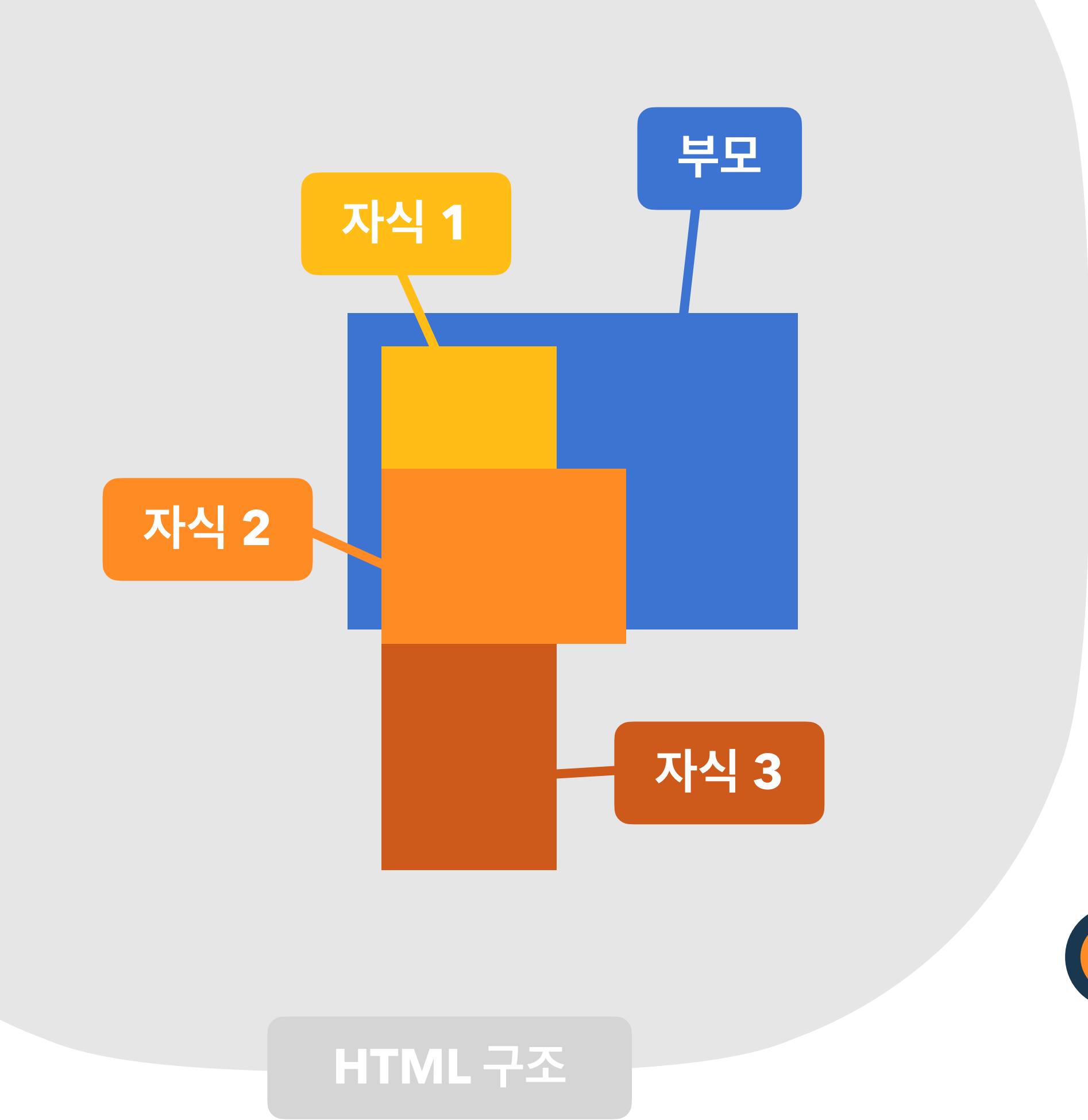
부모 요소와 동일한 쌓임 정도

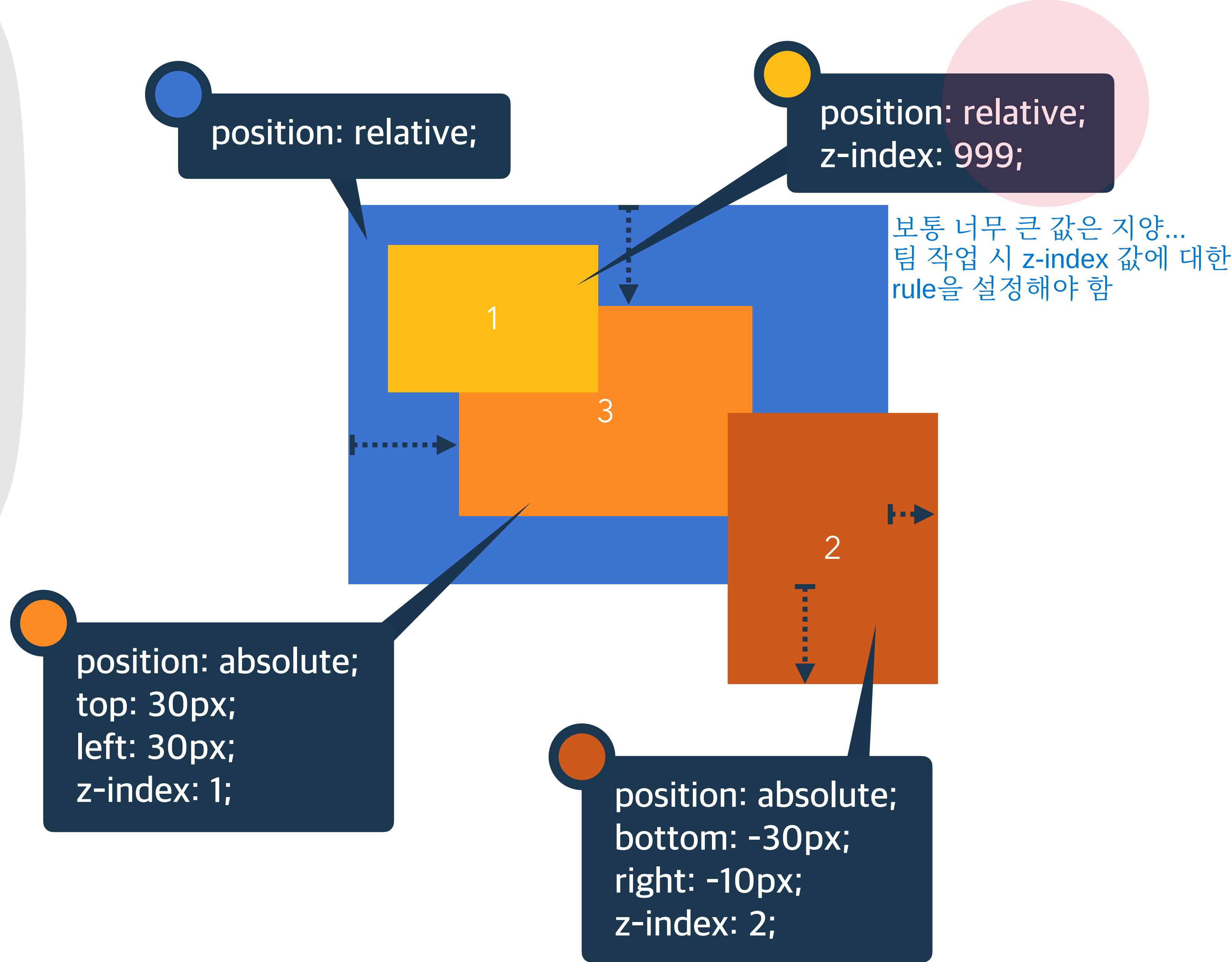
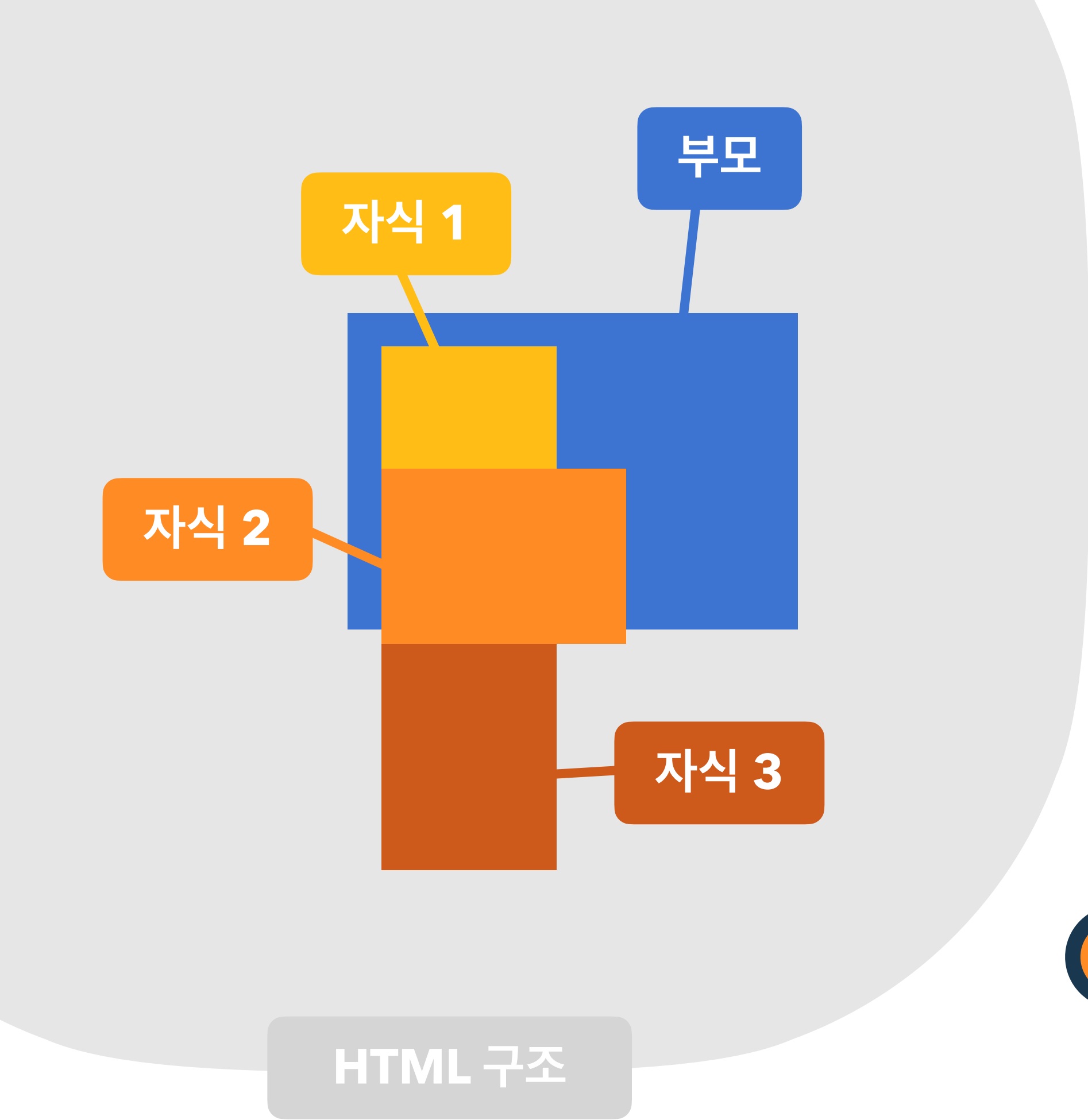
숫자

숫자가 높을 수록 위에 쌓임

↳ 음수도 사용 가능

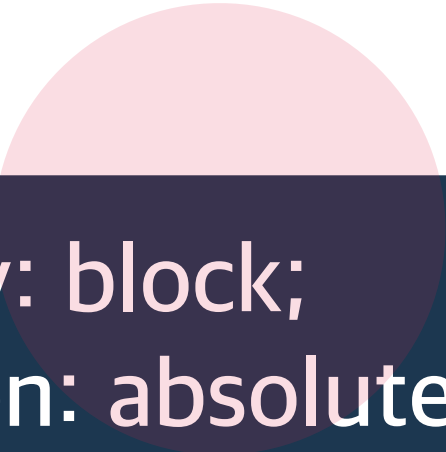






요소의 display가 변경됨

position 속성의 값으로 absolute, fixed가 지정된 요소는,
display 속성이 block으로 변경됨.

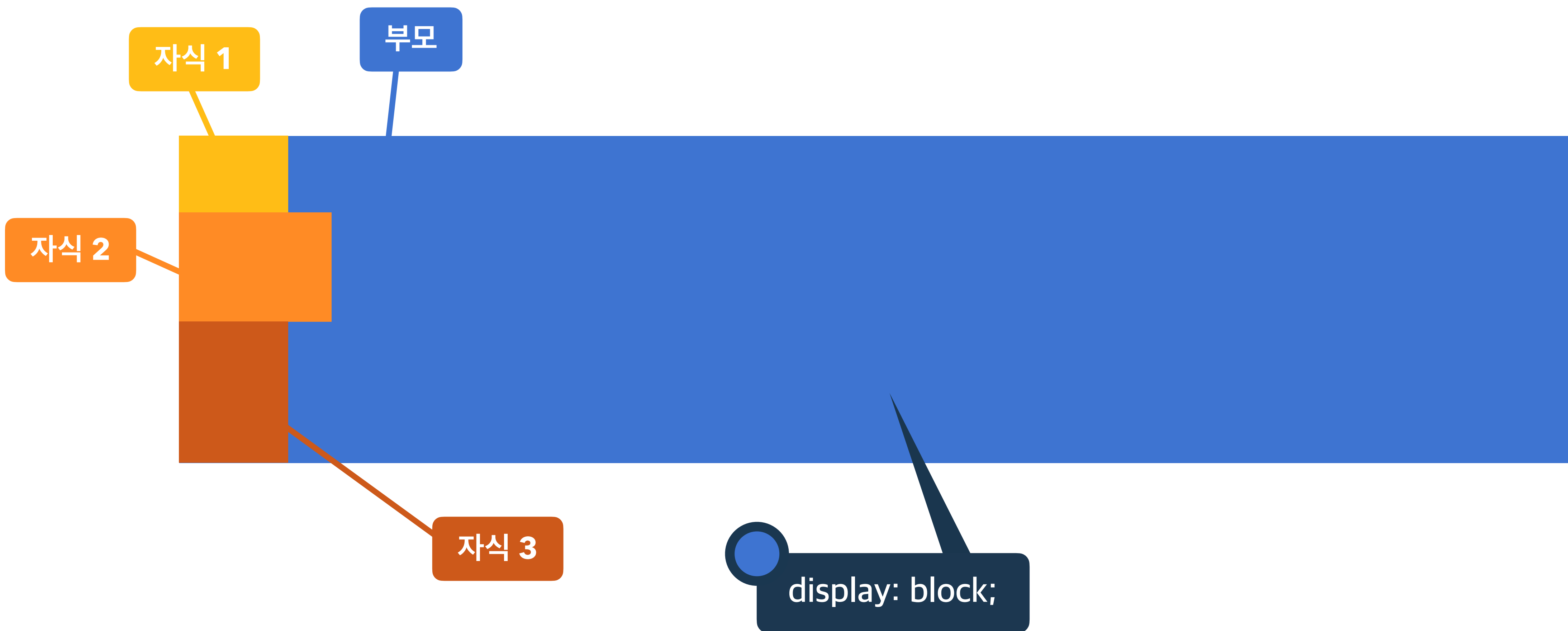


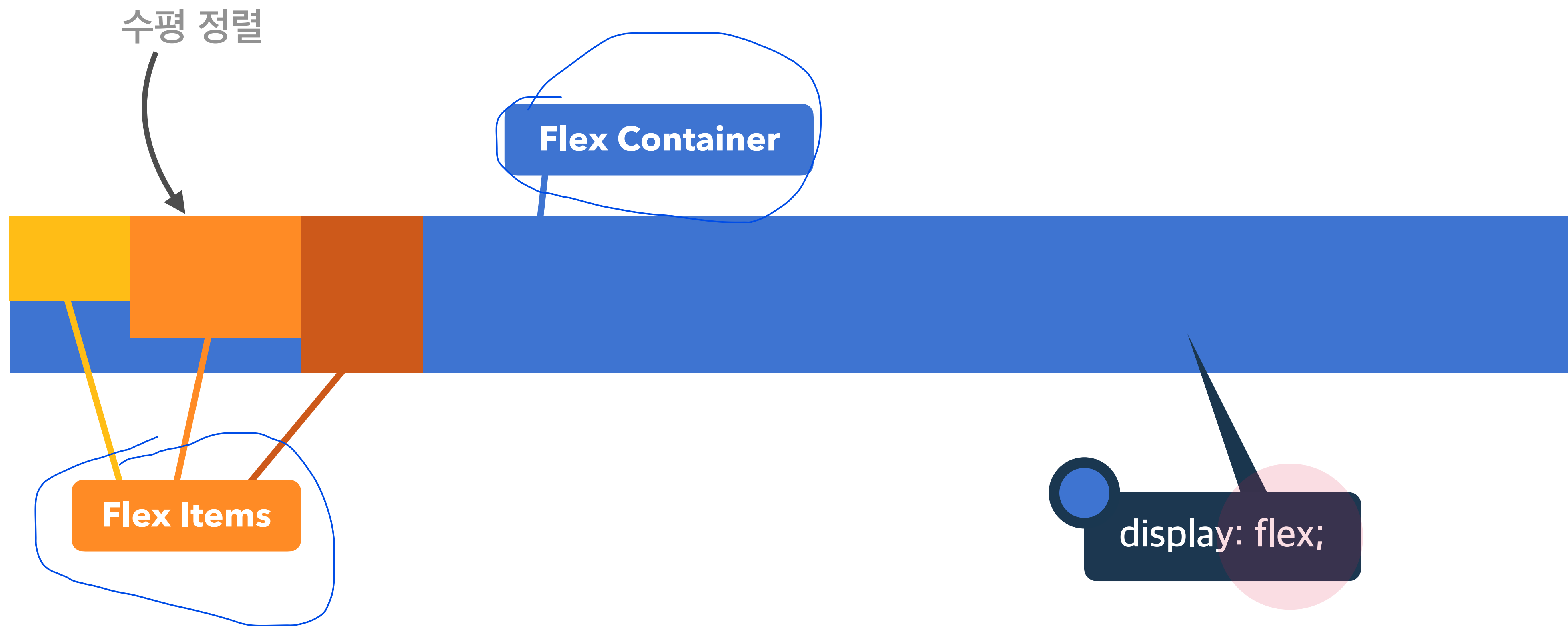
```
display: block;  
position: absolute;  
top: 30px;  
left: 30px;  
z-index: 1;
```

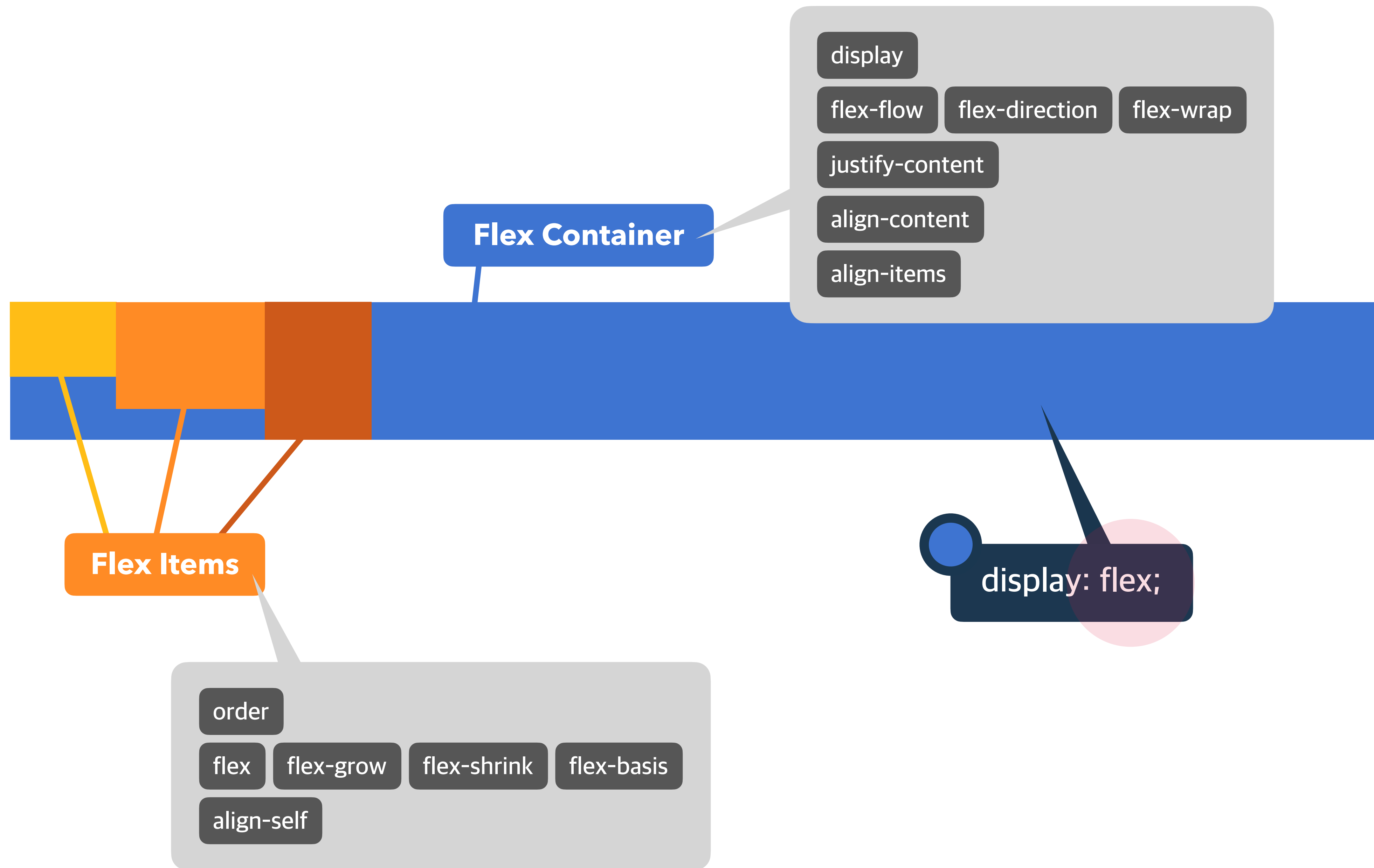
=

```
position: absolute;  
top: 30px;  
left: 30px;  
z-index: 1;
```


플렉스(정렬)







Flex Container의 화면 출력(보여짐) 특성

display

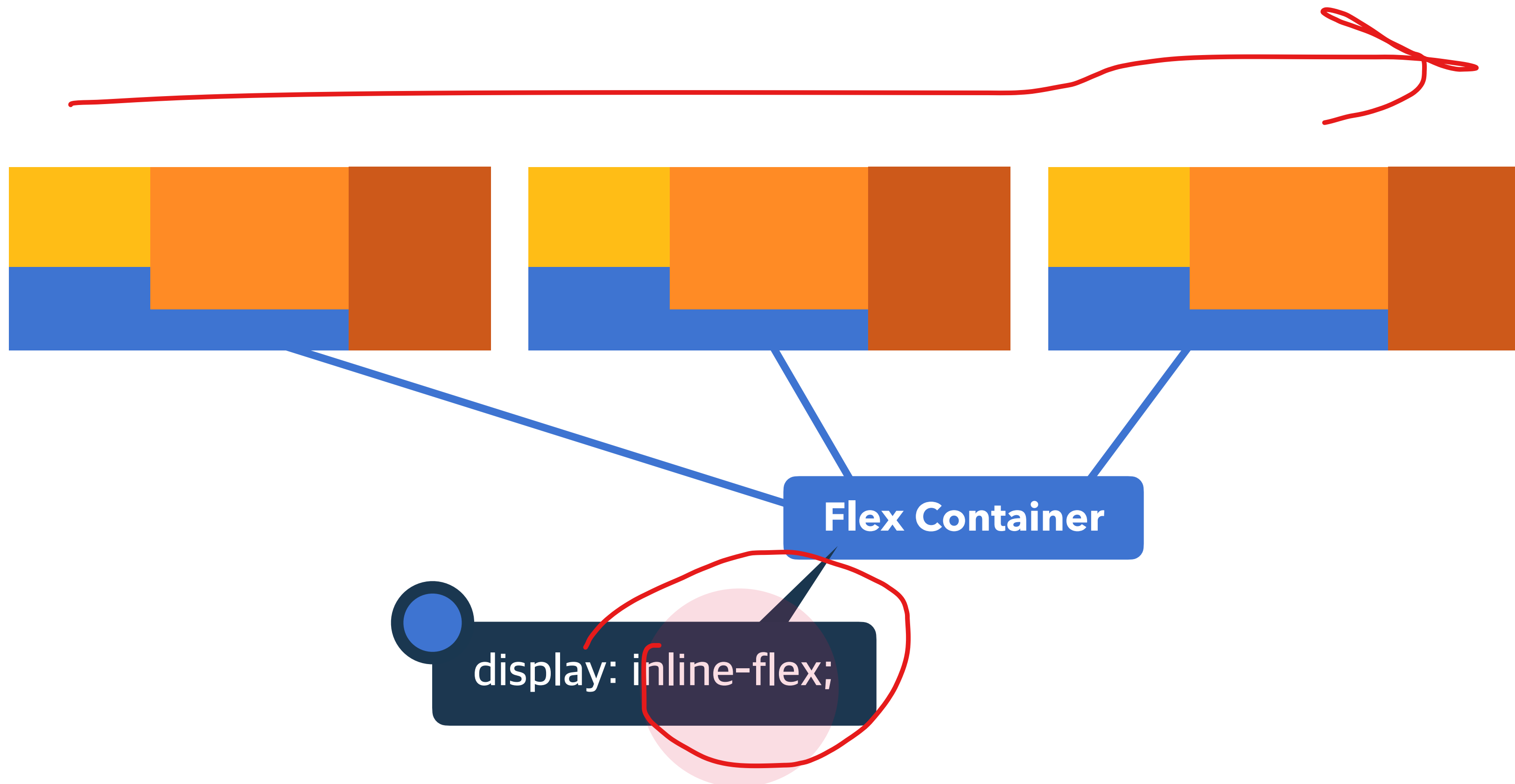
flex

블록 요소와 같이 Flex Container 정의

inline-flex

인라인 요소와 같이 Flex Container 정의





주 축을 설정

flex-direction

row 행 축 (좌 => 우) 수평

row-reverse 행 축 (우 => 좌)

column 열 축 (위 => 아래) 수직

column-reverse 열 축 (아래 => 위)

행, Row

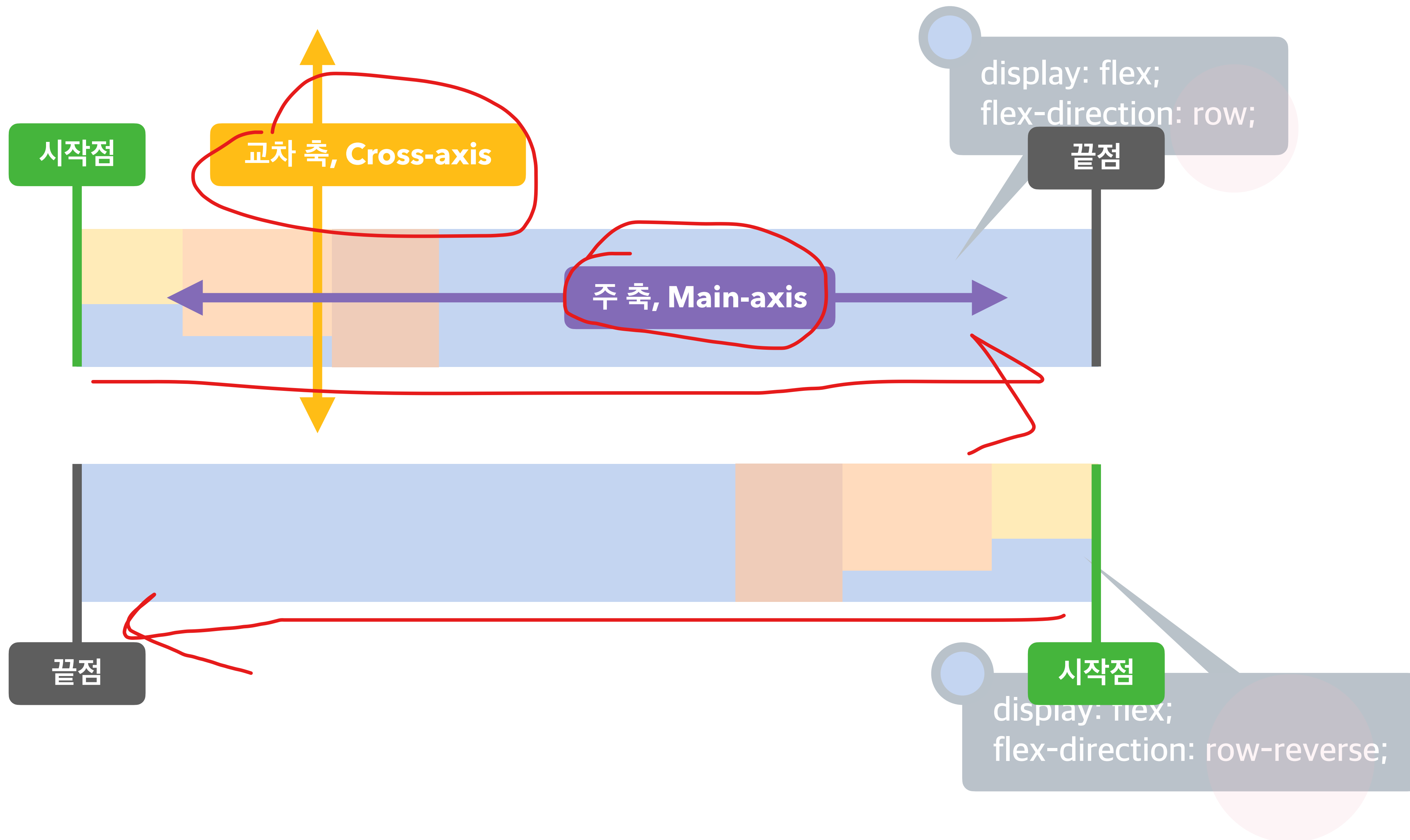
열, Column

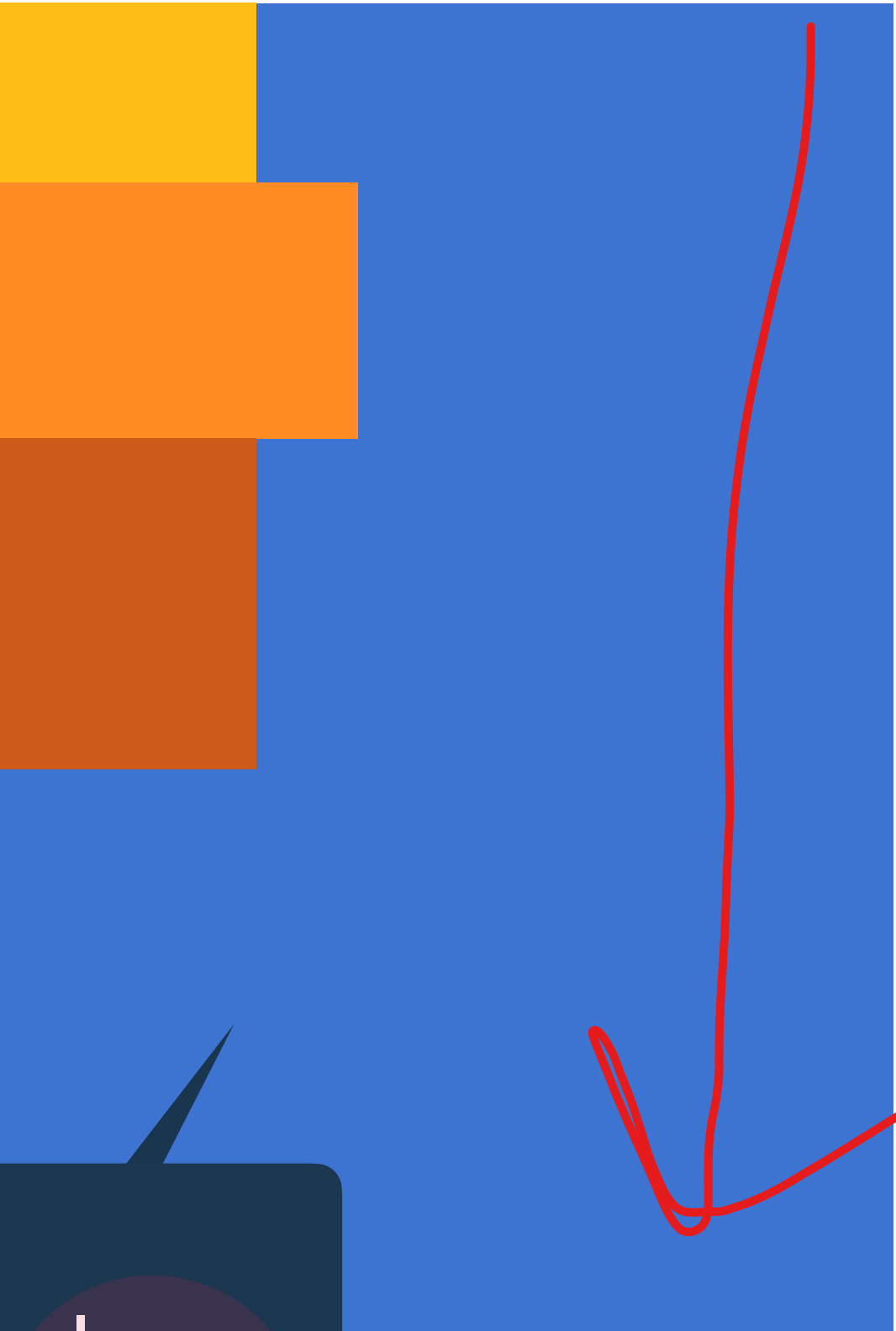


display: flex;
flex-direction: row;



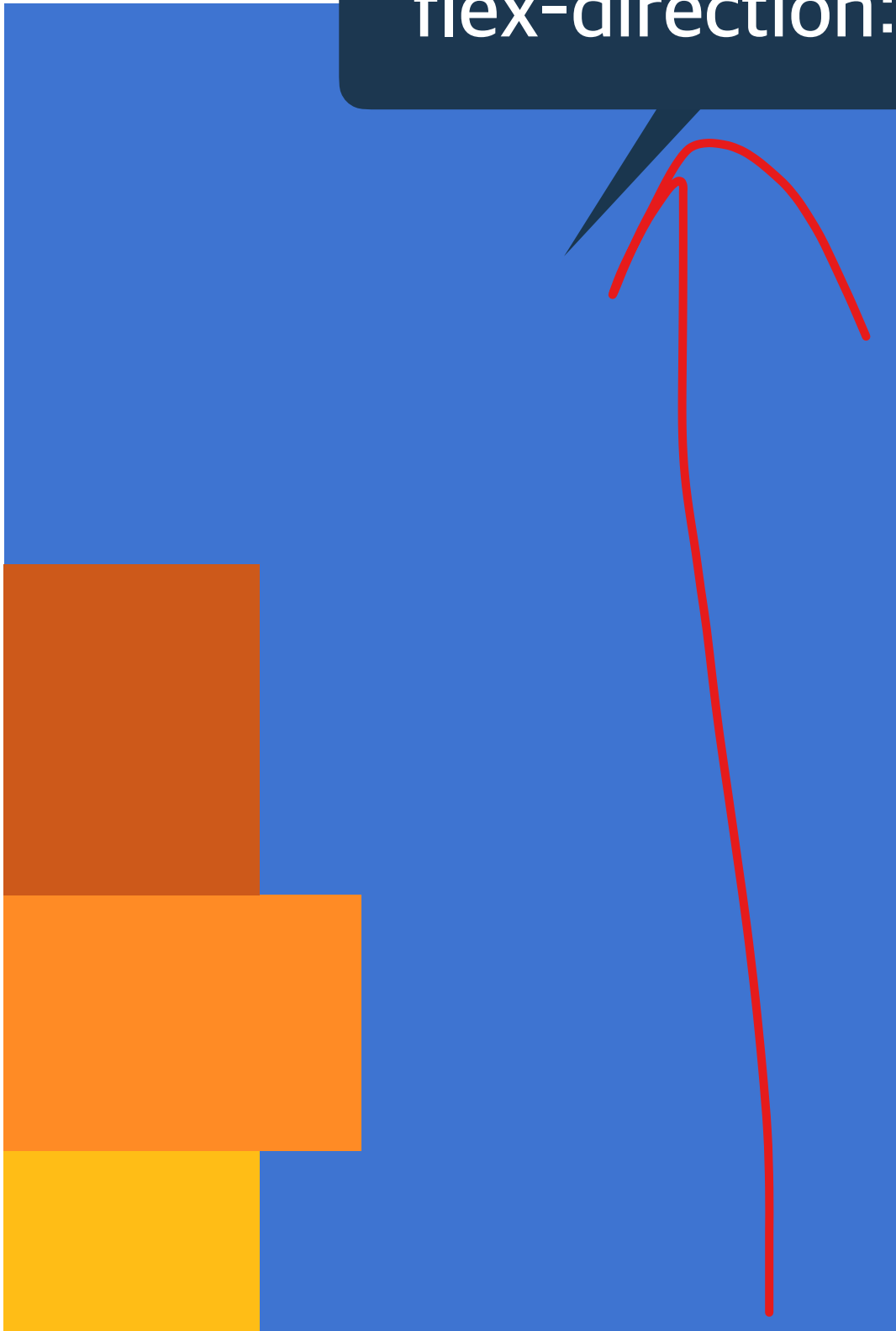
display: flex;
flex-direction: row-reverse;





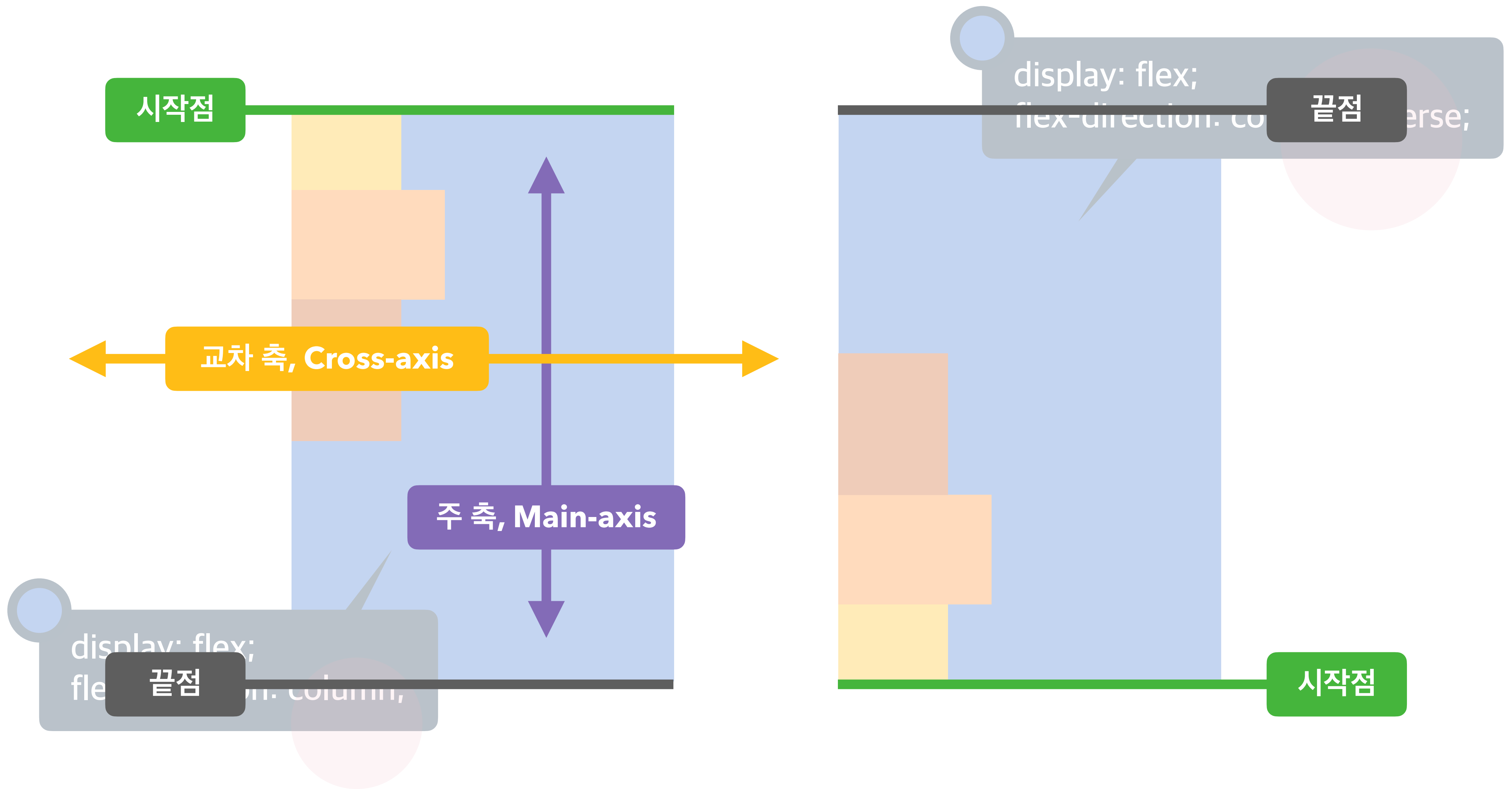
A blue square represents a flex container. Inside, three colored rectangles (yellow, orange, and brown) are stacked vertically on the left side. A red arrow points downwards from the top of the stack to the bottom, indicating the direction of the flex axis.

`display: flex;`
`flex-direction: column;`



A blue square represents a flex container. Inside, three colored rectangles (brown, orange, and yellow) are stacked vertically on the left side. A red arrow points upwards from the bottom of the stack to the top, indicating the direction of the flex axis.

`display: flex;`
`flex-direction: column-reverse;`



Flex Items 묶음(줄 바꿈) 여부

flex-wrap

nowrap

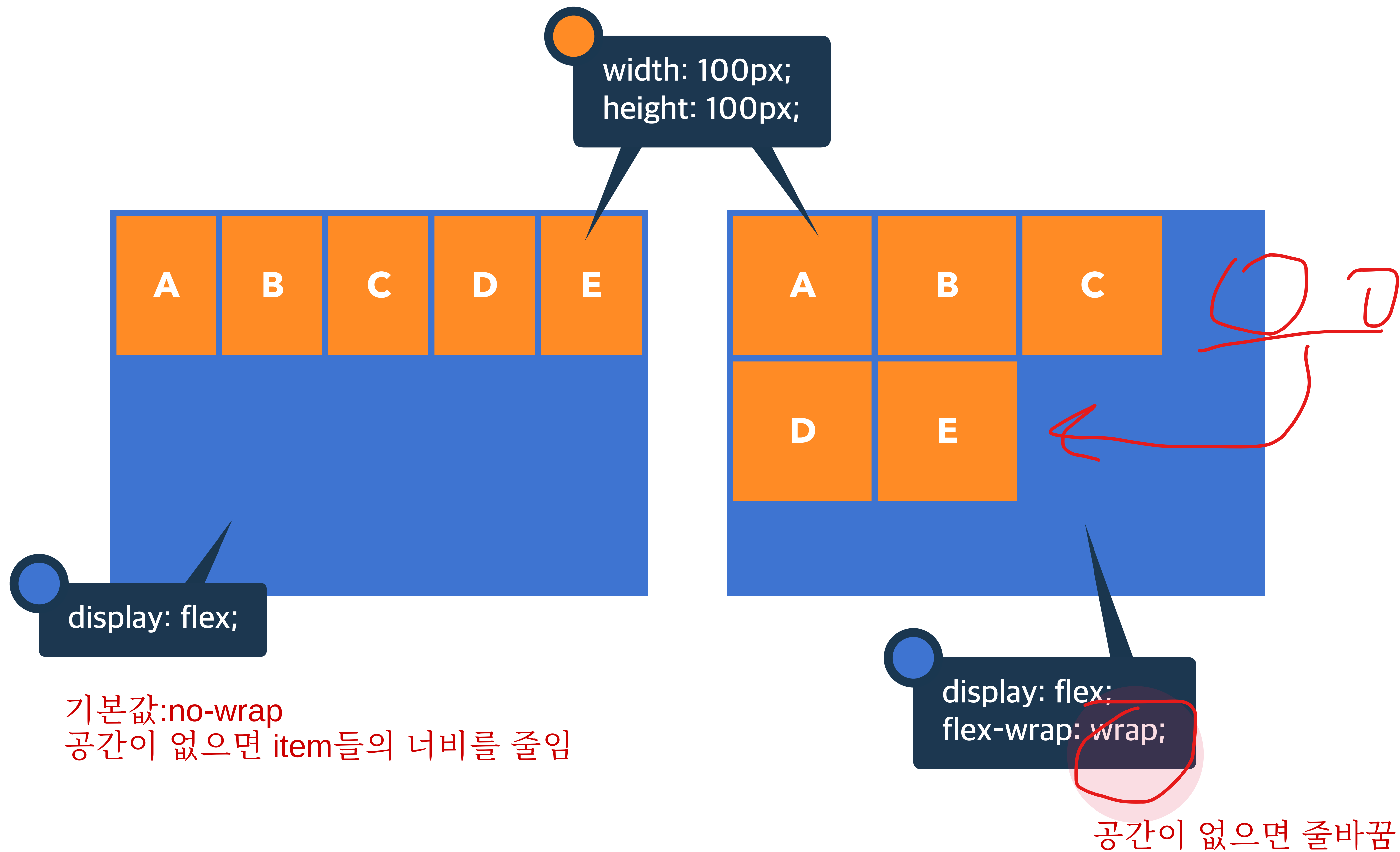
묶음(줄 바꿈) 없음

wrap

여러 줄로 묶음

wrap-reverse

wrap의 반대 방향으로 묶음



주 축의 정렬 방법

justify-content

주 축이
row -> 수평 정렬
column -> 수직 정렬
이 됨

left, right 로도 할 수 있지만
비표준임 (chrome에서만 동작)

flex-start

Flex Items를 시작점으로 정렬

flex-end

Flex Items를 끝점으로 정렬

center

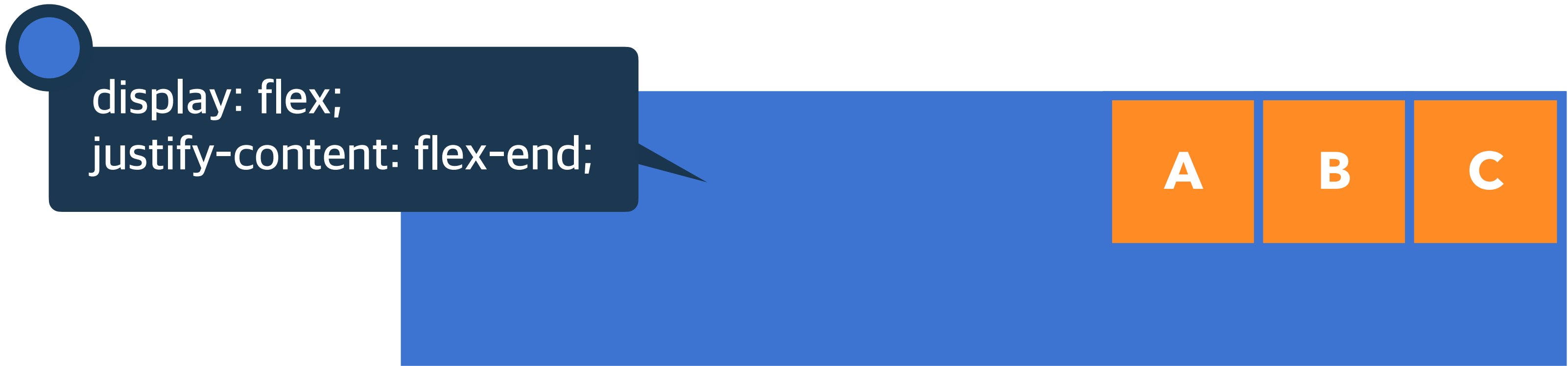
Flex Items를 가운데 정렬

space-between

각 Flex Item 사이를 균등하게 정렬

space-around

각 Flex Item의 외부 여백을 균등하게 정렬



교차 축의 여러 줄 정렬 방법

align-content

row라고 가정했을 때,
수직으로 여러줄 정렬하는 것

stretch

Flex Items를 시작점으로 정렬

flex-start

Flex Items를 시작점으로 정렬

flex-end

Flex Items를 끝점으로 정렬

center

Flex Items를 가운데 정렬

space-between

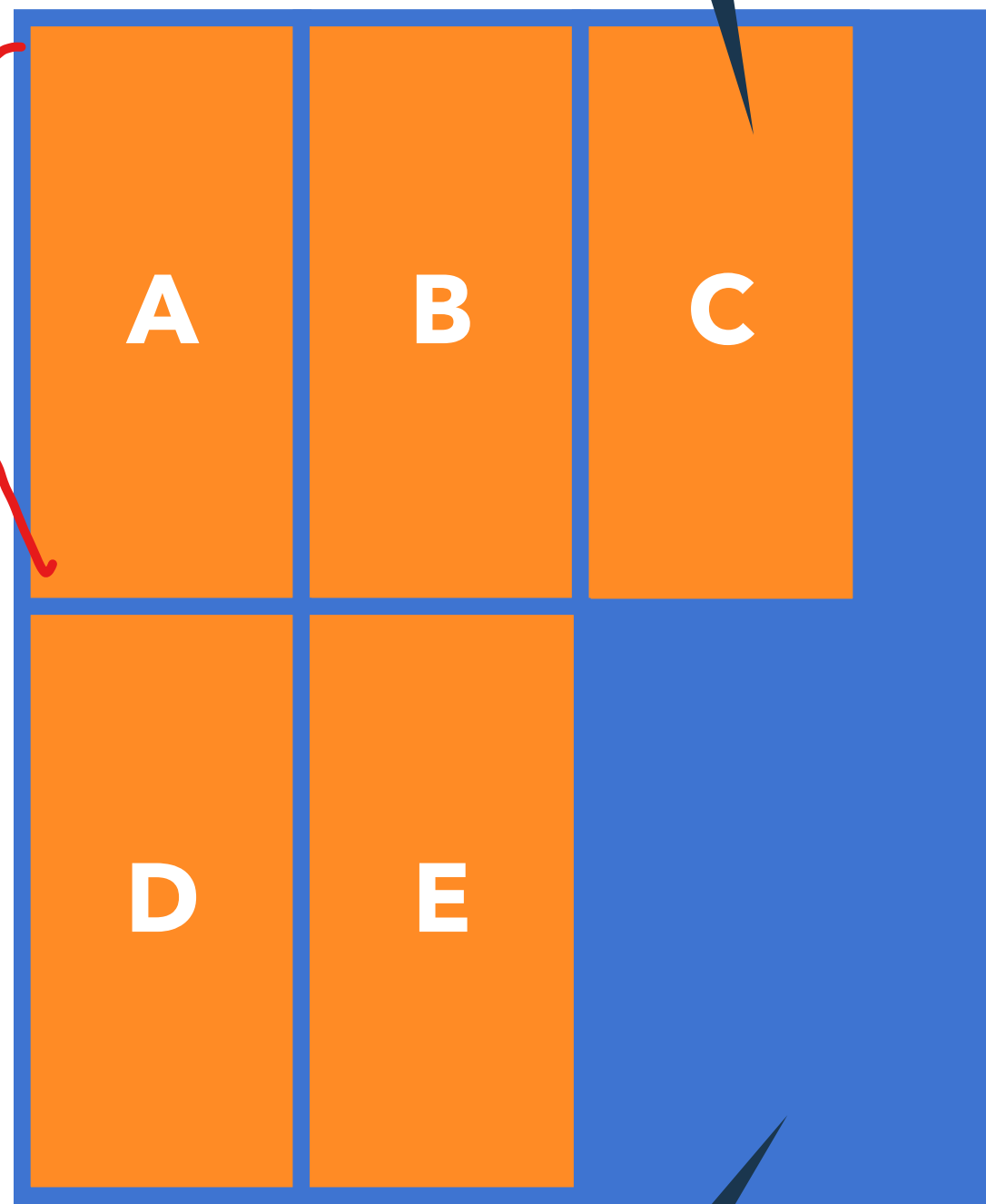
각 Flex Item 사이를 균등하게 정렬

space-around

각 Flex Item의 외부 여백을 균등하게 정렬

width: 100px;

높이를 지정 안했을 때,
늘어날 수 있는 만큼 늘어남

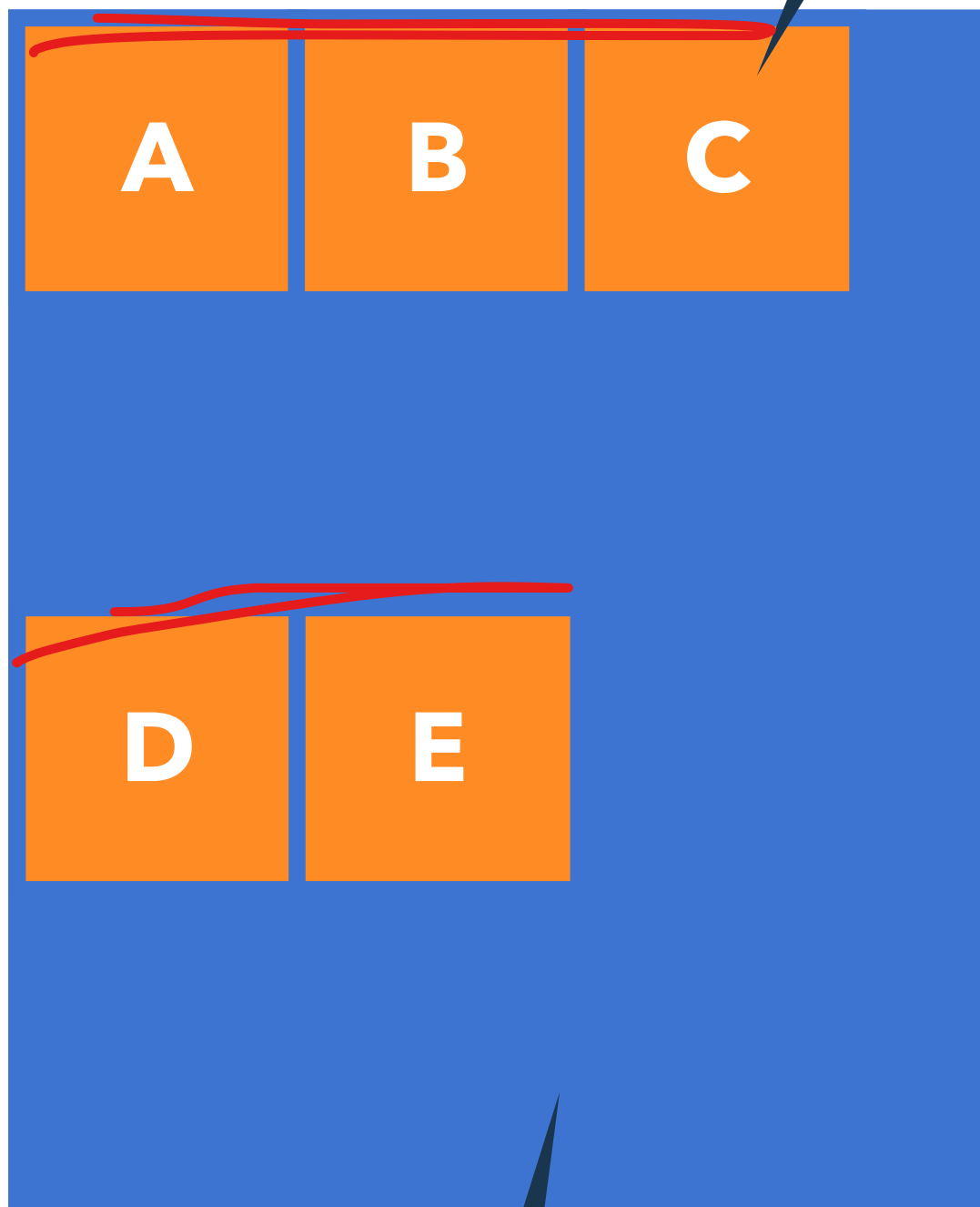


display: flex;
flex-wrap: wrap;

동작 전제조건
(여러줄 정렬이기 때문에, 일단 여러줄로 만들어야 함)

기본값

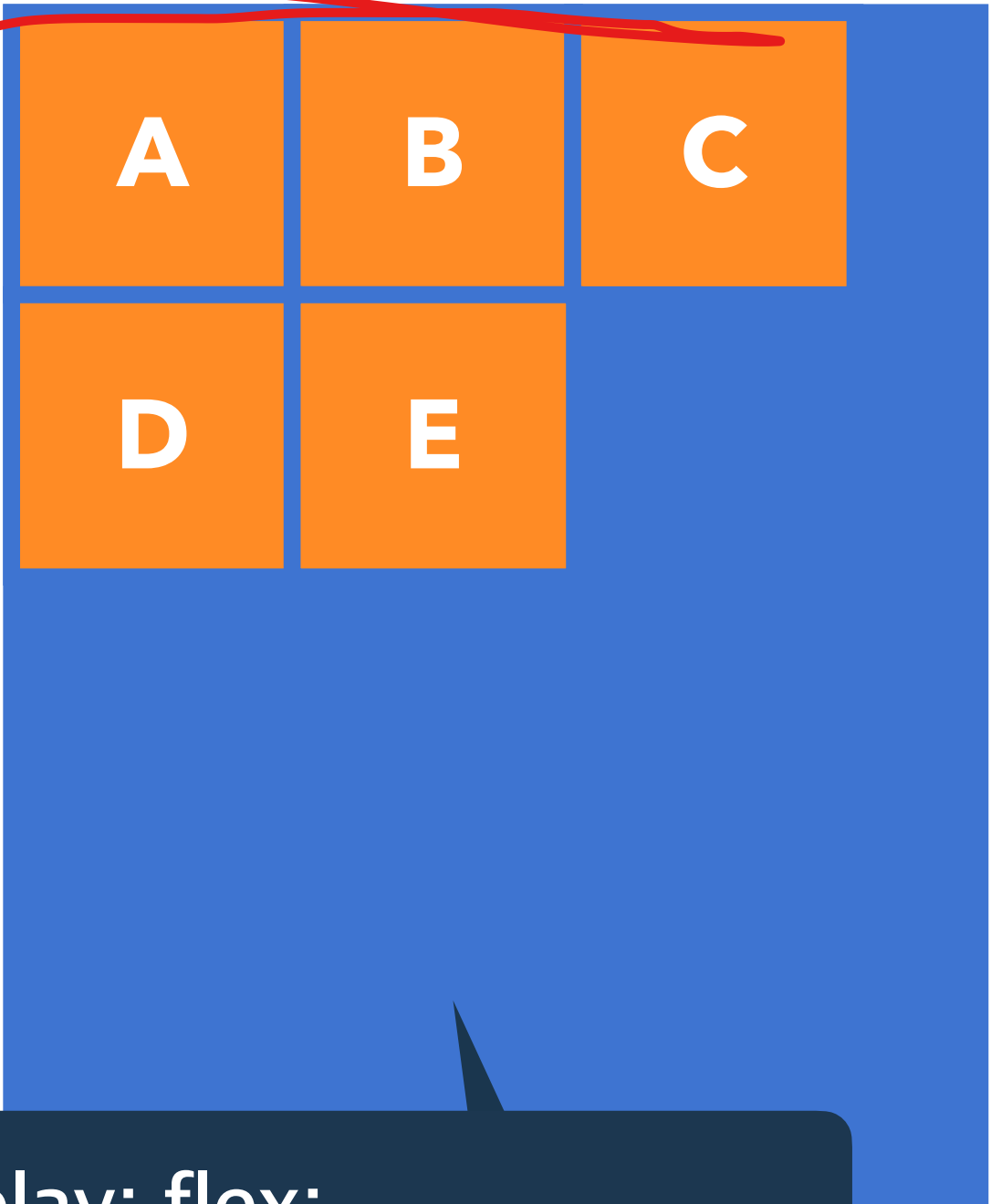
width: 100px;
height: 100px;



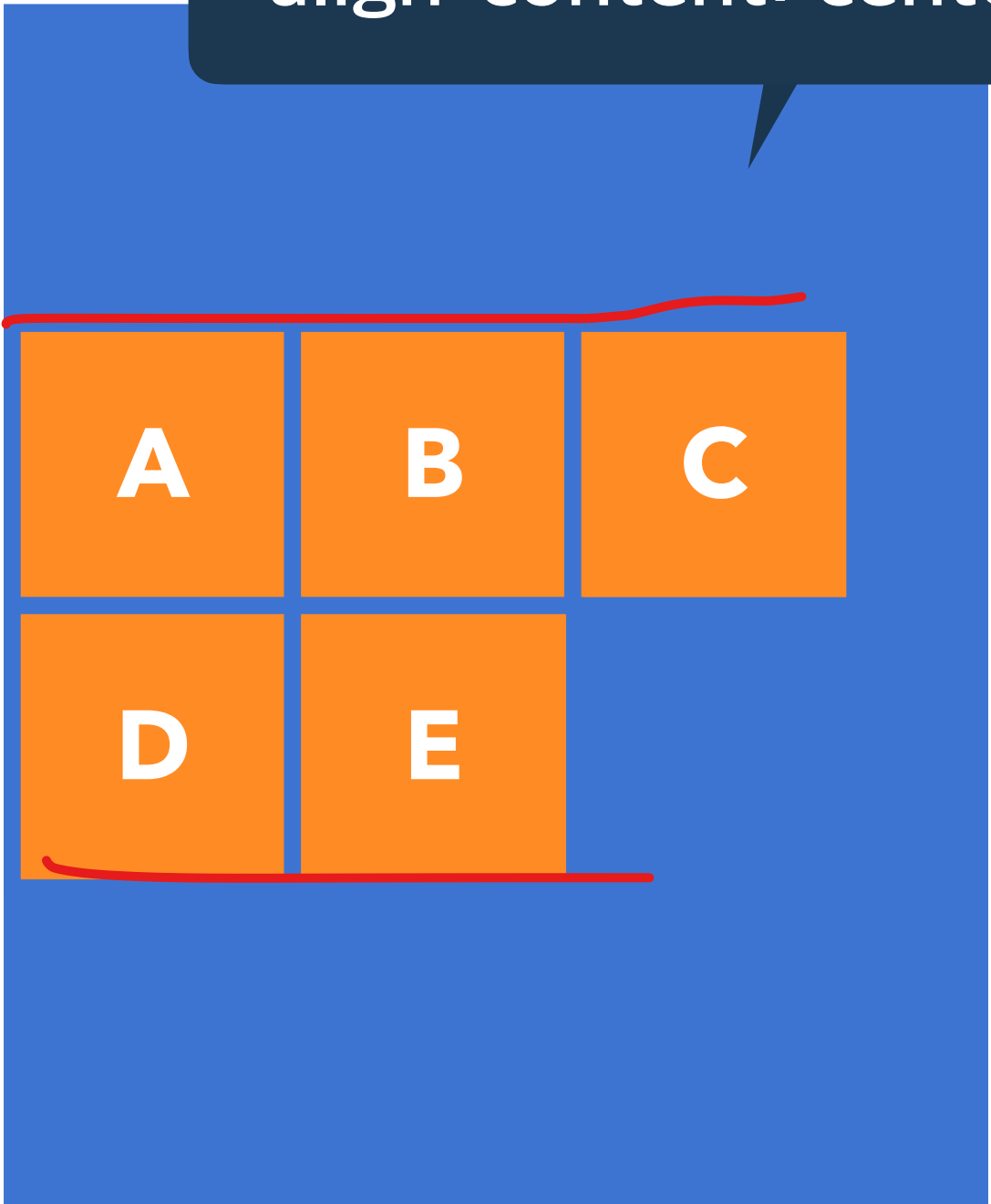
display: flex;
flex-wrap: wrap;

display: flex;
flex-wrap: wrap;
align-content: flex-start;

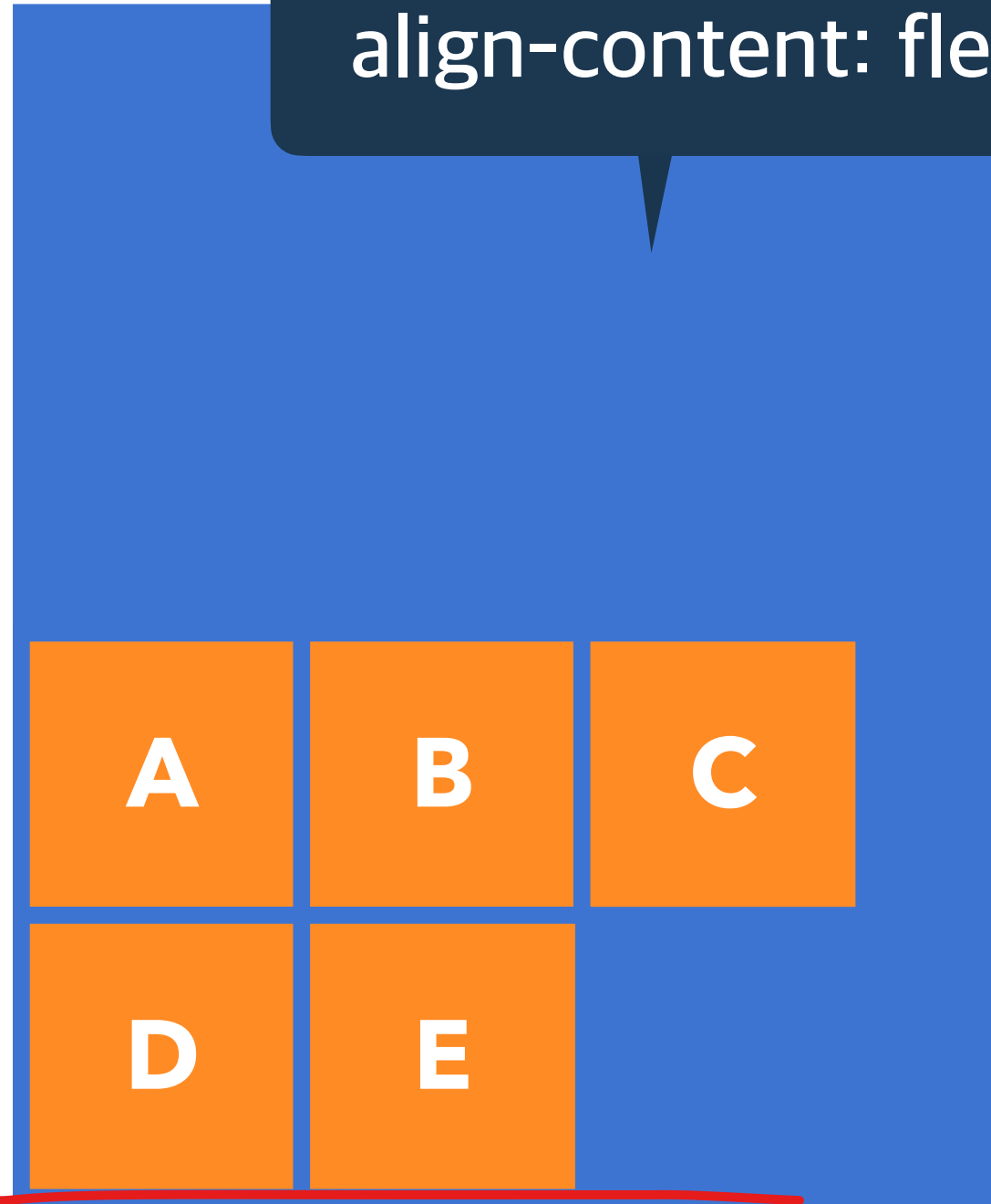
`display: flex;`
`flex-wrap: wrap;`
`align-content: flex-start;`

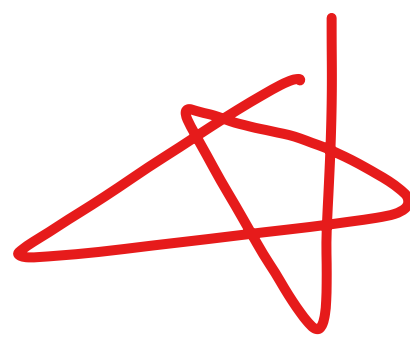


`display: flex;`
`flex-wrap: wrap;`
`align-content: center;`



`display: flex;`
`flex-wrap: wrap;`
`align-content: flex-end;`





교차 축의 한 줄 정렬 방법



align-items

stretch

Flex Items를 교차 축으로 늘림

flex-start

Flex Items를 각 줄의 시작점으로 정렬

flex-end

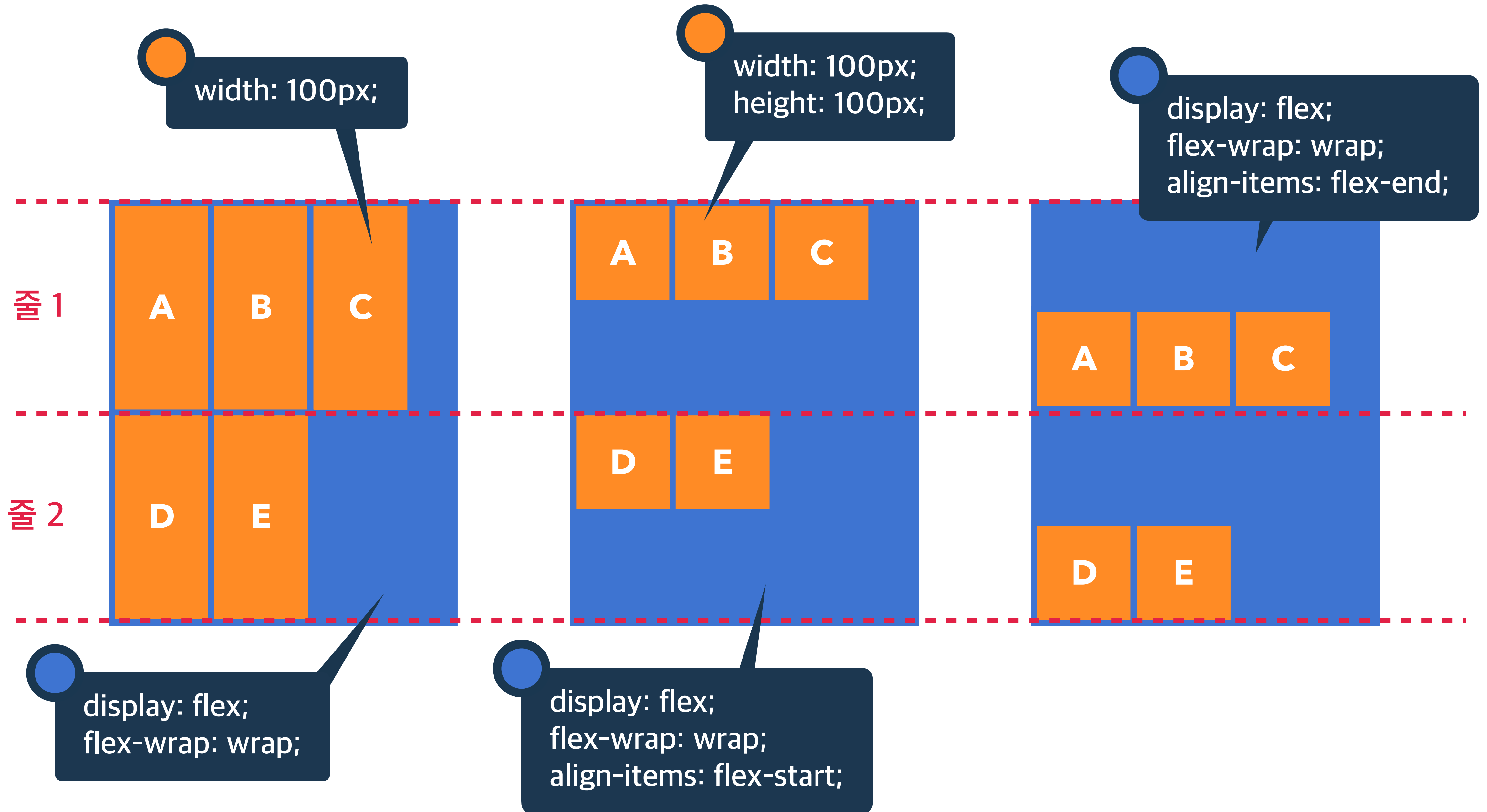
Flex Items를 각 줄의 끝점으로 정렬

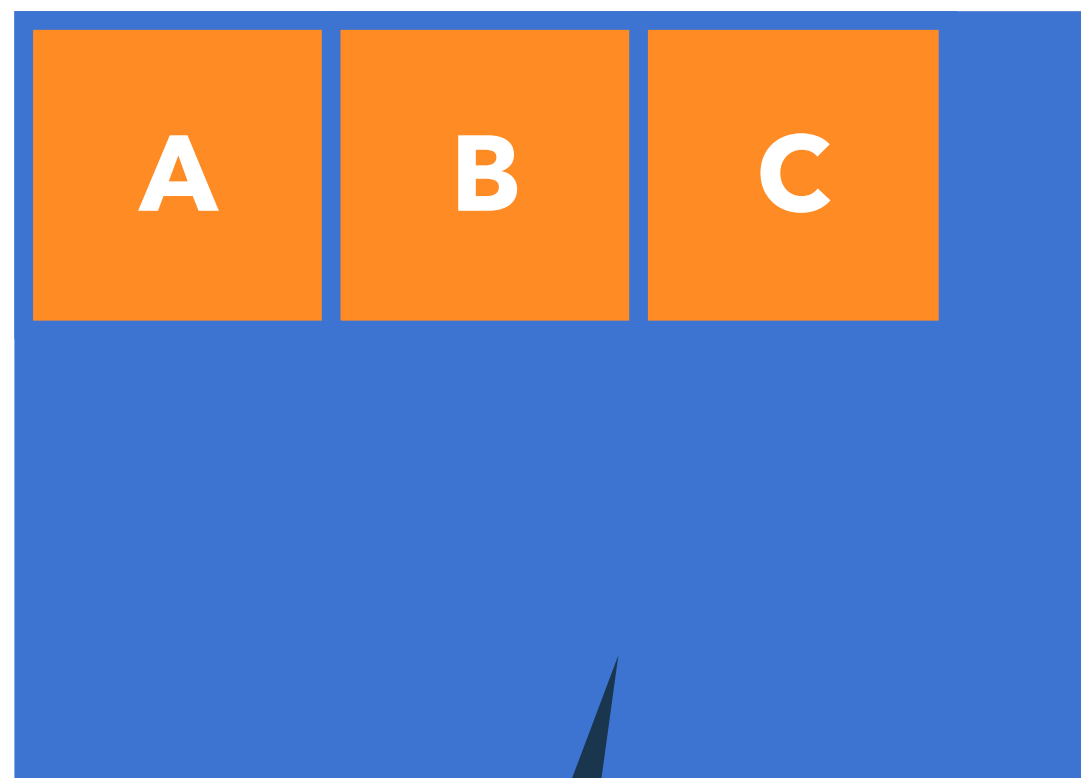
center

Flex Items를 각 줄의 가운데 정렬

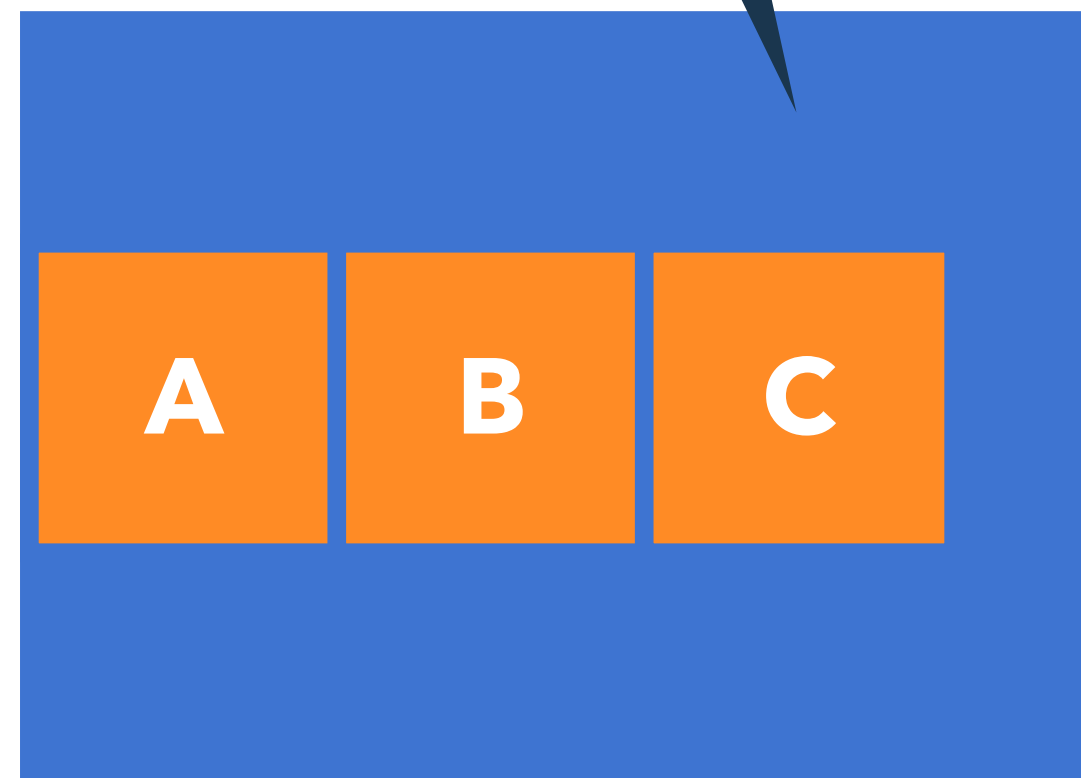
baseline

Flex Items를 각 줄의 문자 기준선에 정렬

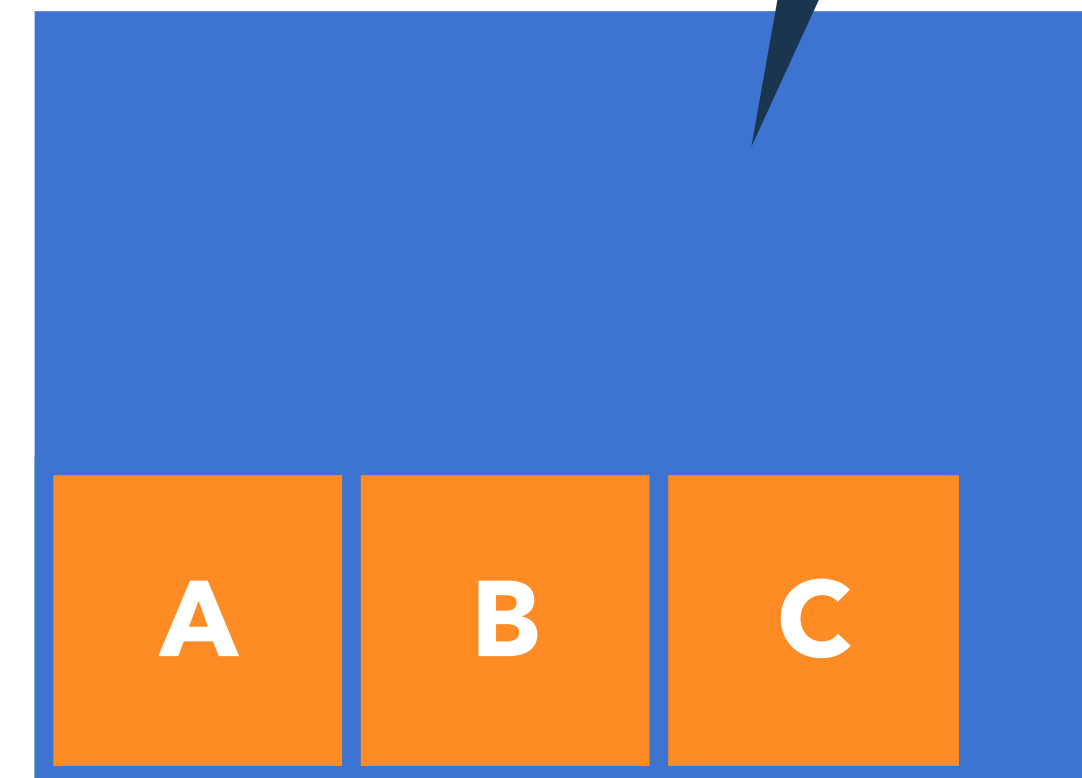




display: flex;
align-items: flex-start;



display: flex;
align-items: center;



display: flex;
align-items: flex-end;

Flex Item의 순서

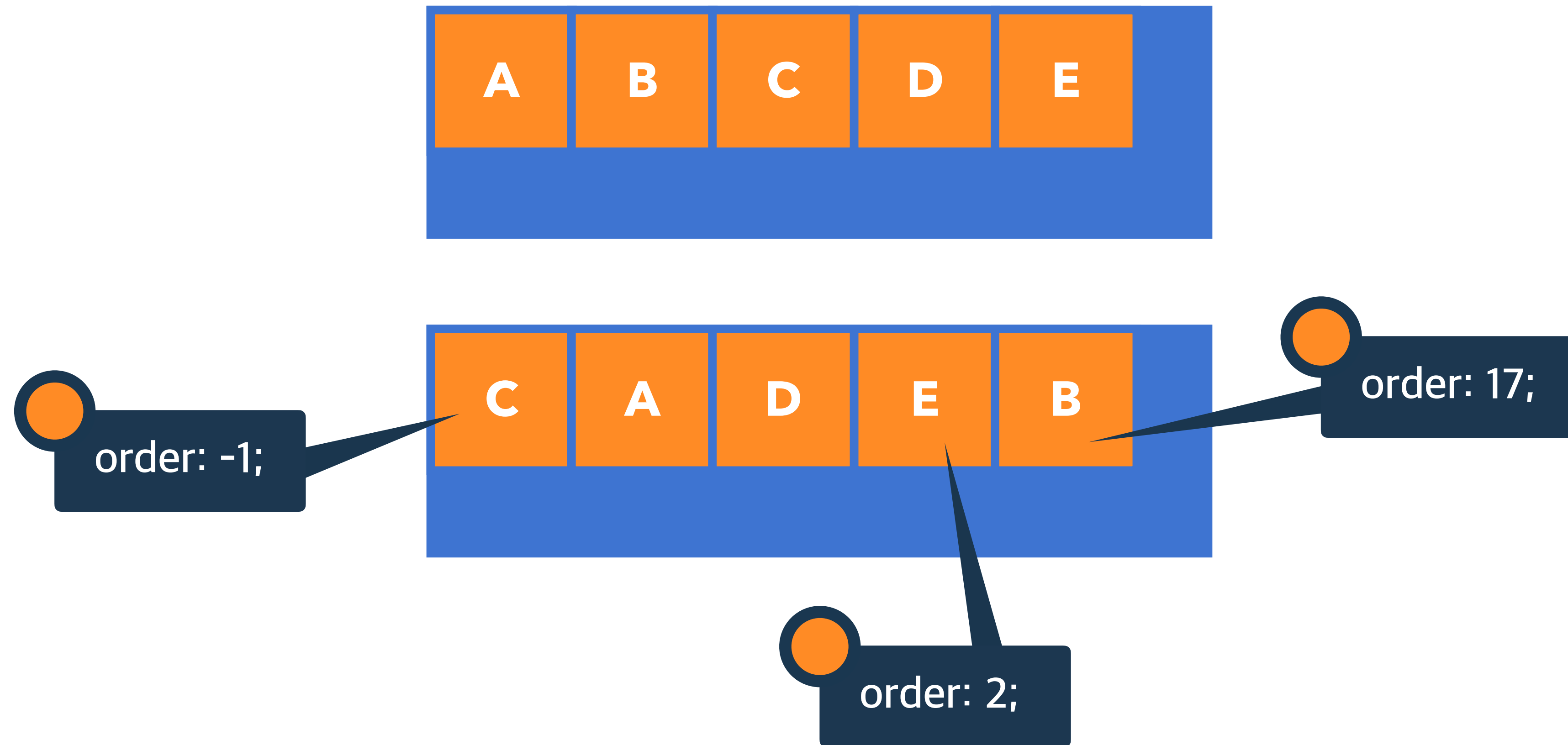
order

0

순서 없음

숫자

숫자가 작을 수록 먼저



-> 숫자가 작은 것부터 큰 순서대로 정렬됨

Flex Item의 증가 너비 비율

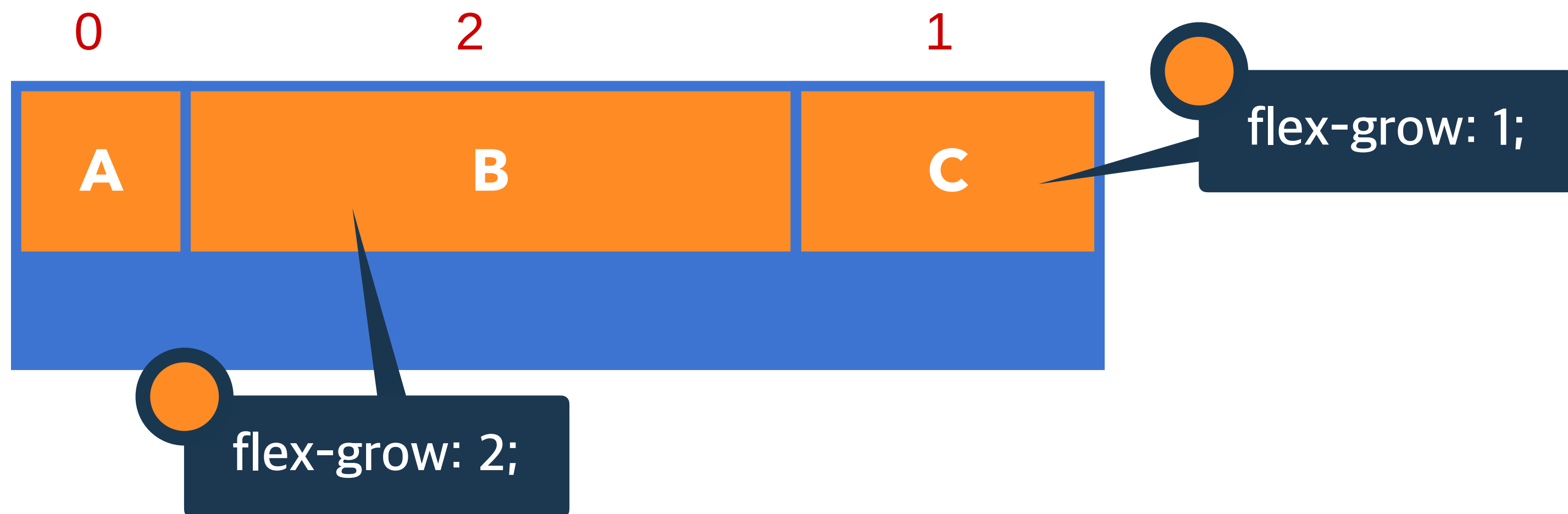
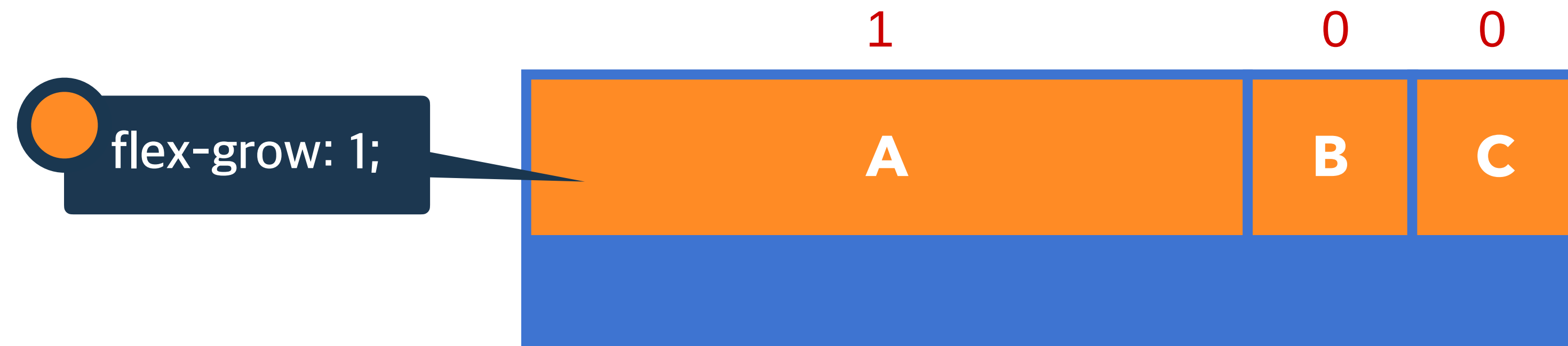
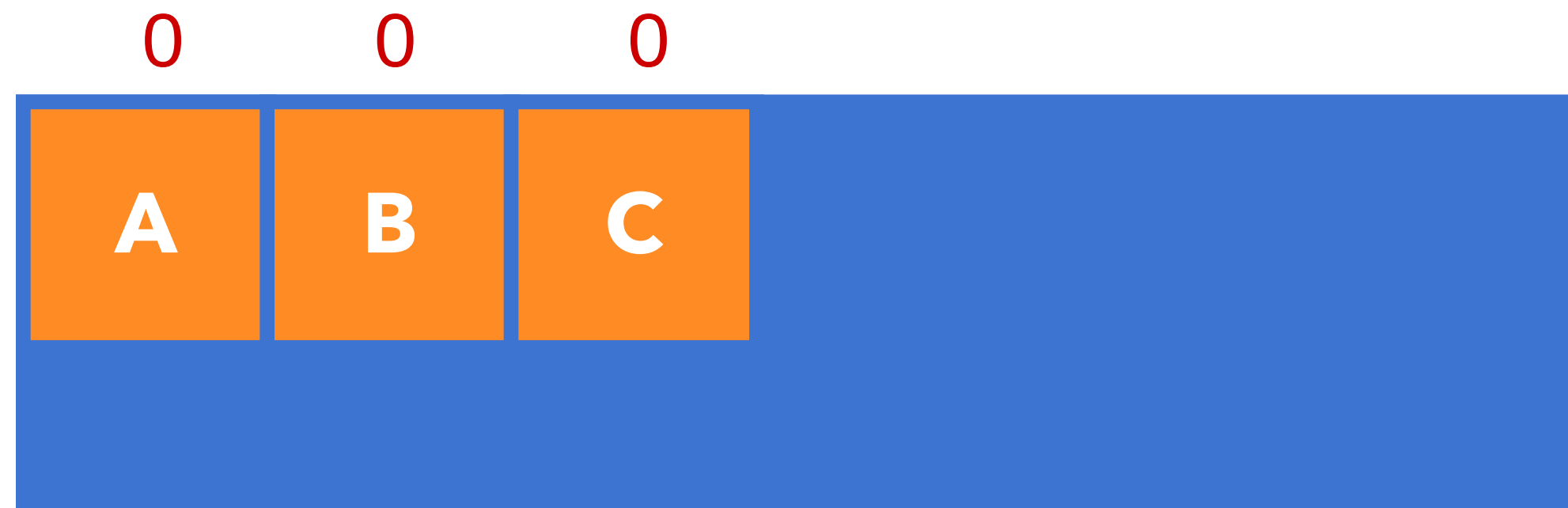
flex-grow

0

증가 비율 없음

숫자

증가 비율



Flex Item의 감소 너비 비율

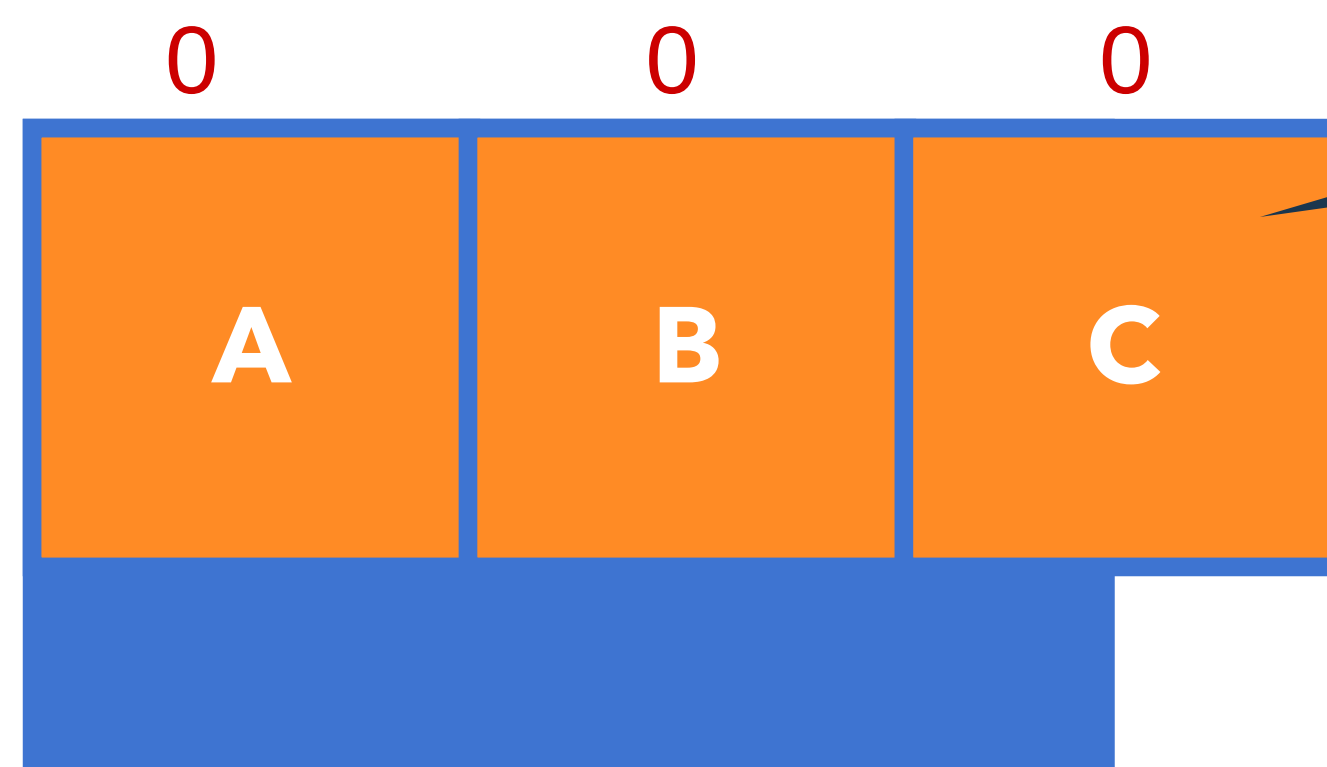
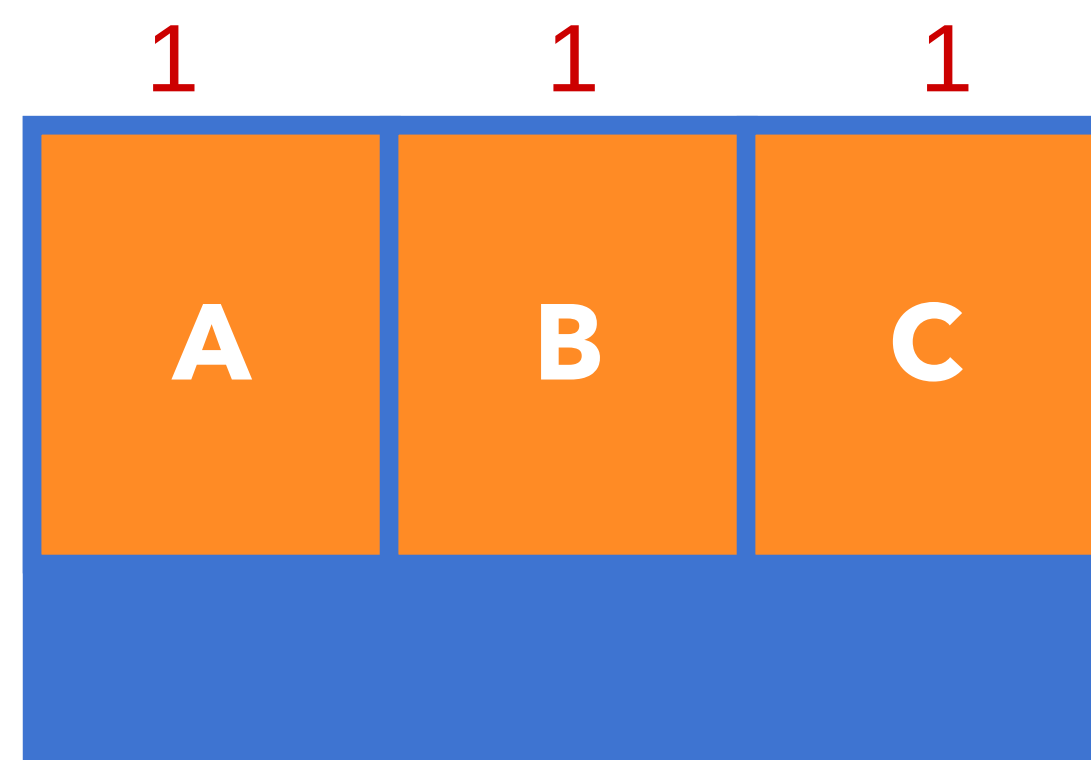
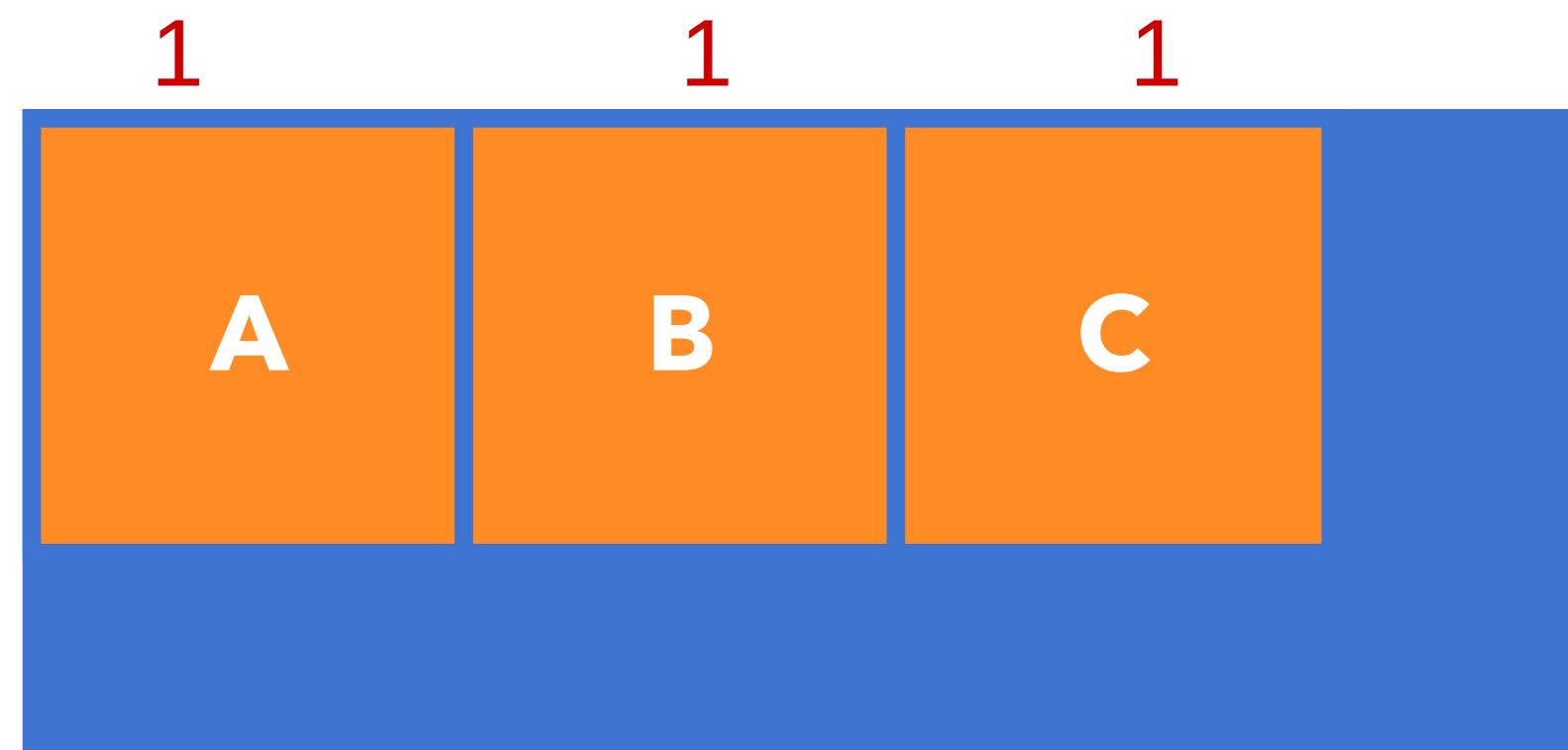
flex-shrink

1

Flex Container 너비에 따라 감소 비율 적용

숫자

감소 비율



flex-shrink: 0;

Flex Item의 공간 배분 전 기본 너비

flex-basis

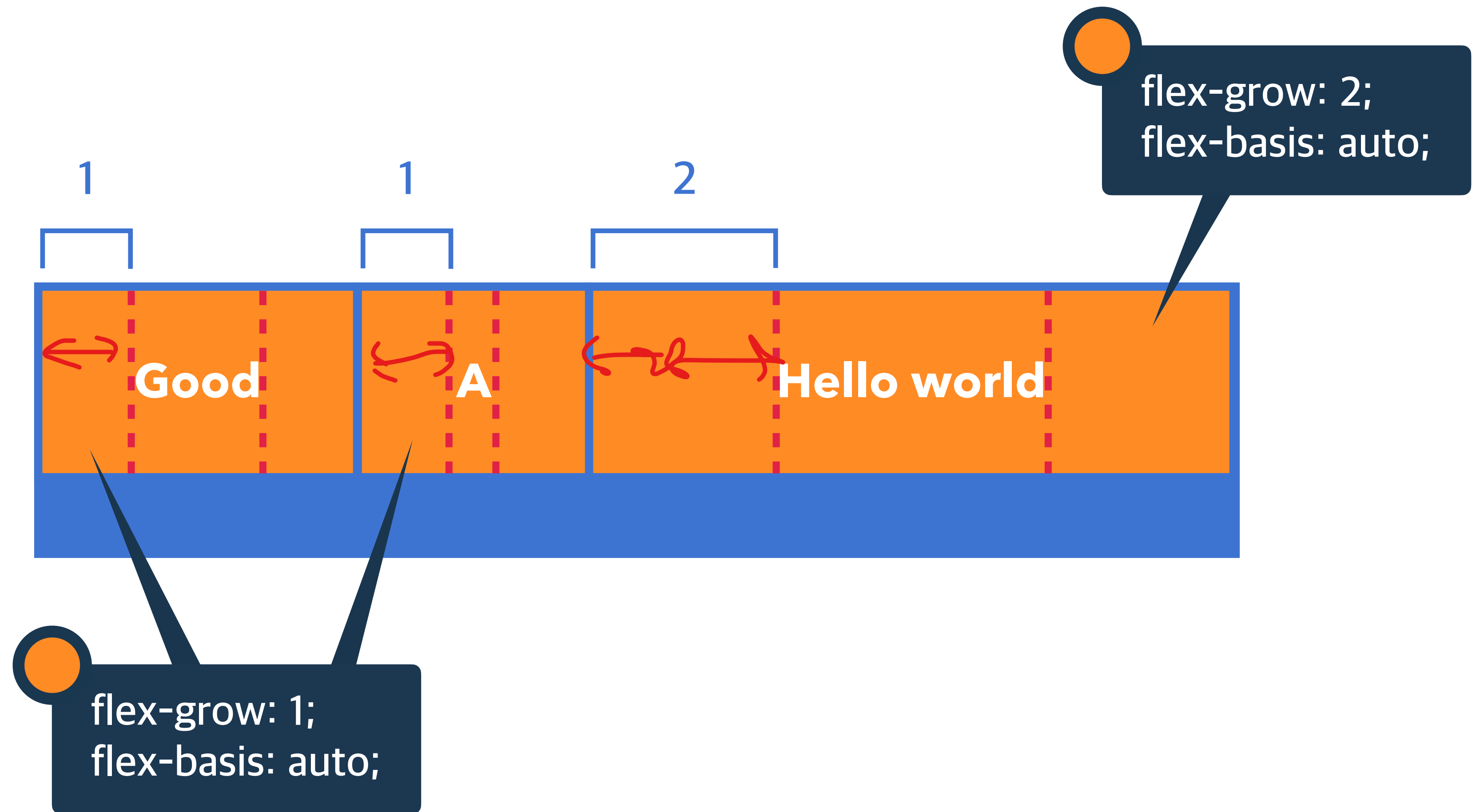
-> 애를 제외한 나머지 부분이 비율로 쓰임

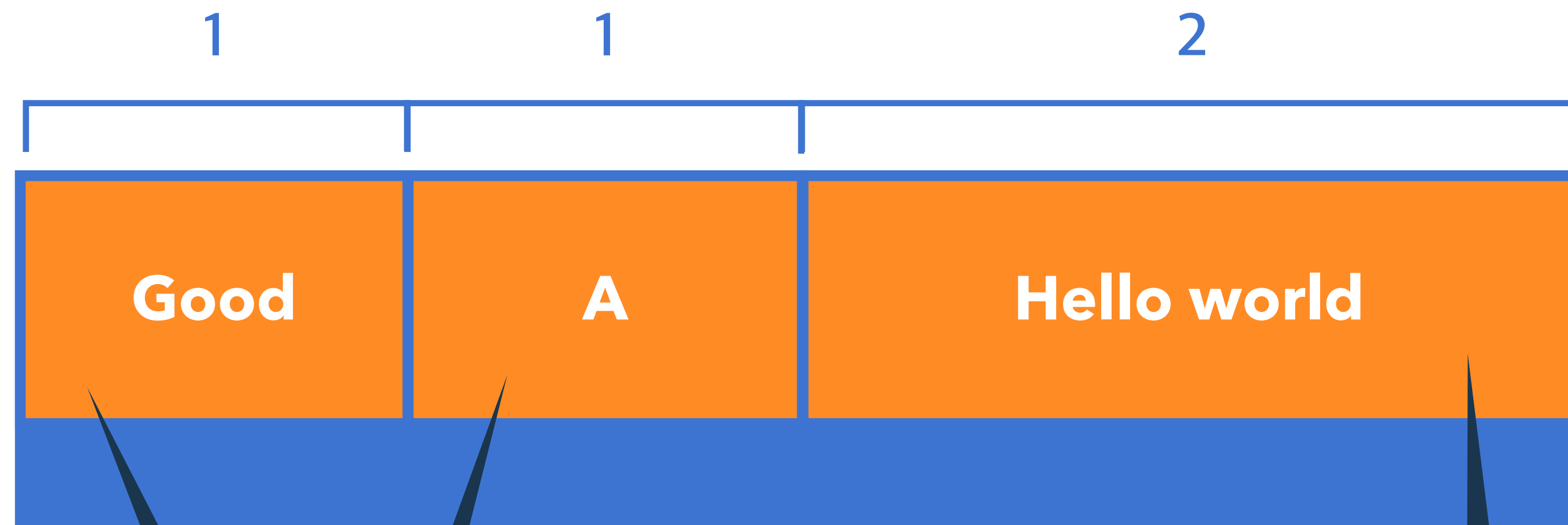
auto

요소의 Content 너비

단위

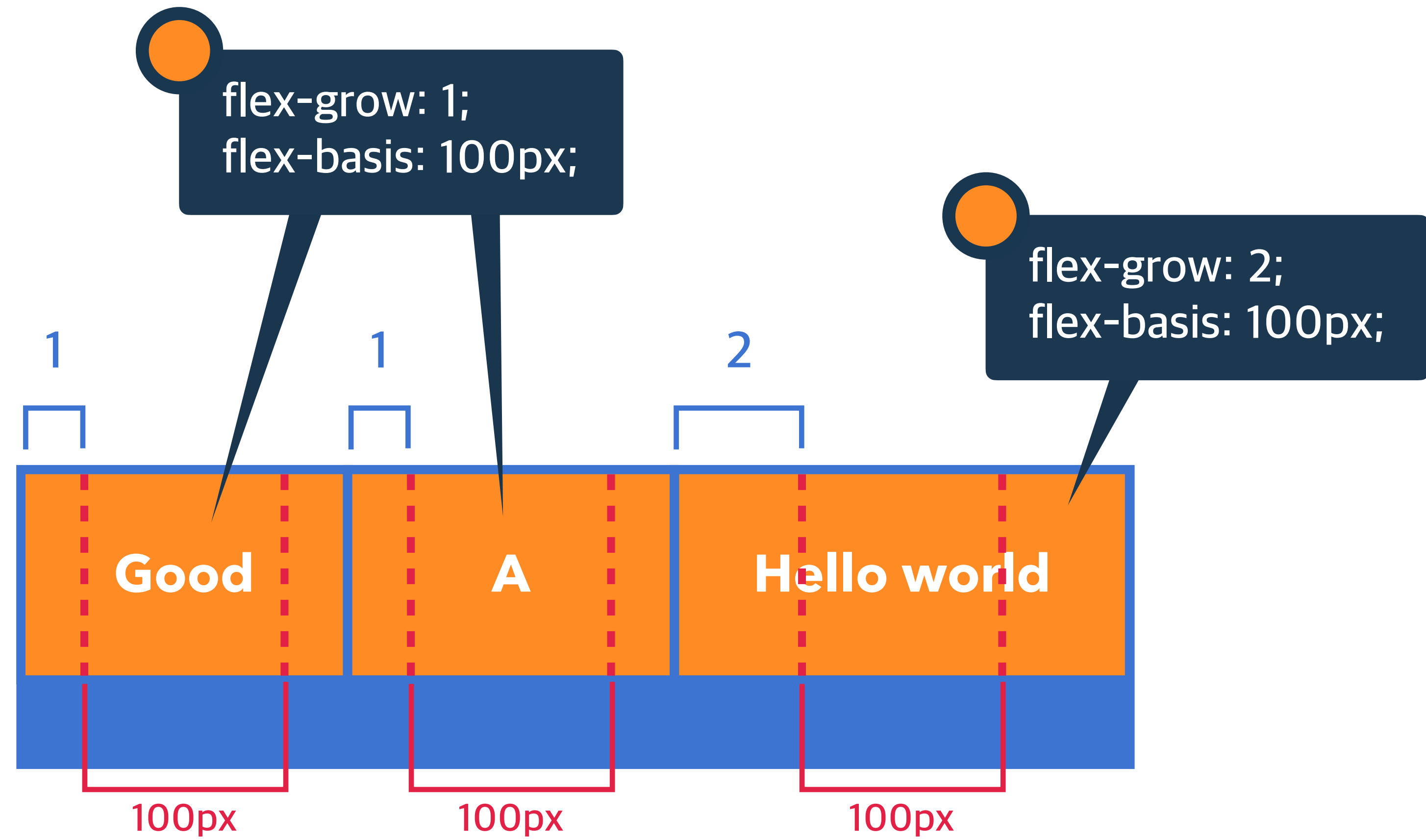
px, em, rem 등 단위로 지정





`flex-grow: 1;`
`flex-basis: 0;`

`flex-grow: 2;`
`flex-basis: 0;`



전화

단축형으로 작성할 때,
필수 포함 속성!

요소의 전환(시작과 끝) 효과를 지정하는 단축 속성

transition: 속성명 **지속시간** 타이밍함수 대기시간;

transition-property

transition-duration

transition-timing-function

transition-delay

전환 효과를 사용할 속성 이름을 지정

transition-property

all

모든 속성에 적용

속성이름

전환 효과를 사용할 속성 이름 명시

전환 효과의 지속시간을 지정

transition-duration

0s

전환 효과 없음

시간

지속시간(s)을 지정

전환 효과의 타이밍(Easing) 함수를 지정

transition-timing-function



전환 효과가 몇 초 뒤에 시작할지 대기시간을 지정

transition-delay

0s

대기시간 없음

시간

대기시간(s)을 지정

변화

요소의 변환 효과

transform: 변환함수1 변환함수2 변환함수3 ... ;

transform: 원근법 이동 크기 회전 기울임;

2D 변환 함수

px

translate(x, y) 이동(x축, y축)

translateX(x) 이동(x축)

translateY(y) 이동(y축)

scale(x, y) 크기(x축, y축)

scaleX(x) 크기(x축)

scaleY(y) 크기(y축)

없음(배수)

deg

rotate(degree) 회전(각도)

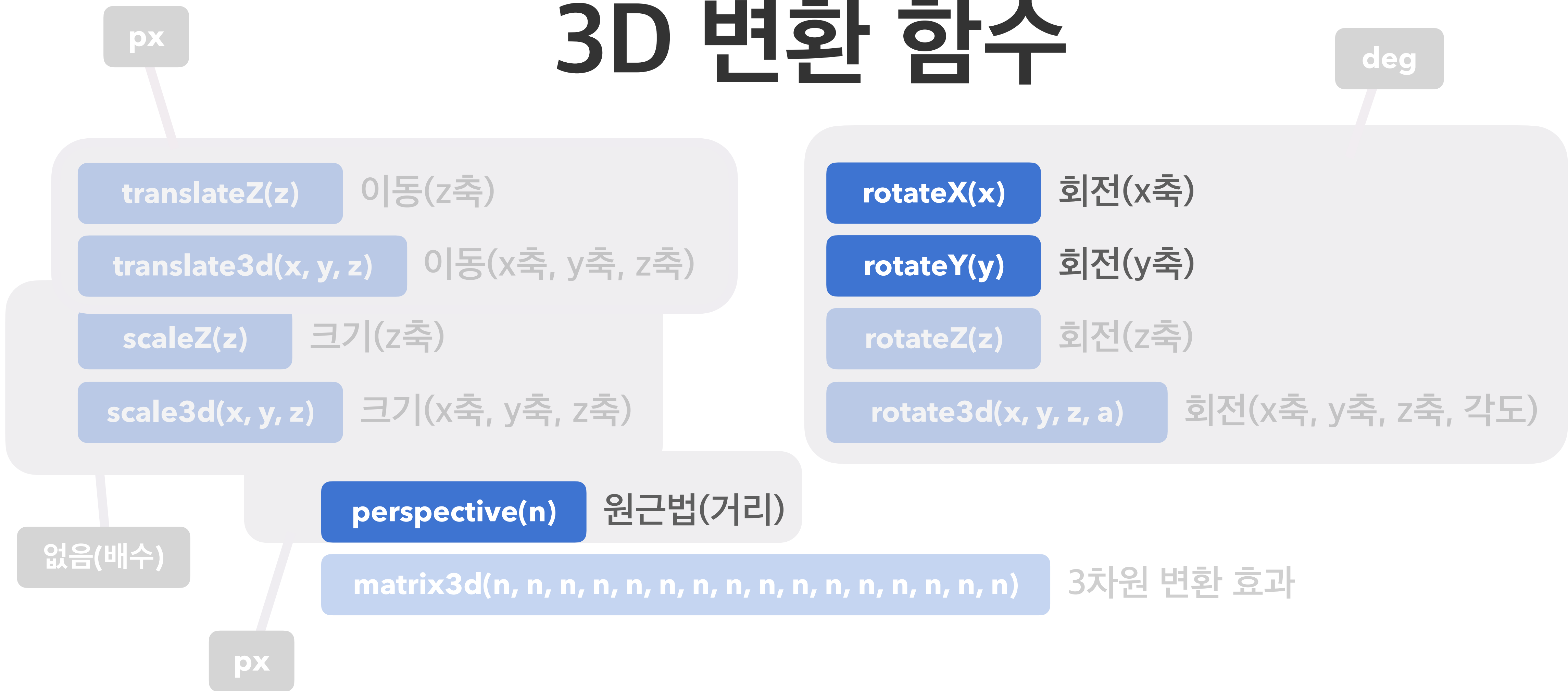
skew(x, y) 기울임(x축, y축)

skewX(x) 기울임(x축)

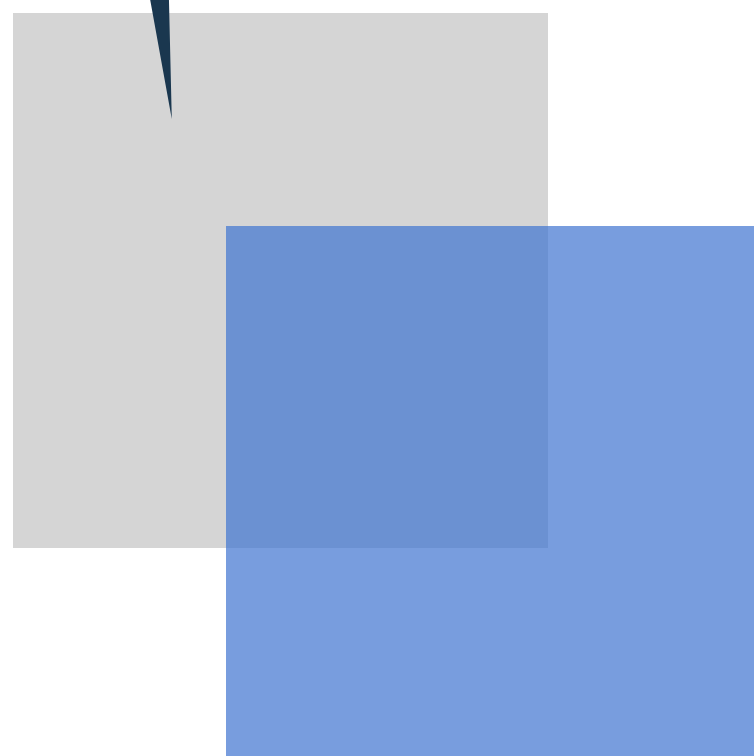
skewY(y) 기울임(y축)

matrix(n,n,n,n,n,n) 2차원 변환 효과

3D 변환 함수



transform: translate(40px, 40px);



transform: translateY(40px);



transform: translateX(40px);



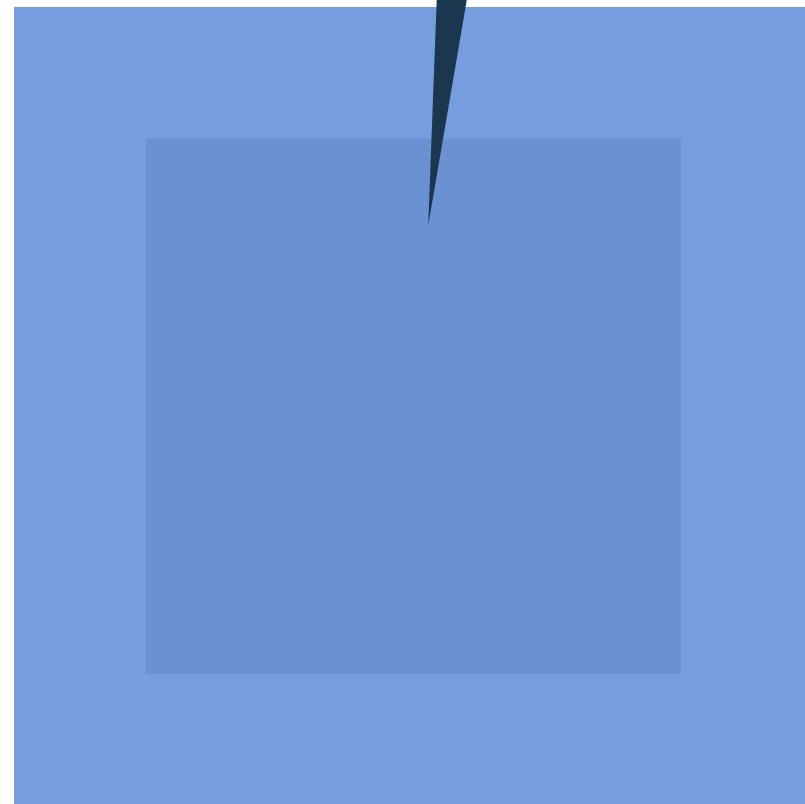
transform: translate(40px, 0);

또는

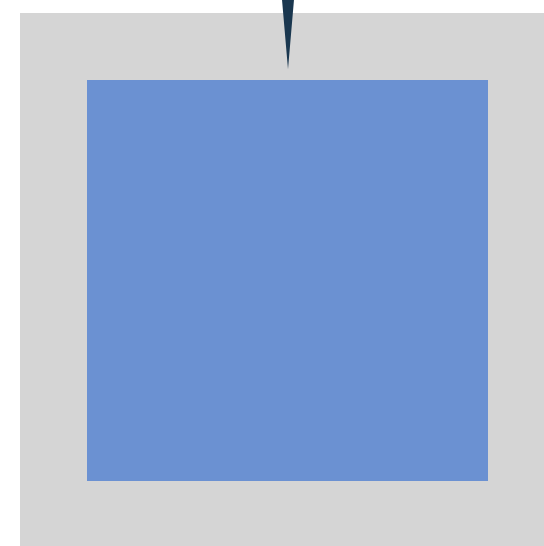

```
transform: scale(1.5, 1.5);
```

또는

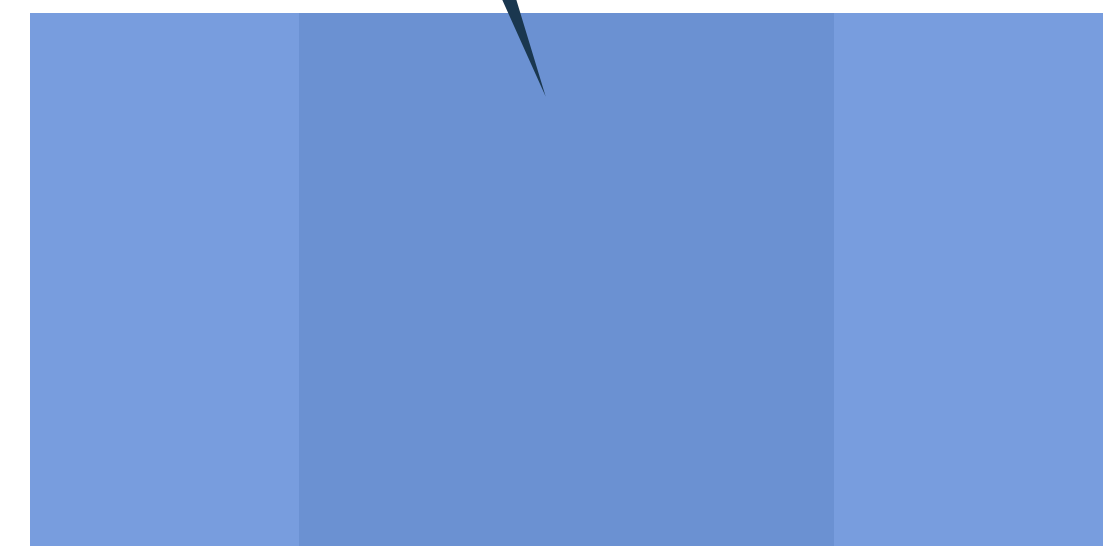
```
transform: scale(1.5);
```



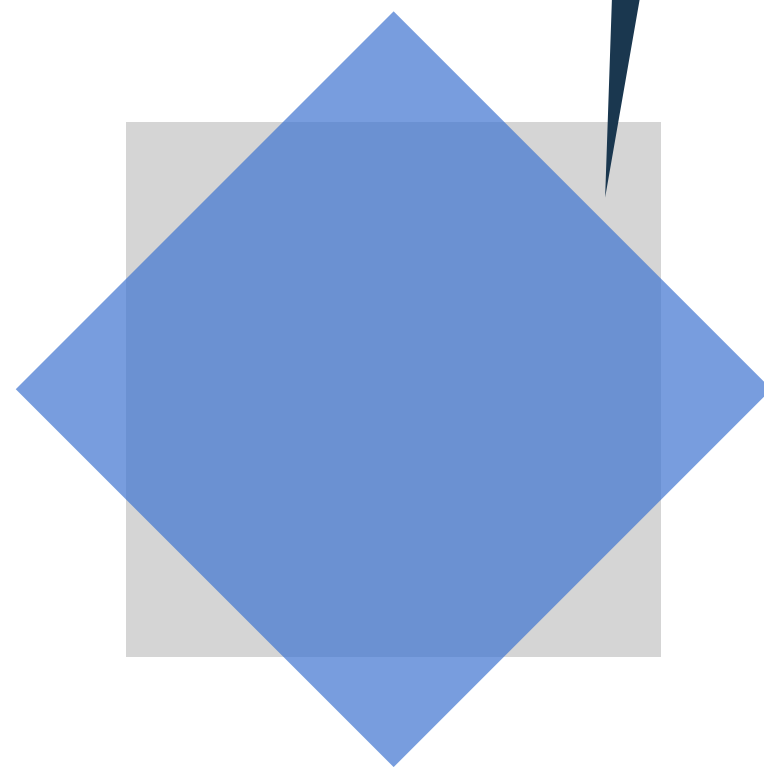
```
transform: scale(0.7);
```



```
transform: scaleX(2);
```

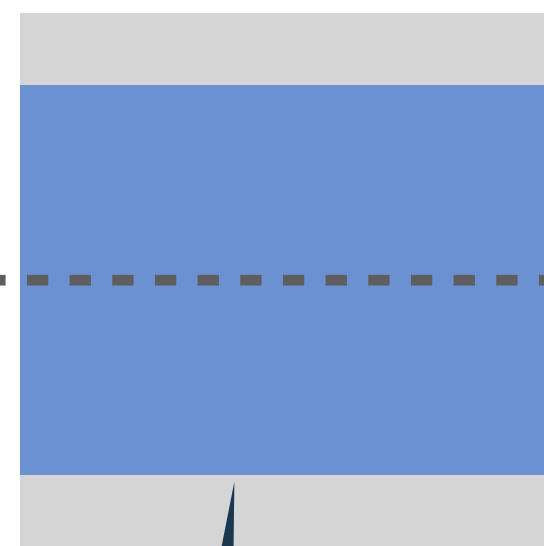


`transform: rotate(45deg);`



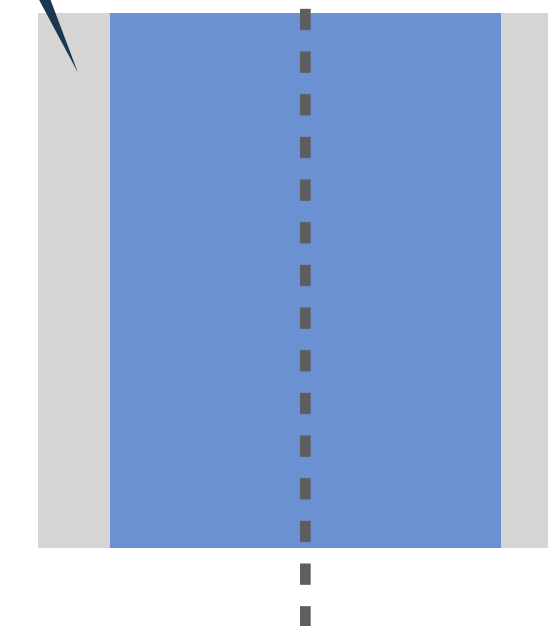
x

`transform: rotateX(45deg);`



`transform: rotateY(45deg);`

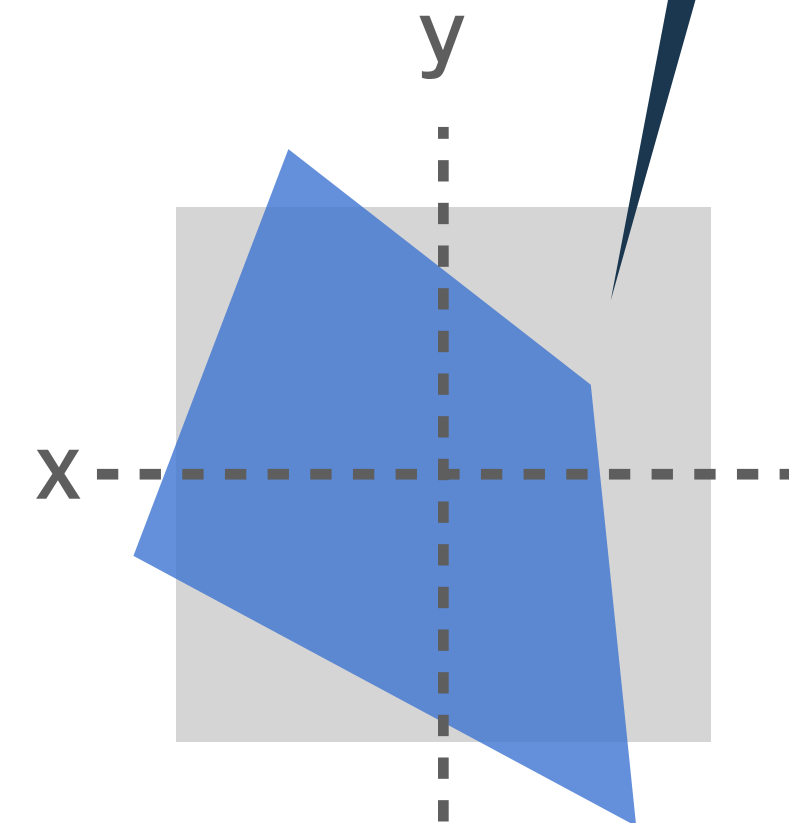
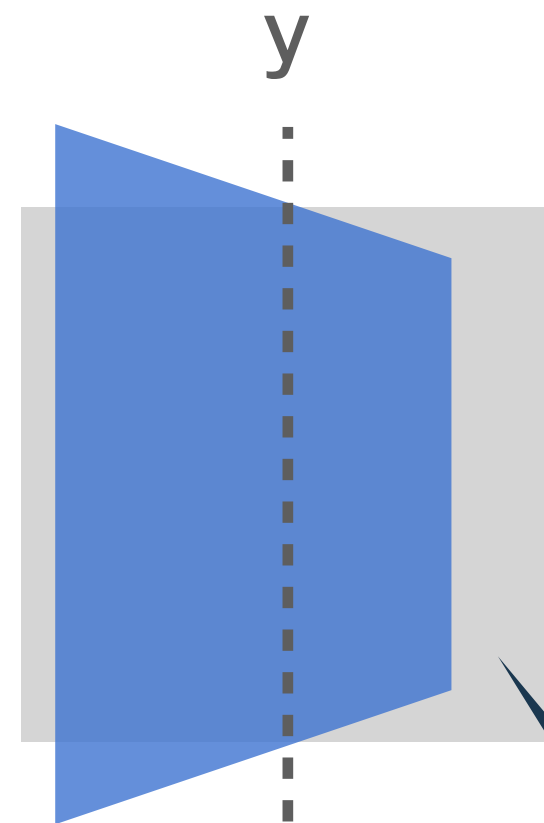
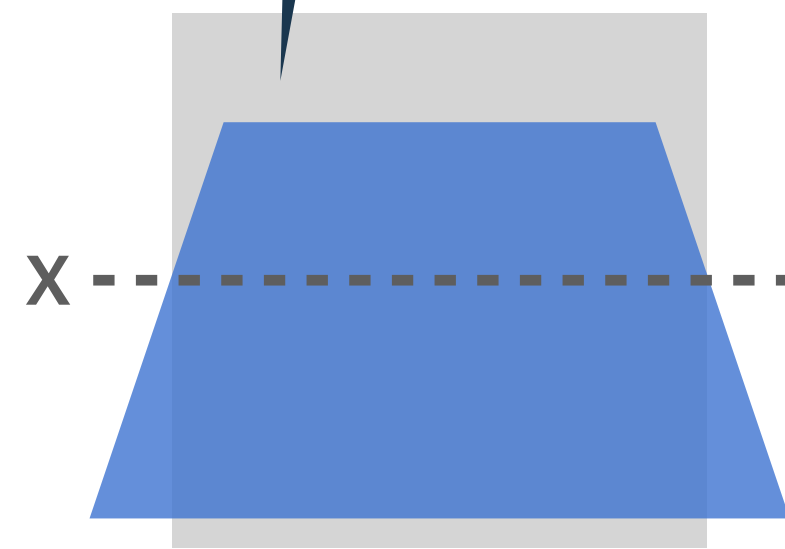
y



원근법 함수는
제일 앞에 작성해야 합니다!!

```
transform: perspective(500px) rotateX(45deg) rotateY(45deg);
```

```
transform: perspective(500px) rotateX(45deg);
```



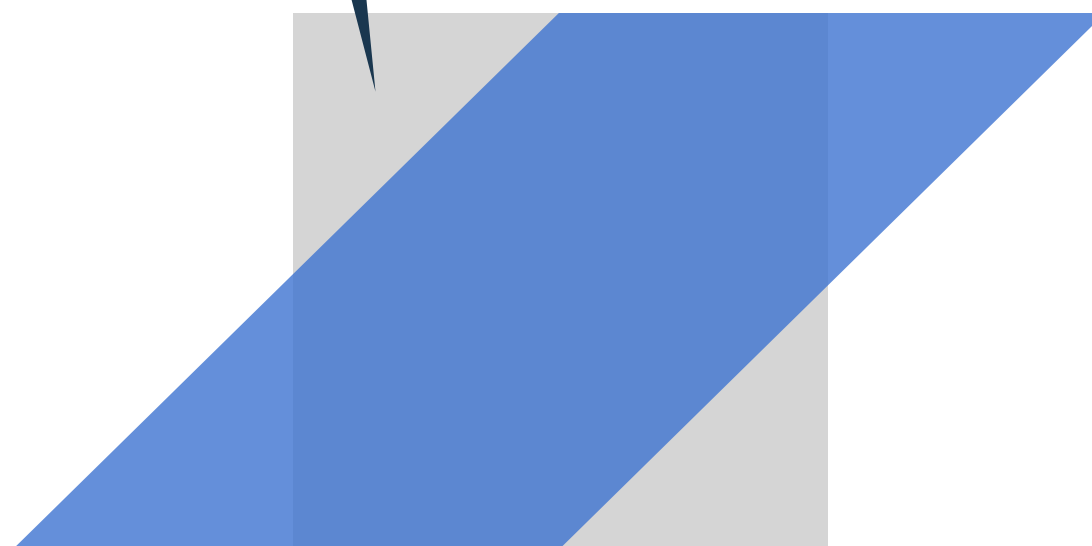
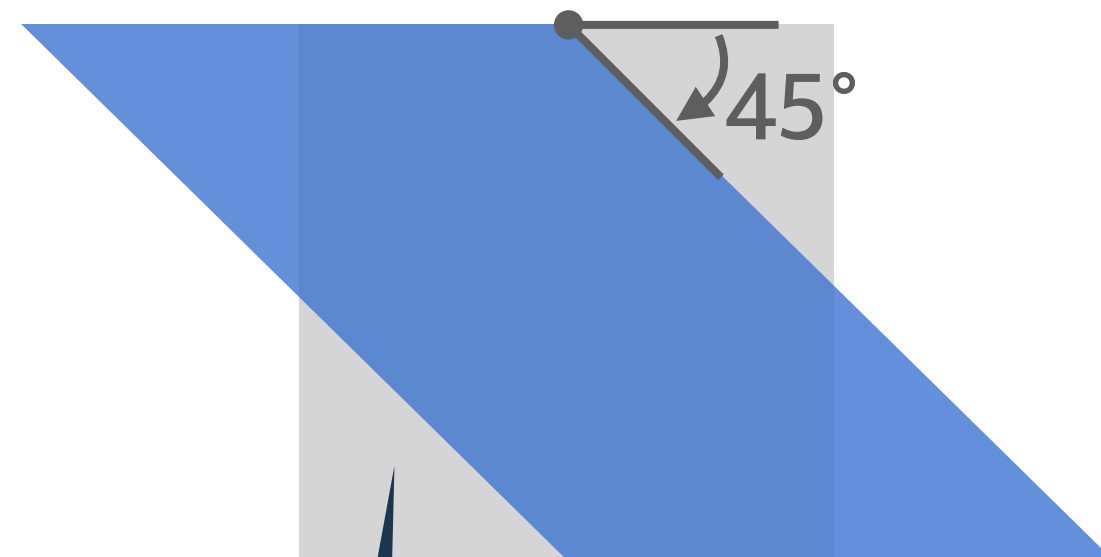
```
transform: perspective(500px) rotateY(45deg);
```


`transform: skewY(45deg);`

`transform: skewX(-45deg);`

`transform: skewX(45deg);`

또는
`transform: skew(45deg, 0);`



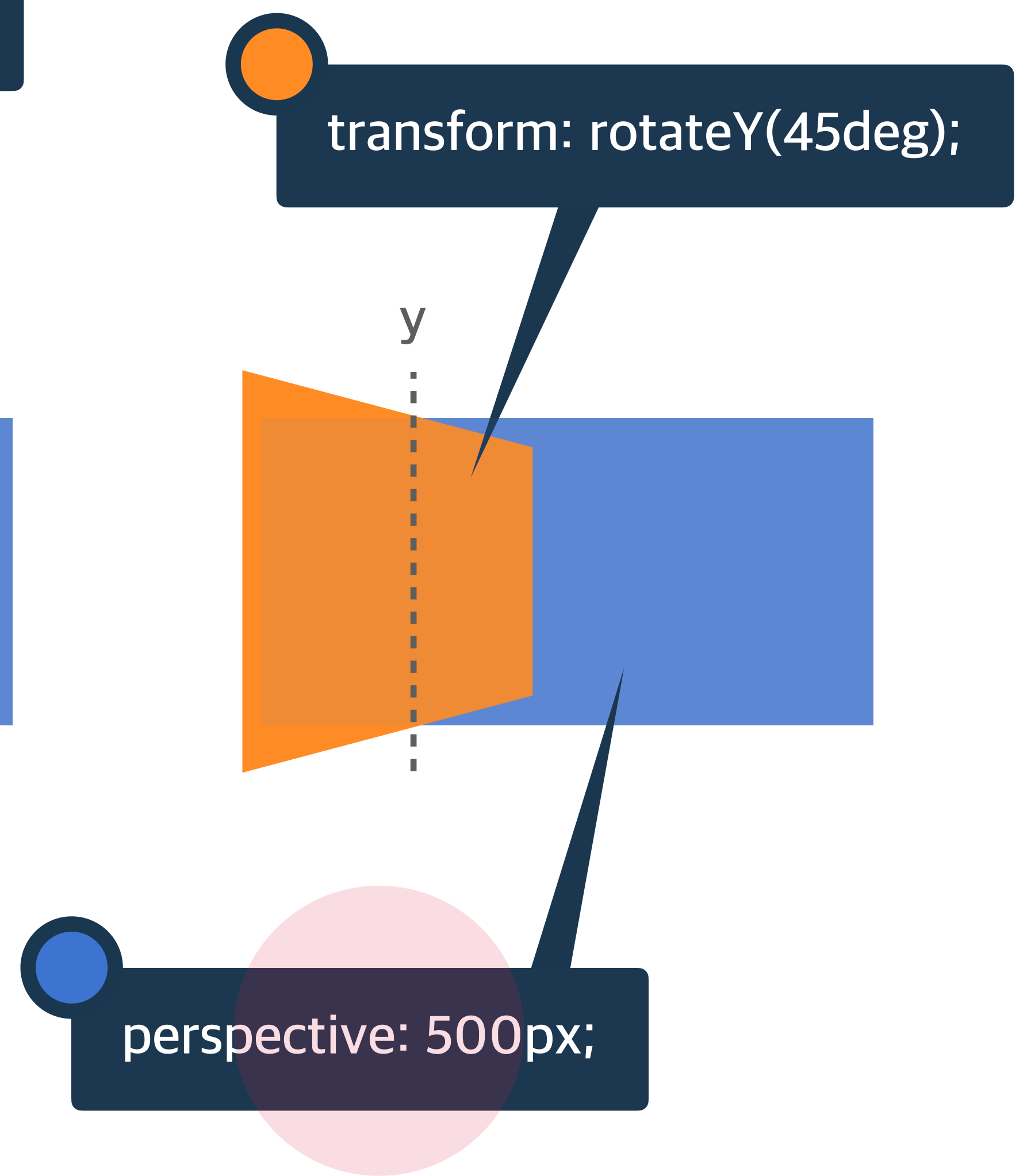
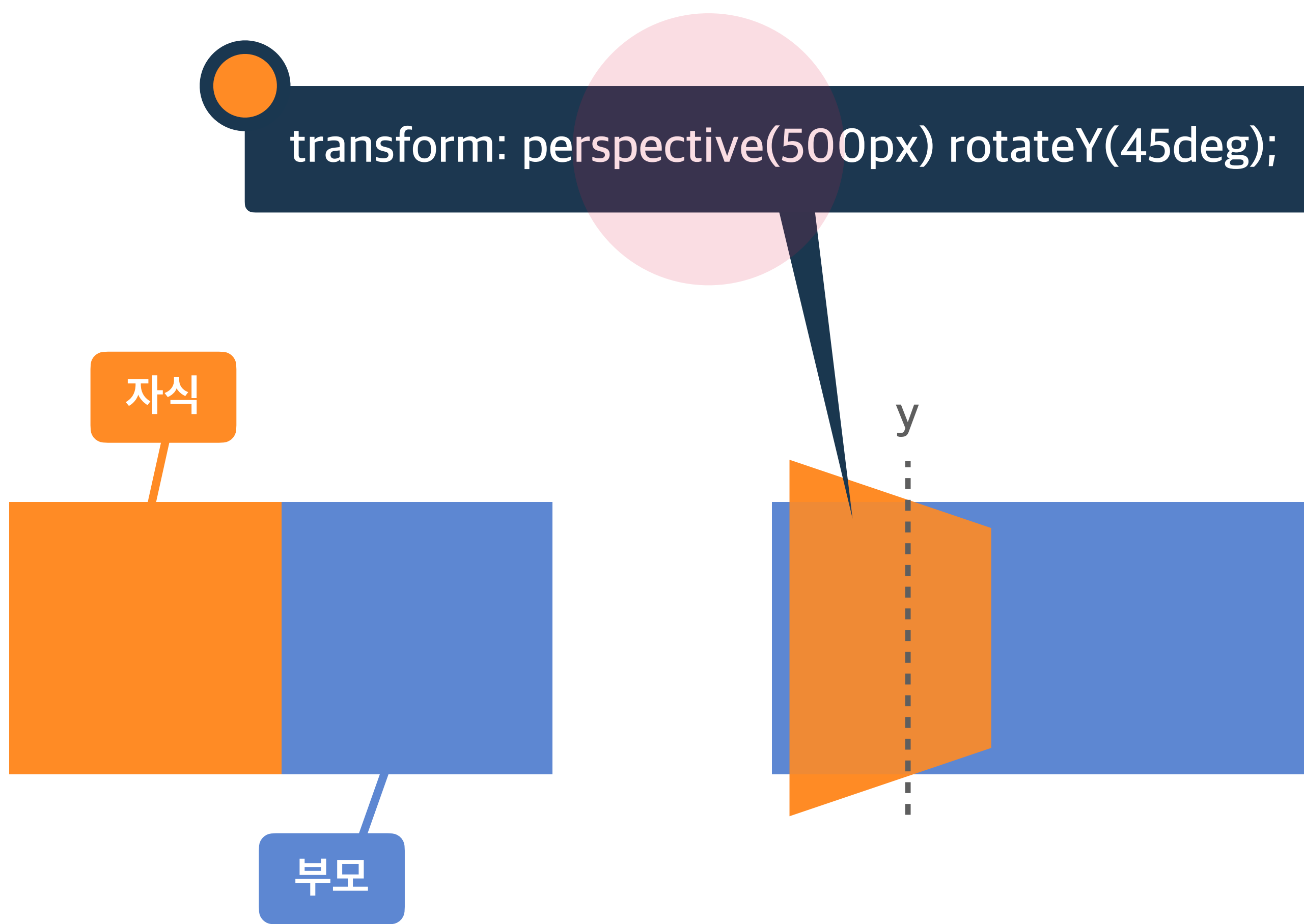
하위 요소를 관찰하는 원근 거리를 지정

perspective

단위 px 등 단위로 지정

perspective 속성과 함수 차이점

속성 / 함수	적용 대상	기준점 설정
<code>perspective: 600px;</code>	관찰 대상의 부모	<code>perspective-origin</code>
<code>transform: perspective(600px)</code>	관찰 대상	<code>transform-origin</code>



3D 변환으로 회전된 요소의 뒷면 숨김 여부

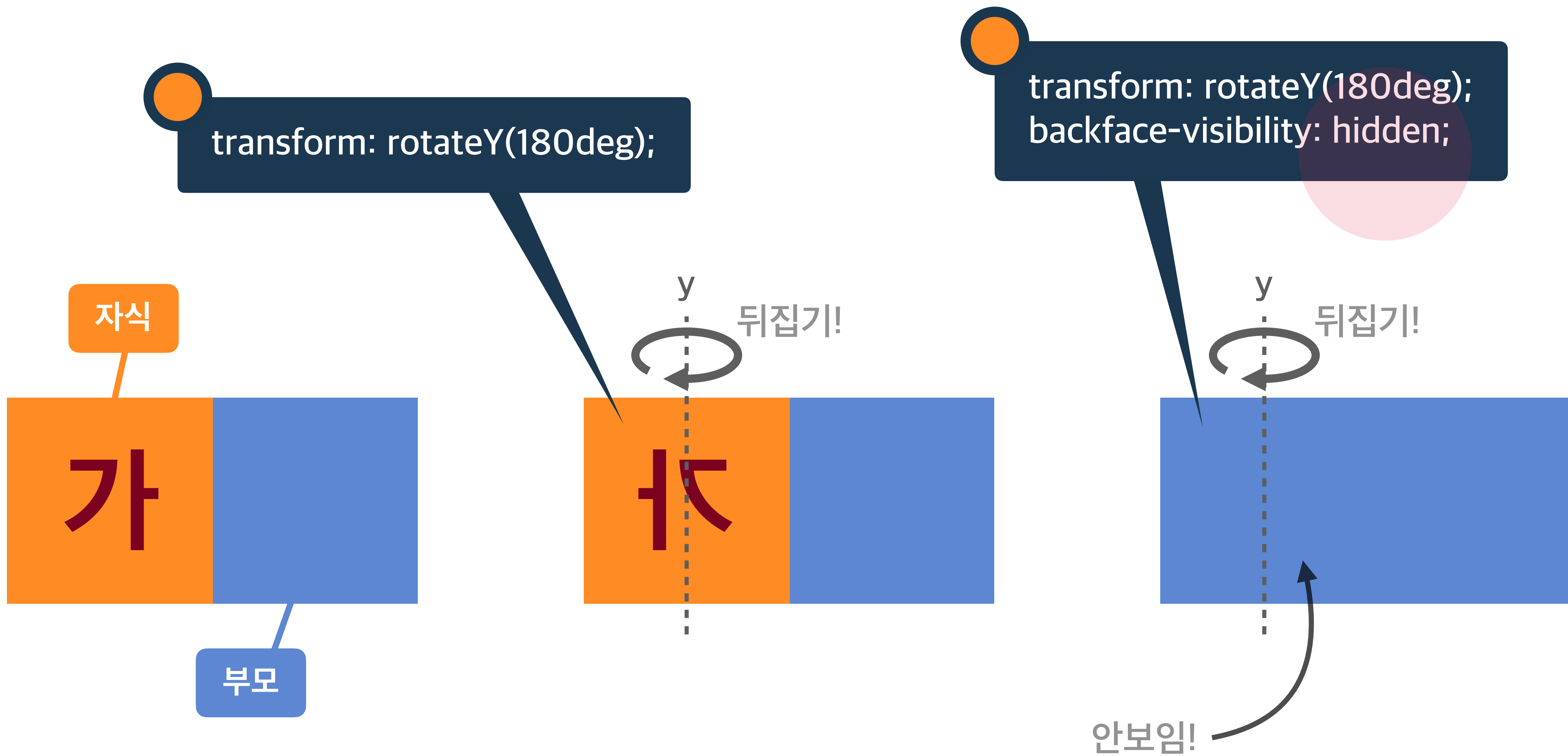
backface-visibility

visible

뒷면 보임

hidden

뒷면 숨김



JavaScript 선행 학습

VS Code에서 main.js 연결 테스트!

표기법

dash-case(kebab-case)

snake_case

camelCase

ParcelCase

HTML

CSS

dash-case(kebab-case)

the-quick-brown-fox-jumps-over-the-lazy-dog

HTML

CSS

snake_case

the_quick_brown_fox_jumps_over_the_lazy_dog

JS

camelCase

theQuickBrownFoxJumpsOverTheLazyDog

JS

PascalCase

TheQuickBrownFoxJumpsOverTheLazyDog

Zero-based Numbering

0 기반 번호 매기기!

특수한 경우를 제외하고 0부터 숫자를 시작합니다.

```
let fruits = ['Apple', 'Banana', 'Cherry'];
```

```
console.log(fruits[0]); // 'Apple'
```

```
console.log(fruits[1]); // 'Banana'
```

```
console.log(fruits[2]); // 'Cherry'
```

```
console.log(new Date('2021-01-30').getDay()); // 6, 토요일
```

```
console.log(new Date('2021-01-31').getDay()); // 0, 일요일
```

```
console.log(new Date('2021-02-01').getDay()); // 1, 월요일
```

주석

Comments

데이터 종류(자료형)

String

Number

Boolean

Undefined

Null

Object

Array

```
let a = 123;  
const n = obj.name;  
document.querySelector('.data-abc')  
{ name: 'Heropy', age: 85 }  
function mount(params) {  
  return this;  
}
```




```
// String(문자 데이터)
// 따옴표를 사용합니다.
let myName = "HEROPY";
let email = 'thesecon@gmail.com';
let hello = `Hello ${myName}?!`

console.log(myName); // HEROPY
console.log(email); // thesecon@gmail.com
console.log(hello); // Hello HEROPY?!
```

```
// Number(숫자 데이터)  
// 정수 및 부동소수점 숫자를 나타냅니다.  
let number = 123;  
let opacity = 1.57;  
  
console.log(number); // 123  
console.log(opacity); // 1.57
```



```
// Boolean(불린 데이터)  
// true, false 두 가지 값밖에 없는 논리 데이터입니다.  
let checked = true;  
let isShow = false;  
  
console.log(checked); // true  
console.log(isShow); // false
```

```
// Undefined
// 값이 할당되지 않은 상태를 나타냅니다.
let undef;
let obj = { abc: 123 };

console.log(undef); // undefined
console.log(obj.abc); // 123
console.log(obj.xyz); // undefined
```

```
// Null  
// 어떤 값이 의도적으로 비어있음을 의미합니다.  
let empty = null;  
  
console.log(empty); // null
```



```
// Object(객체 데이터)
// 여러 데이터를 Key:Value 형태로 저장합니다. { }

let user = {
  // Key: Value,
  name: 'HEROPY',
  age: 85,
  isValid: true
};

console.log(user.name); // HEROPY
console.log(user.age); // 85
console.log(user.isValid); // true
```

```
// Array(배열 데이터)  
// 여러 데이터를 순차적으로 저장합니다. [ ]  
let fruits = ['Apple', 'Banana', 'Cherry'];  
  
console.log(fruits[0]); // 'Apple'  
console.log(fruits[1]); // 'Banana'  
console.log(fruits[2]); // 'Cherry'
```

변수

데이터를 저장하고 참조(사용)하는 데이터의 이름

`var, let, const`


```
// 재사용이 가능!
```

```
// 변수 선언!
```

```
let a = 2;
```

```
let b = 5;
```

```
console.log(a + b); // 7
```

```
console.log(a - b); // -3
```

```
console.log(a * b); // 10
```

```
console.log(a / b); // 0.4
```

// 값(데이터)의 재할당 가능!

```
let a = 12;
```

```
console.log(a); // 12
```

```
a = 999;
```

```
console.log(a); // 999
```


// 값(데이터)의 재할당 불가!

```
const a = 12;
```

```
console.log(a); // 12
```

```
a = 999;
```

```
console.log(a); // TypeError: Assignment to constant variable.
```

예약어

특별한 의미를 가지고 있어, 변수나 함수 이름 등으로 사용할 수 없는 단어

Reserved Word

```
let this = 'Hello!'; // SyntaxError
```

```
let if = 123; // SyntaxError
```

```
let break = true; // SyntaxError
```


break, case, catch, continue, default, delete, do, else, false, finally, for, function, if, in, instanceof, new, null, return, switch, this, throw, true, try, typeof, var, void, while, with,
abstract, boolean, byte, char, class, const, debugger, double, enum, export, extends, final, float, goto, implements, import, int, interface, long, native, package, private, protected, public, short, static, super, synchronized, throws, transient, volatile, as, is,
namespace, use, arguments, Array, Boolean, Date, decodeURI, decodeURIComponent, encodeURI, Error, escape, eval,
EvalError, Function, Infinity, isFinite, isNaN, Math, NaN, Number, Object, parseFloat, parseInt, RangeError,
ReferenceError, RegExp, String, SyntaxError, TypeError,
undefined, unescape, URIError ...

함수

특정 동작(기능)을 수행하는 일부 코드의 집합(부분)

function

```
// 함수 선언
```

```
function helloFunc() {
```

```
    // 실행 코드
```

```
    console.log(1234);
```

```
}
```

```
// 함수 호출
```

```
helloFunc(); // 1234
```



```
function returnFunc() {  
    return 123;  
}
```

```
let a = returnFunc();
```

```
console.log(a); // 123
```

```
// 함수 선언!
```

```
function sum(a, b) { // a와 b는 매개변수(Parameters)  
    return a + b;  
}
```

```
// 재사용!
```

```
let a = sum(1, 2); // 1과 2는 인수(Arguments)
```

```
let b = sum(7, 12);
```

```
let c = sum(2, 4);
```

```
console.log(a, b, c); // 3, 19, 6
```



```
// 기명(이름이 있는) 함수  
// 함수 선언!  
function hello() {  
    console.log('Hello~');  
}
```

```
// 익명(이름이 없는) 함수  
// 함수 표현!  
let world = function () {  
    console.log('World~');  
}
```

```
// 함수 호출!  
hello(); // Hello~  
world(); // World~
```

```
// 객체 데이터
```

```
const heropy = {  
  name: 'HEROPY',  
  age: 85,  
  // 메소드(Method)  
  getName: function () {  
    return this.name;  
  }  
};
```

```
const hisName = heropy.getName();
```

```
console.log(hisName); // HEROPY
```

```
// 혹은
```

```
console.log(heropy.getName()); // HEROPY
```

조건문

조건에 따라 다른 코드를 실행하는 구문
if, else


```
let isShow = true;  
let checked = false;
```

```
if (isShow) {  
    console.log( 'Show! ' ); // Show!  
}
```

```
if (checked) {  
    console.log( 'Checked! ' );  
}
```

```
let isShow = true;
```

```
if (isShow) {  
    console.log( 'Show! ' );  
} else {  
    console.log( 'Hide? ' );  
}
```

DOM API

Document Object Model, Application Programming Interface


```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
  <script src="./main.js"></script>
</head>
<body>
  <div class="box">Box!!</div>
</body>
</html>
```

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
</head>
<body>
  <div class="box">Box!!</div>
  <script src="./main.js"></script>
</body>
</html>
```



```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
  <script defer src="./main.js"></script>
</head>
<body>
  <div class="box">Box!!</div>
</body>
</html>
```

```
// HTML 요소(Element) 1개 검색/찾기
const boxEl = document.querySelector( '.box' );

// HTML 요소에 적용할 수 있는 메소드!
boxEl.addEventListener();

// 인수(Arguments)를 추가 가능!
boxEl.addEventListener(1, 2);

// 1 - 이벤트(Event, 상황)
boxEl.addEventListener('click', 2);

// 2 - 핸들러(Handler, 실행할 함수)
boxEl.addEventListener('click', function () {
    console.log('Click~!');
});
```

// HTML 요소(Element) 검색/찾기

```
const boxEl = document.querySelector( '.box' );
```

// 요소의 클래스 정보 객체 활용!

```
boxEl.classList.add( 'active' );
```

```
let isContains = boxEl.classList.contains( 'active' );
```

```
console.log( isContains ); // true
```

```
boxEl.classList.remove( 'active' );
```

```
isContains = boxEl.classList.contains( 'active' );
```

```
console.log( isContains ); // false
```



```
// HTML 요소(Element) 모두 검색/찾기
const boxEls = document.querySelectorAll('.box');
console.log(boxEls);

// 찾은 요소들 반복해서 함수 실행!
// 익명 함수를 인수로 추가!
boxEls.forEach(function () {});

// 첫 번째 매개변수(boxEl): 반복 중인 요소.
// 두 번째 매개변수(index): 반복 중인 번호
boxEls.forEach(function (boxEl, index) {});

// 출력!
boxEls.forEach(function (boxEl, index) {
    boxEl.classList.add(`order-${index + 1}`);
    console.log(index, boxEl);
});
```

```
const boxEl = document.querySelector( '.box' );
```

```
// Getter, 값을 얻는 용도
```

```
console.log(boxEl.textContent); // Box!!
```

```
// Setter, 값을 지정하는 용도
```

```
boxEl.textContent = 'HEROPY?!';
```

```
console.log(boxEl.textContent); // HEROPY?!
```

메소드 체이닝

Method Chaining



```
const a = 'Hello~';
```

```
// split: 문자를 인수 기준으로 쪼개서 배열로 반환.
```

```
// reverse: 배열을 뒤집기.
```

```
// join: 배열을 인수 기준으로 문자로 병합해 반환.
```

```
const b = a.split('').reverse().join(''); // 메소드 체이닝...
```

```
console.log(a); // Hello~
```

```
console.log(b); // ~olleH
```

Q.

The quick brown fox

위 문장을 camelCase(낙타 표기법)로 작성하시오!

A.

theQuickBrownFox

Q.

```
let fruits = ['Apple', 'Banana', 'Cherry'];
```

위 데이터를 활용해 'Banana'를 **콘솔 출력**하시오!

A.

```
console.log(fruits[1]);
```


Q.

불린 데이터(Boolean)에서
거짓을 의미하는 데이터는?

A.

false

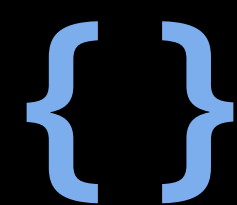
Q.

'값이 의도적으로 비어있음'을 의미하는 데이터는?

A.

null

Q.



위 데이터의 종류는?

A.

Object(객체 데이터)

Q.

```
let obj = { abc: 123 };  
console.log(obj.xyz);
```

위 코드를 통해 **콘솔 출력될 값**(데이터)은?

A.

undefined

Q.

값(데이터)을 재할당할 수 없는
변수 선언 키워드는?

A.

const

Q.

함수에서 값(데이터)을 반환하기 위해
사용하는 키워드는?

A.

return

Q.

`sum(2, 4);`

위 함수 호출에서 2, 4를 무엇이라 하는가?

A.

인수(Arguments)

Q.

```
function sum(a, b) {  
    return a + b;  
}
```

위 함수 선언의 **a, b**와 같이,

함수 호출에서 전달받은 **인수**를

함수 내부로 **전달**하기 위한 **변수**를 무엇이라 하는가?

A.

매개변수(Parameters)

Q.

이름이 없는 함수를 무엇이라 하는가?

A.

익명 함수(Anonymous Function)

Q.

hello 이름의 함수 표현을 작성하고 호출하시오!

A.

```
const hello = function () {};  
hello();
```

Q.

```
const user = {  
  getName: function () {}  
}
```

위 코드의 `getName`과 같이,
함수가 할당된 객체 데이터의
속성(Property)을 무엇이라 하는가?

A.

메소드(Method)

Q.

조건이 참(true)인 조건문을 작성하시오!

A.

```
if (true) { }
```


Q.

가져온 JS 파일을 HTML 문서 분석 이후에 실행하도록
지시하는 HTML 속성(Attribute)은?

A.

defer

Q.

```
<div class="box">Box!!</div>
```

위 HTML 요소의 내용(Content)을 콘솔 출력하시오!

A.

```
const boxEl = document.querySelector('.box');  
console.log(boxEl.textContent);
```

Q.

값(데이터)을 재할당할 목적의
변수 선언 키워드는?

A.

let

Q.

```
const boxEl = document.querySelector('.box');
```

위 코드의 `boxEl` 요소에 **클릭(Click) 이벤트**를 **추가**해,
클릭 시 **'Hello~'**를 **콘솔 출력**하시오!

A.

```
boxEl.addEventListener('click', function () {  
    console.log('Hello~');  
});
```


Q.

```
<div>1</div>
```

```
<div>2</div>
```

위 2개의 DIV 요소에 JS로
class="hello"를 추가하시오!

A.

```
const divEls = document.querySelectorAll('div');  
divEls.forEach(function (divEl) {  
    divEl.classList.add('hello');  
});
```

Q.

```
'HEROPY'.split(' ').reverse().join(' ');
```

위와 같이, 메소드를 이어 작성하는 방법을
무엇이라 하는가?

A.

메소드 체이닝(Method Chaining)

Q.

```
const boxEl = document.querySelector('.box');
```

위 코드의 `boxEl` 요소에 HTML 클래스 속성의
값으로 `'active'`가 포함되어 있으면,
`'포함됨!'`을 **콘솔 출력**하시오!

A.

```
if (boxEl.classList.contains('active')) {  
    console.log('포함됨!');  
}
```